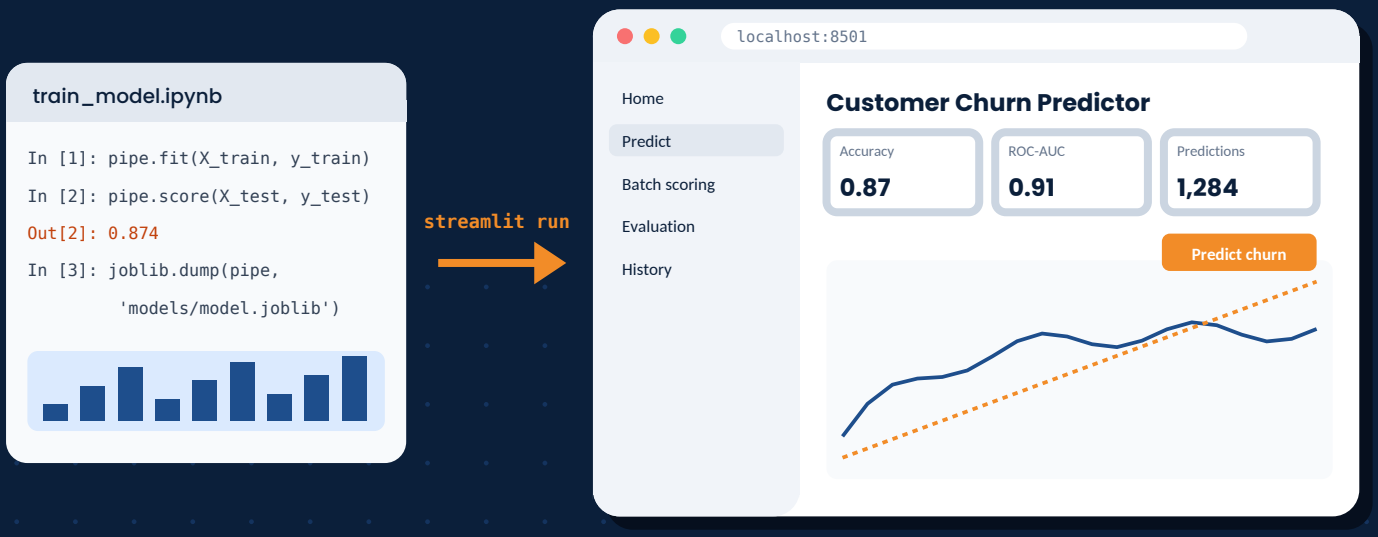
SPEEDUP INFOTECH | DATA SCIENCE & AI TRAINING SERIES

Complete Streamlit Guide for Machine Learning Deployment

From Jupyter Notebook to Interactive ML Web Applications



22 chapters | 10 hands-on projects | Capstone platform | Docker & cloud deployment | 30+ exercises

Streamlit 1.65 | Python 3.10+ | scikit-learn | TensorFlow/Keras | PyTorch | FastAPI | Docker

Ashwin Chaudhari
Data Scientist & AI Trainer, SpeedUp Infotech, Pune

OCTOBER 2026 EDITION
Verified against Streamlit 1.65.0

About This Guide

Most Data Science courses end at the notebook: a model is trained, a metric is printed, and the `.joblib` file sits in a folder that nobody else can use. This guide starts exactly there. It teaches you to turn a trained Machine Learning or Deep Learning model into an interactive web application with **Streamlit**, and then to test, secure, containerise and deploy it so that other people can use it.

The book moves from beginner to advanced. Chapters 1-5 teach Streamlit itself: how it runs, every major component, and how to design a clean interface. Chapters 6-12 connect real models to the UI: classification, regression, clustering, forecasting, image and text models, batch scoring, state management, databases, APIs and reports. Chapters 13-16 cover deployment, Docker, security and project architecture. Chapters 17-22 are practice: ten complete projects, an evaluation dashboard, a troubleshooting guide, a quick reference, exercises and a capstone.

Who this guide is for

- Data Science students who can train models in Jupyter and want a portfolio app.
- ML beginners who know pandas, NumPy and scikit-learn but no HTML, CSS or JavaScript.
- ML engineers who need a fast way to put a model in front of stakeholders.
- Trainers who teach deployment and want complete, tested classroom material.

What you need

Python 3.10 or newer, a code editor (VS Code or PyCharm), a terminal, and the ability to run `pip install`. A free GitHub account is needed for the cloud-deployment chapter. A GPU is not required: every model in the book trains on a laptop CPU in under a minute.

How the code in this book was verified

Every Python listing is parsed for valid syntax when the PDF is built. The project applications were executed end-to-end: datasets were generated, models were trained, and each Streamlit app was run with Streamlit's own testing framework (`streamlit.testing.v1.AppTest`) and in a real browser. The screenshots you see are of those running apps, not mock-ups. Where a listing is a deliberate fragment (for example, one function shown out of context) its code block has a purple edge and is labelled **PSEUDOCODE / FRAGMENT**.

Software	Version used	Notes
Streamlit	1.65.0	Latest stable release at the time of writing (Oct 2026)
Python	3.13	All code also runs on 3.10+ unless noted
pandas / NumPy	3.0 / 2.5	pandas 3 uses a dedicated string dtype
scikit-learn	1.9.1	Pipelines, ColumnTransformer, metrics
TensorFlow / Keras	2.21 / 3.15	Image classification chapter
PyTorch	2.14	PyTorch adaptation of the image model
Plotly / Altair	7.1 / 6.3	Interactive charts
statsmodels	0.15	SARIMAX forecasting with prediction intervals
FastAPI / SQLAlchemy	0.143 / 2.1	API and database chapters

Table 1: Versions used to test every example

VERSION NOTE

Streamlit releases often (roughly monthly). Names such as `st.navigation`, `st.dialog`, `st.fragment`, `st.datetime_input`, `st.space`, horizontal containers and the `width="stretch"` argument are recent additions. If an example fails on an older install, run `pip install --upgrade streamlit` or check the release notes at docs.streamlit.io.

Deprecated in current Streamlit: `use_container_width=` (use `width="stretch"` or `width="content"`), `use_column_width=` on images (removed), `st.experimental_rerun()` (use `st.rerun()`), and `st.bokeh_chart()` (removed; use the `streamlit-bokeh` component). This book uses only the current APIs.

Conventions used in this book

Box	Meaning
NOTE	Background information worth knowing.
TIP	A shortcut or productivity idea.
BEST PRACTICE	The way professionals usually do it.
WARNING	Something that commonly goes wrong.
SECURITY WARNING	A risk to users, data or servers.
COMMON MISTAKE	An error students make often, with the fix.
VERSION NOTE	Behaviour that depends on the Streamlit or library version.
RESPONSIBLE ML	Fairness, privacy and human-oversight considerations.
TRY IT	A short hands-on activity.

Code blocks show the file name in their header bar, for example `churn_app/app.py`. Unless stated otherwise, run commands from the project folder that contains that file. Terminal commands are labelled TERMINAL; Windows-specific commands are labelled WINDOWS CMD or POWERSHELL.

Companion project folders

Folder layout of the companion code — TEXT

```
streamlit_ml_guide/
├── iris_app/          Chapter 17, Project 1   (first ML app)
├── house_app/         Chapter 6 + Project 2   (regression)
├── churn_app/         Chapters 6, 8-12, 14    (classification, batch, DB, API, reports)
├── segment_app/       Chapter 6 + Project 4   (clustering)
├── forecast_app/      Chapter 6 + Project 5   (time series)
├── sentiment_app/     Chapter 6 + Project 6   (NLP)
├── image_app/         Chapter 6 + Project 7   (Keras + PyTorch)
├── fraud_app/         Chapter 6 + Project 8   (anomaly detection)
├── resume_app/        Chapter 17, Project 9   (document classification)
├── eval_app/          Chapter 18 + Project 10 (evaluation dashboard)
├── multipage_demo/    Chapter 4               (st.navigation)
├── demos/             Chapters 3-5            (component catalogue, design examples)
└── capstone/          Chapter 22              (production-style platform)
```

All datasets are generated by small scripts inside each project (for example `data/make_dataset.py`) so every example works offline. Each generator produces the same column names as a well-known public dataset, so you can later swap in real data from Kaggle or the UCI repository without changing the app.

Table of Contents

1. Introduction to Streamlit 1

1.1 What is Streamlit? 1

1.2 Why Machine Learning engineers use Streamlit 1

1.3 What problems does Streamlit solve? 2

1.4 Streamlit vs Flask vs FastAPI vs Django vs Gradio 2

1.5 Streamlit architecture and execution workflow 3

1.6 How Streamlit differs from traditional front-end development 3

1.7 How Streamlit interacts with Python ML models 4

1.8 The rerun model: how Streamlit reacts to widgets 4

1.9 Understanding the Streamlit server and the browser 5

1.10 What Streamlit can and cannot do 5

1.11 When to choose Streamlit for ML deployment 6

1.12 Real-world examples of Streamlit applications 6

1.13 From Jupyter Notebook to deployed application 8

Chapter summary 8

2. Environment Setup 9

2.1 Python environment setup 9

2.2 Creating a virtual environment 9

2.3 Installing Streamlit 10

2.4 Installing common ML libraries 10

2.5 Creating the project directory 10

2.6 Creating requirements.txt 11

2.7 Writing the first Streamlit application 12

2.8 Running an application locally 12

2.9 Understanding the terminal output 12

2.10 Stopping the Streamlit server 13

2.11 Reloading the application 13

2.12 Troubleshooting installation errors 13

Chapter summary 14

3. Streamlit Fundamentals 15

3.1 Importing Streamlit 15

3.2 A complete tour in one file 15

3.3 Component reference cards 18

3.4 Writing text: st.write, st.text and magic 21

3.5 Placeholders and live updates 21

3.6 Containers: writing out of order . . . 22
3.7 Fragments: rerunning only part of the page . . . 22
Chapter summary . . . 23

4. Complete Streamlit UI Components Catalog . . . 24
4A. Text and content components . . . 24
4B. Input widgets . . . 25
4C. Layout components . . . 32
4D. Data display components . . . 38
4E. Chart and visualisation components . . . 41
4F. Media and file components . . . 45
4G. Navigation components . . . 47
4H. Status and feedback components . . . 51
Chapter summary . . . 54

5. Advanced UI Customization . . . 55
5.1 Page configuration, titles and icons . . . 55
5.2 Sidebar configuration . . . 55
5.3 Custom themes with .streamlit/config.toml . . . 56
5.4 Colour, typography, spacing and hierarchy . . . 57
5.5 Images and logos . . . 57
5.6 Custom CSS: a card-style interface . . . 57
5.7 Design example 1: clean business dashboard . . . 58
5.8 Design example 2: dark-themed AI application . . . 59
5.9 Design example 3: modern prediction form . . . 61
5.10 Design example 4: model monitoring dashboard . . . 62
5.11 Responsive layout considerations . . . 64
5.12 Reusable UI functions . . . 64
5.13 Custom components and their limitations . . . 64
5.14 Professional application branding checklist . . . 65

6. Connecting Streamlit with Machine Learning Models . . . 66
6.1 The fourteen-step deployment workflow . . . 66
6.2 Why training-time and inference-time preprocessing must match . . . 67
6.3 Saving and loading models . . . 68
6.4 Classification project: Customer Churn Prediction . . . 70
6.5 Regression project: House Price Prediction . . . 83
6.6 Clustering project: Customer Segmentation . . . 89
6.7 Time-series forecasting project: Sales Forecasting . . . 93
6.8 Deep learning project: Image Classification (TensorFlow/Keras) . . . 97

6.9 NLP project: Sentiment Analysis . . . 104
6.10 Anomaly detection project: Transaction Fraud Screening . . . 107
6.11 Saving prediction history and preparing to deploy . . . 112

7. Building UI for Different Types of ML Projects . . . 113
7.1 Project-to-UI mapping (summary) . . . 113
7.2 Blueprint cards for all 25 application types . . . 114
7.3 Wireframes for the major categories . . . 123
7.4 High-impact applications: medical, financial and hiring . . . 128

8. Dataset Upload and Batch Prediction . . . 129
8.1 Training data versus inference data . . . 129
8.2 The complete batch prediction app . . . 129
8.3 How each step works . . . 133
8.4 Handling large files and memory . . . 133
8.5 Error reporting that users understand . . . 134

9. State Management and Performance . . . 135
9.1 Rerun behaviour, precisely . . . 135
9.2 Widget keys and st.session_state . . . 135
9.3 Session-specific prediction history . . . 136
9.4 Multi-step workflows . . . 137
9.5 st.cache_data vs st.cache_resource . . . 137
9.6 Caching ML models and avoiding repeated loading . . . 138
9.7 Caching datasets and cache invalidation . . . 138
9.8 Preventing duplicate work and handling expensive predictions . . . 139
9.9 Memory usage and session-state limitations . . . 140
9.10 Long-running tasks and progress . . . 140
9.11 Error handling and performance checklist . . . 141

10. Forms, Callbacks and Interactive Workflows . . . 142
10.1 Forms and submit buttons . . . 142
10.2 Callbacks: on_click and on_change . . . 142
10.3 Dependent widgets and conditional UI . . . 143
10.4 Input validation and reset buttons . . . 143
10.5 Project: a five-step prediction wizard . . . 144

11. Database and API Integration . . . 150
11.1 Architecture . . . 150
11.2 Database helper module (SQLite, PostgreSQL, MySQL) . . . 151

11.3 Secrets and environment variables . . . 152
11.4 A Streamlit app that stores predictions and feedback . . . 153
11.5 Serving the model with FastAPI . . . 156
11.6 Calling a REST API from Streamlit . . . 157
11.7 When should the model live behind an API? . . . 158

12. Report Generation and Downloads . . . 159
12.1 Choosing a report format . . . 159
12.2 Report-building helpers . . . 159
12.3 The report app . . . 161
12.4 Verifying generated files . . . 163
12.5 Model evaluation reports . . . 164

13. Deployment to the Cloud . . . 165
13.1 The deployment lifecycle . . . 165
13.2 Preparing the application . . . 165
13.3 requirements.txt for deployment . . . 166
13.4 Organising model files . . . 166
13.5 Environment variables and secrets . . . 167
13.6 Testing the production configuration locally . . . 167
13.7 Step by step: deploy to Streamlit Community Cloud . . . 168
13.8 Deploying to a virtual machine . . . 169
13.9 Deploying with Docker and managed cloud platforms . . . 170
13.10 Git, GitHub and CI/CD basics . . . 170
13.11 Logs and error monitoring . . . 171
13.12 Updating deployed apps and managing artifacts . . . 171
13.13 Large models and public versus private apps . . . 171

14. Docker Deployment . . . 173
14.1 What Docker is and why it helps . . . 173
14.2 The Dockerfile . . . 173
14.3 Build, run, inspect and stop . . . 175
14.4 Model files: bake in or mount? . . . 175
14.5 Docker Compose: app + PostgreSQL . . . 176
14.6 Production considerations . . . 176

15. Security and Production Best Practices . . . 178
15.1 Classroom demo versus production application . . . 178
15.2 Input validation . . . 178
15.3 File upload restrictions and malicious files . . . 179

15.4 Preventing unsafe model deserialisation . . . 179
15.5 Protecting secrets . . . 179
15.6 Authentication and authorisation . . . 180
15.7 Avoiding exposure of sensitive data and logging safely . . . 180
15.8 Dependency management and model versioning . . . 181
15.9 Error handling and resource management . . . 181
15.10 Data privacy and responsible ML predictions . . . 182
15.11 Production monitoring . . . 182

16. Streamlit Project Architecture . . . 183

16.1 Principles . . . 183
16.2 Architecture 1: beginner (single file) . . . 183
16.3 Architecture 2: intermediate (modular single app) . . . 184
16.4 Architecture 3: advanced production-style . . . 184
16.5 Importing functions correctly . . . 185
16.6 Configuration files . . . 185
16.7 Logging . . . 186
16.8 Testing . . . 187
16.9 Model version management . . . 188

17. Complete Hands-On Projects . . . 190

Project 1: Iris Flower Classification . . . 190
Project 2: House Price Prediction . . . 195
Project 3: Customer Churn Prediction . . . 198
Project 4: Customer Segmentation Dashboard . . . 201
Project 5: Sales Forecasting Application . . . 203
Project 6: Sentiment Analysis Application . . . 207
Project 7: Image Classification Application . . . 209
Project 8: Fraud Detection Application . . . 212
Project 9: Resume Classification Application . . . 215
Project 10: End-to-End ML Model Evaluation and Deployment Dashboard . . . 221

18. ML Model Evaluation Dashboard . . . 224

18.1 Evaluation versus prediction . . . 224
18.2 Which metrics apply to which task . . . 224
18.3 Building the model registry . . . 225
18.4 Metric functions with safeguards . . . 226
18.5 The dashboard app . . . 229
18.6 Tests . . . 237

19. Troubleshooting Guide . . . 239
19.1 A systematic debugging routine . . . 239
19.2 Quick lookup table . . . 239
19.3 Detailed fixes . . . 240
20. Streamlit Quick Reference . . . 247
20.1 Common commands . . . 247
20.2 Categorised component table . . . 247
20.3 Widgets, layout and state . . . 248
20.4 Caching, files and charts . . . 249
20.5 Navigation, model loading, prediction and errors . . . 249
20.6 Error-handling patterns . . . 250
20.7 Deployment checklist . . . 250
21. Exercises and Assessments . . . 251
21.1 Beginner exercises . . . 251
21.2 Intermediate exercises . . . 252
21.3 Advanced exercises . . . 253
21.4 Solutions and solution outlines . . . 254
22. Final Capstone Project . . . 257
22.1 Business requirements . . . 257
22.2 Functional requirements . . . 257
22.3 UI wireframes . . . 258
22.4 Project folder structure . . . 259
22.5 Implementation plan . . . 260
22.6 Source files . . . 260
22.7 Running the capstone . . . 276
22.8 Testing checklist . . . 283
22.9 Deployment checklist . . . 284
22.10 Future enhancements . . . 284
22.11 Presenting the capstone as a portfolio project . . . 284
A. Glossary, Resources and Final Checklist . . . 285
A.1 Glossary . . . 285
A.2 Official documentation resources . . . 286
A.3 Final deployment checklist . . . 286

CHAPTER 1

Introduction to Streamlit

01

Why ML engineers ship apps in pure Python

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Explain what Streamlit is and which problems it solves for ML work.
- ✓ Compare Streamlit with Flask, FastAPI, Django and Gradio.
- ✓ Describe the browser-server architecture and the top-to-bottom rerun model.
- ✓ Map the full journey from Jupyter Notebook to a deployed ML application.
- ✓ Decide when Streamlit is (and is not) the right tool.

Before you start: basic Python and a model you have trained in a notebook.

1.1 What is Streamlit?

Streamlit is an open-source Python framework for building interactive web applications for data and machine learning. You write an ordinary Python script; each call such as `st.title()`, `st.slider()` or `st.dataframe()` adds an element to a web page. There is no HTML, CSS or JavaScript to write, no URL routing to configure and no separate front-end project to build.

Streamlit was released in 2019 and acquired by Snowflake in 2022. It remains open source (Apache 2.0 licence) and installs with `pip install streamlit`.

hello.py (run with: streamlit run hello.py) — PYTHON

```python
import streamlit as st

st.title("Hello, Streamlit")
name = st.text_input("Your name")
if name:
    st.write(f"Welcome, {name}!")
```

EXPECTED OUTPUT

A page with a large title and a text box. When you type a name and press Enter, a greeting appears below the box. Those five lines are a complete web application.

1.2 Why Machine Learning engineers use Streamlit

- **Same language as the model** — Your preprocessing, `joblib.load()` and `model.predict()` code moves from the notebook into the app almost unchanged.
- **Minutes, not weeks** — A working prediction form takes about 30 lines. A polished multipage tool takes a few hundred.
- **Rich data widgets built in** — Interactive tables, charts, file upload, download buttons, metrics and maps need no extra front-end code.

- **Live reload** — Save the file and the browser updates. This tight loop makes experimenting with a UI as quick as experimenting in a notebook.
- **Free and simple hosting options** — Streamlit Community Cloud deploys straight from a GitHub repository; Docker images run anywhere.
- **Portfolio value** — A recruiter can click a link and *use* your model, which is far more convincing than a notebook screenshot.

1.3 What problems does Streamlit solve?

Problem without Streamlit	How Streamlit helps
Models live in notebooks only data scientists can run	A browser UI anyone can use, with no Python install on their side
Building a web front-end needs HTML/CSS/JS skills	Widgets are Python function calls that return Python values
Stakeholder demos are static slides	Live what-if analysis: move a slider and see the prediction change
Batch scoring needs a script run by an engineer	Upload a CSV, click a button, download predictions
Evaluating models means re-running notebooks	Shareable dashboards for metrics, confusion matrices and drift
Internal tools take a team weeks to build	One person can build a useful internal tool in a day

1.4 Streamlit vs Flask vs FastAPI vs Django vs Gradio

All five are Python frameworks, but they solve different problems. Streamlit and Gradio generate the user interface for you. Flask and Django are general web frameworks where you design every page. FastAPI builds **APIs**: machine-readable endpoints that other programs call, with no user interface at all.

Criterion	Streamlit	Gradio	Flask	FastAPI	Django
Primary purpose	Data / ML apps and dashboards	ML model demos	General web apps	REST APIs	Full-stack web platforms
UI written in	Python only	Python only	HTML templates + JS	None (JSON only)	HTML templates + JS
Learning curve	Very low	Very low	Medium	Medium	High
Layout control	Good (columns, tabs, containers)	Moderate (Blocks)	Total	n/a	Total
Multipage apps	Built in (st.navigation)	Tabs / mounted apps	Manual routes	n/a	Full URL routing
Dashboards and charts	Excellent	Basic	Manual	n/a	Manual
Serving predictions to other software	Not designed for it	Possible via API	Possible	Excellent	Possible
Concurrency / scale	Moderate (one server process, sessions per user)	Moderate (queue)	Good with WSGI server	Excellent (async)	Excellent
Authentication	OIDC login (st.login) or proxy	Basic / OAuth	Extensions	Libraries	Built in
Typical ML use	Internal tools, demos, batch scoring, monitoring	Quick model demos, Hugging Face Spaces	Custom web products	Model microservices	Large products with users and admin

Table 1.1: Choosing a Python framework for ML deployment

BEST PRACTICE

A common professional pattern combines two of them: **FastAPI serves the model** (fast, testable, callable by any system) and **Streamlit provides the human interface** that calls the API. Chapter 11 builds exactly this.

1.5 Streamlit architecture and execution workflow

A Streamlit app has two halves that talk over a WebSocket connection:

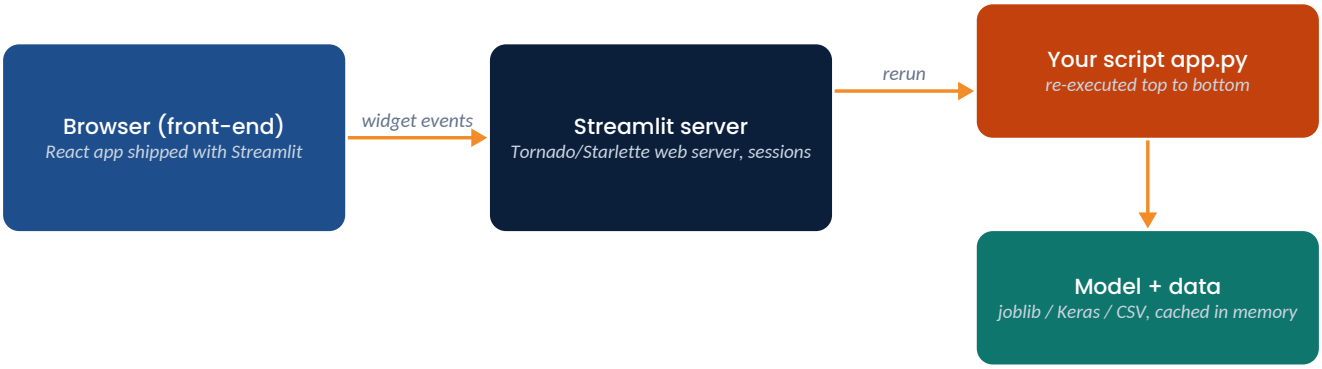


Figure 1.1: Streamlit's runtime: the browser renders, the server runs your Python

1. You run `streamlit run app.py`. A web server starts (port 8501 by default).
2. A user opens the URL. The browser downloads Streamlit's front-end and opens a WebSocket to the server. The server creates a **session** for that browser tab.
3. The server executes `app.py` from top to bottom. Each `st.*` call sends a small message describing an element; the browser draws it.
4. When the user changes a widget, the browser sends the new value. The server **re-runs the whole script** with that value, and the page updates.

1.6 How Streamlit differs from traditional front-end development

Traditional web stack	Streamlit
Separate front-end (HTML/CSS/JavaScript) and back-end code	One Python script produces both
Event handlers update parts of the page	The whole script re-runs; Streamlit updates only what changed
You design URLs, forms, and JSON endpoints	Widgets return values directly: `age = st.slider(...)`
State lives in the browser, database or cookies	State lives in `st.session_state` on the server, per browser tab
Pixel-level design control	Opinionated, consistent design with theming and layout primitives; custom CSS is possible but fragile

1.7 How Streamlit interacts with Python ML models

The model never runs in the browser. It runs in the server's Python process, exactly as it would in your notebook. The browser only sends inputs and receives outputs. A minimal prediction loop looks like this:

model_loop.py (illustrative: requires a model trained on these two columns) PYTHON

```python
import joblib
import pandas as pd
import streamlit as st


@st.cache_resource                    # load ONCE, reuse on every rerun
def load_model():
    return joblib.load("models/model.joblib")


model = load_model()
tenure = st.slider("Tenure (months)", 0, 72, 12)
charges = st.number_input("Monthly charges", 18.0, 120.0, 70.0)
X = pd.DataFrame([{"tenure_months": tenure, "monthly_charges": charges}])
st.metric("Churn probability", f"{model.predict_proba(X)[0, 1]:.1%}")
```

NOTE

Without `@st.cache_resource` the model file would be read from disk on every widget change, because the script re-runs each time. Caching is covered in depth in Chapter 9.

1.8 The rerun model: how Streamlit reacts to widgets

This is the single most important idea in Streamlit. **Every interaction re-runs your script from the first line to the last.** Variables are recreated each time. A widget remembers its own value between runs, and `st.session_state` remembers anything else you store in it.

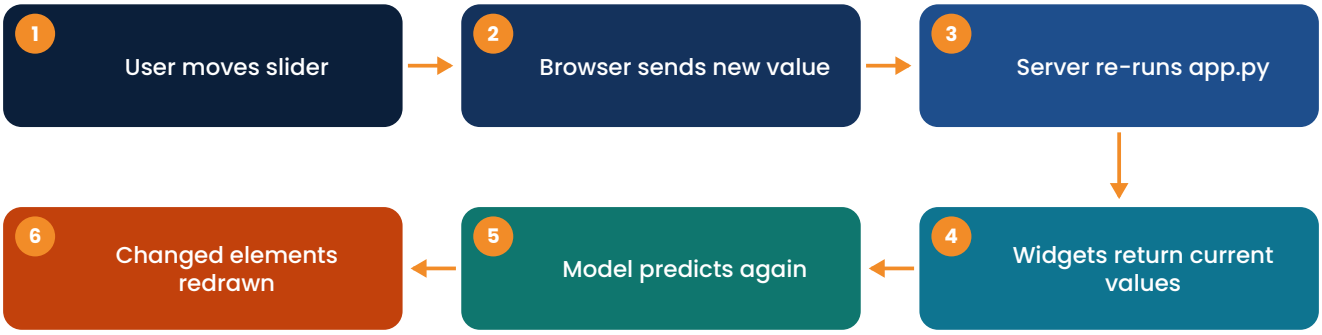


Figure 1.2: One interaction = one full script run

rerun_demo.py PYTHON

```python
import streamlit as st

st.write("Script started")                  # printed on EVERY run
count = 0                                   # reset to 0 on every run
if st.button("Click me"):
    count += 1                              # always ends up 1 at most
st.write("count =", count)

if "clicks" not in st.session_state:        # session_state survives reruns
    st.session_state.clicks = 0
if st.button("Click me too"):
    st.session_state.clicks += 1
st.write("clicks =", st.session_state.clicks)
```

EXPECTED OUTPUT

`count` never goes above 1, because it is reset at the top of every run. `clicks` keeps increasing because it lives in `st.session_state`.

COMMON MISTAKE

Putting expensive work (reading a 2 GB CSV, loading a model, calling an API) at the top level without caching. It runs again on every click and makes the app feel frozen. Wrap it in `@st.cache_data` or `@st.cache_resource`.

1.9 Understanding the Streamlit server and the browser

- **One server process, many sessions** — Each browser tab gets its own session and its own `st.session_state`. Two users never see each other's widget values.
- **Shared memory for caches** — `@st.cache_resource` objects (like a loaded model) are shared by all sessions in the process; that is why they must be read-only.
- **Local address** — `http://localhost:8501` only works on your computer. The Network URL printed in the terminal works for other devices on the same Wi-Fi.
- **Stateless restarts** — Restarting the server clears every session and in-memory cache. Anything that must survive belongs in a file or database.

1.10 What Streamlit can and cannot do

Streamlit is great for	Streamlit is not designed for
Interactive prediction forms and what-if tools	High-traffic consumer websites with millions of users
Dashboards, reports and model monitoring	Pixel-perfect marketing pages or complex custom animations
Batch scoring of uploaded CSV/Excel files	Being the API that other software calls (use FastAPI)
Internal ML tools for analysts and managers	Long-running background jobs inside a request (use a job queue)
Portfolio demos and teaching	Offline mobile apps
Chat interfaces for LLM and RAG prototypes	Fine-grained, role-based permission systems out of the box

1.11 When to choose Streamlit for ML deployment

Choose Streamlit when the main users are **people** who need to interact with a model or its results, the audience is in the tens to low thousands, and you want to ship quickly in Python. Choose FastAPI when the main users are **programs**. Choose a full web framework when you are building a product with complex accounts, payments and custom design. When in doubt, prototype in Streamlit: the model code you write is reusable in any of the others.

1.12 Real-world examples of Streamlit applications

Domain	Example application	Typical components
Retail	Demand forecasting tool for store managers	date_input, line chart with intervals, download_button
Telecom / SaaS	Churn risk dashboard for retention teams	metrics, dataframe with progress columns, threshold slider
Banking	Credit-risk pre-check for analysts (human-in-the-loop)	forms, validation messages, model card page
Healthcare research	Educational image classifier with confidence display	file_uploader, image, bar chart of probabilities
HR tech	Resume routing assistant	multi-file upload, text extraction, fairness warnings
MLOps	Model evaluation and drift monitoring	tabs, Plotly charts, fragments with auto-refresh
GenAI	Chat with company documents (RAG)	chat_input, chat_message, file_uploader, sources expander

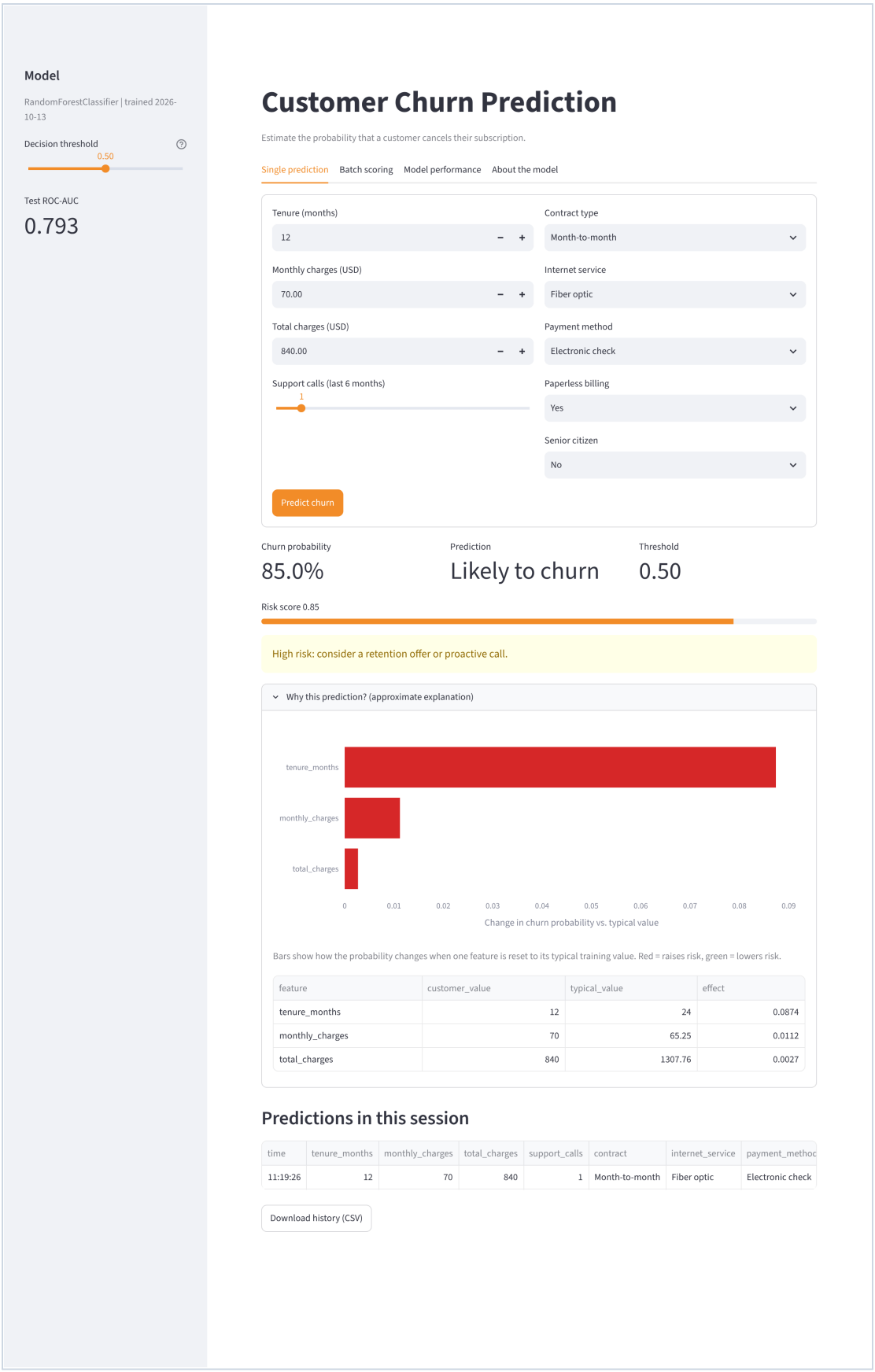


Figure 1.3: A Streamlit churn app built in Chapter 6: sidebar controls, tabs, a form, metrics and session history

1.13 From Jupyter Notebook to deployed application

Every project in this book follows the same eight-stage workflow. The first four stages happen in your notebook or a training script; the last four are what this book teaches.

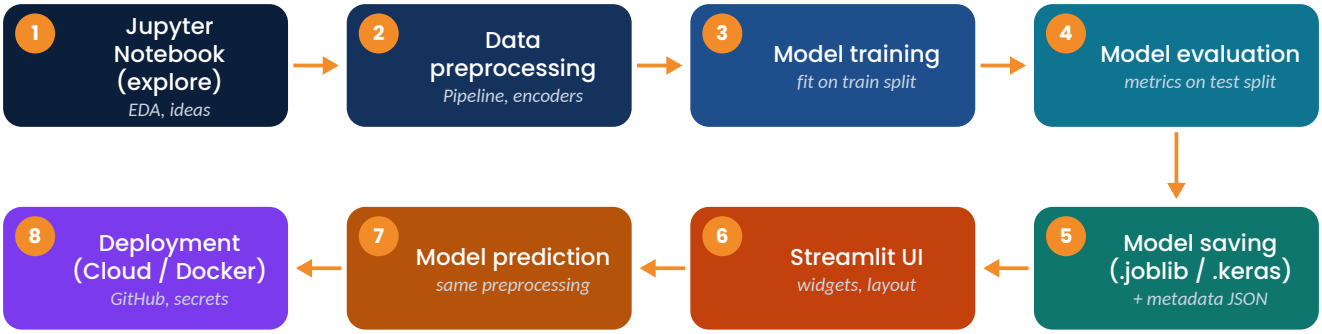


Figure 1.4: The end-to-end workflow followed throughout the book



Figure 1.5: Artifacts flow from training to the app; users only touch the app

Chapter summary

- Streamlit turns a Python script into a web app; widgets return Python values.
- The whole script re-runs on every interaction. Cache expensive work and keep persistent values in `st.session_state`.
- Streamlit is ideal for human-facing ML tools; FastAPI is better for program-facing model services. They combine well.
- The workflow is: notebook, preprocessing, training, evaluation, saving, UI, prediction, deployment.

CHAPTER 2

Environment Setup

02

Install, organise and run your first app on Windows, macOS and Linux

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Create an isolated virtual environment for an ML app.
- ✓ Install Streamlit and the common ML libraries, and pin them in requirements.txt.
- ✓ Create a clean project structure and explain the role of each folder.
- ✓ Run, reload and stop a Streamlit app, and read its terminal output.
- ✓ Fix the most common installation errors.

2.1 Python environment setup

Streamlit 1.65 supports Python 3.10 and newer. Check what you have:

● Check the Python version — TERMINAL

```
python --version        # Windows (or: py --version)
python3 --version       # macOS / Linux
```

If Python is missing or older than 3.10, install it from python.org (Windows/macOS) or your package manager (`sudo apt install python3 python3-venv` on Ubuntu). On Windows, tick **Add python.exe to PATH** during installation.

TIP

Anaconda users can use `conda create -n mlapp python=3.12` instead of venv. Do not mix conda and pip installs of the same package in one environment.

2.2 Creating a virtual environment

A **virtual environment** is a private folder of packages for one project. It prevents version conflicts between projects and makes `requirements.txt` accurate, which is essential for deployment.

● Windows (Command Prompt) — WINDOWS CMD

```
mkdir ml_deployment_project
cd ml_deployment_project
python -m venv .venv
.venv\Scripts\activate
```

● Windows (PowerShell) — POWERSHELL

```
python -m venv .venv
.venv\Scripts\Activate.ps1
# If blocked: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

macOS / Linux TERMINAL

```
mkdir ml_deployment_project && cd ml_deployment_project
python3 -m venv .venv
source .venv/bin/activate
```

Your prompt now starts with `(.venv)`. Every `pip install` goes into this environment. Type `deactivate` to leave it.

2.3 Installing Streamlit

Install and verify TERMINAL

```
pip install --upgrade pip
pip install streamlit
streamlit version
```

Expected output OUTPUT

```
Streamlit, version 1.65.0
```

`pip install streamlit` downloads Streamlit and its dependencies (including pandas, NumPy, PyArrow, Altair and a web server). `streamlit version` confirms the command-line tool is on your PATH.

2.4 Installing common ML libraries

Libraries used in most chapters TERMINAL

```
pip install pandas numpy scikit-learn matplotlib seaborn plotly joblib
```

Package	Why an ML app needs it
pandas	Reading CSV/Excel uploads, building the input DataFrame for the model
numpy	Array maths, probabilities, image tensors
scikit-learn	Pipelines, preprocessing, models and evaluation metrics
matplotlib / seaborn	Static charts such as confusion matrices and heatmaps shown with `st.pyplot`
plotly	Interactive charts (zoom, hover) shown with `st.plotly_chart`
joblib	Saving and loading trained scikit-learn models

Install extras only when a chapter needs them: `openpyxl` (Excel), `tensorflow` or `torch` (deep learning), `statsmodels` (forecasting), `sqlalchemy` (databases), `fastapi uvicorn` (APIs), `reportlab` (PDF reports).

2.5 Creating the project directory

Recommended project structure TEXT

```
ml_deployment_project/
│
├── app.py                  # the Streamlit app (entry point)
├── train_model.py          # trains and saves the model
├── predict.py              # reusable prediction function (no Streamlit code)
├── requirements.txt        # pinned dependencies for deployment
├── README.md               # how to install, run and use the project
├── .gitignore              # keeps .venv, secrets and caches out of Git
```

● Recommended project structure TEXT

```
├── .streamlit/
│   ├── config.toml         # theme and server settings (safe to commit)
│   └── secrets.toml        # passwords, API keys (NEVER commit)
├── models/
│   ├── model.joblib        # trained pipeline
│   └── metadata.json       # feature list, metrics, versions, fingerprint
├── data/
│   └── sample_data.csv     # small sample for demos and tests
├── utils/
│   ├── __init__.py
│   └── preprocessing.py    # helpers shared by training and the app
└── assets/
    └── logo.png            # images used in the UI
```

File / folder	Purpose
`app.py`	Entry point you pass to `streamlit run`. Contains UI code only.
`train_model.py`	Reproducible training: loads data, fits the pipeline, evaluates it, saves the model. Never imported by the app.
`predict.py`	Pure Python prediction logic, importable from the app, tests or an API. Keeps business logic out of the UI.
`requirements.txt`	Exact library versions; hosting platforms install from it.
`README.md`	Instructions for anyone (including future you) to run the project.
`models/`	Serialised model artifacts plus metadata. Version them with care.
`data/`	Small sample data only. Large or private datasets do not belong in Git.
`utils/`	Shared helpers such as preprocessing, validation and formatting.
`assets/`	Logos and static images displayed with `st.image` or `st.logo`.
`.streamlit/`	Configuration (`config.toml`) and secrets (`secrets.toml`).

2.6 Creating requirements.txt

Two approaches are common. Writing it by hand keeps the file short and readable; `pip freeze` captures every transitive dependency exactly.

● requirements.txt (hand-written, pinned) REQUIREMENTS

```
streamlit==1.65.0
pandas==3.0.5
numpy==2.5.3
scikit-learn==1.9.1
plotly==7.1.0
joblib==1.6.0
```

● Generate / install requirements TERMINAL

```
pip freeze > requirements.txt          # snapshot of the active environment
pip install -r requirements.txt        # recreate it elsewhere
```

WARNING

Pin **scikit-learn** to the exact version you trained with. A model saved with one version may fail to load, or behave subtly differently, on another.

2.7 Writing the first Streamlit application

app.py PYTHON

```python
import pandas as pd
import streamlit as st

st.set_page_config(page_title="My first ML app", layout="centered")
st.title("My first Streamlit app")
st.write("This app squares a number and shows a small table.")

number = st.slider("Pick a number", min_value=0, max_value=20, value=5)
st.metric("Square", number ** 2)

table = pd.DataFrame({"x": range(number + 1)})
table["x_squared"] = table["x"] ** 2
st.line_chart(table, x="x", y="x_squared")
```

2.8 Running an application locally

Run commands TERMINAL

```
streamlit hello            # built-in demo gallery: proves the install works
streamlit run app.py       # run YOUR app
python -m streamlit run app.py     # same, if 'streamlit' is not on PATH
streamlit run app.py --server.port 8502   # choose another port
```

Command	What it does
`pip install streamlit`	Installs the Streamlit library and CLI into the active environment.
`pip install pandas numpy scikit-learn matplotlib seaborn plotly joblib`	Installs the data and ML stack used by most apps in this book.
`streamlit hello`	Starts a demo app with animation, plotting, mapping and DataFrame examples. Use it to check the installation.
`streamlit run app.py`	Starts a local server, runs `app.py`, opens your browser at http://localhost:8501 and watches the file for changes.

2.9 Understanding the terminal output

Terminal after streamlit run app.py OUTPUT

```
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.23:8501
```

- **Local URL** — Works on this computer only.
- **Network URL** — Other devices on the same network can open it, which is handy for testing on a phone. Firewalls may block it.
- **Errors** — Python tracebacks appear in the terminal **and** in the browser during development.

2.10 Stopping the Streamlit server

Click the terminal and press **Ctrl + C**. Closing the browser tab does not stop the server; the app keeps running until you stop the process.

2.11 Reloading the application

Streamlit watches your source files. After you save `app.py`, the app shows a **Rerun** prompt (or reruns automatically if you choose **Always rerun**). You can also press **R** in the browser. Changes to imported modules in your project folder are detected too. A full restart (Ctrl + C, then run again) is needed after changing `.streamlit/config.toml` or installing packages.

● .streamlit/config.toml (optional) TOML

```toml
[server]
runOnSave = true        # rerun automatically when a source file is saved
```

2.12 Troubleshooting installation errors

Symptom	Likely cause	Fix
`'streamlit' is not recognized as an internal or external command`	Virtual environment not active, or Scripts folder not on PATH	Activate `.venv`, or run `python -m streamlit run app.py`
`ModuleNotFoundError: No module named 'sklearn'`	Package installed in a different Python	Activate the venv and run `pip install scikit-learn`; check `which python` / `where python`
`error: Microsoft Visual C++ 14.0 is required`	No pre-built wheel for your Python version	Use a Python version with wheels (3.10-3.13), upgrade pip, then retry
PowerShell: *running scripts is disabled*	Execution policy	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`Port 8501 is already in use`	Another app is running	Stop it, or `--server.port 8502`
Browser does not open	Headless server or SSH session	Open the Local URL manually
`pip` installs into system Python on Linux (externally-managed error)	PEP 668 protection	Always use a virtual environment

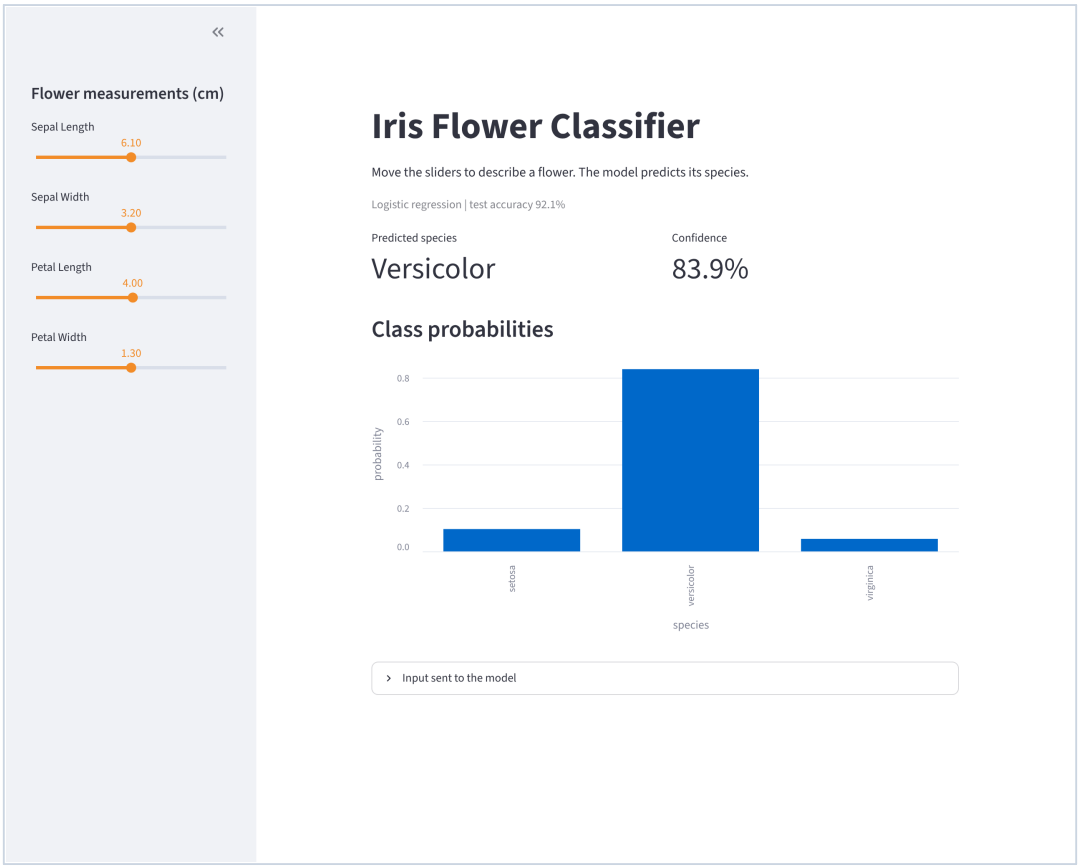


Figure 2.1: Your goal by Chapter 17: a working Iris classifier running locally

Chapter summary

- One virtual environment per project; activate it before installing or running.
- Pin versions in `requirements.txt`, especially scikit-learn.
- Use the standard layout: `app.py`, `train_model.py`, `models/`, `data/`, `utils/`, `.streamlit/`.
- `streamlit run app.py` starts the server; Ctrl + C stops it; saving reruns it.

CHAPTER 3

Streamlit Fundamentals

03

Text, messages, containers and the execution model

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Display text, headings, Markdown, code, maths and JSON.
- ✓ Communicate outcomes with success, info, warning, error and exception elements.
- ✓ Organise output with dividers, placeholders, containers and expanders.
- ✓ Use fragments to rerun only part of an app.

3.1 Importing Streamlit

app.py — PYTHON

```python
import streamlit as st          # the universal convention

st.set_page_config(page_title="Churn Predictor", layout="wide")   # optional, first
st.title("Customer Churn Predictor")
```

Every function lives in the `st` namespace. `st.set_page_config` is optional; if used it can be called anywhere but is usually placed first so the tab title and layout are set before anything is drawn.

3.2 A complete tour in one file

The following app uses every element in this chapter in the context of a churn model. Run it, then read the reference cards that follow.

demos/text_demo.py — PYTHON

```python
"""Text, content and message elements.  Run: streamlit run text_demo.py"""
import streamlit as st

st.title("Customer Churn Model", icon=":material/insights:")
st.header("1. Model overview")
st.subheader("Random forest, 300 trees")
st.markdown("The model predicts **churn risk** from *nine* customer features. "
            "See `train_model.py` for details. :orange[Threshold = 0.5]")
st.caption("Last trained: 13 Oct 2026 on 4,000 customers")
st.write("st.write renders many types, e.g. a dict:", {"accuracy": 0.87, "f1": 0.81})
st.divider()

st.header("2. Code, maths and JSON")
st.code("model = joblib.load('models/churn_pipeline.joblib')\n"
        "proba = model.predict_proba(X)[:, 1]", language="python")
st.latex(r"F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}"
         r"{\text{precision} + \text{recall}}")
st.json({"model": "RandomForestClassifier", "features": 9,
         "metrics": {"roc_auc": 0.91, "recall": 0.74}}, expanded=1)
st.divider()
```

demos/text_demo.py PYTHON

```python
st.header("3. Status messages")
st.success("Model loaded successfully.", icon=":material/check_circle:")
st.info("Predictions are estimates, not decisions.")
st.warning("3 rows have values outside the training range.")
st.error("Column 'contract' is missing from the uploaded file.")
try:
    float("not-a-number")
except ValueError as exc:
    st.exception(exc)

st.header("4. Containers and placeholders")
with st.expander("Technical details", expanded=True):
    st.write("Preprocessing: median imputation, scaling, one-hot encoding.")
box = st.container(border=True)
box.write("This text is inside a bordered container.")
placeholder = st.empty()
placeholder.info("Placeholder content: replaced when the prediction is ready.")
```

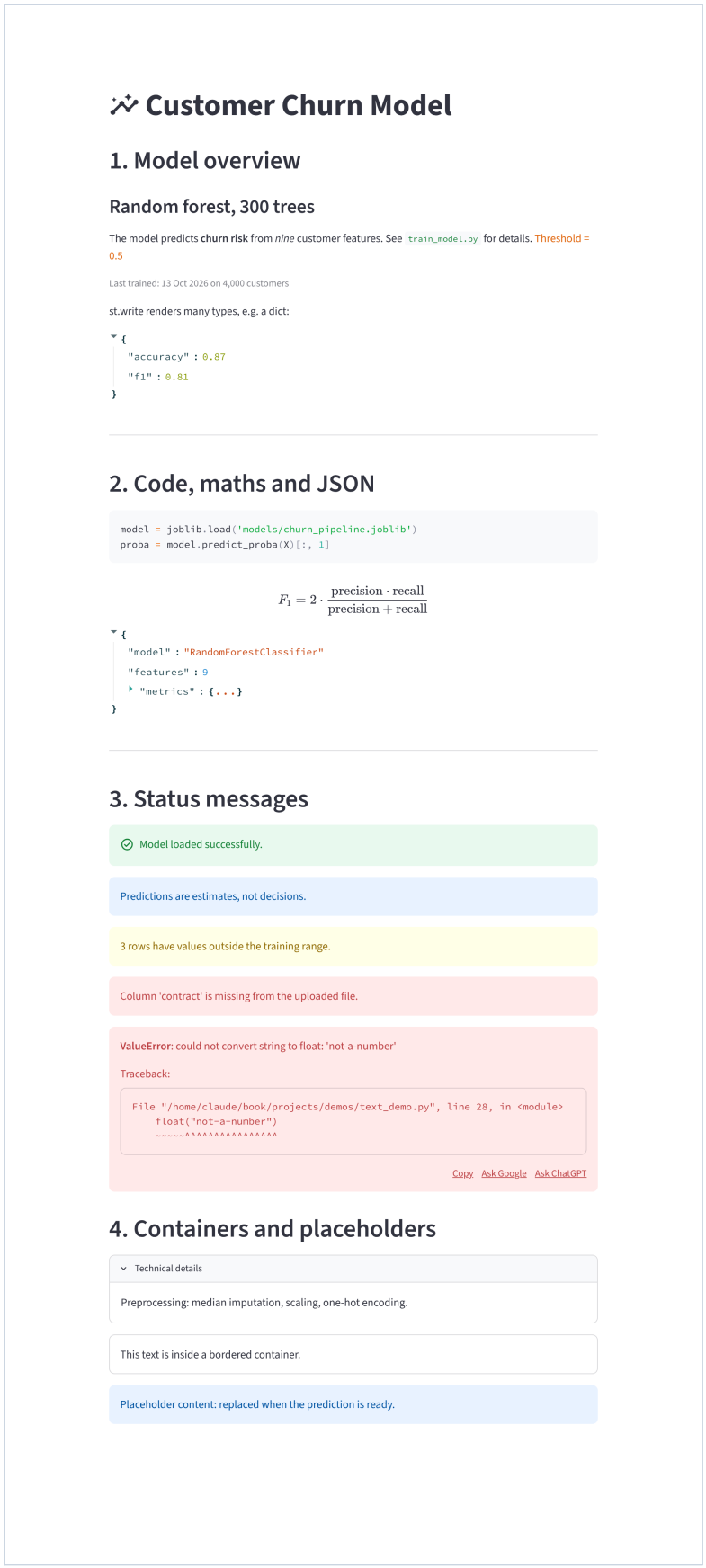


Figure 3.1: Output of text_demo.py: headings, Markdown, code, LaTeX, JSON, messages, exception, expander, container and placeholder

3.3 Component reference cards

Each card lists what the element does, when to use it, its syntax, the most useful parameters, its return value, an ML use case and the most common mistake.

`st.write()`

Purpose	The 'Swiss army knife': renders text, numbers, DataFrames, dicts, charts and more, choosing a display method from the argument's type.
When to use	Quick output while prototyping, or mixing text and objects in one call.
Syntax	`st.write(*args, unsafe_allow_html=False)`
Key parameters	`*args`: any number of objects, rendered in order.
Returns	None
ML use case	`st.write("Accuracy:", acc, "on", n_test, "rows")`
Common mistake	Relying on it in production. Prefer the explicit element (`st.dataframe`, `st.metric`) so you control formatting and width.

`st.title()` / `st.header()` / `st.subheader()`

Purpose	Three levels of headings that build a visual hierarchy and anchors.
When to use	One `st.title` per page; `st.header` for sections; `st.subheader` inside sections.

Syntax	`st.title(body, anchor=None, *, help=None, icon=None, width="stretch")`
	`st.header(body, anchor=None, *, help=None, divider=False, icon=None)`
	`st.subheader(body, anchor=None, *, help=None, divider=False, icon=None)`

Key parameters	`help` adds a tooltip; `divider=True` or a colour name draws a coloured rule under headers; `icon` takes an emoji or `:material/name:`.
Returns	None (a DeltaGenerator you rarely need)
ML use case	`st.header("Model performance", divider="orange")`
Common mistake	Using Markdown `#` inside `st.write` for every heading, which loses consistent spacing and anchors.

`st.markdown()`

Purpose	Renders GitHub-flavoured Markdown: bold, italics, lists, links, tables, emoji shortcodes, coloured text and Material icons.
When to use	Formatted explanatory text, model descriptions, instructions.
Syntax	`st.markdown(body, unsafe_allow_html=False, *, help=None, width="auto")`
Key parameters	`body` supports `:orange[text]`, `:blue-background[text]`, `:material/check:`; `unsafe_allow_html` allows raw HTML (see Chapter 5).
Returns	None
ML use case	A model card: algorithm, training data, intended use, limits.
Common mistake	Forgetting that leading spaces in triple-quoted strings can turn text into a code block. Use `textwrap.dedent` or start lines at column 0.

st.caption() / help=

Purpose	Small, muted text for footnotes; `help=` on most elements shows a tooltip icon.
When to use	Data sources, timestamps, units, disclaimers.
Syntax	`st.caption(body, unsafe_allow_html=False, *, help=None)` `st.slider("Threshold", 0.0, 1.0, help="Rows above this are flagged")`
Key parameters	`body` accepts Markdown.
Returns	None
ML use case	`st.caption("Metrics on a held-out test set of 1,000 rows.")`
Common mistake	Hiding important warnings in captions. Use `st.warning`.

st.code()

Purpose	Displays code with syntax highlighting and a copy button.
When to use	Showing an API request, a SQL query, the model's pipeline structure or a command to run.
Syntax	`st.code(body, language="python", *, line_numbers=False,` `wrap_lines=False, height="content")`
Key parameters	`language=None` for plain text; `line_numbers`; `wrap_lines`.
Returns	None
ML use case	`st.code(str(pipeline), language=None)` shows preprocessing steps.
Common mistake	Using it to display *results*. Use `st.text` or `st.json`.

st.latex()

Purpose	Renders mathematical expressions written in LaTeX (via KaTeX).
When to use	Explaining a metric, loss function or scoring formula.
Syntax	`st.latex(body, *, help=None)`
Key parameters	`body`: a raw string such as `r"\frac{a}{b}"`.
Returns	None
ML use case	Show the F1-score or the logistic function on an 'About' page.
Common mistake	Not using a raw string `r"..."`, so `\f` or `\t` become escape characters.

st.json()

Purpose	Pretty-prints a dict, list or JSON string as a collapsible tree.
When to use	Model metadata, API responses, configuration.
Syntax	`st.json(body, *, expanded=True, width="stretch")`
Key parameters	`expanded`: True, False, or an integer depth to expand.
Returns	None
ML use case	`st.json(metadata, expanded=1)` on a model-information page.
Common mistake	Passing NumPy types that are not JSON serialisable inside a JSON *string*. Pass a dict (Streamlit converts it) or cast with `float()`.

st.success() / st.info() / st.warning() / st.error()

Purpose	Coloured status boxes for outcomes, guidance, risks and failures.
When to use	success = operation finished; info = neutral guidance; warning = result usable but risky; error = cannot continue.
Syntax	`st.success(body, *, icon=None, width="stretch") # same for others`
Key parameters	`icon`: emoji or `:material/...:` name.
Returns	None
ML use case	warning: 'Input outside the training range'; error: 'Missing column contract'.
Common mistake	Showing raw exceptions to end users. Use a friendly `st.error` and log the details.

st.exception()

Purpose	Displays an exception object with its traceback.
When to use	Development and internal tools, inside an `except` block.
Syntax	`st.exception(exception, width="stretch")`
Key parameters	`exception`: the caught exception instance.
Returns	None
ML use case	A debug expander on a batch-scoring page used by the data team.
Common mistake	Leaving it in public apps, leaking file paths and code. Set `client.showErrorDetails` appropriately for production (Chapter 15).

st.divider()

Purpose	Draws a horizontal rule to separate sections.
When to use	Between the input area and the results area.
Syntax	`st.divider(*, width="stretch")`
Key parameters	`width`
Returns	None
ML use case	Separate the prediction form from model performance charts.
Common mistake	Overusing it; headers with `divider=` are often cleaner.

st.empty()

Purpose	A single-element placeholder you can fill, replace or clear later in the same run.
When to use	Live updates: a status line, a countdown, a chart redrawn in a loop.

Syntax	`slot = st.empty()`
	`slot.info("Waiting...")`
	`slot.metric("Score", 0.91)`
	`slot.empty() # clear`

Key parameters	none
Returns	A container holding at most one element.
ML use case	Show 'Scoring...' then replace it with the result.
Common mistake	Expecting it to hold several elements; use `with slot.container():` for multiple.

st.container()	
Purpose	Groups elements; lets you write into a region out of order; adds optional border, fixed height (scrolling) and horizontal layout.
When to use	Cards, scrollable logs, toolbars, inserting results above the form that produced them.
Syntax	`st.container(*, border=None, key=None, width="stretch", height="content", horizontal=False, gap="small")`
Key parameters	`border`, `height` (pixels = scroll), `horizontal`, `key` (adds CSS class `st-key-<key>`).
Returns	A container usable with `with` or method calls.
ML use case	`results = st.container()` at the top; fill it after predicting.
Common mistake	Forgetting the `with` block, so elements appear in the main area instead of the container.

st.expander()	
Purpose	A collapsible section.
When to use	Technical details, raw data, explanations that most users skip.
Syntax	`st.expander(label, expanded=False, *, icon=None, type="default")`
Key parameters	`expanded`, `icon`; `type="compact"` (new) for a lighter style.
Returns	A container.
ML use case	'Why this prediction?' or 'Input sent to the model'.
Common mistake	Putting required inputs inside a collapsed expander, where users never see them.

3.4 Writing text: st.write, st.text and magic

`st.text()` prints fixed-width text with no Markdown processing. Streamlit also supports **magic**: a bare variable or literal on its own line is written to the app automatically, like in a notebook.

magic_demo.py PYTHON

```python
import pandas as pd
import streamlit as st

df = pd.DataFrame({"model": ["RF", "LR"], "auc": [0.91, 0.86]})
df                                  # magic: same as st.write(df)
"**Best model:** random forest"     # magic Markdown string
st.text("Raw text: no *markdown* here")
```

TIP

Magic is handy while prototyping, but explicit `st.*` calls make code easier to read and review. You can turn magic off with `runner.magicEnabled = false` in config.toml.

3.5 Placeholders and live updates

live_updates.py PYTHON

```python
import time

import streamlit as st

status = st.empty()
chart = st.empty()
values = []
for step in range(1, 11):
    values.append(1 / step)                    # pretend training loss
    status.info(f"Epoch {step}/10 - loss {values[-1]:.3f}")
    chart.line_chart(values)
    time.sleep(0.2)
status.success("Training finished")
```

EXPECTED OUTPUT

A message and a line chart that update ten times, then the message turns green. Each `slot.<element>()` call replaces what was in the slot.

3.6 Containers: writing out of order

container_order.py PYTHON

```python
import streamlit as st

result_area = st.container(border=True)   # reserved near the top
with st.form("inputs"):
    tenure = st.number_input("Tenure (months)", 0, 72, 12)
    go = st.form_submit_button("Predict")
if go:
    result_area.metric("Churn probability", f"{max(0.05, 0.9 - tenure / 80):.0%}")
```

The metric is created after the form in code, but it appears **above** the form because it is written into a container created earlier.

3.7 Fragments: rerunning only part of the page

Normally any widget change reruns the whole script. Decorating a function with `@st.fragment` makes widgets **inside** it rerun only that function. This is valuable when the rest of the page is expensive, for example a dashboard with large charts and a small what-if calculator.

fragment_demo.py PYTHON

```python
import time

import streamlit as st


@st.cache_data
def heavy_report():
    time.sleep(2)                          # pretend this is slow
    return {"rows": 120_000, "auc": 0.91}


st.title("Fragment demo")
report = heavy_report()
st.metric("Rows scored today", f"{report['rows']:,}")


@st.fragment
```

fragment_demo.py PYTHON

```python
def what_if():
    discount = st.slider("Discount offered (%)", 0, 30, 10)
    st.write(f"Estimated churn reduction: {discount * 0.4:.1f} points")


what_if()                               # moving the slider reruns ONLY what_if()


@st.fragment(run_every="5s")            # auto-refresh every 5 seconds
def live_clock():
    st.caption(f"Last refreshed: {time.strftime('%H:%M:%S')}")


live_clock()
```

VERSION NOTE

`@st.fragment` became stable in Streamlit 1.37. Version 1.58 added `parallel=True` for running fragments concurrently, and later releases added event-scoped reruns. Call `st.rerun(scope="fragment")` inside a fragment to rerun just that fragment, or `st.rerun()` for the full app.

- A fragment cannot write to containers created outside it (except by returning values and letting the next full run use them).
- Use `@st.fragment` together with caching: fragments limit *what* reruns, caching makes reruns *cheap*.
- Forms (Chapter 10) are another optimisation: they batch several inputs into a single rerun when the submit button is pressed.

Chapter summary

Need	Use
Page and section headings	`st.title`, `st.header`, `st.subheader`
Formatted explanations	`st.markdown`, `st.caption`
Code, formulas, configuration	`st.code`, `st.latex`, `st.json`
Outcome messages	`st.success`, `st.info`, `st.warning`, `st.error`
Debug information	`st.exception` (internal apps only)
Structure	`st.divider`, `st.container`, `st.expander`
Replaceable output	`st.empty`
Partial reruns	`@st.fragment`

CHAPTER 4

Complete Streamlit UI Components Catalog

04

Every component you need to build a professional ML application

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Choose the right text, input, layout, data, chart, media, navigation and status components for an ML task.
- ✓ Know each widget's return value and how it is stored in session state.
- ✓ Validate inputs and handle files safely.
- ✓ Build multipage apps with st.navigation and st.Page.

VERSION NOTE

All signatures in this chapter were checked against Streamlit 1.65.0 by introspecting the installed package. `st.bokeh_chart` no longer exists in current Streamlit, so it is not covered; use `pip install streamlit-bokeh` and `streamlit_bokeh(fig)` if you need Bokeh.

4A. Text and content components

Chapter 3 covered these in depth. The table maps each one to its job in an ML application.

Component	Function	When to use it in an ML app
Title	`st.title`	Page name: 'House Price Estimator'
Header / Subheader	`st.header`, `st.subheader`	Sections: 'Inputs', 'Results', 'Model performance'
Markdown	`st.markdown`	Model card, instructions, coloured highlights
Text	`st.text`	Fixed-width output such as a raw log line
Caption	`st.caption`	Data source, units, 'metrics on held-out data'
Code	`st.code`	API request examples, pipeline structure
LaTeX	`st.latex`	Metric or loss formulas on an About page
JSON	`st.json`	Metadata, API responses, configuration
Divider	`st.divider`	Separate inputs from outputs
Empty placeholder	`st.empty`	'Scoring...' replaced by the result
Status messages	`st.success/info/warning/error`	Validation results, model warnings, failures
Exception display	`st.exception`	Developer-facing debug panel

4B. Input widgets

Widgets are functions that **draw a control and return its current value**. On the first run they return the default; after the user interacts they return the new value on every rerun.

How a widget's value is stored

widget_keys.py PYTHON

```python
import streamlit as st

threshold = st.slider("Threshold", 0.0, 1.0, 0.5, key="threshold")
st.write(threshold)                    # the return value
st.write(st.session_state.threshold)   # the same value, via its key
```

- Every widget has an internal identity computed from its parameters; giving it a `key=` makes the identity explicit and mirrors the value into `st.session_state[key]`.
- Two widgets with identical parameters on one page raise `DuplicateWidgetID`: give them different keys.
- You can set a keyed widget's value from code **before** it is drawn (`st.session_state.threshold = 0.3`), but not after it has been created in the same run.

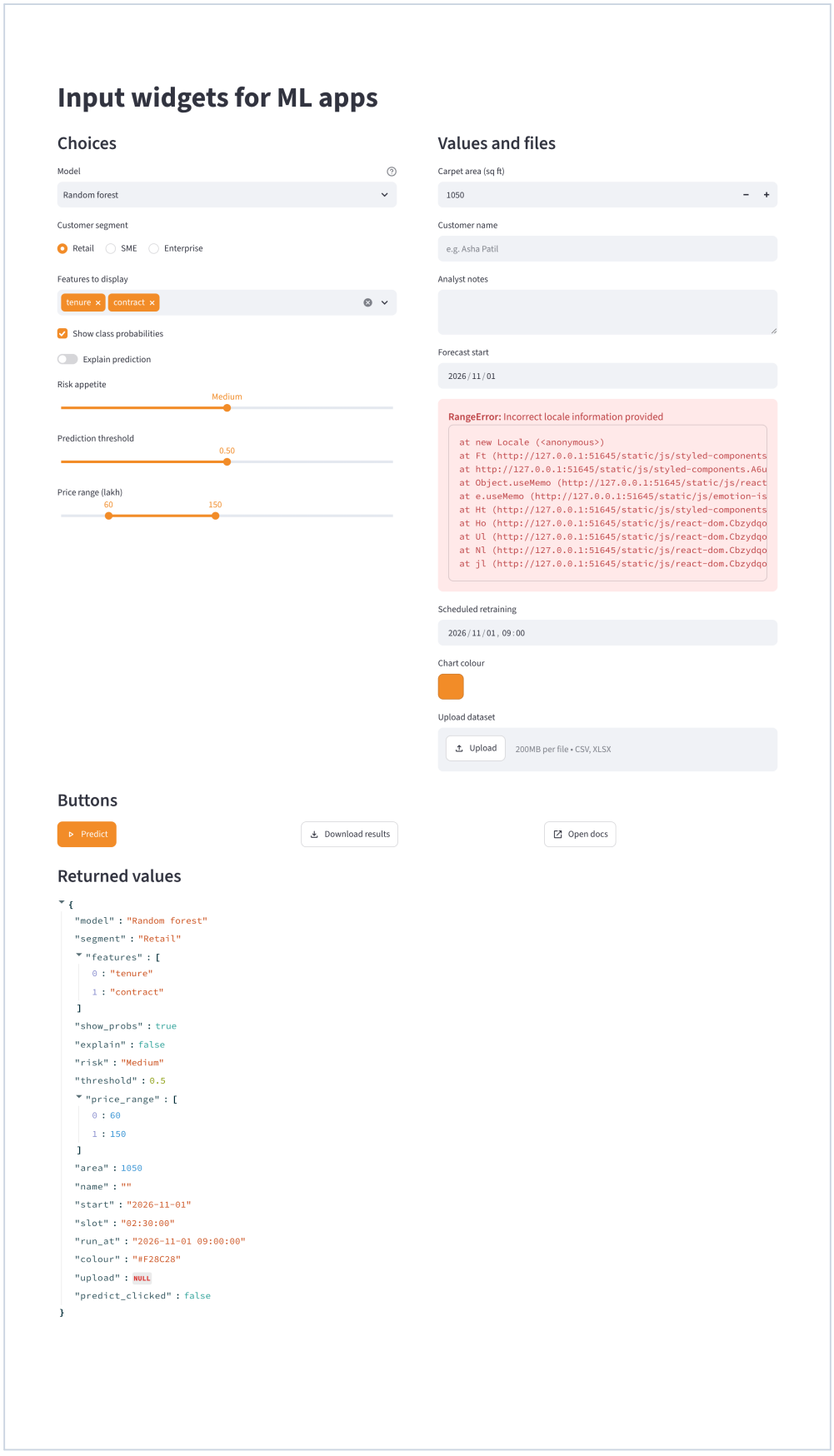


Figure 4.1: Every input widget with ML labels, and the values they return (widgets_demo.py)

demos/widgets_demo.py PYTHON

```python
"""Every input widget used in ML apps.  Run: streamlit run widgets_demo.py"""
import datetime as dt

import pandas as pd
import streamlit as st

st.set_page_config(page_title="Input widgets", layout="wide")
st.title("Input widgets for ML apps")
left, right = st.columns(2, gap="large")

with left:
    st.subheader("Choices")
    model_name = st.selectbox("Model", ["Random forest", "Logistic regression", "XGBoost"],
                              help="Which trained model to use")
    segment = st.radio("Customer segment", ["Retail", "SME", "Enterprise"],
                       horizontal=True)
    features = st.multiselect("Features to display",
                              ["tenure", "monthly_charges", "contract", "support_calls"],
                              default=["tenure", "contract"])
    show_probs = st.checkbox("Show class probabilities", value=True)
    explain = st.toggle("Explain prediction")
    risk = st.select_slider("Risk appetite", ["Low", "Medium", "High"], value="Medium")
    threshold = st.slider("Prediction threshold", 0.0, 1.0, 0.5, 0.05)
    price_range = st.slider("Price range (lakh)", 20, 300, (60, 150))

with right:
    st.subheader("Values and files")
    area = st.number_input("Carpet area (sq ft)", min_value=250, max_value=6000,
                           value=1050, step=25)
    name = st.text_input("Customer name", placeholder="e.g. Asha Patil", max_chars=60)
    notes = st.text_area("Analyst notes", height=80)
    start = st.date_input("Forecast start", dt.date(2026, 11, 1))
    slot = st.time_input("Batch run time", dt.time(2, 30))
    run_at = st.datetime_input("Scheduled retraining", dt.datetime(2026, 11, 1, 9, 0))
    brand = st.color_picker("Chart colour", "#F28C28")
    upload = st.file_uploader("Upload dataset", type=["csv", "xlsx"])

st.subheader("Buttons")
b1, b2, b3 = st.columns(3)
clicked = b1.button("Predict", type="primary", icon=":material/play_arrow:")
results = pd.DataFrame({"customer": ["C1", "C2"], "churn_probability": [0.82, 0.12]})
b2.download_button("Download results", results.to_csv(index=False), "results.csv",
                   "text/csv", icon=":material/download:")
b3.link_button("Open docs", "https://docs.streamlit.io", icon=":material/open_in_new:")

st.subheader("Returned values")
st.json({"model": model_name, "segment": segment, "features": features,
         "show_probs": show_probs, "explain": explain, "risk": risk,
         "threshold": threshold, "price_range": list(price_range), "area": area,
         "name": name, "start": str(start), "slot": str(slot), "run_at": str(run_at),
         "colour": brand, "upload": None if upload is None else upload.name,
         "predict_clicked": clicked}, expanded=True)
```

Widget reference

st.button()

Syntax	`clicked = st.button(label, key=None, help=None, on_click=None, *, type="secondary", icon=None, disabled=False, width="content")`
Returns	`bool`: True only on the rerun triggered by the click, then False again.
Key parameters	`type="primary"` for the main action; `on_click` callback; `icon`.
ML use case	Trigger a prediction or a batch job.
Common mistake	Expecting it to stay True. Store results in `st.session_state` if they must survive the next interaction.

st.download_button()

Syntax	`st.download_button(label, data, file_name=None, mime=None, key=None, on_click="rerun", *, type="secondary", icon=None)`
Returns	`bool` (True on the click rerun).
Key parameters	`data`: str, bytes or file-like; `mime`, e.g. `text/csv`; `on_click="ignore"` downloads without rerunning the app.
ML use case	Download predictions as CSV, a PDF report or a model card.
Common mistake	Generating a big file on every rerun. Cache the generation function.

st.link_button()

Syntax	`st.link_button(label, url, *, type="secondary", icon=None, disabled=False, width="content")`
Returns	Opens the URL in a new tab.
Key parameters	`url`, `type`, `icon`.
ML use case	Link to the model documentation or a dashboard.
Common mistake	Using it for in-app navigation; use `st.page_link`.

st.checkbox()

Syntax	`value = st.checkbox(label, value=False, key=None, on_change=None)`
Returns	`bool`
Key parameters	`value` sets the default.
ML use case	'Show class probabilities'
Common mistake	Many checkboxes for mutually exclusive options; use radio or selectbox.

st.toggle()

Syntax	`value = st.toggle(label, value=False, key=None, on_change=None)`
Returns	`bool`
Key parameters	Same as checkbox, switch style.
ML use case	'Explain prediction', 'Use log scale'
Common mistake	Using it for actions; use a button.

st.radio()

Syntax	`choice = st.radio(label, options, index=0, format_func=str, key=None, *, horizontal=False, captions=None)`
Returns	The selected option (or None if `index=None`).
Key parameters	`horizontal=True`; `captions` per option; `format_func` for display text.
ML use case	Choose a customer segment or data source.
Common mistake	More than ~5 options; use selectbox.

st.selectbox()

Syntax	`choice = st.selectbox(label, options, index=0, format_func=str, key=None, *, placeholder=None, accept_new_options=False)`
Returns	The selected option (any type, e.g. a model object or id).
Key parameters	`index=None` starts empty; `format_func` shows names for ids; type-to-search (`filter_mode`).
ML use case	Select the ML model or a locality.
Common mistake	Passing a DataFrame column with NaN values as options.

st.multiselect()

Syntax	`chosen = st.multiselect(label, options, default=None, format_func=str, key=None, *, max_selections=None, placeholder=None)`
Returns	`list` of selected options.
Key parameters	`default`, `max_selections`, `accept_new_options`.
ML use case	Choose features to display or filter categories.
Common mistake	`default` values that are not in `options` raise an error.

st.slider()

Syntax	`x = st.slider(label, min_value=None, max_value=None, value=None, step=None, format=None, key=None)`
Returns	A number, date/time, or a tuple if `value` is a tuple (range slider).
Key parameters	Types of min/max/value/step must match (all int or all float).
ML use case	Prediction threshold 0.0-1.0; price range (low, high).
Common mistake	Mixing `int` and `float`: `st.slider('x', 0, 1.0)` raises an error.

st.select_slider()

Syntax	`x = st.select_slider(label, options=(), value=None, format_func=str, key=None)`
Returns	One option, or a tuple for a range.
Key parameters	Any ordered list of options.
ML use case	Risk appetite Low/Medium/High; loan tenure 1-20 years.
Common mistake	Options that are not in a meaningful order.

st.text_input()

Syntax	`text = st.text_input(label, value="", max_chars=None, key=None, type="default", *, placeholder=None, required=False, validate=None)`
Returns	`str` (empty string if nothing typed).
Key parameters	`type` = default/password/email/url/phone/search; `max_chars`; `placeholder`.
ML use case	Customer id, search query, API key (type="password").
Common mistake	Trusting it: always strip, check length and pattern before use.

st.text_area()

Syntax	`text = st.text_area(label, value="", height=None, max_chars=None, key=None, *, placeholder=None)`
Returns	`str`
Key parameters	`height` in pixels; `max_chars`.
ML use case	Paste a review for sentiment analysis.
Common mistake	No `max_chars`: users can paste megabytes of text.

st.number_input()

Syntax	`n = st.number_input(label, min_value=None, max_value=None, value="min", step=None, format=None, key=None)`
Returns	`int` or `float` (matching the argument types).
Key parameters	`min_value`/`max_value` enforce bounds; `format="%.2f"`.
ML use case	Carpet area, income, tenure.
Common mistake	Leaving bounds unset; out-of-range values reach the model.

st.date_input()

Syntax	`d = st.date_input(label, value="today", min_value=None, max_value=None, key=None, *, format="YYYY/MM/DD")`
Returns	`datetime.date`, or a tuple when `value` is a tuple (range).
Key parameters	Pass `(start, end)` for a range picker.
ML use case	Forecast start date or training window.
Common mistake	Using the range before the user has picked both ends (tuple of length 1).

st.time_input()

Syntax	`t = st.time_input(label, value="now", key=None, *, step=timedelta(minutes=15), format="24h")`
Returns	`datetime.time`
Key parameters	`step` in seconds or a timedelta.
ML use case	Time of a scheduled batch run.
Common mistake	Assuming a timezone; it is a naive time.

st.datetime_input()

Syntax	`dt = st.datetime_input(label, value="now", min_value=None, max_value=None, key=None, *, format="YYYY/MM/DD", step=900)`
Returns	`datetime.datetime`
Key parameters	Combines date and time in one widget.
ML use case	Schedule a retraining job.
Common mistake	Using it on old Streamlit versions where it does not exist.

st.file_uploader()

Syntax	`f = st.file_uploader(label, type=None, accept_multiple_files=False, key=None, *, max_upload_size=None)`
Returns	`UploadedFile` (file-like, with `.name`, `.size`, `.type`, `.getvalue()`), a list if multiple, or None.
Key parameters	`type=["csv", "xlsx"]`; `accept_multiple_files=True` or "directory"; `max_upload_size` in MB (per widget).
ML use case	Upload a CSV for batch scoring or an image to classify.
Common mistake	Trusting the extension. Parse inside try/except and validate the content.

st.camera_input()

Syntax	`photo = st.camera_input(label, key=None, *, disabled=False)`
Returns	`UploadedFile` (a JPEG image) or None.
Key parameters	Requires camera permission in the browser (and HTTPS when not on localhost).
ML use case	Take a photo for image classification.
Common mistake	Leaving the camera always on; put it behind a toggle.

st.audio_input()

Syntax	`rec = st.audio_input(label, *, sample_rate=16000, key=None)`
Returns	`UploadedFile` containing a WAV recording, or None.
Key parameters	`sample_rate` (default 16 kHz suits most speech models).
ML use case	Record speech for speech-to-text or emotion recognition.
Common mistake	Assuming MP3; it is WAV.

st.color_picker()

Syntax	`hex_colour = st.color_picker(label, value=None, key=None)`
Returns	`str` such as '#F28C28'.
Key parameters	`value` default colour.
ML use case	Let users pick chart colours in a report builder.
Common mistake	Rarely needed in ML apps.

VERSION NOTE

`st.datetime_input` and `st.audio_input` exist in current Streamlit (both verified in 1.65). If you must support older servers, fall back to `st.date_input` + `st.time_input`, and `st.file_uploader(type=["wav"])`.

`st.text_input(type=...)` gained `email`, `url`, `phone` and `search` types and a `validate=` option in 2026 releases. Older versions only accept `default` and `password`.

Input validation pattern

validation_pattern.py PYTHON

```python
import re

import streamlit as st

customer_id = st.text_input("Customer ID", max_chars=12, placeholder="CUST-00042")
area = st.number_input("Carpet area (sq ft)", min_value=250, max_value=6000, value=1050)
bedrooms = st.number_input("Bedrooms", 1, 6, 2)
bathrooms = st.number_input("Bathrooms", 1, 6, 2)

errors = []
if customer_id and not re.fullmatch(r"CUST-\d{5}", customer_id.strip()):
    errors.append("Customer ID must look like CUST-00042.")
if bathrooms > bedrooms + 1:
    errors.append("Bathrooms cannot exceed bedrooms + 1.")
if area / bedrooms < 150:
    errors.append("Less than 150 sq ft per bedroom looks like a typo.")

if st.button("Predict", type="primary"):
    if errors:
        for e in errors:
            st.error(e)
    else:
        st.success("Inputs valid - ready to call model.predict().")
```

BEST PRACTICE

Validate in three layers: **widget bounds** (min/max, options), **business rules** in Python (cross-field checks like bathrooms vs bedrooms), and **training-range warnings** that tell users when the model is extrapolating.

File uploader: types, return values and limits

Argument / attribute	Meaning
`type=["csv", "xlsx"]`	Accepted extensions (checked in the browser and on the server). This checks the *name*, not the content.
`accept_multiple_files=True`	Returns a list of UploadedFile objects (empty list when none). `"directory"` lets users upload a folder.
`max_upload_size=50`	Per-widget limit in MB; the global default is `server.maxUploadSize = 200` in config.toml.
`f.name`, `f.size`, `f.type`	Original file name, bytes, MIME type reported by the browser (not trustworthy).
`f.getvalue()`	All bytes. `pd.read_csv(f)` and `Image.open(f)` also accept the object directly.
Storage	Files are held in server memory for the session, not written to disk; they disappear when the user removes them or the session ends.

4C. Layout components

st.sidebar

Syntax	`with st.sidebar:`
	`  st.selectbox("Model", models)`
	`st.sidebar.slider("Threshold", 0.0, 1.0, 0.5)`

When to use	Global settings shared by the whole page: model choice, thresholds, filters, file upload.
Advantages	Always visible, collapsible on mobile.
Limitations	Narrow; not for wide tables or charts.

st.columns()

Syntax	`left, right = st.columns([2, 1], gap="large",` `  vertical_alignment="top", border=False)`
When to use	Side-by-side forms, KPI rows, chart pairs.
Advantages	Weights control relative widths; can nest one level.
Limitations	Columns stack vertically on narrow screens.

st.container()

Syntax	`with st.container(border=True, height=300):`
	`  ...`
	`with st.container(horizontal=True, horizontal_alignment="right"):`
	`  st.button("Export")`

When to use	Cards, scrollable areas, toolbars (horizontal=True), out-of-order writing.
Advantages	Flexible; `key=` gives a stable CSS class.
Limitations	Fixed heights can hide content.

st.expander()

Syntax	`with st.expander("Technical details", icon=":material/code:"):` `  st.code(str(pipeline), language=None)`
When to use	Optional detail.
Advantages	Keeps pages short.
Limitations	Content is still computed even when collapsed.

st.tabs()

Syntax	`pred, analytics, info = st.tabs(["Prediction", "Analytics",` `  "Model info"])`
When to use	Separate modes of one page.
Advantages	Clear organisation.
Limitations	All tabs run on every rerun (hidden tabs still compute). Use `on_change` / `default` (new) or pages for heavy content.

st.popover()

Syntax	`with st.popover("Filters", icon=":material/filter_list:"):` `  st.multiselect("Contract", options)`
When to use	Secondary controls that should not take space.
Advantages	Compact.
Limitations	Hidden controls are easy to miss.

@st.dialog()

Syntax	`@st.dialog("Confirm batch run")`
	`def confirm():`
	`  if st.button("Run"):`
	`  st.rerun()`

When to use	Confirmation, detail views, small forms in a modal.
Advantages	Focused interaction; a dialog function reruns like a fragment.
Limitations	Only one dialog can be open at a time.

st.form() + st.form_submit_button()

Syntax	`with st.form("predict"):`
	`  a = st.number_input("Area", 250, 6000)`
	`  go = st.form_submit_button("Predict", type="primary")`

When to use	Collect several inputs, then predict once.
Advantages	Prevents a rerun on every keystroke.
Limitations	Widgets inside cannot have callbacks except the submit button.

st.empty()

Syntax	`slot = st.empty()`
	`slot.info("Scoring...")`
	`slot.success("Done")`

When to use	Single replaceable element.
Advantages	Simple live updates.
Limitations	Holds one element.

st.space()

Syntax	`st.space("medium") # "small" | "medium" | "large" | pixels`
When to use	Vertical breathing room between sections.
Advantages	No CSS needed.
Limitations	Recent addition (check your version).

The following app combines nine layout tools in one churn-monitoring page.

demos/layout_demo.py PYTHON

```python
"""Layout primitives in one ML dashboard.  Run: streamlit run layout_demo.py"""
import numpy as np
import pandas as pd
import streamlit as st

st.set_page_config(page_title="Layouts", layout="wide")

with st.sidebar:                                  # 1. sidebar for global controls
    st.header("Controls")
    model = st.selectbox("Model", ["Random forest", "Logistic regression"])
    threshold = st.slider("Threshold", 0.1, 0.9, 0.5)

st.title("Churn monitoring")
k1, k2, k3 = st.columns(3)                        # 2. three-column KPI row
k1.metric("Accuracy", "0.87", "+0.02", border=True)
k2.metric("Customers at risk", "1,284", "-56", delta_color="inverse", border=True)
k3.metric("Avg. probability", "0.31", border=True)

with st.container(horizontal=True, horizontal_alignment="right"):   # 3. toolbar
    st.button("Refresh", icon=":material/refresh:")
```

demos/layout_demo.py PYTHON

```python
    with st.popover("Filters", icon=":material/filter_list:"):     # 4. popover
        st.multiselect("Contract", ["Month-to-month", "One year", "Two year"])
    st.button("Export", icon=":material/download:")

tab_pred, tab_analytics, tab_info = st.tabs(["Prediction", "Analytics", "Model info"])
with tab_pred:                                      # 5. tabs
    with st.form("predict_form"):                   # 6. form inside a tab
        a, b = st.columns(2)                        #    two-column form
        a.number_input("Tenure (months)", 0, 72, 12)
        a.number_input("Monthly charges", 18.0, 120.0, 70.0)
        b.selectbox("Contract", ["Month-to-month", "One year", "Two year"])
        b.selectbox("Internet service", ["Fiber optic", "DSL", "No"])
        st.form_submit_button("Predict", type="primary")
with tab_analytics:
    rng = np.random.default_rng(0)
    st.line_chart(pd.DataFrame(rng.normal(0, 1, (30, 2)).cumsum(0),
                               columns=["actual", "predicted"]))
with tab_info:
    st.write(f"Active model: **{model}**, threshold {threshold}")

st.space("medium")                                  # 7. vertical spacing
with st.expander("Technical details"):              # 8. expander
    st.code("Pipeline(preprocess -> RandomForestClassifier)", language=None)


@st.dialog("Confirm batch run")                     # 9. modal dialog
def confirm():
    st.write("Score 12,000 customers now?")
    if st.button("Yes, run it", type="primary"):
        st.session_state.confirmed = True
        st.rerun()


if st.button("Run batch scoring"):
    confirm()
if st.session_state.get("confirmed"):
    st.success("Batch job submitted.")
```

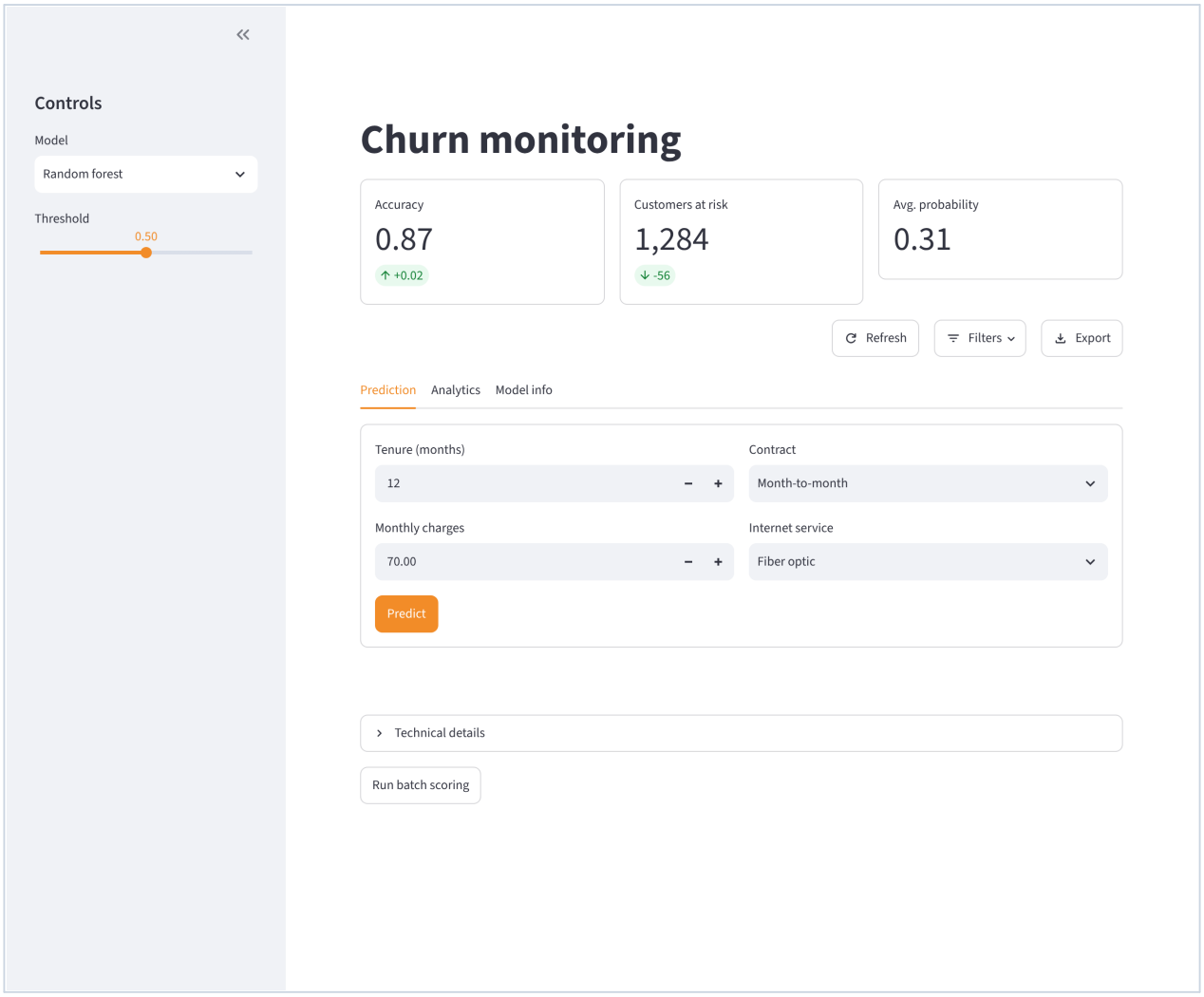


Figure 4.2: layout_demo.py: sidebar, KPI columns, a horizontal toolbar with a popover, tabs with a two-column form, spacing, an expander and a dialog trigger

Layout patterns for ML apps

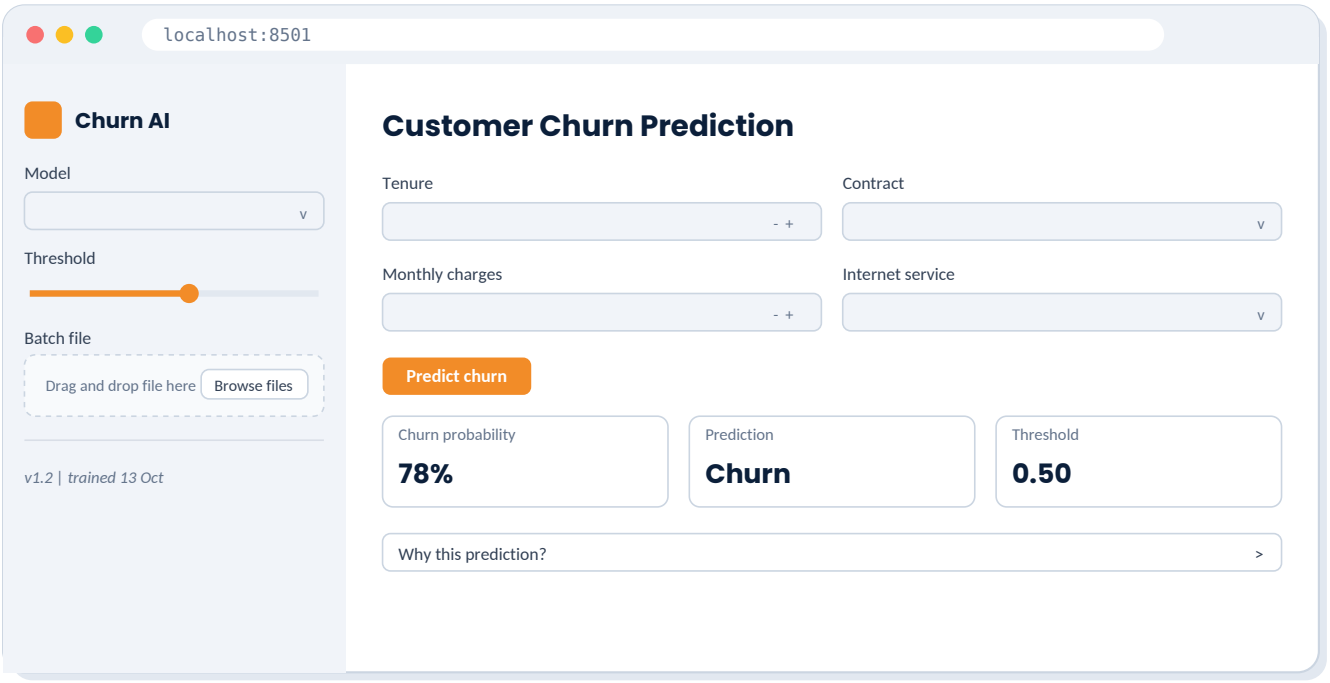


Figure 4.3: Sidebar for global settings, two-column form, three-column result row

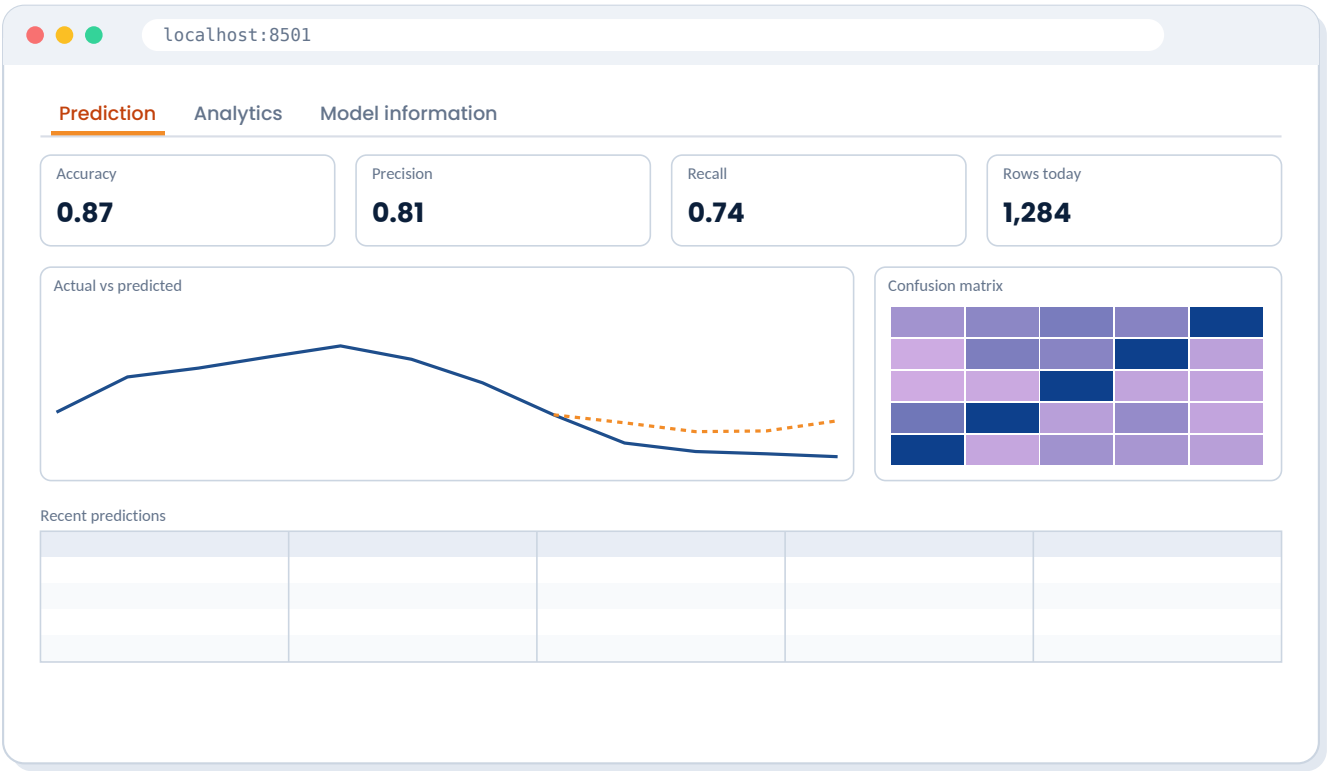


Figure 4.4: Tabs for prediction, analytics and model information with a KPI row

4D. Data display components

demos/data_demo.py PYTHON

```python
"""Data display: dataframe, data_editor, table, metric, column_config."""
import numpy as np
import pandas as pd
import streamlit as st

st.set_page_config(page_title="Data display", layout="wide")
rng = np.random.default_rng(1)
preds = pd.DataFrame({
    "customer_id": [f"C{i:03d}" for i in range(1, 9)],
    "contract": rng.choice(["Month-to-month", "One year", "Two year"], 8),
    "monthly_charges": rng.uniform(20, 110, 8).round(2),
    "churn_probability": rng.uniform(0, 1, 8).round(3),
    "trend": [list(rng.normal(0, 1, 12).cumsum()) for _ in range(8)],
    "reviewed": [False] * 8,
})

st.title("Data display components")
st.subheader("st.metric - KPI cards")
c1, c2, c3 = st.columns(3)
c1.metric("ROC-AUC", 0.912, 0.015, border=True, format="%.3f")
c2.metric("Predictions today", 1284, -56, border=True,
          chart_data=[1100, 1180, 1250, 1340, 1284], chart_type="bar")
c3.metric("Mean churn probability", "31%", "2 pts", delta_color="inverse", border=True)

st.subheader("st.dataframe with column_config")
st.dataframe(
    preds, hide_index=True, width="stretch",
    column_config={
        "customer_id": st.column_config.TextColumn("Customer", pinned=True),
        "monthly_charges": st.column_config.NumberColumn("Monthly ($)", format="dollar"),
        "churn_probability": st.column_config.ProgressColumn(
            "Churn probability", min_value=0, max_value=1, format="percent"),
        "trend": st.column_config.LineChartColumn("Usage trend"),
        "reviewed": st.column_config.CheckboxColumn("Reviewed"),
    })

st.subheader("st.data_editor - editable review queue")
edited = st.data_editor(
    preds.drop(columns="trend"), hide_index=True, width="stretch", key="review",
    disabled=["customer_id", "churn_probability"],
    column_config={
        "reviewed": st.column_config.CheckboxColumn("Reviewed?"),
        "contract": st.column_config.SelectboxColumn(
            "Contract", options=["Month-to-month", "One year", "Two year"]),
        "churn_probability": st.column_config.NumberColumn(format="%.2f"),
    })
st.write(f"Rows marked reviewed: **{int(edited['reviewed'].sum())}**")

st.subheader("st.table - small static summary")
st.table(pd.DataFrame({"metric": ["Accuracy", "Precision", "Recall"],
                       "value": [0.87, 0.81, 0.74]}).set_index("metric"))

st.subheader("Data quality summary")
dq = pd.DataFrame({"missing": preds.isna().sum(), "dtype": preds.dtypes.astype(str)})
st.dataframe(dq.style.highlight_max(subset=["missing"], color="#fde68a"), width="stretch")
```

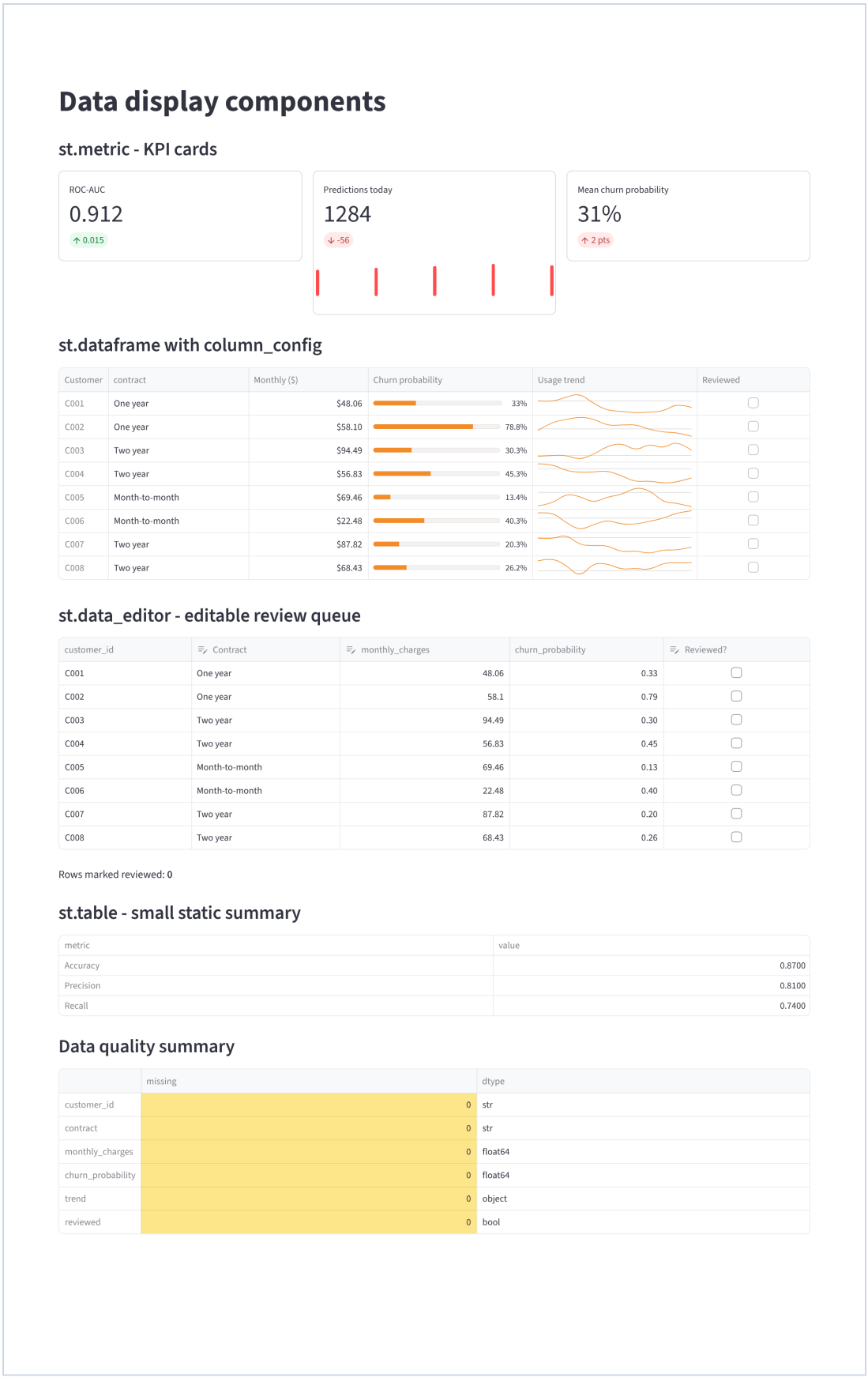


Figure 4.5: data_demo.py: metric cards with sparklines, a configured dataframe, an editable review queue and a static table

Element	Interactive?	Best for	Avoid when
`st.dataframe`	Sort, search, resize, select rows, download	Uploaded datasets, prediction tables (thousands of rows)	You need users to change values
`st.data_editor`	Edit cells, add/delete rows (`num_rows`)	Review queues, what-if tables, labelling, correcting inputs	Read-only results (adds noise)
`st.table`	No (static)	Small summaries: metrics, a classification report	More than ~20 rows
`st.metric`	No	KPI cards with delta and optional sparkline	Long text
`st.json`	Collapse/expand	Metadata and API payloads	Tabular data

st.column_config reference

Column type	Use it for	Example
`TextColumn`	Ids, names; `pinned=True` keeps it visible	`TextColumn("Customer", pinned=True)`
`NumberColumn`	Currency, percentages, decimals	`NumberColumn(format="dollar")`, `format="%.2f"`, `format="percent"`
`ProgressColumn`	Probabilities and scores as bars	`ProgressColumn(min_value=0, max_value=1, format="percent")`
`CheckboxColumn`	Booleans; editable in data_editor	`CheckboxColumn("Reviewed?")`
`SelectboxColumn`	Editable categorical values	`SelectboxColumn(options=["Yes", "No"])`
`DateColumn`, `DatetimeColumn`	Formatted dates	`DateColumn(format="DD MMM YYYY")`
`LineChartColumn`, `BarChartColumn`	Sparklines from list cells	`LineChartColumn("Usage trend")`
`ImageColumn`, `LinkColumn`	Thumbnails from URLs, clickable links	`LinkColumn(display_text="Open")`

Highlighting and data-quality summaries

quality_summary.py (expects data/sample_data.csv) PYTHON

```python
import pandas as pd
import streamlit as st

df = pd.read_csv("data/sample_data.csv")
quality = pd.DataFrame({
    "dtype": df.dtypes.astype(str),
    "missing": df.isna().sum(),
    "missing_%": (df.isna().mean() * 100).round(1),
    "unique": df.nunique(),
})
st.dataframe(quality.style.background_gradient(subset=["missing_%"], cmap="Oranges"),
             width="stretch")
st.dataframe(df.style.highlight_max(subset=df.select_dtypes("number").columns,
                                    color="#d1fae5"), width="stretch")
```

NOTE

Pandas Styler objects are supported by `st.dataframe` and `st.table`. `background_gradient` needs matplotlib installed.

4E. Chart and visualisation components

Function	Native or library?	Requires	Typical ML use
`st.line_chart`, `st.area_chart`	Native (Vega-Lite inside)	-	Forecasts, training curves
`st.bar_chart`	Native	-	Feature importance, class counts
`st.scatter_chart`	Native	-	Clusters, actual vs predicted
`st.map`	Native	-	Points with lat/lon columns
`st.pyplot`	Library	matplotlib (seaborn optional)	Confusion matrix, heatmaps, publication figures
`st.plotly_chart`	Library	plotly	Interactive ROC/PR, residuals, dashboards; supports selections
`st.altair_chart`	Library	altair (installed with Streamlit)	Declarative statistical charts
`st.vega_lite_chart`	Spec (no Python lib)	-	Charts from a JSON spec
`st.graphviz_chart`	Library	graphviz (DOT string works without the Python package)	Pipeline and decision-tree diagrams
`st.pydeck_chart`	Library	pydeck	3D/geospatial layers (hexbins, arcs)
`st.image`, `st.audio`, `st.video`	Native	Pillow for some image types	Inputs/outputs of vision and speech models
`st.bokeh_chart`	Removed	Use the streamlit-bokeh component	-

demos/charts_demo.py PYTHON

```python
"""ML visualisations with native and library charts.  Run: streamlit run charts_demo.py
Extra deps: matplotlib seaborn plotly altair graphviz pydeck
"""
import altair as alt
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import pydeck as pdk
import seaborn as sns
import streamlit as st
from sklearn.cluster import KMeans
from sklearn.datasets import make_classification, make_regression
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (ConfusionMatrixDisplay, auc, precision_recall_curve,
                             roc_curve)
from sklearn.model_selection import train_test_split

st.set_page_config(page_title="ML charts", layout="wide")


@st.cache_data
def classification_results():
    X, y = make_classification(1500, n_features=8, n_informative=5, random_state=0)
    Xtr, Xte, ytr, yte = train_test_split(X, y, random_state=0)
    clf = RandomForestClassifier(200, random_state=0).fit(Xtr, ytr)
    return yte, clf.predict(Xte), clf.predict_proba(Xte)[:, 1], clf.feature_importances_


y_test, y_pred, y_score, importances = classification_results()
st.title("Visualising ML results")
```

demos/charts_demo.py PYTHON

```python
st.header("Native charts (no extra library)")
rng = np.random.default_rng(2)
months = pd.date_range("2025-01-01", periods=18, freq="MS")
sales = pd.DataFrame({"month": months, "actual": 100 + rng.normal(0, 6, 18).cumsum(),
                      "forecast": 100 + np.linspace(0, 12, 18)})
c1, c2 = st.columns(2)
c1.line_chart(sales, x="month", y=["actual", "forecast"], x_label="Month",
              y_label="Sales (lakh)")
c2.area_chart(sales, x="month", y="actual", color="#F28C28")
c1, c2 = st.columns(2)
fi = pd.DataFrame({"feature": [f"f{i}" for i in range(8)], "importance": importances})
c1.bar_chart(fi, x="feature", y="importance", horizontal=True, sort="-importance")
X2, _ = make_classification(400, n_features=2, n_redundant=0, random_state=3)
labels = KMeans(3, n_init=10, random_state=0).fit_predict(X2)
c2.scatter_chart(pd.DataFrame({"x": X2[:, 0], "y": X2[:, 1],
                               "cluster": labels.astype(str)}),
                 x="x", y="y", color="cluster")

st.header("Matplotlib and Seaborn (st.pyplot)")
c1, c2 = st.columns(2)
fig, ax = plt.subplots(figsize=(4.5, 3.6))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, ax=ax, cmap="Blues",
                                        colorbar=False)
ax.set_title("Confusion matrix")
c1.pyplot(fig)
plt.close(fig)
corr = pd.DataFrame(make_regression(300, 6, random_state=1)[0],
                    columns=[f"x{i}" for i in range(6)]).corr()
fig, ax = plt.subplots(figsize=(4.5, 3.6))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", vmin=-1, vmax=1, ax=ax)
ax.set_title("Correlation heatmap")
c2.pyplot(fig)
plt.close(fig)

st.header("Plotly (interactive)")
fpr, tpr, _ = roc_curve(y_test, y_score)
prec, rec, _ = precision_recall_curve(y_test, y_score)
c1, c2 = st.columns(2)
roc = go.Figure(go.Scatter(x=fpr, y=tpr, name=f"AUC = {auc(fpr, tpr):.3f}"))
roc.add_scatter(x=[0, 1], y=[0, 1], line=dict(dash="dash"), name="random")
roc.update_layout(title="ROC curve", xaxis_title="FPR", yaxis_title="TPR", height=360)
c1.plotly_chart(roc, width="stretch")
c2.plotly_chart(px.line(x=rec, y=prec, title="Precision-recall curve", height=360,
                        labels={"x": "Recall", "y": "Precision"}), width="stretch")
Xr, yr = make_regression(300, 3, noise=25, random_state=4)
pred_r = LinearRegression().fit(Xr, yr).predict(Xr)
reg = pd.DataFrame({"actual": yr, "predicted": pred_r, "residual": yr - pred_r})
c1, c2 = st.columns(2)
c1.plotly_chart(px.scatter(reg, x="actual", y="predicted", trendline="ols",
                           title="Actual vs predicted", height=360), width="stretch")
fig = px.scatter(reg, x="predicted", y="residual", title="Residual plot", height=360)
fig.add_hline(y=0, line_dash="dash")
c2.plotly_chart(fig, width="stretch")

st.header("Altair, Vega-Lite and Graphviz")
c1, c2 = st.columns(2)
c1.altair_chart(alt.Chart(fi).mark_bar(color="#1f4e8c").encode(
    x="importance:Q", y=alt.Y("feature:N", sort="-x")).properties(
    title="Feature importance (Altair)", height=280), width="stretch")
c2.vega_lite_chart(sales, {"mark": {"type": "line", "point": True},
                           "encoding": {"x": {"field": "month", "type": "temporal"},
                                        "y": {"field": "actual", "type": "quantitative"}},
                           "title": "Sales (Vega-Lite spec)", "height": 280},
                   width="stretch")
st.graphviz_chart("""
digraph { rankdir=LR; node [shape=box, style=rounded];
  "Raw CSV" -> "Imputer" -> "Scaler + OneHot" -> "RandomForest" -> "Probability" }
""")
```

demos/charts_demo.py PYTHON

```python
st.header("Maps")
pune = pd.DataFrame({"lat": 18.52 + rng.normal(0, 0.04, 200),
                     "lon": 73.85 + rng.normal(0, 0.05, 200),
                     "price": rng.uniform(40, 250, 200)})
c1, c2 = st.columns(2)
c1.map(pune, latitude="lat", longitude="lon", size=40, color="#F28C2899")
c2.pydeck_chart(pdk.Deck(
    initial_view_state=pdk.ViewState(latitude=18.52, longitude=73.85, zoom=10, pitch=40),
    layers=[pdk.Layer("HexagonLayer", data=pune, get_position="[lon, lat]", radius=600,
                      elevation_scale=20, extruded=True)]))
```

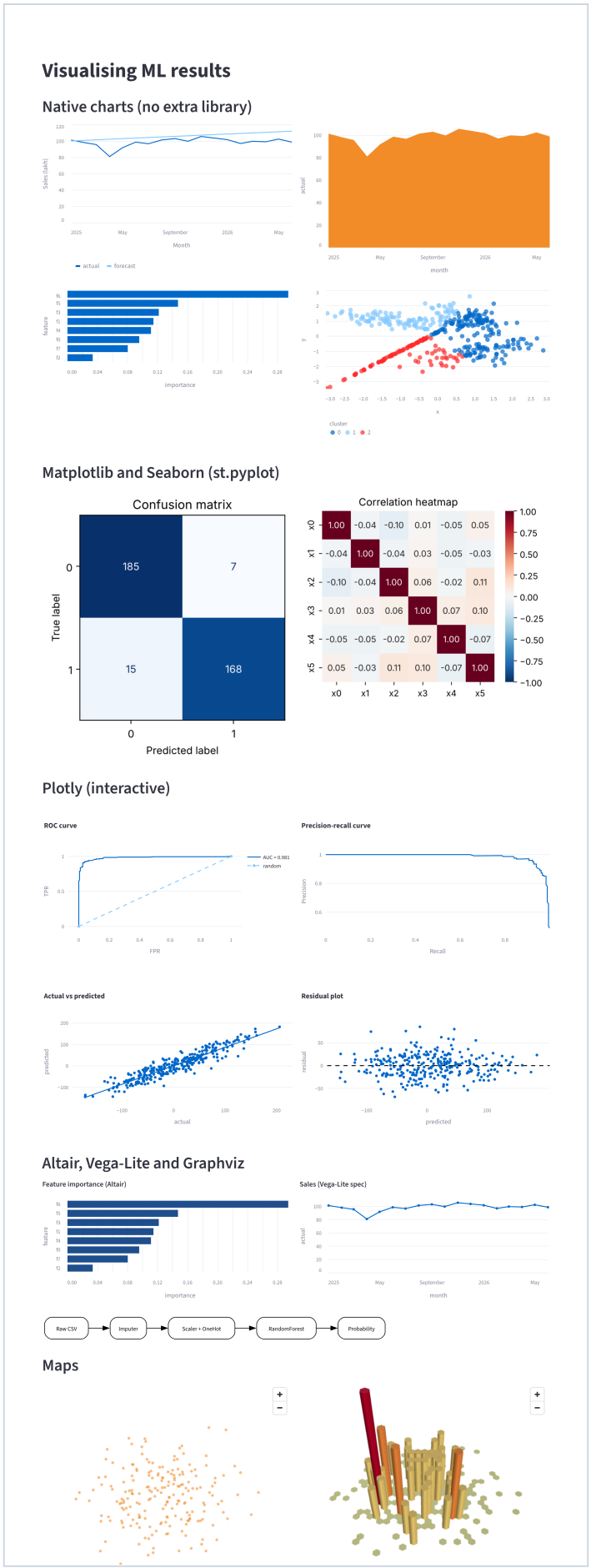

Visualising ML results
Native charts (no extra library)
Sales (lakh)
Month
actual
forecast
actual
month
feature
importance
y
x
cluster
Matplotlib and Seaborn (st.pyplot)
Confusion matrix
True label
Predicted label
185
7
15
168
Correlation heatmap
Plotly (interactive)
ROC curve
TPR
FPR
AUC = 0.981
random
Precision-recall curve
Precision
Recall
Actual vs predicted
predicted
actual
Residual plot
residual
predicted
Altair, Vega-Lite and Graphviz
Feature importance (Altair)
feature
importance
Sales (Vega-Lite spec)
actual
month
Raw CSV
Imputer
Scaler + OneHot
RandomForest
Probability
Maps

Figure 4.6: charts_demo.py: native charts, a confusion matrix and heatmap (Matplotlib/Seaborn), interactive ROC/PR and residual plots (Plotly), Altair, Vega-Lite, Graphviz and two maps

WARNING

Close Matplotlib figures with `plt.close(fig)` after `st.pyplot(fig)`. Because the script reruns constantly, unclosed figures accumulate in memory on a long-running server.

TIP

Prefer `st.plotly_chart(fig, width="stretch")` for ML diagnostics: users can zoom into the ROC curve near the top-left corner or hover over a residual to see the exact value. With `on_select="rerun"` you can even let users select points and show those rows in a table.

4F. Media and file components

demos/media_demo.py PYTHON

```python
"""Images, audio and video.  Run: streamlit run media_demo.py"""
import streamlit as st

st.title("Media elements")
c1, c2 = st.columns(2)
c1.image("assets/logo.png", caption="st.image from a file path", width="stretch")
c2.video("assets/demo.mp4")
st.audio("assets/tone.wav", format="audio/wav")
st.caption("st.audio / st.video accept file paths, URLs, bytes or NumPy arrays (audio).")
```

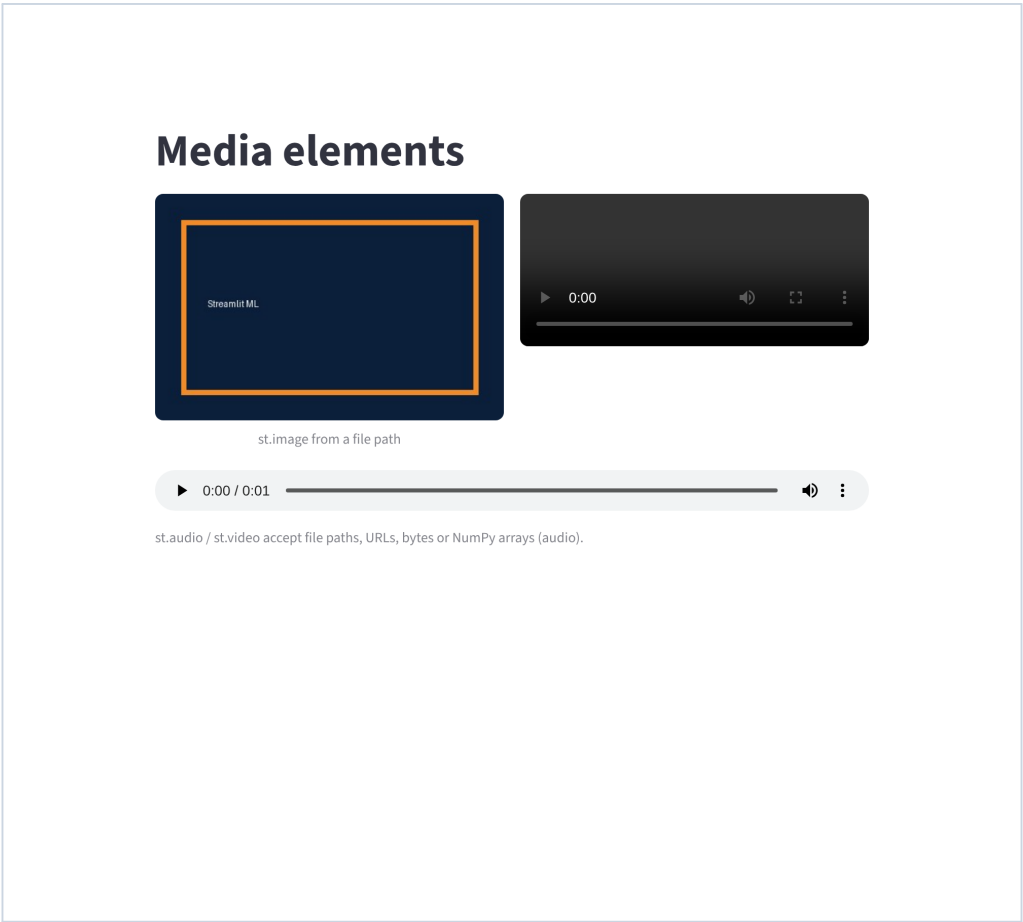


Figure 4.7: st.image, st.video and st.audio

Reading uploaded files of every common type

read_any_file.py PYTHON

```python
import io
import json

import pandas as pd
import streamlit as st
from PIL import Image, UnidentifiedImageError

f = st.file_uploader("Upload a file",
                     type=["csv", "xlsx", "json", "txt", "png", "jpg", "wav", "mp4"])
if f is not None:
    name = f.name.lower()
    try:
        if name.endswith(".csv"):
            df = pd.read_csv(f)                          # CSV -> DataFrame
            st.dataframe(df.head())
        elif name.endswith(".xlsx"):
            df = pd.read_excel(f, sheet_name=0)          # needs openpyxl
            st.dataframe(df.head())
        elif name.endswith(".json"):
            payload = json.loads(f.getvalue().decode("utf-8"))
            st.json(payload)
        elif name.endswith(".txt"):
            text = f.getvalue().decode("utf-8", errors="replace")
            st.text(text[:2000])
        elif name.endswith((".png", ".jpg")):
            img = Image.open(f)
            img.load()                                   # decode now: surfaces errors
            st.image(img, caption=f"{img.width}x{img.height} {img.mode}")
        elif name.endswith(".wav"):
            st.audio(f.getvalue(), format="audio/wav")
        elif name.endswith(".mp4"):
            st.video(f.getvalue())
    except (ValueError, UnicodeDecodeError, UnidentifiedImageError) as exc:
        st.error(f"Could not read {f.name}: {exc}")
```

Downloading CSV, Excel, PDF, predictions and charts

downloads.py PYTHON

```python
import io

import matplotlib.pyplot as plt
import pandas as pd
import streamlit as st

preds = pd.DataFrame({"customer": ["C1", "C2"], "churn_probability": [0.81, 0.12]})

st.download_button("CSV", preds.to_csv(index=False), "predictions.csv", "text/csv")

xlsx = io.BytesIO()
preds.to_excel(xlsx, index=False)                        # needs openpyxl
st.download_button("Excel", xlsx.getvalue(), "predictions.xlsx",
                   "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet")

fig, ax = plt.subplots()
ax.bar(preds["customer"], preds["churn_probability"])
png = io.BytesIO()
fig.savefig(png, format="png", dpi=150)
plt.close(fig)
st.download_button("Chart (PNG)", png.getvalue(), "chart.png", "image/png")
```

PDF report generation with ReportLab is built step by step in Chapter 12.

SECURITY WARNING

Treat every upload as untrusted. Check the extension **and** parse the content inside try/except; limit size (`max_upload_size`) and row counts.

Never `joblib.load()` or `pickle.load()` an uploaded file: unpickling can execute arbitrary code on your server.

Do not write uploads to a shared folder using the user's file name (path traversal, collisions). Process them in memory.

Images can be 'decompression bombs'. Check `width * height` before processing.

4G. Navigation components

API	Purpose	Notes / limitations
`st.set_page_config()`	Tab title, icon, layout, sidebar state, menu items	In multipage apps call it in the entry file; pages may call it again to override the title
`st.Page(page, title, icon, url_path, default)`	Declares a page from a file path or a function	Each page gets a URL path (`/model_prediction`)
`st.navigation(pages, position="sidebar")`	Builds the menu and returns the selected page; call `.run()`	position: sidebar, top or hidden; dict keys become section headers
`st.page_link(page, label, icon)`	A link to a page (or external URL)	Works anywhere: home-page shortcuts, 'Next step' links
`st.switch_page(page)`	Navigate programmatically	Stops the current run; use after a successful action
`pages/` folder	Legacy automatic multipage	Still supported, but `st.navigation` gives more control; it ignores `pages/` when used
`st.sidebar` widgets	Hand-made menu (`st.radio` of page names)	Simple, but no URLs and no back button support

Example: a six-page ML application

multipage_demo/ structure — TEXT

```
multipage_demo/
├── app.py                # router: page config, navigation, shared sidebar
├── common.py             # cached loaders shared by pages
├── train_model.py
├── models/
│   ├── iris_model.joblib
│   └── iris_metadata.json
└── app_pages/
    ├── home.py           # 1. Home dashboard
    ├── data_explorer.py  # 2. Data explorer
    ├── prediction.py     # 3. Model prediction
    ├── evaluation.py     # 4. Model evaluation
    ├── history.py        # 5. Prediction history
    └── about.py          # 6. About the model
```

multipage_demo/app.py — PYTHON

```python
"""Entry point of a multipage ML app.  Run: streamlit run app.py

With st.navigation, app.py is the "router": it runs on EVERY rerun, then runs the
selected page. Put shared setup (page config, sidebar, state) here.
"""
import streamlit as st

from common import init_history

st.set_page_config(page_title="Iris ML Studio", page_icon=":material/local_florist:",
                   layout="wide")
```

multipage_demo/app.py PYTHON

```python
init_history()

home = st.Page("app_pages/home.py", title="Home dashboard", icon=":material/home:",
               default=True)
pages = {
    "Overview": [home],
    "Data": [st.Page("app_pages/data_explorer.py", title="Data explorer",
                     icon=":material/table:")],
    "Model": [
        st.Page("app_pages/prediction.py", title="Model prediction",
                icon=":material/online_prediction:"),
        st.Page("app_pages/evaluation.py", title="Model evaluation",
                icon=":material/analytics:"),
        st.Page("app_pages/history.py", title="Prediction history",
                icon=":material/history:"),
        st.Page("app_pages/about.py", title="About the model", icon=":material/info:"),
    ],
}
current = st.navigation(pages, position="sidebar")
st.sidebar.caption(f"Predictions this session: {len(st.session_state.history)}")
current.run()
```

multipage_demo/common.py PYTHON

```python
"""Shared loaders and helpers used by every page."""
import json

import joblib
import pandas as pd
import streamlit as st
from sklearn.datasets import load_iris


@st.cache_resource
def load_model():
    model = joblib.load("models/iris_model.joblib")
    with open("models/iris_metadata.json") as f:
        return model, json.load(f)


@st.cache_data
def load_data() -> pd.DataFrame:
    iris = load_iris(as_frame=True)
    df = iris.frame.copy()
    df["species"] = df["target"].map(dict(enumerate(iris.target_names)))
    return df.drop(columns="target")


def init_history() -> None:
    st.session_state.setdefault("history", [])
```

multipage_demo/app_pages/home.py PYTHON

```python
import streamlit as st

from common import load_data, load_model

model, meta = load_model()
df = load_data()
st.title("Iris ML Studio")
st.write("A small multipage app: explore the data, predict a species, check the model.")
c1, c2, c3 = st.columns(3)
c1.metric("Training rows", len(df))
c2.metric("Classes", df["species"].nunique())
c3.metric("Test accuracy", f"{meta['accuracy']:.1%}")
st.subheader("Quick links")
q1, q2, q3 = st.columns(3)
```

multipage_demo/app_pages/home.py PYTHON

```python
q1.page_link("app_pages/prediction.py", label="Make a prediction",
             icon=":material/online_prediction:")
q2.page_link("app_pages/data_explorer.py", label="Explore the data",
             icon=":material/table:")
q3.page_link("app_pages/evaluation.py", label="See evaluation",
             icon=":material/analytics:")
```

multipage_demo/app_pages/data_explorer.py PYTHON

```python
import streamlit as st

from common import load_data

df = load_data()
st.title("Data explorer")
species = st.multiselect("Species", sorted(df["species"].unique()),
                         default=sorted(df["species"].unique()))
view = df[df["species"].isin(species)]
st.dataframe(view, width="stretch", hide_index=True)
x = st.selectbox("X axis", df.columns[:4], index=2)
y = st.selectbox("Y axis", df.columns[:4], index=3)
st.scatter_chart(view, x=x, y=y, color="species")
```

multipage_demo/app_pages/prediction.py PYTHON

```python
from datetime import datetime

import pandas as pd
import streamlit as st

from common import load_model

model, meta = load_model()
st.title("Model prediction")
with st.form("predict"):
    cols = st.columns(2)
    values = {}
    for i, feat in enumerate(meta["feature_names"]):
        lo, hi = meta["feature_ranges"][feat]
        values[feat] = cols[i % 2].number_input(feat, lo, hi, round((lo + hi) / 2, 1))
    submitted = st.form_submit_button("Predict", type="primary")
if submitted:
    proba = model.predict_proba(pd.DataFrame([values]))[0]
    label = meta["target_names"][proba.argmax()]
    st.success(f"Predicted species: **{label}** ({proba.max():.1%})")
    st.session_state.history.append({"time": datetime.now().strftime("%H:%M:%S"),
                                     **values, "prediction": label,
                                     "confidence": round(float(proba.max()), 3)})
    # A button nested under `if submitted:` would vanish on the next rerun, so its
    # click would never register. A page link works because it is a plain link.
    st.page_link("app_pages/history.py", label="Open prediction history",
                 icon=":material/history:")
```

multipage_demo/app_pages/evaluation.py PYTHON

```python
import pandas as pd
import plotly.express as px
import streamlit as st
from sklearn.metrics import confusion_matrix

from common import load_data, load_model

model, meta = load_model()
df = load_data()
st.title("Model evaluation")
st.caption("Computed on the full Iris dataset for illustration; real evaluation must "
```

multipage_demo/app_pages/evaluation.py PYTHON

```
            "use held-out data (see the evaluation dashboard chapter).")
pred_idx = model.predict(df[meta["feature_names"]])
pred = pd.Series(pred_idx).map(dict(enumerate(meta["target_names"])))
cm = confusion_matrix(df["species"], pred, labels=meta["target_names"])
st.plotly_chart(px.imshow(cm, text_auto=True, x=meta["target_names"],
                          y=meta["target_names"], labels=dict(x="Predicted", y="Actual")),
                width="stretch")
```

multipage_demo/app_pages/history.py PYTHON

```
import pandas as pd
import streamlit as st

st.title("Prediction history")
if not st.session_state.history:
    st.info("No predictions yet in this session.")
    st.page_link("app_pages/prediction.py", label="Go to prediction page")
else:
    hist = pd.DataFrame(st.session_state.history)
    st.dataframe(hist, hide_index=True, width="stretch")
    st.download_button("Download CSV", hist.to_csv(index=False), "history.csv")
    if st.button("Clear history"):
        st.session_state.history = []
        st.rerun()
```

multipage_demo/app_pages/about.py PYTHON

```
import streamlit as st

from common import load_model

_, meta = load_model()
st.title("About the model")
st.markdown("""
- **Algorithm:** standardised features + multinomial logistic regression
- **Dataset:** Fisher's Iris (150 flowers, 4 measurements, 3 species)
- **Limitations:** trained on one small dataset; it cannot recognise other species.
""")
st.json(meta, expanded=False)
```

`train_model.py` is the same as the Iris training script in Project 1 (Chapter 17). Run it once before starting the app.

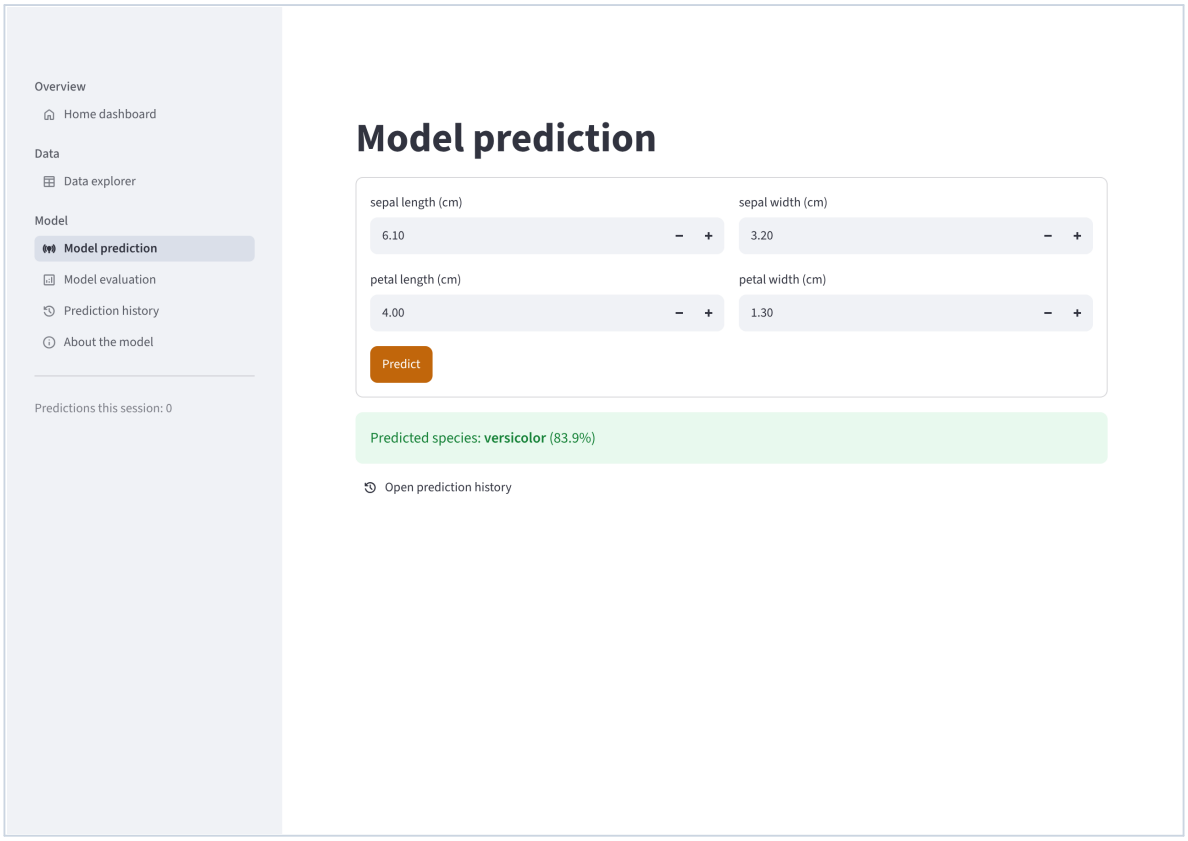


Figure 4.8: The prediction page with grouped navigation in the sidebar

COMMON MISTAKE

Placing a button inside `if submitted:` (or inside another button's branch). On the next rerun the outer condition is False, so the inner button is never drawn and its click is lost. The demo uses `st.page_link` instead, which is a plain link.

4H. Status and feedback components

Component	Use for	Example
`st.spinner(text, show_time=True)`	Short waits with unknown duration	Loading a model
`st.progress(value, text)`	Loops with known length (0-100 or 0.0-1.0)	Batch prediction in chunks
`st.status(label, state, expanded)`	Multi-step jobs with a log	Training: load, fit, evaluate, save
`st.toast(body, icon, duration)`	Non-blocking confirmation	'Report generated'
`st.balloons()`, `st.snow()`	Celebration (use sparingly)	A finished capstone demo
`st.success/info/warning/error`	Persistent messages	Validation results
`st.exception(exc)`	Developer diagnostics	Prediction failure details

demos/status_demo.py PYTHON

```python
"""Progress and feedback elements during ML workflows.  Run: streamlit run status_demo.py"""
import time

import streamlit as st

st.title("Status and feedback")

if st.button("Load model", icon=":material/download:"):
    with st.spinner("Loading model artifacts...", show_time=True):
        time.sleep(1.0)
    st.success("Model loaded.")

if st.button("Run batch predictions", icon=":material/play_arrow:"):
    bar = st.progress(0, text="Scoring rows...")
    for pct in range(0, 101, 20):
        time.sleep(0.15)
        bar.progress(pct, text=f"Scored {pct}% of 10,000 rows")
    st.toast("Batch scoring finished", icon=":material/task_alt:")

with st.status("Training workflow", expanded=True, state="complete") as status:
    st.write("1. Loaded 5,000 rows")
    st.write("2. Fitted preprocessing pipeline")
    st.write("3. Trained random forest (300 trees)")
    st.write("4. Saved models/churn_pipeline.joblib")

st.info("Report generation can take a few seconds for large files.")
st.warning("Model was trained with scikit-learn 1.8; you are running 1.9.")
try:
    raise ValueError("Input contains NaN in column 'total_charges'")
except ValueError as exc:
    st.error("Prediction failed. Check the highlighted inputs.")
    st.exception(exc)
if st.button("Celebrate a successful deployment"):
    st.balloons()
```



Figure 4.9: status_demo.py after running a batch: progress, a completed status log, messages and an exception

status_with_errors.py PYTHON

```python
import streamlit as st

with st.status("Generating report...", expanded=True) as status:
    try:
        st.write("Collecting predictions")
        rows = 1200
        st.write("Rendering charts")
        if rows == 0:
            raise ValueError("No predictions to report")
        status.update(label="Report ready", state="complete", expanded=False)
    except ValueError as exc:
        status.update(label="Report failed", state="error")
        st.error(str(exc))
```

Chapter summary

- Widgets return values; keys mirror them into `st.session_state`.
- Use forms to batch inputs, columns/tabs/expanders to organise, and the sidebar for global settings.
- Choose `st.dataframe` for reading, `st.data_editor` for editing, `st.table` for small summaries, `st.metric` for KPIs.
- Prefer Plotly for interactive diagnostics; close Matplotlib figures.
- `st.navigation` + `st.Page` is the modern way to build multipage apps.
- Use spinner/progress/status for long work and toasts for quick confirmations.

CHAPTER 5

Advanced UI Customization

05

Themes, branding, CSS and dashboard design

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Configure page settings, layouts and the sidebar.
- ✓ Create light and dark themes with .streamlit/config.toml.
- ✓ Add logos, typography, spacing and safe custom CSS/HTML.
- ✓ Build reusable UI functions and dashboard layouts.
- ✓ Understand what custom components can and cannot do.

5.1 Page configuration, titles and icons

page_config.py — PYTHON

```python
import streamlit as st

st.set_page_config(
    page_title="Churn Predictor | SpeedUp Analytics",   # browser tab text
    page_icon=":material/insights:",                     # emoji, image path or Material icon
    layout="wide",                                       # "centered" (default) or "wide"
    initial_sidebar_state="expanded",                    # "auto" | "expanded" | "collapsed"
    menu_items={
        "Get help": "https://docs.streamlit.io",
        "Report a bug": "mailto:ml-team@example.com",
        "About": "Churn model v1.2 - trained 13 Oct 2026",
    },
)
```

Layout	Best for
`centered`	Forms, chat apps, single predictions, articles: a readable column of about 730 px
`wide`	Dashboards, tables with many columns, side-by-side charts

Material Symbols are referenced as `:material/icon_name:` (for example `:material/analytics:`, `:material/upload_file:`). They render consistently on every operating system, unlike emoji, and are accepted by `page_icon`, button `icon=` parameters, `st.Page(icon=...)`, headings and messages.

5.2 Sidebar configuration

- `initial_sidebar_state` controls whether it starts open.
- `st.logo(image, size="medium", link=None, icon_image=None)` places a logo at the top of the sidebar (and a smaller icon when collapsed).
- Put global controls (model, threshold, data source) in the sidebar and keep page-specific content in the main area.

- Theme the sidebar separately with `[theme.sidebar]` in config.toml.

5.3 Custom themes with .streamlit/config.toml

Themes are the safest way to brand an app because they use Streamlit's supported configuration, not internal HTML structure.

.streamlit/config.toml (complete example) TOML

```toml
[theme]
base = "light"                        # starting point: "light" or "dark"
primaryColor = "#F28C28"              # buttons, sliders, focus rings, active tabs
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#F3F6FA" # widget backgrounds, sidebar, code blocks
textColor = "#1F2937"
borderColor = "#D9DEE7"
baseRadius = "0.6rem"                 # corner roundness
font = "sans-serif"                   # "sans-serif" | "serif" | "monospace" | font name
headingFont = "sans-serif"
codeFont = "monospace"
baseFontSize = 15                     # pixels
showSidebarBorder = true

[theme.sidebar]                       # optional overrides for the sidebar only
backgroundColor = "#0B1F3A"
textColor = "#E2E8F0"
secondaryBackgroundColor = "#14325C"

[theme.light]                         # used when the viewer picks light mode
primaryColor = "#C2410C"

[theme.dark]                          # used when the viewer picks dark mode
primaryColor = "#F28C28"
backgroundColor = "#0E1117"
textColor = "#E5E7EB"

[server]
maxUploadSize = 50                    # MB

[client]
toolbarMode = "minimal"               # hide developer options from viewers
```

VERSION NOTE

The `[theme.light]` / `[theme.dark]` sections, `borderColor`, `baseRadius`, `headingFont`, `baseFontSize` and `[theme.sidebar]` were added during 2025. Older releases support only the six basic colour and font options. Check `streamlit config show` to list options for your installed version.

Light and dark themes

Viewers can switch between light, dark and system theme from the app menu. If you set only `[theme]`, that is used for both. To start an app dark for testing, run `streamlit run app.py --theme.base dark`.

Custom fonts

● Loading a self-hosted font TOML

```toml
[[theme.fontFaces]]
family = "Inter"
url = "static/Inter-Regular.woff2"   # requires server.enableStaticServing = true
weight = 400

[theme]
font = "Inter, sans-serif"

[server]
enableStaticServing = true           # serves files from ./static at /app/static/...
```

5.4 Colour, typography, spacing and hierarchy

Principle	In practice
One accent colour	Use `primaryColor` for the main action only (one primary button per screen).
Semantic colours	Green = success/low risk, amber = warning, red = error/high risk. Never rely on colour alone: add text or icons.
Contrast	Body text should reach WCAG AA contrast (4.5:1). Test dark themes too.
Type scale	title > header > subheader > body > caption; do not skip levels.
Spacing	Group related inputs in containers/forms; separate sections with `st.space`, headers with `divider=` or `st.divider()`.
Emoji and icons	Use sparingly and consistently, ideally Material icons. One icon per heading at most; none in body text of serious apps.
Numbers	Format consistently: `f"{p:.1%}"`, thousands separators, units in labels ("Monthly charges (USD)").

5.5 Images and logos

● branding.py (expects files in assets/) PYTHON

```python
import streamlit as st

st.logo("assets/logo.png", size="large", link="https://example.com")
st.image("assets/model_architecture.png", caption="Model architecture", width=600)
```

5.6 Custom CSS: a card-style interface

Streamlit 1.65 offers a stable hook for styling: `st.container(key=...)` **adds the CSS class** `st-key-<key>` to that container. Target those classes instead of Streamlit's internal `data-testid` attributes or generated class names.

● demos/ui_kit.py PYTHON

```python
"""Reusable UI helpers: one place to change the look of every page."""
import streamlit as st

CARD_CSS = """
<style>
/* st.container(key="...") adds a stable CSS class: .st-key-<key> */
[class*="st-key-card"] {
    background: var(--card-bg, #ffffff);
    border: 1px solid #e2e8f0;
    border-left: 5px solid #f28c28;
    border-radius: 12px;
    padding: 1rem 1.2rem;
```

demos/ui_kit.py PYTHON

```
    box-shadow: 0 2px 8px rgba(15, 23, 42, 0.06);
}
.kpi-label { font-size: 0.8rem; color: #64748b; text-transform: uppercase; }
.kpi-value { font-size: 1.8rem; font-weight: 700; color: #0b1f3a; }
.kpi-note  { font-size: 0.8rem; color: #059669; }
</style>
"""


def inject_css() -> None:
    st.html(CARD_CSS)       # st.html renders without an iframe; scripts are stripped


def kpi_card(key: str, label: str, value: str, note: str = "") -> None:
    with st.container(key=f"card_{key}"):
        st.html(f'<div class="kpi-label">{label}</div>'
                f'<div class="kpi-value">{value}</div>'
                f'<div class="kpi-note">{note}</div>')


def page_header(title: str, subtitle: str, logo: str | None = None) -> None:
    left, right = st.columns([5, 1], vertical_alignment="center")
    left.title(title)
    left.caption(subtitle)
    if logo:
        right.image(logo, width=110)
```

SECURITY WARNING

CSS selectors that target Streamlit's internal DOM (`div.css-1d391kg`, `[data-testid="stSidebar"] > div`) can break in any release. Use theme options first, `st-key-` classes second, internal selectors last.

`st.markdown(..., unsafe_allow_html=True)` and `st.html()` must never render text that came from users or uploaded files without escaping (`html.escape(text)`), or you create an XSS (cross-site scripting) hole.

`st.html` strips `<script>` by default. Executing JavaScript requires `unsafe_allow_javascript=True` or a component; avoid both unless you fully control the content.

5.7 Design example 1: clean business dashboard

demos/business_dashboard.py PYTHON

```
"""Clean business dashboard with branded KPI cards.  Run: streamlit run business_dashboard.py"""
import numpy as np
import pandas as pd
import plotly.express as px
import streamlit as st

from ui_kit import inject_css, kpi_card, page_header

st.set_page_config(page_title="Sales Insights", page_icon=":material/monitoring:",
                   layout="wide")
st.logo("assets/logo.png")                  # logo in the sidebar / top-left corner
inject_css()
page_header("Sales Insights", "Forecasts refreshed daily at 02:30 IST",
            "assets/logo.png")

c1, c2, c3, c4 = st.columns(4)
with c1:
    kpi_card("rev", "Revenue (MTD)", "₹4.82 Cr", "+6.1% vs last month")
with c2:
    kpi_card("fc", "Forecast (month end)", "₹6.10 Cr", "MAPE 7.4%")
with c3:
    kpi_card("orders", "Orders", "18,402", "+3.2%")
with c4:
```

demos/business_dashboard.py PYTHON

```python
    kpi_card("churn", "Churn risk customers", "1,284", "-56 since Monday")

rng = np.random.default_rng(7)
days = pd.date_range("2026-09-01", periods=42)
df = pd.DataFrame({"date": days, "actual": 15 + rng.normal(0, 1.2, 42).cumsum() * 0.3})
df["forecast"] = df["actual"].rolling(7, min_periods=1).mean() + 0.4
left, right = st.columns([2, 1], gap="large")
left.plotly_chart(px.line(df, x="date", y=["actual", "forecast"], height=330,
                          title="Daily revenue (lakh)"), width="stretch")
region = pd.DataFrame({"region": ["Pune", "Mumbai", "Nagpur", "Nashik"],
                       "revenue": [1.9, 1.6, 0.8, 0.5]})
right.plotly_chart(px.bar(region, x="revenue", y="region", orientation="h", height=330,
                          title="Revenue by region (Cr)"), width="stretch")
```



Figure 5.1: Business dashboard with logo, branded KPI cards and two charts

5.8 Design example 2: dark-themed AI application

demos/dark_ai_app.py PYTHON

```python
"""Dark-themed AI assistant layout.  Run: streamlit run dark_ai_app.py --theme.base dark

The responder below is a rule-based stub so the example runs offline. In a real app
replace `answer()` with a call to your model or LLM API.
"""
import streamlit as st

st.set_page_config(page_title="Model Assistant", page_icon=":material/smart_toy:")
st.title("Model Assistant", icon=":material/smart_toy:")
st.caption("Ask about the deployed churn model. Answers come from its model card.")

MODEL_CARD = {"algorithm": "Random forest with 300 trees",
              "features": "tenure, charges, support calls, contract, payment method",
```

demos/dark_ai_app.py PYTHON

```
            "metric": "ROC-AUC 0.79 on 1,000 held-out customers",
            "limit": "Trained on one telecom dataset; do not use for other industries"}


def answer(question: str) -> str:
    q = question.lower()
    for key, text in MODEL_CARD.items():
        if key in q or (key == "metric" and "accur" in q):
            return text
    return "I can answer questions about the algorithm, features, metric and limits."


st.session_state.setdefault("messages", [
    {"role": "assistant", "content": "Hi! What would you like to know about the model?"}])
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.write(msg["content"])
if prompt := st.chat_input("e.g. What features does the model use?"):
    st.session_state.messages.append({"role": "user", "content": prompt})
    st.session_state.messages.append({"role": "assistant", "content": answer(prompt)})
    st.rerun()
```

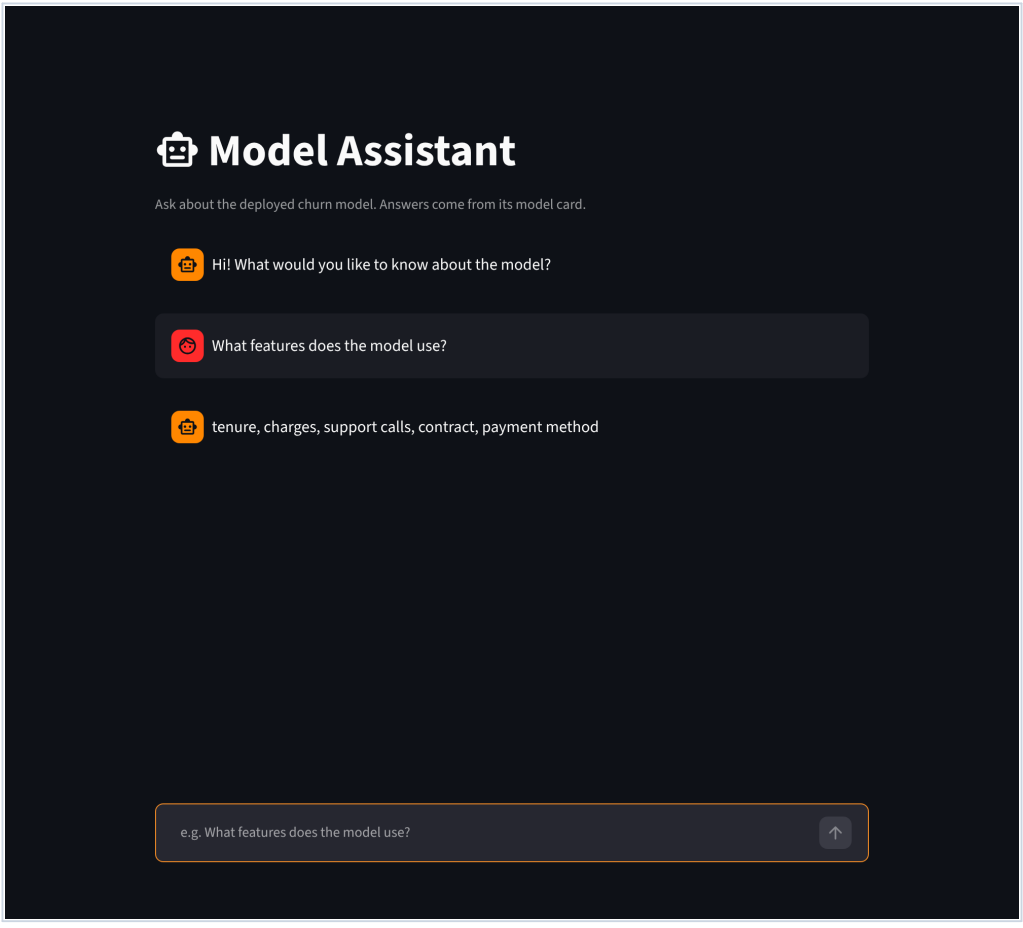


Figure 5.2: Chat-style assistant in the dark theme (`--theme.base dark`)

5.9 Design example 3: modern prediction form

demos/prediction_form_modern.py PYTHON

```python
"""A modern, card-style prediction form.  Run: streamlit run prediction_form_modern.py"""
import streamlit as st

from ui_kit import inject_css

st.set_page_config(page_title="Loan Pre-check", layout="centered")
inject_css()
st.title("Loan pre-check", icon=":material/account_balance:")
st.caption("Educational demo: a real credit decision needs validated models, "
           "regulatory review and a human underwriter.")

with st.container(key="card_form"):
    with st.form("loan", border=False):
        st.subheader("Applicant")
        a, b = st.columns(2)
        income = a.number_input("Monthly income (INR)", 0, 1_000_000, 85_000, step=5_000)
        emi = b.number_input("Existing EMIs (INR)", 0, 500_000, 12_000, step=1_000)
        tenure = a.select_slider("Loan tenure (years)", [1, 2, 3, 5, 7, 10, 15, 20],
                                 value=5)
        amount = b.number_input("Loan amount (INR)", 10_000, 10_000_000, 800_000,
                                step=50_000)
        employment = st.segmented_control("Employment", ["Salaried", "Self-employed",
                                                         "Other"], default="Salaried")
        ok = st.form_submit_button("Check eligibility", type="primary", width="stretch")

if ok:
    monthly_rate = 0.11 / 12
    n = tenure * 12
    new_emi = amount * monthly_rate * (1 + monthly_rate) ** n / ((1 + monthly_rate) ** n - 1)
    dti = (emi + new_emi) / max(income, 1)
    with st.container(key="card_result"):
        c1, c2 = st.columns(2)
        c1.metric("Estimated EMI", f"₹{new_emi:,.0f}")
        c2.metric("Debt-to-income", f"{dti:.0%}")
        if dti <= 0.4:
            st.success("Within the 40% debt-to-income guideline.")
        else:
            st.warning("Above the 40% guideline; refer to an underwriter.")
```

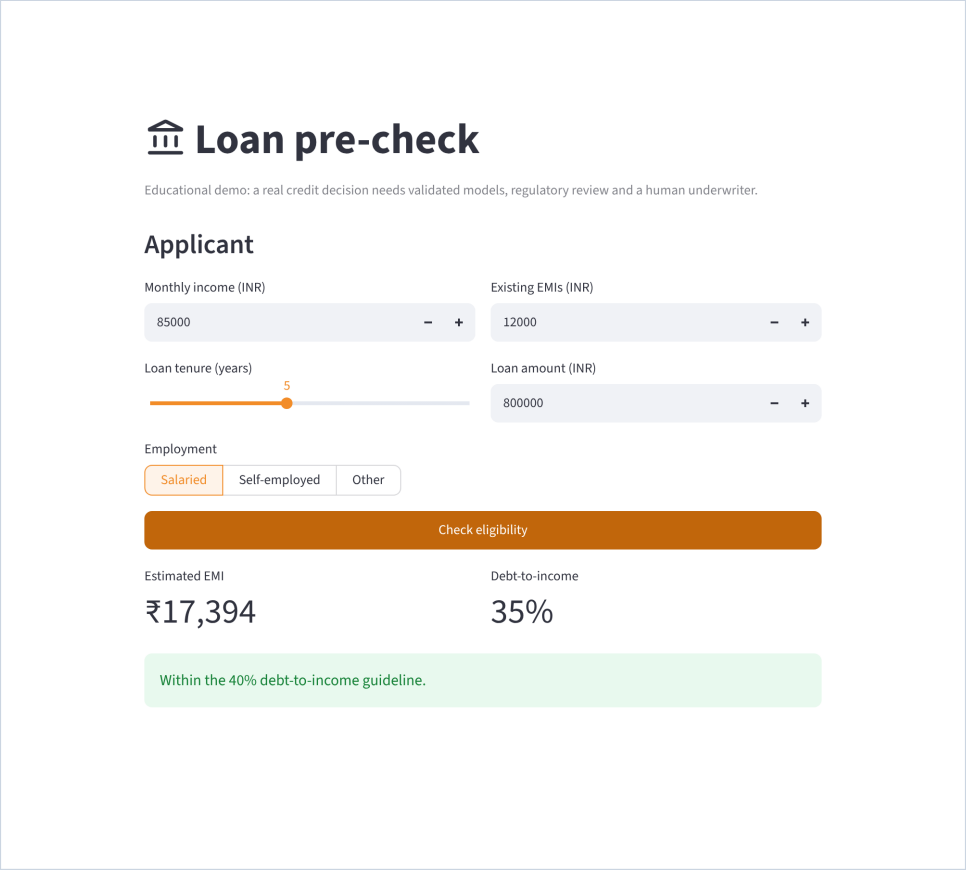


Figure 5.3: Card-style form using st-key CSS classes, segmented control and a result card

5.10 Design example 4: model monitoring dashboard

demos/monitoring_dashboard.py — PYTHON

```python
"""Model monitoring dashboard with an auto-refreshing fragment.

Run: streamlit run monitoring_dashboard.py
The metrics are simulated; in production read them from your logging database.
"""
from datetime import datetime

import numpy as np
import pandas as pd
import plotly.express as px
import streamlit as st

st.set_page_config(page_title="Model Monitoring", page_icon=":material/monitor_heart:",
                   layout="wide")
st.title("Model monitoring", icon=":material/monitor_heart:")


def simulated_metrics(n_days: int = 30) -> pd.DataFrame:
    rng = np.random.default_rng(int(datetime.now().timestamp()) // 10)
    days = pd.date_range(end=pd.Timestamp.today().normalize(), periods=n_days)
    drift = np.clip(np.linspace(0.04, 0.17, n_days) + rng.normal(0, 0.01, n_days), 0, 1)
    return pd.DataFrame({
        "date": days, "requests": rng.poisson(1200, n_days),
        "latency_ms": rng.normal(85, 9, n_days).round(1),
        "psi_drift": drift.round(3),
        "auc_on_labeled": (0.80 - drift * 0.4 + rng.normal(0, 0.005, n_days)).round(3),
    })
```

demos/monitoring_dashboard.py PYTHON

```python
@st.fragment(run_every="10s")              # only this function reruns every 10 s
def live_panel():
    m = simulated_metrics()
    last = m.iloc[-1]
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Requests (today)", f"{last.requests:,}", border=True)
    c2.metric("p50 latency", f"{last.latency_ms:.0f} ms", border=True)
    c3.metric("Feature drift (PSI)", f"{last.psi_drift:.2f}",
              f"{last.psi_drift - m.psi_drift.iloc[-8]:+.2f} vs last week",
              delta_color="inverse", border=True)
    c4.metric("AUC (labeled sample)", f"{last.auc_on_labeled:.3f}", border=True)
    if last.psi_drift > 0.15:
        st.error("Drift above 0.15: schedule a retraining review.",
                 icon=":material/warning:")
    left, right = st.columns(2)
    left.plotly_chart(px.line(m, x="date", y="psi_drift", title="Population stability "
                              "index", height=300), width="stretch")
    right.plotly_chart(px.line(m, x="date", y="auc_on_labeled",
                               title="AUC on delayed labels", height=300),
                       width="stretch")
    st.caption(f"Last refreshed {datetime.now():%H:%M:%S}")


live_panel()
with st.expander("How is drift measured?"):
    st.write("PSI compares the distribution of each input feature today with the "
             "training data. Rule of thumb: < 0.1 stable, 0.1-0.25 moderate, > 0.25 "
             "major shift. Thresholds should be agreed per project.")
```

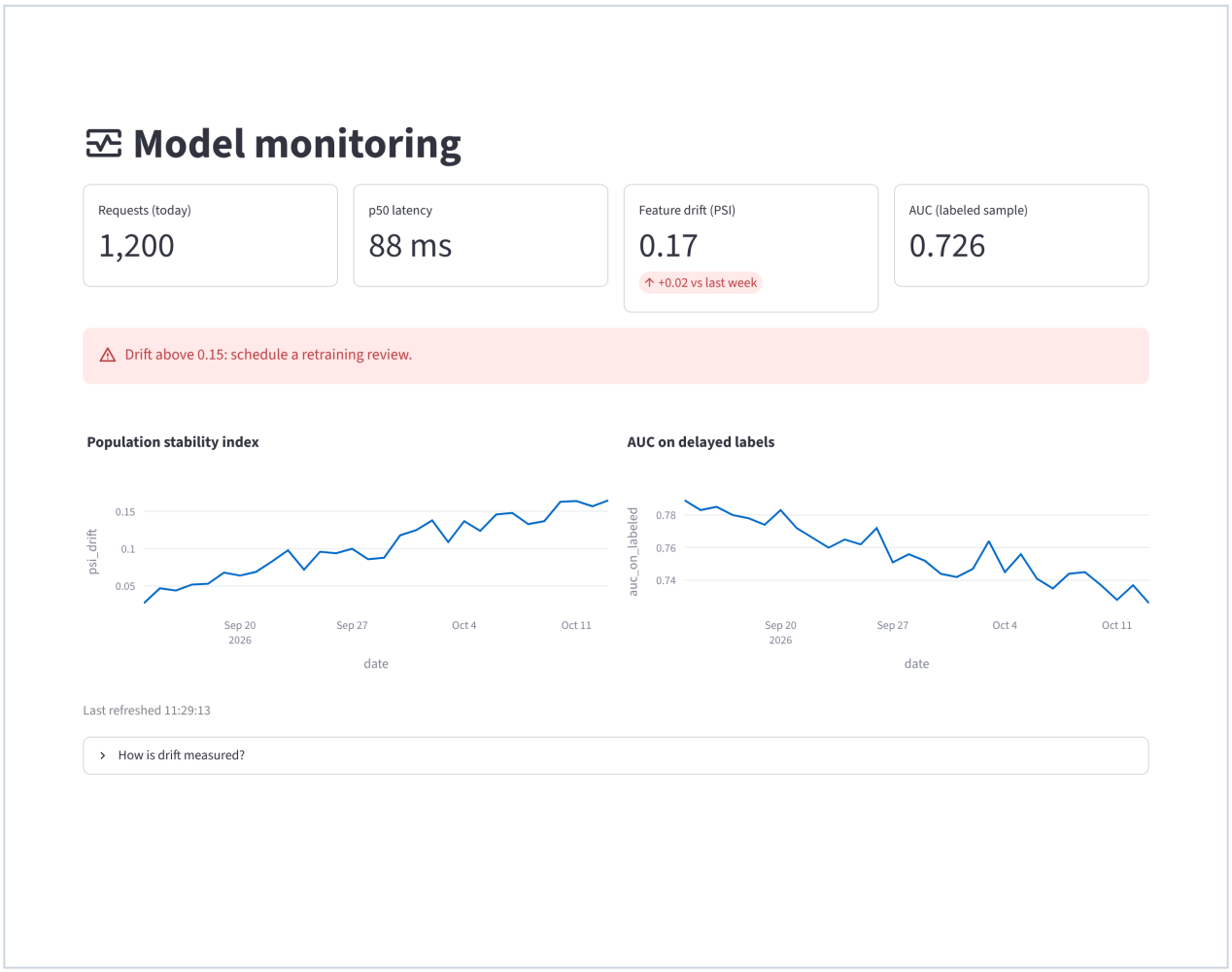


Figure 5.4: Monitoring dashboard: an auto-refreshing fragment with drift and performance KPIs

5.11 Responsive layout considerations

- Columns stack vertically on narrow screens; order them so the stacked version still reads correctly (inputs before results).
- Avoid more than four columns of KPIs; use `st.container(horizontal=True, wrap=True)` for chips and buttons that should wrap.
- Give charts `width="stretch"` and a sensible `height`; avoid fixed pixel widths.
- Test with the browser's device toolbar (phone width ~ 390 px).
- Wide tables: hide the index, pin the id column, and choose columns with `column_order`.

5.12 Reusable UI functions

ui_helpers.py PYTHON

```python
import streamlit as st


def kpi_row(metrics: dict[str, str]) -> None:
    for col, (label, value) in zip(st.columns(len(metrics)), metrics.items()):
        col.metric(label, value, border=True)


def risk_badge(probability: float) -> None:
    if probability >= 0.7:
        st.badge("High risk", icon=":material/error:", color="red")
    elif probability >= 0.4:
        st.badge("Medium risk", icon=":material/warning:", color="orange")
    else:
        st.badge("Low risk", icon=":material/check:", color="green")


kpi_row({"Accuracy": "0.87", "Recall": "0.74", "Rows": "1,284"})
risk_badge(0.82)
```

BEST PRACTICE

Put helpers like these in `ui/components.py`. Every page then looks consistent, and a design change happens in one place. The capstone (Chapter 22) uses exactly this structure.

5.13 Custom components and their limitations

When no built-in element fits, Streamlit **components** embed custom HTML/JS (for example `streamlit-aggrid`, `streamlit-folium`, `streamlit-bokeh`). You can also build your own with React or plain JS.

Consideration	Detail
Maintenance	Third-party components may lag behind new Streamlit releases.
Security	They run JavaScript in your users' browsers; install only well-maintained packages and pin versions.
Performance	Each component is an iframe (or custom element) with its own loading cost.
Theming	Components may not follow your Streamlit theme automatically.
Alternatives first	`st.dataframe` selections, `st.plotly_chart` selections, `st.html`, `st.iframe` and fragments cover many needs.

5.14 Professional application branding checklist

- ☐ Descriptive `page_title` and a Material `page_icon`.
- ☐ Logo via `st.logo`; brand colours in config.toml, not hard-coded everywhere.
- ☐ One primary action per screen; consistent button labels (verb + noun).
- ☐ Units and help tooltips on every numeric input.
- ☐ A footer or About page with model version, training date and limitations.
- ☐ Toolbar set to `minimal` and error details hidden for public apps.
- ☐ Checked in both light and dark mode, and at phone width.

CHAPTER 6

Connecting Streamlit with Machine Learning Models

06

From a notebook model to a validated, explainable prediction app

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Fit preprocessing and model together so inference matches training.
- ✓ Save and safely load scikit-learn, Keras and PyTorch models.
- ✓ Build complete apps for classification, regression, clustering, forecasting, image classification, NLP and anomaly detection.
- ✓ Validate inputs, explain predictions and handle errors.
- ✓ Keep a prediction history and prepare the app for deployment.

Before you start: Chapters 1-4. Install: `pip install scikit-learn plotly joblib statsmodels tensorflow pillow` (and `torch` for Section 6.8.1).

6.1 The fourteen-step deployment workflow

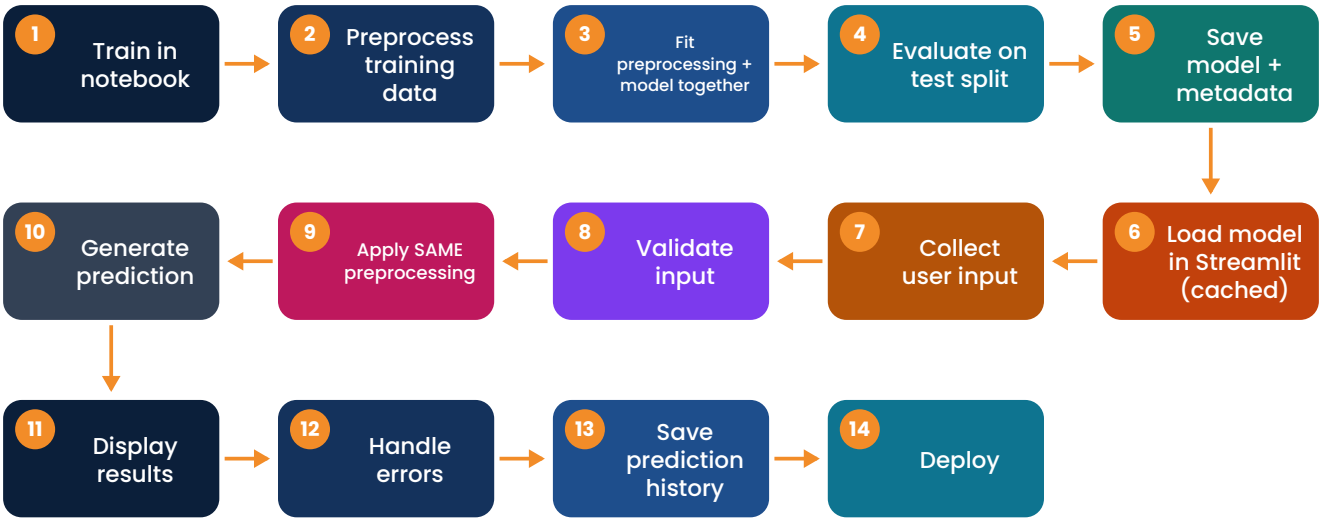


Figure 6.1: The steps every project in this chapter follows

Steps 1-5 belong in a reproducible `train_model.py` (you can prototype them in Jupyter first). Steps 6-13 are the Streamlit app. Step 14 is Chapters 13-14.

TIP

Moving from notebook to script: copy the cells that load data, build the pipeline, fit, evaluate and save into `train_model.py`; delete exploratory plots; replace hard-coded paths with `Path(__file__).parent / ...`; run it from a clean kernel to prove it works top to bottom.

6.2 Why training-time and inference-time preprocessing must match

A model learns patterns in **transformed** numbers. If the scaler subtracted a mean of 32.4 months during training, the app must subtract exactly 32.4 too; if one-hot encoding created a column `contract_Two year`, the app must produce the same column in the same position. Any difference produces predictions that look valid but are wrong, and no error is raised.

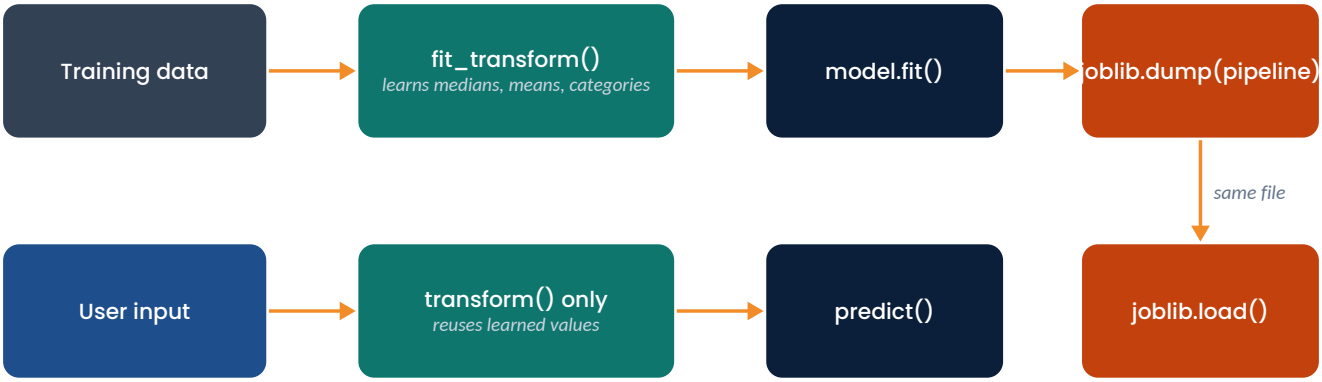


Figure 6.2: Top: training. Bottom: inference reuses the fitted objects

anti_patterns.py — PSEUDOCODE / FRAGMENT

```
# WRONG: preprocessing re-fitted inside the app on a single row
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_user_scaled = scaler.fit_transform(X_user)   # mean/std of ONE row -> all zeros!
prediction = model.predict(X_user_scaled)

# WRONG: manual encoding that differs from training
X_user["contract"] = X_user["contract"].map({"Month-to-month": 0, "One year": 1})
```

pipeline_pattern.py — PSEUDOCODE / FRAGMENT

```
# RIGHT: one Pipeline holds every fitted step; save and load it as a unit
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

pipe = Pipeline([
    ("preprocess", ColumnTransformer([
        ("num", StandardScaler(), ["tenure_months", "monthly_charges"]),
        ("cat", OneHotEncoder(handle_unknown="ignore"), ["contract"]),
    ])),
    ("model", RandomForestClassifier(random_state=42)),
])
pipe.fit(X_train, y_train)                # fit on TRAINING data only
# in the app: pipe.predict(X_user) -> transform with stored values, then predict
```

BEST PRACTICE

Keep **all** preprocessing (imputation, scaling, encoding, text vectorising, feature engineering) inside a scikit-learn `Pipeline` or a single function imported by both training and the app.

Store the exact feature list and order in a **schema module** or metadata file. Build the inference DataFrame with `pd.DataFrame([record])[FEATURES]`.

Fit only on the training split. Fitting on all data before splitting leaks test information and inflates your metrics.

6.3 Saving and loading models

Framework	Save	Load	Notes
scikit-learn (joblib)	`joblib.dump(pipe, "m.joblib")`	`joblib.load("m.joblib")`	Standard choice; efficient for NumPy arrays
scikit-learn (pickle)	`pickle.dump(pipe, f)`	`pickle.load(f)`	Same risks as joblib; slightly larger files
TensorFlow / Keras 3	`model.save("m.keras")`	`keras.models.load_model("m.keras")`	Native zip format; `.h5` is legacy
PyTorch	`torch.save(model.state_dict(), "m.pt")`	`model.load_state_dict(torch.load(p, weights_only=True))`	Save weights, rebuild the class in code
Hugging Face	`model.save_pretrained(dir)`	`from_pretrained(dir)`	Prefer `safetensors` weights
Cross-framework	skl2onnx / torch.onnx	onnxruntime	Portable, no pickle, fast CPU inference

save_load_sklearn.py (fragment: assumes a fitted `pipe`) — PSEUDOCODE / FRAGMENT

```
import pickle

import joblib

joblib.dump(pipe, "models/model.joblib")          # joblib (recommended for sklearn)
pipe = joblib.load("models/model.joblib")

with open("models/model.pkl", "wb") as f:         # pickle (standard library)
    pickle.dump(pipe, f)
with open("models/model.pkl", "rb") as f:
    pipe = pickle.load(f)
```

SECURITY WARNING

Never load a pickle or joblib file you did not create yourself or obtain from a source you fully trust. Unpickling can execute arbitrary code, so a malicious `.joblib` file can take over your server. This includes files uploaded by users and files downloaded from the internet.

Never offer an 'upload your model' feature in a public app that calls `joblib.load()` on the upload.

For PyTorch always pass `weights_only=True` to `torch.load` (the default in recent PyTorch versions).

Version compatibility

- A joblib/pickle file stores references to classes such as `sklearn.ensemble._forest.RandomForestClassifier`. Loading it with a different scikit-learn version may fail or warn (`InconsistentVersionWarning`). Pin the training version in `requirements.txt`.
- Custom functions used inside a pipeline (for example a text cleaner) are stored **by reference**: the module that defines them must be importable when loading (Section 6.9).

- Record versions, features, metrics and a file fingerprint in a metadata JSON next to the model. The helper below does this and refuses to load a file whose fingerprint has changed.

churn_app/utils/model_io.py PYTHON

```python
"""Save and load model artifacts with an integrity check.

joblib/pickle files can execute arbitrary code when loaded, so an app should only
load artifacts that YOU produced. We record a SHA-256 fingerprint at training time
and refuse to load a file whose fingerprint no longer matches.
"""
import hashlib
import json
import platform
from datetime import datetime, timezone
from pathlib import Path

import joblib
import sklearn


class ModelLoadError(RuntimeError):
    """Raised when a model artifact is missing, modified, or incompatible."""


def file_sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with open(path, "rb") as fh:
        for chunk in iter(lambda: fh.read(1 << 20), b""):
            digest.update(chunk)
    return digest.hexdigest()


def save_model(model, model_path: Path, meta_path: Path, metadata: dict) -> dict:
    model_path.parent.mkdir(parents=True, exist_ok=True)
    joblib.dump(model, model_path)
    metadata = {
        **metadata,
        "artifact": model_path.name,
        "sha256": file_sha256(model_path),
        "sklearn_version": sklearn.__version__,
        "python_version": platform.python_version(),
        "trained_at": datetime.now(timezone.utc).isoformat(timespec="seconds"),
    }
    meta_path.write_text(json.dumps(metadata, indent=2))
    return metadata


def load_metadata(meta_path: Path) -> dict:
    if not meta_path.exists():
        raise ModelLoadError(f"Metadata file not found: {meta_path}")
    return json.loads(meta_path.read_text())


def load_trusted_model(model_path: Path, meta_path: Path):
    """Return (model, metadata, warnings). Raises ModelLoadError on failure."""
    if not model_path.exists():
        raise ModelLoadError(
            f"Model file not found: {model_path}. Run train_model.py first."
        )
    meta = load_metadata(meta_path)
    if file_sha256(model_path) != meta.get("sha256"):
        raise ModelLoadError(
            "Model fingerprint does not match metadata. The file may have been "
            "replaced or corrupted, so it will not be loaded."
        )
    warnings = []
    if meta.get("sklearn_version") != sklearn.__version__:
        warnings.append(
```

churn_app/utils/model_io.py PYTHON

```
            f"Model trained with scikit-learn {meta.get('sklearn_version')}, "
            f"running {sklearn.__version__}. Pin the training version in "
            "requirements.txt to avoid subtle behaviour changes."
        )
    try:
        model = joblib.load(model_path)
    except Exception as exc:  # corrupted file, missing class, version clash
        raise ModelLoadError(f"Could not load model: {exc}") from exc
    return model, meta, warnings
```

6.4 Classification project: Customer Churn Prediction

A telecom company wants to know which customers are likely to cancel so the retention team can contact them first. We build the full stack: data, training script, saved pipeline with metadata, and a Streamlit app with a form, probabilities, an explanation, batch scoring and a metrics dashboard.

churn_app/ structure TEXT

```
churn_app/
├── app.py                    # Streamlit UI
├── schema.py                 # feature names, categories, ranges (single source of truth)
├── train_model.py            # train + evaluate + save
├── requirements.txt
├── .streamlit/config.toml
├── data/
│   ├── make_dataset.py       # generates churn.csv and new_customers.csv
│   ├── churn.csv             # training data (with labels)
│   └── new_customers.csv     # inference data (no labels)
├── models/                   # created by train_model.py
│   ├── churn_pipeline.joblib
│   ├── churn_metadata.json
│   └── churn_test_predictions.csv
├── utils/
│   ├── __init__.py
│   ├── model_io.py           # save/load with fingerprint (Section 6.3)
│   ├── validation.py         # record + DataFrame validation
│   ├── explain.py            # per-prediction explanation
│   └── charts.py             # Plotly figures
└── tests/test_app.py
```

Step 1: the schema

churn_app/schema.py PYTHON

```
"""Single source of truth for the churn model's input schema.

Both train_model.py and app.py import from this file, so the columns used
at training time are guaranteed to match the columns sent at inference time.
"""

TARGET = "churn"
POSITIVE_CLASS = "Yes"
ID_COLUMN = "customer_id"

NUMERIC_FEATURES = ["tenure_months", "monthly_charges", "total_charges", "support_calls"]
CATEGORICAL_FEATURES = [
    "contract",
    "internet_service",
    "payment_method",
    "paperless_billing",
    "senior_citizen",
]
FEATURES = NUMERIC_FEATURES + CATEGORICAL_FEATURES
```

churn_app/schema.py PYTHON

```python
CATEGORY_OPTIONS = {
    "contract": ["Month-to-month", "One year", "Two year"],
    "internet_service": ["Fiber optic", "DSL", "No"],
    "payment_method": ["Electronic check", "Mailed check", "Bank transfer", "Credit card"],
    "paperless_billing": ["Yes", "No"],
    "senior_citizen": ["No", "Yes"],
}

# (min, max) accepted by the UI and by batch validation
NUMERIC_RANGES = {
    "tenure_months": (0, 72),
    "monthly_charges": (18.0, 120.0),
    "total_charges": (0.0, 9000.0),
    "support_calls": (0, 10),
}

FEATURE_LABELS = {
    "tenure_months": "Tenure (months)",
    "monthly_charges": "Monthly charges (USD)",
    "total_charges": "Total charges (USD)",
    "support_calls": "Support calls (last 6 months)",
    "contract": "Contract type",
    "internet_service": "Internet service",
    "payment_method": "Payment method",
    "paperless_billing": "Paperless billing",
    "senior_citizen": "Senior citizen",
}
```

Step 2: the dataset

churn_app/data/make_dataset.py PYTHON

```python
"""Generate a synthetic telecom churn dataset with a realistic schema.

Run from the project root:  python data/make_dataset.py
The output data/churn.csv has the same column names as schema.py, so you can
replace it with a real dataset later as long as you keep those columns.
"""
from pathlib import Path

import numpy as np
import pandas as pd

RNG = np.random.default_rng(42)
N = 5000


def make_churn_data(n: int = N) -> pd.DataFrame:
    contract = RNG.choice(["Month-to-month", "One year", "Two year"], n, p=[0.55, 0.25, 0.20])
    internet = RNG.choice(["Fiber optic", "DSL", "No"], n, p=[0.45, 0.35, 0.20])
    payment = RNG.choice(
        ["Electronic check", "Mailed check", "Bank transfer", "Credit card"],
        n,
        p=[0.35, 0.2, 0.22, 0.23],
    )
    paperless = RNG.choice(["Yes", "No"], n, p=[0.6, 0.4])
    senior = RNG.choice(["No", "Yes"], n, p=[0.84, 0.16])
    tenure = np.clip(RNG.gamma(2.0, 14.0, n).round(), 0, 72).astype(int)
    base = np.where(internet == "Fiber optic", 85, np.where(internet == "DSL", 55, 25))
    monthly = np.clip(base + RNG.normal(0, 12, n), 18, 120).round(2)
    total = np.clip(monthly * tenure * RNG.uniform(0.9, 1.05, n), 0, 9000).round(2)
    calls = np.clip(RNG.poisson(1.6, n), 0, 10)

    logit = (
        -1.4
        + 1.3 * (contract == "Month-to-month")
```

churn_app/data/make_dataset.py PYTHON

```
        - 0.9 * (contract == "Two year")
        + 0.7 * (internet == "Fiber optic")
        + 0.6 * (payment == "Electronic check")
        + 0.3 * (paperless == "Yes")
        + 0.4 * (senior == "Yes")
        - 0.045 * tenure
        + 0.012 * (monthly - 60)
        + 0.35 * calls
    )
    prob = 1 / (1 + np.exp(-logit))
    churn = np.where(RNG.random(n) < prob, "Yes", "No")

    df = pd.DataFrame(
        {
            "customer_id": [f"CUST-{i:05d}" for i in range(1, n + 1)],
            "tenure_months": tenure,
            "monthly_charges": monthly,
            "total_charges": total,
            "support_calls": calls,
            "contract": contract,
            "internet_service": internet,
            "payment_method": payment,
            "paperless_billing": paperless,
            "senior_citizen": senior,
            "churn": churn,
        }
    )
    # a few realistic missing values for the imputers to handle
    missing_idx = RNG.choice(n, 40, replace=False)
    df.loc[missing_idx, "total_charges"] = np.nan
    return df


if __name__ == "__main__":
    out = Path(__file__).parent / "churn.csv"
    data = make_churn_data()
    data.to_csv(out, index=False)
    # an unlabeled file that mimics new customers arriving for scoring
    new = data.drop(columns=["churn"]).sample(200, random_state=7)
    new.to_csv(Path(__file__).parent / "new_customers.csv", index=False)
    print(f"Saved {len(data):,} rows to {out}")
    print(data["churn"].value_counts(normalize=True).round(3))
```

Step 3: preprocessing, split, training, evaluation and serialisation

churn_app/train_model.py PYTHON

```
"""Train, evaluate and save the churn pipeline.

Run from the project root:
    python data/make_dataset.py   # once, creates data/churn.csv
    python train_model.py
"""
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import (accuracy_score, classification_report, confusion_matrix,
                             f1_score, precision_score, recall_score, roc_auc_score)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from schema import CATEGORICAL_FEATURES, FEATURES, NUMERIC_FEATURES, POSITIVE_CLASS
```

churn_app/train_model.py PYTHON

```python
from schema import TARGET
from utils.model_io import save_model

ROOT = Path(__file__).parent
DATA = ROOT / "data" / "churn.csv"
MODEL_DIR = ROOT / "models"


def build_pipeline() -> Pipeline:
    numeric = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ])
    categorical = Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ])
    preprocess = ColumnTransformer([
        ("num", numeric, NUMERIC_FEATURES),
        ("cat", categorical, CATEGORICAL_FEATURES),
    ])
    model = RandomForestClassifier(
        n_estimators=300, max_depth=8, min_samples_leaf=5,
        class_weight="balanced", random_state=42, n_jobs=-1,
    )
    return Pipeline([("preprocess", preprocess), ("model", model)])


def main() -> None:
    df = pd.read_csv(DATA)
    X, y = df[FEATURES], df[TARGET]
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, stratify=y, random_state=42
    )

    pipe = build_pipeline()
    pipe.fit(X_train, y_train)  # the imputers/scaler/encoder learn ONLY from train

    classes = list(pipe.classes_)
    pos_idx = classes.index(POSITIVE_CLASS)
    proba = pipe.predict_proba(X_test)[:, pos_idx]
    pred = pipe.predict(X_test)

    metrics = {
        "accuracy": accuracy_score(y_test, pred),
        "precision": precision_score(y_test, pred, pos_label=POSITIVE_CLASS),
        "recall": recall_score(y_test, pred, pos_label=POSITIVE_CLASS),
        "f1": f1_score(y_test, pred, pos_label=POSITIVE_CLASS),
        "roc_auc": roc_auc_score((y_test == POSITIVE_CLASS).astype(int), proba),
    }
    metrics = {k: round(float(v), 4) for k, v in metrics.items()}

    names = pipe.named_steps["preprocess"].get_feature_names_out()
    importances = pipe.named_steps["model"].feature_importances_
    order = np.argsort(importances)[::-1]
    feature_importance = {str(names[i]): round(float(importances[i]), 4) for i in order}

    # store numeric medians / category modes: used as the baseline for explanations
    baseline = {c: float(X_train[c].median()) for c in NUMERIC_FEATURES}
    baseline.update({c: str(X_train[c].mode()[0]) for c in CATEGORICAL_FEATURES})

    metadata = {
        "model_name": "Customer churn classifier",
        "model_type": type(pipe.named_steps["model"]).__name__,
        "task": "binary_classification",
        "features": FEATURES,
        "target": TARGET,
        "classes": classes,
```

churn_app/train_model.py PYTHON

```
        "positive_class": POSITIVE_CLASS,
        "n_train": len(X_train),
        "n_test": len(X_test),
        "metrics": metrics,
        "confusion_matrix": confusion_matrix(y_test, pred, labels=classes).tolist(),
        "classification_report": classification_report(y_test, pred, output_dict=True),
        "feature_importance": feature_importance,
        "baseline": baseline,
        "default_threshold": 0.5,
    }
    MODEL_DIR.mkdir(exist_ok=True)
    meta = save_model(pipe, MODEL_DIR / "churn_pipeline.joblib",
                      MODEL_DIR / "churn_metadata.json", metadata)

    # keep held-out predictions so the app can draw ROC / PR curves
    pd.DataFrame({"y_true": y_test.values, "y_proba": proba.round(5)}).to_csv(
        MODEL_DIR / "churn_test_predictions.csv", index=False
    )
    print("Saved model", meta["artifact"], "sha256", meta["sha256"][:12])
    for k, v in metrics.items():
        print(f"  {k:<10} {v:.4f}")


if __name__ == "__main__":
    main()
```

python data/make_dataset.py && python train_model.py OUTPUT

```
Saved 5,000 rows to .../churn_app/data/churn.csv
churn
No     0.623
Yes    0.377
Saved model churn_pipeline.joblib sha256 58113c564301
  accuracy   0.7210
  precision  0.6070
  recall     0.7374
  f1         0.6659
  roc_auc    0.7934
```

Notice the order: the split happens **before** fitting, the imputers and scalers are part of the pipeline, and `class_weight="balanced"` trades some precision for recall, which suits a retention use case where missing a churner is costlier than an unnecessary phone call.

Step 4: validation helpers

churn_app/utils/validation.py PYTHON

```
"""Input validation shared by the single-record form and batch scoring."""
import pandas as pd

from schema import CATEGORICAL_FEATURES, CATEGORY_OPTIONS, FEATURES, NUMERIC_FEATURES
from schema import NUMERIC_RANGES


def validate_record(record: dict) -> list[str]:
    """Return a list of human-readable problems (empty list = valid)."""
    errors = []
    for col in FEATURES:
        if col not in record or record[col] is None:
            errors.append(f"'{col}' is required.")
    for col in NUMERIC_FEATURES:
        lo, hi = NUMERIC_RANGES[col]
        val = record.get(col)
        if val is not None and not lo <= val <= hi:
            errors.append(f"'{col}' must be between {lo} and {hi} (got {val}).")
```

churn_app/utils/validation.py PYTHON

```
    for col in CATEGORICAL_FEATURES:
        val = record.get(col)
        if val is not None and val not in CATEGORY_OPTIONS[col]:
            errors.append(f"'{col}' has unknown value '{val}'.")
    # business rule: total charges cannot be far below one month of charges
    if record.get("tenure_months", 0) >= 1 and record.get("total_charges") is not None:
        if record["total_charges"] < 0.5 * record.get("monthly_charges", 0):
            errors.append("Total charges look too low for the given tenure.")
    return errors


def validate_dataframe(df: pd.DataFrame) -> tuple[list[str], list[str]]:
    """Return (errors, warnings) for an uploaded batch file."""
    errors, warnings = [], []
    missing = [c for c in FEATURES if c not in df.columns]
    if missing:
        errors.append(f"Missing required columns: {', '.join(missing)}")
        return errors, warnings
    if df.empty:
        errors.append("The file has no rows.")
        return errors, warnings
    for col in NUMERIC_FEATURES:
        converted = pd.to_numeric(df[col], errors="coerce")
        bad = converted.isna() & df[col].notna()
        if bad.any():
            errors.append(f"Column '{col}' has {int(bad.sum())} non-numeric value(s).")
        lo, hi = NUMERIC_RANGES[col]
        out = ((converted < lo) | (converted > hi)).sum()
        if out:
            warnings.append(f"{int(out)} row(s) have '{col}' outside [{lo}, {hi}].")
    for col in CATEGORICAL_FEATURES:
        unknown = set(df[col].dropna().astype(str)) - set(CATEGORY_OPTIONS[col])
        if unknown:
            warnings.append(
                f"'{col}' contains unseen categories {sorted(unknown)[:5]}; the "
                "encoder will treat them as all-zero."
            )
    n_missing = int(df[FEATURES].isna().sum().sum())
    if n_missing:
        warnings.append(f"{n_missing} missing value(s) will be imputed by the pipeline.")
    extra = [c for c in df.columns if c not in FEATURES]
    if extra:
        warnings.append(f"Ignoring extra columns: {', '.join(extra[:8])}")
    return errors, warnings
```

Step 5: explaining a single prediction

churn_app/utils/explain.py PYTHON

```
"""A simple, model-agnostic local explanation.

For each feature we replace the customer's value with a 'typical' baseline value
(median for numbers, most frequent category for text) and measure how much the
predicted churn probability changes. Large positive numbers mean the customer's
actual value pushes the risk UP. This is an approximation, not a SHAP value.
"""
import pandas as pd


def perturbation_effects(model, record: dict, baseline: dict, features: list,
                         pos_idx: int) -> pd.DataFrame:
    row = pd.DataFrame([record])[features]
    p_actual = model.predict_proba(row)[0, pos_idx]
    rows = []
    for col in features:
        if record[col] == baseline[col]:
            continue
```

churn_app/utils/explain.py PYTHON

```
        modified = row.copy()
        modified[col] = baseline[col]
        p_base = model.predict_proba(modified)[0, pos_idx]
        rows.append({
            "feature": col,
            "customer_value": record[col],
            "typical_value": baseline[col],
            "effect": round(float(p_actual - p_base), 4),
        })
    effects = pd.DataFrame(rows, columns=["feature", "customer_value",
                                          "typical_value", "effect"])
    return effects.sort_values("effect", key=abs, ascending=False)
```

NOTE

Global feature importance tells you what matters **on average**. A local explanation tells a user why **this** customer is high risk. The perturbation method above is simple and model-agnostic. For rigorous attributions use the `shap` library (`shap.TreeExplainer(model)` for tree models) and display its plots with `st.pyplot`.

Step 6: chart helpers

churn_app/utils/charts.py PYTHON

```
"""Plotly figures used by the churn app."""
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from sklearn.metrics import precision_recall_curve, roc_curve


def confusion_matrix_fig(cm: list, labels: list) -> go.Figure:
    fig = px.imshow(cm, x=[f"Pred {c}" for c in labels], y=[f"True {c}" for c in labels],
                    text_auto=True, color_continuous_scale="Blues", aspect="auto")
    fig.update_layout(height=340, margin=dict(l=10, r=10, t=30, b=10),
                      coloraxis_showscale=False)
    return fig


def roc_fig(y_true: pd.Series, y_score: pd.Series, auc: float) -> go.Figure:
    fpr, tpr, _ = roc_curve(y_true, y_score)
    fig = go.Figure()
    fig.add_scatter(x=fpr, y=tpr, mode="lines", name=f"Model (AUC={auc:.3f})")
    fig.add_scatter(x=[0, 1], y=[0, 1], mode="lines", name="Random",
                    line=dict(dash="dash", color="grey"))
    fig.update_layout(xaxis_title="False positive rate", yaxis_title="True positive rate",
                      height=340, margin=dict(l=10, r=10, t=30, b=10))
    return fig


def pr_fig(y_true: pd.Series, y_score: pd.Series) -> go.Figure:
    precision, recall, _ = precision_recall_curve(y_true, y_score)
    fig = go.Figure(go.Scatter(x=recall, y=precision, mode="lines", name="PR curve"))
    fig.add_hline(y=float(y_true.mean()), line_dash="dash", line_color="grey",
                  annotation_text="baseline (positive rate)")
    fig.update_layout(xaxis_title="Recall", yaxis_title="Precision", height=340,
                      margin=dict(l=10, r=10, t=30, b=10))
    return fig


def importance_fig(importance: dict, top_n: int = 10) -> go.Figure:
    s = pd.Series(importance).head(top_n).sort_values()
    s.index = [i.replace("num__", "").replace("cat__", "") for i in s.index]
    fig = px.bar(s, orientation="h", labels={"value": "Importance", "index": ""})
    fig.update_layout(showlegend=False, height=360, margin=dict(l=10, r=10, t=30, b=10))
    return fig
```

churn_app/utils/charts.py PYTHON

```python
def effects_fig(effects: pd.DataFrame) -> go.Figure:
    df = effects.head(8).iloc[::-1]
    colors = ["#d62728" if v > 0 else "#2ca02c" for v in df["effect"]]
    fig = go.Figure(go.Bar(x=df["effect"], y=df["feature"], orientation="h",
                           marker_color=colors))
    fig.update_layout(xaxis_title="Change in churn probability vs. typical value",
                      height=320, margin=dict(l=10, r=10, t=30, b=10))
    return fig
```

Step 7: the Streamlit app

churn_app/app.py PYTHON

```python
"""Customer Churn Prediction app.  Run:  streamlit run app.py"""
from datetime import datetime
from pathlib import Path

import pandas as pd
import streamlit as st

from schema import CATEGORICAL_FEATURES, CATEGORY_OPTIONS, FEATURE_LABELS, FEATURES
from schema import NUMERIC_RANGES, POSITIVE_CLASS
from utils.charts import (confusion_matrix_fig, effects_fig, importance_fig, pr_fig,
                          roc_fig)
from utils.explain import perturbation_effects
from utils.model_io import ModelLoadError, load_trusted_model
from utils.validation import validate_dataframe, validate_record

ROOT = Path(__file__).parent
MODEL_PATH = ROOT / "models" / "churn_pipeline.joblib"
META_PATH = ROOT / "models" / "churn_metadata.json"
TEST_PRED_PATH = ROOT / "models" / "churn_test_predictions.csv"

st.set_page_config(page_title="Churn Predictor", page_icon=":material/insights:",
                   layout="wide")


@st.cache_resource(show_spinner="Loading model...")
def get_model():
    """Loaded once per server process and shared by every user session."""
    return load_trusted_model(MODEL_PATH, META_PATH)


@st.cache_data
def get_test_predictions() -> pd.DataFrame:
    return pd.read_csv(TEST_PRED_PATH)


try:
    model, meta, load_warnings = get_model()
except ModelLoadError as exc:
    st.error(f"The model could not be loaded. {exc}")
    st.stop()

POS_IDX = meta["classes"].index(POSITIVE_CLASS)
if "history" not in st.session_state:
    st.session_state.history = []

# ---------------------------------------------------------------- sidebar
with st.sidebar:
    st.header("Model")
    st.caption(f"{meta['model_type']} | trained {meta['trained_at'][:10]}")
    threshold = st.slider(
        "Decision threshold", 0.05, 0.95, float(meta["default_threshold"]), 0.05,
        help="Customers with churn probability at or above this value are flagged.",
```

churn_app/app.py PYTHON

```python
    )
    st.metric("Test ROC-AUC", f"{meta['metrics']['roc_auc']:.3f}")
    for w in load_warnings:
        st.warning(w)

st.title("Customer Churn Prediction")
st.caption("Estimate the probability that a customer cancels their subscription.")

tab_single, tab_batch, tab_perf, tab_about = st.tabs(
    ["Single prediction", "Batch scoring", "Model performance", "About the model"]
)

# ---------------------------------------------------------------- single
with tab_single:
    with st.form("customer_form"):
        c1, c2 = st.columns(2)
        with c1:
            tenure = st.number_input(FEATURE_LABELS["tenure_months"],
                                     *NUMERIC_RANGES["tenure_months"], value=12, step=1)
            monthly = st.number_input(FEATURE_LABELS["monthly_charges"],
                                      *NUMERIC_RANGES["monthly_charges"], value=70.0,
                                      step=0.5)
            total = st.number_input(FEATURE_LABELS["total_charges"],
                                    *NUMERIC_RANGES["total_charges"], value=840.0,
                                    step=10.0)
            calls = st.slider(FEATURE_LABELS["support_calls"],
                              *NUMERIC_RANGES["support_calls"], value=1)
        with c2:
            cats = {c: st.selectbox(FEATURE_LABELS[c], CATEGORY_OPTIONS[c])
                    for c in CATEGORICAL_FEATURES}
        submitted = st.form_submit_button("Predict churn", type="primary")

    if submitted:
        record = {"tenure_months": int(tenure), "monthly_charges": float(monthly),
                  "total_charges": float(total), "support_calls": int(calls), **cats}
        problems = validate_record(record)
        if problems:
            for p in problems:
                st.error(p)
        else:
            X = pd.DataFrame([record])[FEATURES]
            proba = float(model.predict_proba(X)[0, POS_IDX])
            label = "Likely to churn" if proba >= threshold else "Likely to stay"
            m1, m2, m3 = st.columns(3)
            m1.metric("Churn probability", f"{proba:.1%}")
            m2.metric("Prediction", label)
            m3.metric("Threshold", f"{threshold:.2f}")
            st.progress(proba, text=f"Risk score {proba:.2f}")
            if proba >= threshold:
                st.warning("High risk: consider a retention offer or proactive call.")
            else:
                st.success("Low risk at the current threshold.")

            effects = perturbation_effects(model, record, meta["baseline"], FEATURES,
                                           POS_IDX)
            with st.expander("Why this prediction? (approximate explanation)"):
                if effects.empty:
                    st.info("This customer matches the typical profile on every feature.")
                else:
                    st.plotly_chart(effects_fig(effects), width="stretch")
                    st.caption("Bars show how the probability changes when one feature "
                               "is reset to its typical training value. Red = raises "
                               "risk, green = lowers risk.")
                    st.dataframe(effects, hide_index=True, width="stretch")

            st.session_state.history.append({
                "time": datetime.now().strftime("%H:%M:%S"), **record,
                "churn_probability": round(proba, 4), "prediction": label,
```

churn_app/app.py PYTHON

```
            })

    if st.session_state.history:
        st.subheader("Predictions in this session")
        hist = pd.DataFrame(st.session_state.history)
        st.dataframe(hist, hide_index=True, width="stretch")
        st.download_button("Download history (CSV)", hist.to_csv(index=False),
                           "prediction_history.csv", "text/csv")

# ---------------------------------------------------------------- batch
with tab_batch:
    st.write("Upload a CSV with these columns:", ", ".join(f"`{c}`" for c in FEATURES))
    upload = st.file_uploader("Customer file", type=["csv"], key="batch_file")
    if upload is not None:
        try:
            df = pd.read_csv(upload)
            errors, warns = validate_dataframe(df)
        except Exception as exc:  # malformed CSV, wrong encoding, etc.
            df, errors, warns = None, [f"Could not read the CSV file: {exc}"], []
        for w in warns:
            st.warning(w)
        if errors:
            for e in errors:
                st.error(e)
        else:
            X = df[FEATURES].copy()
            for col in ("tenure_months", "monthly_charges", "total_charges",
                        "support_calls"):
                X[col] = pd.to_numeric(X[col], errors="coerce")
            with st.spinner(f"Scoring {len(X):,} customers..."):
                probs = model.predict_proba(X)[:, POS_IDX]
            result = df.copy()
            result["churn_probability"] = probs.round(4)
            result["prediction"] = (probs >= threshold).astype(int)
            result["prediction"] = result["prediction"].map({1: "Yes", 0: "No"})
            k1, k2, k3 = st.columns(3)
            k1.metric("Customers scored", f"{len(result):,}")
            k2.metric("Flagged as churn", int((result["prediction"] == "Yes").sum()))
            k3.metric("Average probability", f"{probs.mean():.1%}")
            st.dataframe(
                result.sort_values("churn_probability", ascending=False),
                hide_index=True, width="stretch",
                column_config={"churn_probability": st.column_config.ProgressColumn(
                    "Churn probability", min_value=0.0, max_value=1.0, format="%.2f")},
            )
            st.download_button("Download scored file", result.to_csv(index=False),
                               "churn_predictions.csv", "text/csv", type="primary")

# ---------------------------------------------------------------- performance
with tab_perf:
    st.caption(f"Metrics on a held-out test set of {meta['n_test']:,} customers.")
    cols = st.columns(5)
    for col, (name, value) in zip(cols, meta["metrics"].items()):
        col.metric(name.replace("_", " ").upper(), f"{value:.3f}")
    left, right = st.columns(2)
    with left:
        st.subheader("Confusion matrix")
        st.plotly_chart(confusion_matrix_fig(meta["confusion_matrix"], meta["classes"]),
                        width="stretch")
    with right:
        st.subheader("Feature importance")
        st.plotly_chart(importance_fig(meta["feature_importance"]), width="stretch")
    preds = get_test_predictions()
    y_bin = (preds["y_true"] == POSITIVE_CLASS).astype(int)
    left, right = st.columns(2)
    with left:
        st.subheader("ROC curve")
        st.plotly_chart(roc_fig(y_bin, preds["y_proba"], meta["metrics"]["roc_auc"]),
```

churn_app/app.py PYTHON

```
                        width="stretch")
    with right:
        st.subheader("Precision-recall curve")
        st.plotly_chart(pr_fig(y_bin, preds["y_proba"]), width="stretch")
    st.subheader("Classification report")
    report = pd.DataFrame(meta["classification_report"]).T.round(3)
    st.dataframe(report, width="stretch")

# ------------------------------------------------------------------ about
with tab_about:
    st.markdown(
        f"""
**Model:** {meta['model_type']} inside a scikit-learn `Pipeline`
(median/most-frequent imputation, scaling, one-hot encoding).

**Training rows:** {meta['n_train']:,} | **scikit-learn:** {meta['sklearn_version']}
| **Fingerprint:** `{meta['sha256'][:16]}...`

**Intended use:** prioritise customers for retention outreach. A prediction is a
risk estimate, not a fact about a customer; review flagged accounts before acting.
"""
    )
    st.json({"features": meta["features"], "classes": meta["classes"]}, expanded=False)
```

Section of app.py	Workflow step it implements
`get_model()` with `@st.cache_resource`	6. Load once, verify fingerprint
`st.form("customer_form")` with bounded widgets	7. Collect input
`validate_record(record)`	8. Validate (ranges, categories, business rule)
`pd.DataFrame([record])[FEATURES]` -> pipeline	9-10. Same preprocessing, predict
metrics, progress bar, warning/success, expander	11. Display + explain
`try/except ModelLoadError`, batch error list	12. Handle errors
`st.session_state.history` + download	13. Session prediction history

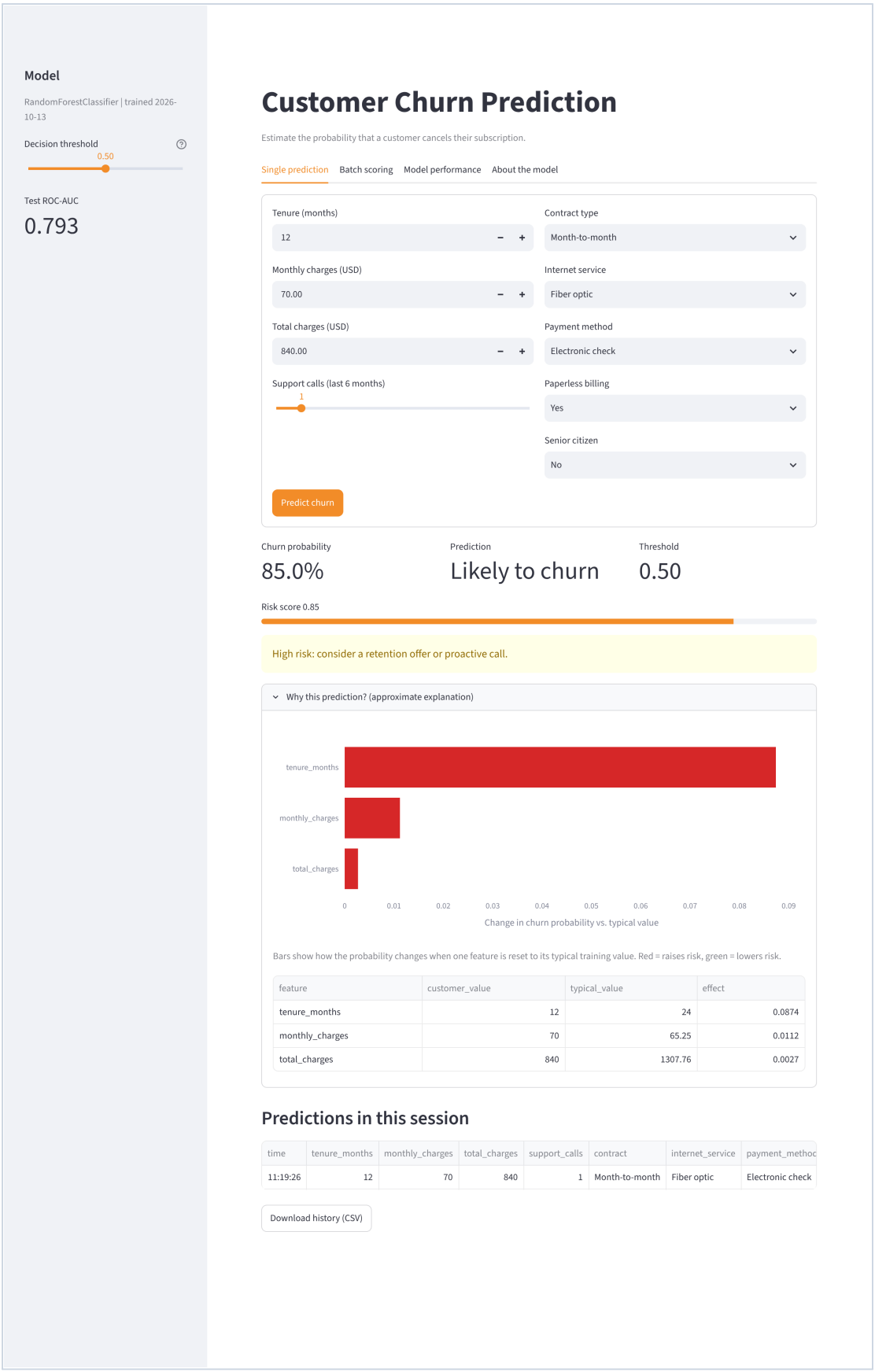


Figure 6.3: Single prediction with probability, decision, explanation and session history



	precision	recall	f1-score	support
No	0.817	0.711	0.761	623
Yes	0.607	0.737	0.666	377
accuracy	0.721	0.721	0.721	0.721
macro avg	0.712	0.724	0.713	1000
weighted avg	0.738	0.721	0.725	1000

Figure 6.4: Model performance tab: metrics, confusion matrix, feature importance, ROC and precision-recall curves, classification report

churn_app/tests/test_app.py PYTHON

```python
"""Smoke tests for the churn app. Run from the project root:  pytest -q

AppTest.from_file() resolves paths relative to THIS test file, hence "../app.py".
"""
from streamlit.testing.v1 import AppTest


def test_app_loads_without_errors():
    at = AppTest.from_file("../app.py", default_timeout=60).run()
    assert not at.exception
    assert at.title[0].value == "Customer Churn Prediction"


def test_single_prediction_shows_probability():
    at = AppTest.from_file("../app.py", default_timeout=60).run()
    at.number_input[0].set_value(2)           # short tenure
    at.number_input[2].set_value(150.0)       # total charges
    at.button[0].click().run()                # form submit button
    assert not at.exception
    labels = [m.label for m in at.metric]
    assert "Churn probability" in labels
    assert len(at.session_state.history) == 1
```

TRY IT

Lower the decision threshold in the sidebar to 0.3 and score `data/new_customers.csv` on the Batch tab. How many more customers are flagged? What does that do to the retention team's workload?

6.5 Regression project: House Price Prediction

Estimate apartment prices in Pune from size, rooms, age, floor, locality, furnishing and parking. New ideas in this project: a `remainder="passthrough"` pipeline, **training-range checks** that warn when the model would extrapolate, and **currency formatting** in Indian Rupees.

house_app/schema.py PYTHON

```python
"""Input schema for the house price model (shared by training and the app)."""

TARGET = "price_inr"
NUMERIC_FEATURES = ["area_sqft", "bedrooms", "bathrooms", "age_years", "floor"]
CATEGORICAL_FEATURES = ["locality", "furnishing", "parking"]
FEATURES = NUMERIC_FEATURES + CATEGORICAL_FEATURES

CATEGORY_OPTIONS = {
    "locality": ["Baner", "Hadapsar", "Hinjewadi", "Kharadi", "Kothrud", "Viman Nagar",
                 "Wakad"],
    "furnishing": ["Unfurnished", "Semi-furnished", "Fully furnished"],
    "parking": ["Yes", "No"],
}

# hard limits: values outside these are rejected by the UI
HARD_LIMITS = {
    "area_sqft": (250, 6000),
    "bedrooms": (1, 6),
    "bathrooms": (1, 6),
    "age_years": (0, 60),
    "floor": (0, 40),
}
```

house_app/data/make_dataset.py PYTHON

```python
"""Generate a synthetic Pune apartment-price dataset.  Run: python data/make_dataset.py"""
from pathlib import Path

import numpy as np
import pandas as pd

RNG = np.random.default_rng(11)
LOCALITY_RATE = {  # approximate price per sq ft in INR (synthetic)
    "Baner": 9500, "Hadapsar": 6800, "Hinjewadi": 7200, "Kharadi": 8600,
    "Kothrud": 10500, "Viman Nagar": 10000, "Wakad": 7400,
}


def make_house_data(n: int = 3000) -> pd.DataFrame:
    locality = RNG.choice(list(LOCALITY_RATE), n)
    bedrooms = RNG.choice([1, 2, 3, 4], n, p=[0.2, 0.45, 0.28, 0.07])
    area = np.clip(RNG.normal(380 + 420 * bedrooms, 120), 300, 4000).round()
    bathrooms = np.clip(bedrooms + RNG.choice([-1, 0, 0, 1], n), 1, 5)
    age = np.clip(RNG.exponential(8, n), 0, 45).round()
    floor = RNG.integers(0, 25, n)
    furnishing = RNG.choice(["Unfurnished", "Semi-furnished", "Fully furnished"], n,
                            p=[0.35, 0.45, 0.2])
    parking = RNG.choice(["Yes", "No"], n, p=[0.75, 0.25])
    rate = np.array([LOCALITY_RATE[l] for l in locality])
    furn = np.select([furnishing == "Semi-furnished", furnishing == "Fully furnished"],
                     [1.05, 1.12], 1.0)
    price = (area * rate * furn * (1 - 0.006 * age) * (1 + 0.004 * floor)
             * np.where(parking == "Yes", 1.04, 1.0) * RNG.lognormal(0, 0.08, n))
    return pd.DataFrame({
        "area_sqft": area.astype(int), "bedrooms": bedrooms, "bathrooms": bathrooms,
        "age_years": age.astype(int), "floor": floor, "locality": locality,
        "furnishing": furnishing, "parking": parking, "price_inr": price.round(-3),
    })


if __name__ == "__main__":
    out = Path(__file__).parent / "houses.csv"
    df = make_house_data()
    df.to_csv(out, index=False)
    print(f"Saved {len(df):,} rows to {out}")
    print(df.describe().round(1).T[["min", "50%", "max"]])
```

house_app/train_model.py PYTHON

```python
"""Train and save the house price regression pipeline.  Run: python train_model.py"""
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

from schema import CATEGORICAL_FEATURES, FEATURES, NUMERIC_FEATURES, TARGET
from utils.model_io import save_model

ROOT = Path(__file__).parent


def build_pipeline() -> Pipeline:
    preprocess = ColumnTransformer(
        [("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False),
          CATEGORICAL_FEATURES)],
        remainder="passthrough",  # numeric columns go straight to the model
```

house_app/train_model.py PYTHON

```python
    )
    model = HistGradientBoostingRegressor(max_iter=400, learning_rate=0.05,
                                          max_leaf_nodes=31, random_state=42)
    return Pipeline([("preprocess", preprocess), ("model", model)])


def main() -> None:
    df = pd.read_csv(ROOT / "data" / "houses.csv")
    X, y = df[FEATURES], df[TARGET]
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                        random_state=42)
    pipe = build_pipeline().fit(X_train, y_train)
    pred = pipe.predict(X_test)
    metrics = {
        "mae": float(mean_absolute_error(y_test, pred)),
        "rmse": float(np.sqrt(mean_squared_error(y_test, pred))),
        "r2": float(r2_score(y_test, pred)),
        "mape": float(np.mean(np.abs((y_test - pred) / y_test))),
    }
    # the range of values the model has actually seen: used for range warnings
    training_ranges = {
        c: {"p01": float(X_train[c].quantile(0.01)), "p99": float(X_train[c].quantile(0.99)),
            "median": float(X_train[c].median())}
        for c in NUMERIC_FEATURES
    }
    meta = save_model(pipe, ROOT / "models" / "house_pipeline.joblib",
                      ROOT / "models" / "house_metadata.json", {
                          "model_name": "Pune apartment price regressor",
                          "model_type": "HistGradientBoostingRegressor",
                          "task": "regression", "features": FEATURES, "target": TARGET,
                          "n_train": len(X_train), "n_test": len(X_test),
                          "metrics": {k: round(v, 4) for k, v in metrics.items()},
                          "training_ranges": training_ranges,
                      })
    pd.DataFrame({"actual": y_test.values, "predicted": pred.round(-3)}).to_csv(
        ROOT / "models" / "house_test_predictions.csv", index=False)
    print("Saved", meta["artifact"], {k: round(v, 3) for k, v in metrics.items()})


if __name__ == "__main__":
    main()
```

house_app/utils/formatting.py PYTHON

```python
"""Currency helpers. Indian numbering groups digits as 12,34,56,789."""


def format_inr(amount: float) -> str:
    """Format a number as Indian Rupees with lakh/crore digit grouping."""
    negative = amount < 0
    rupees = f"{abs(round(amount)):d}"
    if len(rupees) > 3:
        head, tail = rupees[:-3], rupees[-3:]
        groups = []
        while len(head) > 2:
            groups.insert(0, head[-2:])
            head = head[:-2]
        if head:
            groups.insert(0, head)
        rupees = ",".join(groups) + "," + tail
    return ("-" if negative else "") + "₹" + rupees


def format_lakh(amount: float) -> str:
    """Short form used on KPI cards, e.g. 'Rs 87.4 lakh' or 'Rs 1.32 crore'."""
    if abs(amount) >= 1e7:
        return f"₹{amount / 1e7:.2f} crore"
    return f"₹{amount / 1e5:.1f} lakh"
```

house_app/utils/formatting.py PYTHON

```python
def format_usd(amount: float) -> str:
    return f"${amount:,.0f}"
```

Python's `f"{x:,.0f}"` groups digits in thousands (8,740,000). Indian readers expect lakh/crore grouping (87,40,000) and short forms such as '87.4 lakh'. `format_inr` implements the grouping without depending on operating-system locales, which are often missing on cloud servers.

house_app/app.py PYTHON

```python
"""House Price Prediction app.  Run: streamlit run app.py"""
from pathlib import Path

import pandas as pd
import plotly.express as px
import streamlit as st

from schema import CATEGORY_OPTIONS, FEATURES, HARD_LIMITS, NUMERIC_FEATURES
from utils.formatting import format_inr, format_lakh
from utils.model_io import ModelLoadError, load_trusted_model

ROOT = Path(__file__).parent
st.set_page_config(page_title="House Price Estimator", page_icon=":material/home:",
                   layout="wide")


@st.cache_resource(show_spinner="Loading model...")
def get_model():
    return load_trusted_model(ROOT / "models" / "house_pipeline.joblib",
                              ROOT / "models" / "house_metadata.json")


@st.cache_data
def get_test_predictions() -> pd.DataFrame:
    df = pd.read_csv(ROOT / "models" / "house_test_predictions.csv")
    df["residual"] = df["actual"] - df["predicted"]
    return df


def range_warnings(record: dict, ranges: dict) -> list[str]:
    """Flag inputs the model rarely saw during training (extrapolation risk)."""
    msgs = []
    for col in NUMERIC_FEATURES:
        lo, hi = ranges[col]["p01"], ranges[col]["p99"]
        if not lo <= record[col] <= hi:
            msgs.append(f"{col} = {record[col]} is outside the typical training range "
                        f"[{lo:g}, {hi:g}]; the estimate is less reliable.")
    return msgs


try:
    model, meta, load_warnings = get_model()
except ModelLoadError as exc:
    st.error(str(exc))
    st.stop()

st.title("Pune Apartment Price Estimator")
st.caption("Estimates are based on a synthetic training dataset and are for "
           "learning purposes only.")

form_col, result_col = st.columns([1.1, 1], gap="large")
with form_col:
    with st.form("house_form", border=True):
        st.subheader("Property details")
        a, b = st.columns(2)
        area = a.number_input("Carpet area (sq ft)", *HARD_LIMITS["area_sqft"], value=1050,
```

house_app/app.py PYTHON

```
                                step=25)
        locality = b.selectbox("Locality", CATEGORY_OPTIONS["locality"], index=0)
        bedrooms = a.number_input("Bedrooms (BHK)", *HARD_LIMITS["bedrooms"], value=2)
        bathrooms = b.number_input("Bathrooms", *HARD_LIMITS["bathrooms"], value=2)
        age = a.slider("Building age (years)", *HARD_LIMITS["age_years"], value=5)
        floor = b.number_input("Floor", *HARD_LIMITS["floor"], value=6)
        furnishing = a.radio("Furnishing", CATEGORY_OPTIONS["furnishing"], index=1)
        parking = b.toggle("Covered parking", value=True)
        submitted = st.form_submit_button("Estimate price", type="primary",
                                          width="stretch")

with result_col:
    if not submitted:
        st.info("Fill in the property details and click **Estimate price**.")
    else:
        record = {"area_sqft": area, "bedrooms": bedrooms, "bathrooms": bathrooms,
                  "age_years": age, "floor": floor, "locality": locality,
                  "furnishing": furnishing, "parking": "Yes" if parking else "No"}
        if bathrooms > bedrooms + 1:
            st.error("Bathrooms cannot exceed bedrooms + 1 for this model.")
        else:
            X = pd.DataFrame([record])[FEATURES]
            price = float(model.predict(X)[0])
            mae = meta["metrics"]["mae"]
            st.subheader("Estimated price")
            st.metric("Point estimate", format_lakh(price), border=True)
            st.write(f"**{format_inr(price)}**  ({format_inr(price / area)} per sq ft)")
            st.caption(f"Typical error on test data (MAE): +/- {format_lakh(mae)}. "
                       f"A rough range is {format_lakh(price - mae)} to "
                       f"{format_lakh(price + mae)}; this is not a statistical "
                       "prediction interval.")
            for msg in range_warnings(record, meta["training_ranges"]):
                st.warning(msg)

st.divider()
st.subheader("Model performance on held-out data")
m = meta["metrics"]
k1, k2, k3, k4 = st.columns(4)
k1.metric("R-squared", f"{m['r2']:.3f}")
k2.metric("MAE", format_lakh(m["mae"]))
k3.metric("RMSE", format_lakh(m["rmse"]))
k4.metric("MAPE", f"{m['mape']:.1%}")

preds = get_test_predictions()
left, right = st.columns(2)
with left:
    fig = px.scatter(preds, x="actual", y="predicted", opacity=0.5,
                     labels={"actual": "Actual price (INR)",
                             "predicted": "Predicted price (INR)"},
                     title="Actual vs predicted")
    lo, hi = preds[["actual", "predicted"]].min().min(), preds["actual"].max()
    fig.add_shape(type="line", x0=lo, y0=lo, x1=hi, y1=hi, line=dict(dash="dash"))
    st.plotly_chart(fig, width="stretch")
with right:
    fig = px.scatter(preds, x="predicted", y="residual", opacity=0.5,
                     title="Residuals (actual - predicted)",
                     labels={"predicted": "Predicted price", "residual": "Residual"})
    fig.add_hline(y=0, line_dash="dash")
    st.plotly_chart(fig, width="stretch")
for w in load_warnings:
    st.warning(w)
```

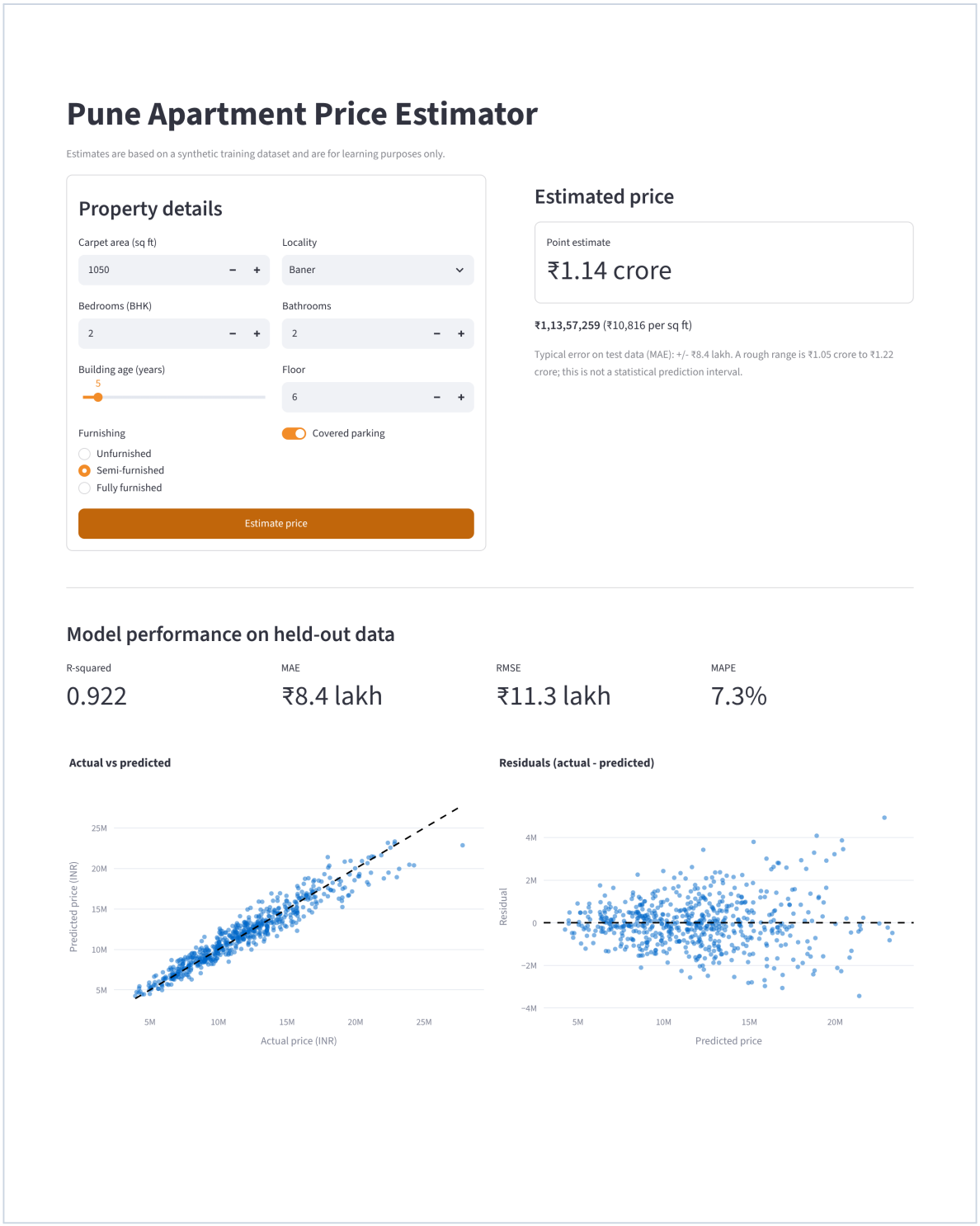


Figure 6.5: Price estimate in lakh and full rupees, typical error, and actual-vs-predicted and residual plots

> **WARNING**
>
> The 'rough range' uses the test-set MAE and is labelled as such. It is **not** a statistical prediction interval. For genuine intervals train quantile models (`HistGradientBoostingRegressor(loss="quantile", quantile=0.1)` and `0.9`) or use conformal prediction (e.g. the MAPIE library).

6.6 Clustering project: Customer Segmentation

KMeans is distance based, so features must be scaled. Saving the scaler and KMeans **together** in one Pipeline guarantees that uploaded customers are scaled with the training means and standard deviations, then assigned to the nearest saved centroid with `pipe.predict()`. Re-fitting KMeans in the app would renumber the clusters and break the segment names.

segment_app/train_model.py — PYTHON

```python
"""Create mall-customer data, fit scaler + KMeans as ONE pipeline, save it.

Run from the project folder:  python train_model.py
"""
import json
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

ROOT = Path(__file__).parent
FEATURES = ["age", "annual_income_k", "spending_score"]
RNG = np.random.default_rng(3)


def make_customers(n_per_group: int = 120) -> pd.DataFrame:
    centers = [(45, 26, 20), (25, 25, 80), (33, 55, 50), (40, 88, 17), (31, 86, 82)]
    rows = []
    for age, inc, score in centers:
        rows.append(np.column_stack([
            RNG.normal(age, 6, n_per_group), RNG.normal(inc, 7, n_per_group),
            RNG.normal(score, 8, n_per_group)]))
    X = np.clip(np.vstack(rows), [18, 10, 1], [70, 140, 100]).round()
    df = pd.DataFrame(X, columns=FEATURES).astype(int)
    df.insert(0, "customer_id", [f"M{i:04d}" for i in range(1, len(df) + 1)])
    df.insert(1, "gender", RNG.choice(["Female", "Male"], len(df)))
    return df.sample(frac=1, random_state=1).reset_index(drop=True)


def name_segment(income: float, score: float) -> str:
    inc = "High income" if income > 70 else ("Low income" if income < 40 else "Mid income")
    spend = "high spend" if score > 65 else ("low spend" if score < 35 else "mid spend")
    return f"{inc}, {spend}"


def main() -> None:
    (ROOT / "data").mkdir(exist_ok=True)
    df = make_customers()
    df.to_csv(ROOT / "data" / "mall_customers.csv", index=False)

    pipe = Pipeline([("scale", StandardScaler()),
                     ("kmeans", KMeans(n_clusters=5, n_init=10, random_state=42))])
    labels = pipe.fit_predict(df[FEATURES])
    sil = silhouette_score(pipe.named_steps["scale"].transform(df[FEATURES]), labels)

    # profiles are computed in ORIGINAL units so business users can read them
    profile = df.assign(cluster=labels).groupby("cluster")[FEATURES].mean().round(1)
    names = {int(k): name_segment(r.annual_income_k, r.spending_score)
             for k, r in profile.iterrows()}
    (ROOT / "models").mkdir(exist_ok=True)
    joblib.dump(pipe, ROOT / "models" / "segment_pipeline.joblib")
    meta = {"features": FEATURES, "n_clusters": 5, "silhouette": round(float(sil), 3),
            "segment_names": names, "profile": profile.reset_index().to_dict("records")}
```

segment_app/train_model.py PYTHON

```
    (ROOT / "models" / "segment_metadata.json").write_text(json.dumps(meta, indent=2))
    print(f"Saved pipeline. Silhouette = {sil:.3f}")
    print(profile.assign(name=pd.Series(names)))


if __name__ == "__main__":
    main()
```

segment_app/app.py PYTHON

```
"""Customer Segmentation dashboard.  Run: streamlit run app.py"""
import json
from pathlib import Path

import joblib
import pandas as pd
import plotly.express as px
import streamlit as st

ROOT = Path(__file__).parent
st.set_page_config(page_title="Customer Segments", page_icon=":material/groups:",
                   layout="wide")


@st.cache_resource
def load_artifacts():
    pipe = joblib.load(ROOT / "models" / "segment_pipeline.joblib")
    meta = json.loads((ROOT / "models" / "segment_metadata.json").read_text())
    return pipe, meta


@st.cache_data
def read_table(file) -> pd.DataFrame:
    if file.name.lower().endswith((".xlsx", ".xls")):
        return pd.read_excel(file)
    return pd.read_csv(file)


pipe, meta = load_artifacts()
FEATURES = meta["features"]
names = {int(k): v for k, v in meta["segment_names"].items()}

st.title("Customer Segmentation Dashboard")
st.caption(f"KMeans with {meta['n_clusters']} clusters | silhouette "
           f"{meta['silhouette']} on training data")

with st.sidebar:
    st.header("1. Data")
    source = st.radio("Data source", ["Use sample data", "Upload a file"])
    upload = None
    if source == "Upload a file":
        upload = st.file_uploader("CSV or Excel", type=["csv", "xlsx"])

if source == "Use sample data":
    df = pd.read_csv(ROOT / "data" / "mall_customers.csv")
elif upload is not None:
    try:
        df = read_table(upload)
    except Exception as exc:
        st.error(f"Could not read file: {exc}")
        st.stop()
else:
    st.info("Upload a CSV/XLSX file in the sidebar to continue.")
    st.stop()

# --- map the user's columns to the features the model was trained on
with st.sidebar:
    st.header("2. Map columns")
```

segment_app/app.py PYTHON

```
    numeric_cols = df.select_dtypes("number").columns.tolist()
    mapping = {}
    for feat in FEATURES:
        default = numeric_cols.index(feat) if feat in numeric_cols else 0
        mapping[feat] = st.selectbox(f"Column for '{feat}'", numeric_cols, index=default,
                                     key=f"map_{feat}")
if len(set(mapping.values())) < len(FEATURES):
    st.error("Each model feature must be mapped to a different column.")
    st.stop()

X = df[[mapping[f] for f in FEATURES]].copy()
X.columns = FEATURES
n_missing = int(X.isna().any(axis=1).sum())
if n_missing:
    st.warning(f"Dropping {n_missing} row(s) with missing values in model features.")
valid = X.dropna()
result = df.loc[valid.index].copy()
result["cluster"] = pipe.predict(valid)          # scaler + kmeans applied together
result["segment"] = result["cluster"].map(names)

k1, k2, k3 = st.columns(3)
k1.metric("Customers", f"{len(result):,}")
k2.metric("Segments found", result["cluster"].nunique())
k3.metric("Largest segment", result["segment"].value_counts().idxmax())

tab_viz, tab_profile, tab_data = st.tabs(["Visualise", "Segment profiles", "Data"])
with tab_viz:
    c1, c2 = st.columns(2)
    x_axis = c1.selectbox("X axis", FEATURES, index=1)
    y_axis = c2.selectbox("Y axis", FEATURES, index=2)
    plot_df = valid.assign(segment=result["segment"])
    fig = px.scatter(plot_df, x=x_axis, y=y_axis, color="segment", opacity=0.75,
                     height=480)
    st.plotly_chart(fig, width="stretch")

with tab_profile:
    prof = (valid.assign(segment=result["segment"]).groupby("segment")[FEATURES]
            .mean().round(1))
    prof["customers"] = result["segment"].value_counts()
    st.dataframe(prof.sort_values("customers", ascending=False), width="stretch")
    st.bar_chart(prof["customers"], horizontal=True)

with tab_data:
    st.dataframe(result, hide_index=True, width="stretch")
    st.download_button("Download segmented customers (CSV)",
                       result.to_csv(index=False), "customer_segments.csv",
                       "text/csv", type="primary")
```

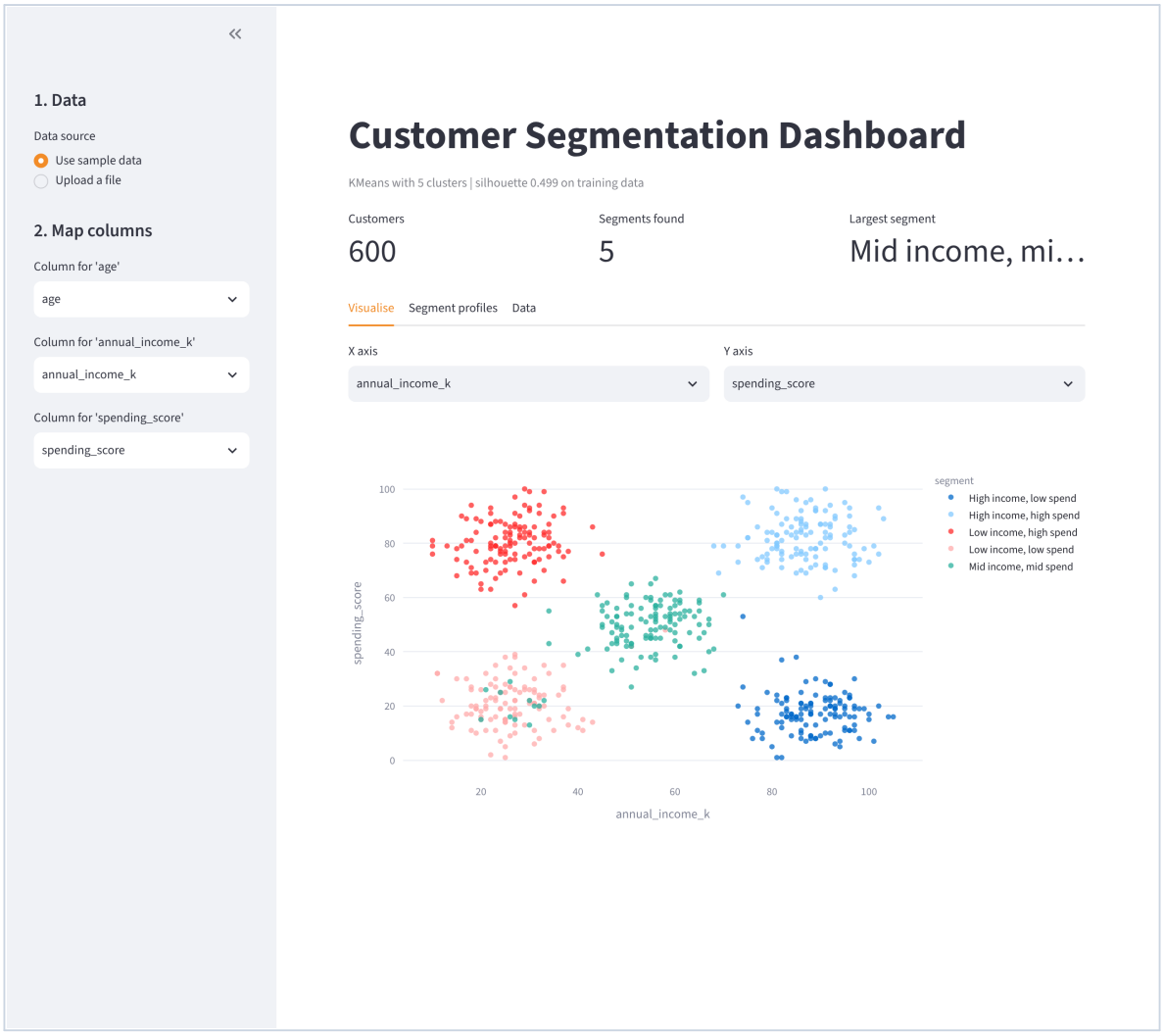


Figure 6.6: Segmentation dashboard: column mapping in the sidebar, KPIs and an interactive cluster scatter plot

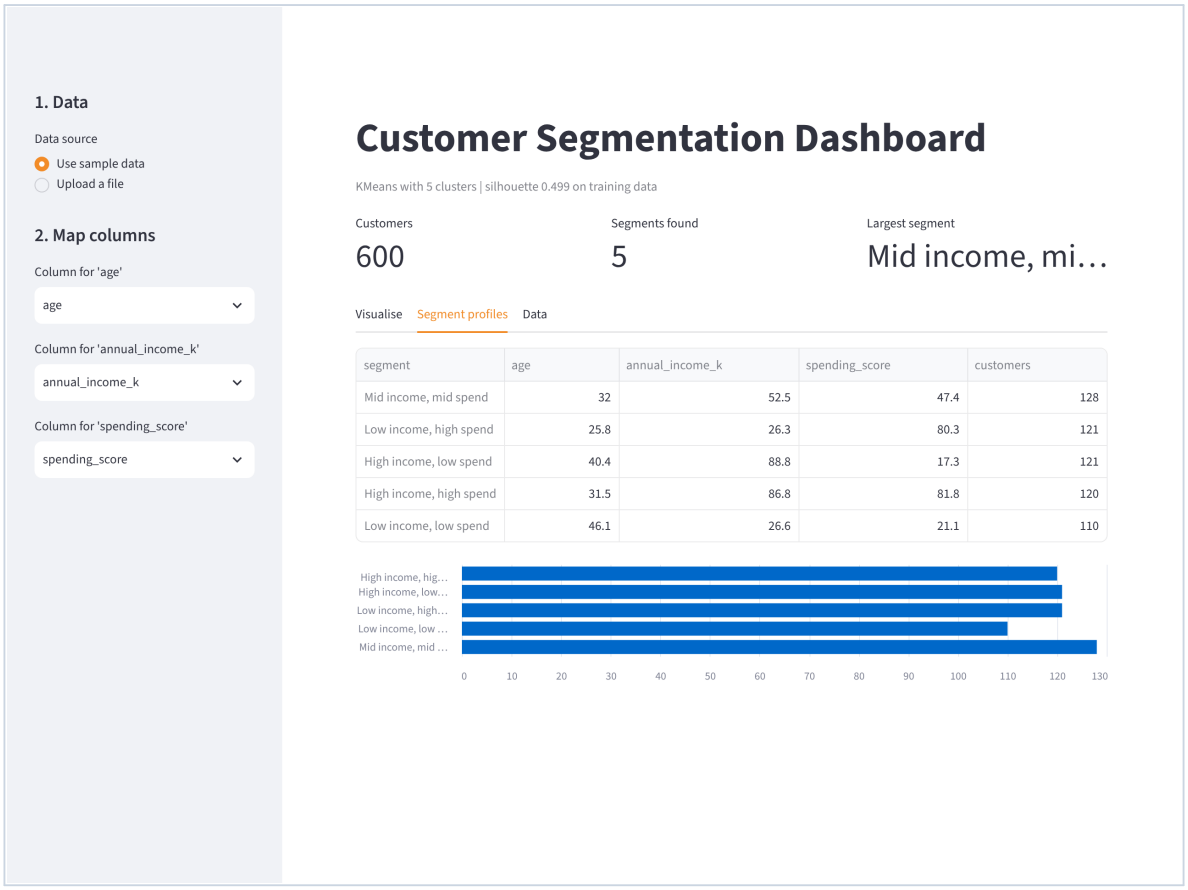


Figure 6.7: Segment profiles in original units with segment sizes

6.7 Time-series forecasting project: Sales Forecasting

Forecasting differs from the other projects: the most recent history is the most valuable input, so production systems usually **re-fit** the model on the latest data before forecasting. This app does that explicitly and caches the fit with `@st.cache_data` keyed on the series, horizon and method.

forecast_app/data/make_dataset.py — PYTHON

```python
"""Generate 3 years of synthetic daily store sales.  Run: python data/make_dataset.py"""
from pathlib import Path

import numpy as np
import pandas as pd

rng = np.random.default_rng(5)
dates = pd.date_range("2023-01-01", "2025-12-31", freq="D")
t = np.arange(len(dates))
weekly = np.array([0.92, 0.88, 0.9, 0.95, 1.08, 1.22, 1.15])[dates.dayofweek]
yearly = 1 + 0.12 * np.sin(2 * np.pi * (dates.dayofyear - 80) / 365.25)
festive = np.where(dates.month.isin([10, 11]), 1.18, 1.0)  # Diwali season uplift
sales = (42000 + 18 * t) * weekly * yearly * festive * rng.lognormal(0, 0.06, len(t))
df = pd.DataFrame({"date": dates, "sales": sales.round(0).astype(int)})

if __name__ == "__main__":
    out = Path(__file__).parent / "daily_sales.csv"
    df.to_csv(out, index=False)
    print(f"Saved {len(df)} days to {out}")
```

forecast_app/forecasting.py PYTHON

```python
"""Forecasting helpers kept separate from the UI so they can be unit-tested."""
import warnings

import numpy as np
import pandas as pd
from statsmodels.tsa.statespace.sarimax import SARIMAX


def prepare_series(df: pd.DataFrame, date_col: str, value_col: str) -> pd.Series:
    s = df[[date_col, value_col]].copy()
    s[date_col] = pd.to_datetime(s[date_col], errors="coerce")
    s[value_col] = pd.to_numeric(s[value_col], errors="coerce")
    s = s.dropna().groupby(date_col)[value_col].sum().sort_index()
    return s.asfreq("D").interpolate(limit=3)  # fill short gaps only


def sarimax_forecast(series: pd.Series, horizon: int, alpha: float = 0.05):
    """Return a DataFrame with forecast, lower and upper interval bounds."""
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        model = SARIMAX(np.log(series), order=(1, 1, 1), seasonal_order=(1, 0, 1, 7),
                        enforce_stationarity=False, enforce_invertibility=False)
        fitted = model.fit(disp=False)
    fc = fitted.get_forecast(steps=horizon)
    ci = fc.conf_int(alpha=alpha)
    out = pd.DataFrame({
        "forecast": np.exp(fc.predicted_mean),
        "lower": np.exp(ci.iloc[:, 0]),
        "upper": np.exp(ci.iloc[:, 1]),
    })
    out.index.name = "date"
    return out


def seasonal_naive_forecast(series: pd.Series, horizon: int) -> pd.DataFrame:
    """Repeat the last observed week. Simple baseline WITHOUT prediction intervals."""
    last_week = series.iloc[-7:].to_numpy()
    values = np.resize(last_week, horizon)
    idx = pd.date_range(series.index[-1] + pd.Timedelta(days=1), periods=horizon, freq="D")
    return pd.DataFrame({"forecast": values}, index=pd.Index(idx, name="date"))


def mape(actual: pd.Series, predicted: pd.Series) -> float:
    return float(np.mean(np.abs((actual - predicted) / actual)))
```

forecast_app/app.py PYTHON

```python
"""Sales Forecasting app.  Run: streamlit run app.py"""
from pathlib import Path

import pandas as pd
import plotly.graph_objects as go
import streamlit as st

from forecasting import mape, prepare_series, sarimax_forecast, seasonal_naive_forecast

ROOT = Path(__file__).parent
st.set_page_config(page_title="Sales Forecast", page_icon=":material/trending_up:",
                   layout="wide")
METHODS = {"SARIMAX (with prediction intervals)": "sarimax",
           "Seasonal naive baseline (no intervals)": "naive"}


@st.cache_data(show_spinner="Fitting forecasting model...")
def run_forecast(series: pd.Series, horizon: int, method: str, alpha: float):
    if method == "sarimax":
        return sarimax_forecast(series, horizon, alpha)
```

forecast_app/app.py PYTHON

```
    return seasonal_naive_forecast(series, horizon)


st.title("Sales Forecasting")
with st.sidebar:
    st.header("Data")
    upload = st.file_uploader("Historical sales CSV", type=["csv"],
                              help="Needs a date column and a numeric sales column.")
    raw = pd.read_csv(upload) if upload else pd.read_csv(ROOT / "data" / "daily_sales.csv")
    date_col = st.selectbox("Date column", raw.columns, index=0)
    value_col = st.selectbox("Sales column", raw.columns, index=min(1, len(raw.columns) - 1))

try:
    series = prepare_series(raw, date_col, value_col)
except Exception as exc:
    st.error(f"Could not build a daily series: {exc}")
    st.stop()
if len(series) < 60:
    st.error("At least 60 days of history are needed for a weekly-seasonal model.")
    st.stop()

with st.sidebar:
    st.header("Forecast settings")
    start, end = st.date_input(
        "Training window", value=(series.index[-365].date(), series.index[-1].date()),
        min_value=series.index[0].date(), max_value=series.index[-1].date())
    horizon = st.slider("Forecast horizon (days)", 7, 90, 30)
    method_label = st.radio("Method", list(METHODS))
    level = st.select_slider("Interval level", [80, 90, 95], value=95,
                             disabled=METHODS[method_label] != "sarimax")
    backtest = st.toggle("Back-test on the last horizon", value=True,
                         help="Hide the last N days, forecast them, compare to actuals.")

train = series.loc[str(start):str(end)]
method = METHODS[method_label]
alpha = 1 - level / 100
if backtest:
    fit_series, actual = train.iloc[:-horizon], train.iloc[-horizon:]
else:
    fit_series, actual = train, None
fc = run_forecast(fit_series, horizon, method, alpha)

fig = go.Figure()
fig.add_scatter(x=fit_series.index[-180:], y=fit_series.iloc[-180:], name="History",
                line=dict(color="#1f4e8c"))
if "lower" in fc:
    fig.add_scatter(x=list(fc.index) + list(fc.index[::-1]),
                    y=list(fc["upper"]) + list(fc["lower"][::-1]), fill="toself",
                    fillcolor="rgba(242,140,40,0.18)", line=dict(width=0),
                    name=f"{level}% prediction interval")
fig.add_scatter(x=fc.index, y=fc["forecast"], name="Forecast",
                line=dict(color="#f28c28", dash="dash"))
if actual is not None:
    fig.add_scatter(x=actual.index, y=actual, name="Actual (held out)",
                    line=dict(color="#2ca02c"))
fig.update_layout(height=460, yaxis_title="Sales", margin=dict(t=30))
st.plotly_chart(fig, width="stretch")

c1, c2, c3 = st.columns(3)
c1.metric("Forecast total", f"{fc['forecast'].sum():,.0f}")
c2.metric("Average per day", f"{fc['forecast'].mean():,.0f}")
if actual is not None:
    c3.metric("Back-test MAPE", f"{mape(actual, fc['forecast'].reindex(actual.index)):.1%}")
if "lower" not in fc:
    st.info("The seasonal naive method does not produce prediction intervals, so none "
            "are shown. Do not invent intervals for methods that cannot estimate them.")

out = fc.round(0).reset_index()
```

forecast_app/app.py PYTHON

```python
out["date"] = out["date"].dt.date
st.dataframe(out, hide_index=True, width="stretch", height=260)
st.download_button("Download forecast (CSV)", out.to_csv(index=False),
                   "sales_forecast.csv", "text/csv", type="primary")
```

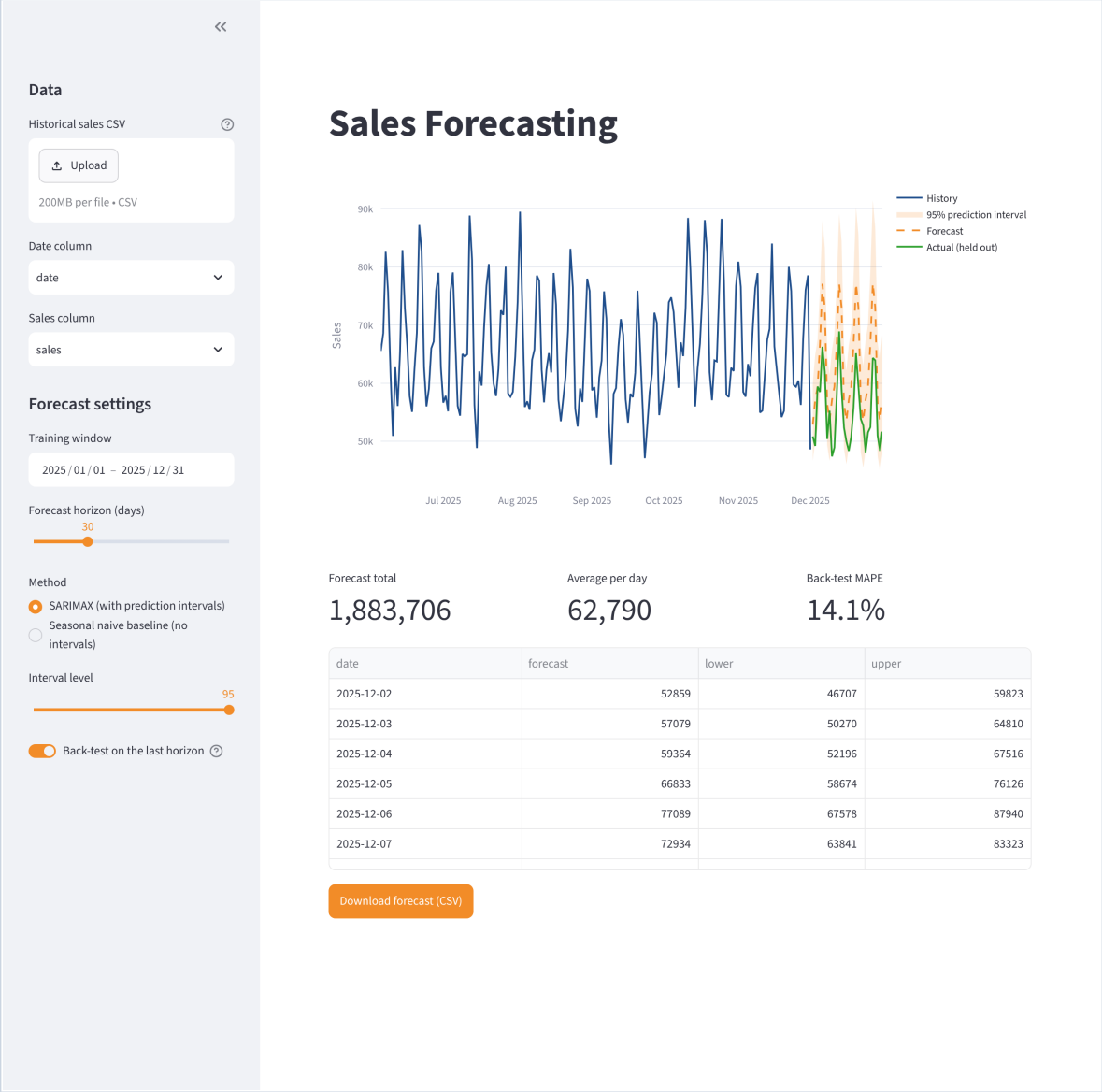


Figure 6.8: SARIMAX forecast with a 95% prediction interval and a back-test against held-out actuals

RESPONSIBLE ML: Honest uncertainty

Prediction intervals are **model-dependent**. SARIMAX produces them from its error model; the seasonal-naive baseline cannot, so the app shows none.

Never draw a shaded band by adding a fixed percentage around a point forecast and call it a confidence interval. If you need intervals for models without them, use residual bootstrapping or conformal methods and document the method.

6.8 Deep learning project: Image Classification (TensorFlow/Keras)

A small convolutional network recognises handwritten-style digits 0-9. The same structure works for any image model (for example a MobileNetV2 transfer learning model on product photos): change the preprocessing function, the class names and the model file.

image_app/preprocessing.py — PYTHON

```python
"""Image preprocessing shared by training AND inference.

If the app prepared images differently from training (different size, colour mode,
scaling or inversion), predictions would silently degrade. Import this one function
in both places.
"""
import numpy as np
from PIL import Image, ImageOps

IMG_SIZE = 28


def preprocess_image(img: Image.Image) -> np.ndarray:
    """PIL image of a single digit -> float32 array of shape (28, 28, 1) in [0, 1]."""
    img = ImageOps.exif_transpose(img)          # respect phone camera rotation
    gray = img.convert("L")
    arr = np.asarray(gray, dtype=np.float32)
    if arr.mean() > 127:                         # dark ink on light paper -> invert
        arr = 255.0 - arr
    arr[arr < 40] = 0                            # remove paper texture / noise
    ys, xs = np.nonzero(arr)
    if len(xs):                                  # crop to the digit's bounding box
        arr = arr[ys.min():ys.max() + 1, xs.min():xs.max() + 1]
    h, w = arr.shape
    side = max(h, w)
    square = np.zeros((side, side), dtype=np.float32)
    square[(side - h) // 2:(side - h) // 2 + h, (side - w) // 2:(side - w) // 2 + w] = arr
    digit = Image.fromarray(square.astype(np.uint8)).resize((20, 20), Image.LANCZOS)
    canvas = np.zeros((IMG_SIZE, IMG_SIZE), dtype=np.float32)
    canvas[4:24, 4:24] = np.asarray(digit, dtype=np.float32)
    return (canvas / 255.0)[..., np.newaxis]
```

image_app/synthetic_digits.py — PYTHON

```python
"""Synthetic digit images rendered with several fonts (no download needed).

Kept in its own module so the Keras AND PyTorch training scripts can share it
without importing each other's frameworks.
"""
import glob

import numpy as np
from PIL import Image, ImageDraw, ImageFilter, ImageFont

from preprocessing import preprocess_image

CLASS_NAMES = [str(d) for d in range(10)]
FONT_FILES = sorted(glob.glob("/usr/share/fonts/truetype/**/*.ttf", recursive=True))
rng = np.random.default_rng(0)


def render_digit(d: int, font_path: str) -> Image.Image:
    size = int(rng.integers(70, 110))
    font = ImageFont.truetype(font_path, size)
    img = Image.new("L", (140, 140), color=255)
    ImageDraw.Draw(img).text((35, 10), str(d), fill=int(rng.integers(0, 90)), font=font)
```

image_app/synthetic_digits.py PYTHON

```python
    img = img.rotate(float(rng.uniform(-15, 15)), fillcolor=255)
    if rng.random() < 0.3:
        img = img.filter(ImageFilter.GaussianBlur(rng.uniform(0.5, 1.5)))
    noise = rng.normal(0, 8, (140, 140))
    return Image.fromarray(np.clip(np.asarray(img) + noise, 0, 255).astype(np.uint8))


def make_dataset(per_font: int = 8):
    fonts = [f for f in FONT_FILES if "Emoji" not in f][:40]
    X, y = [], []
    for font in fonts:
        for d in range(10):
            for _ in range(per_font):
                X.append(preprocess_image(render_digit(d, font)))
                y.append(d)
    return np.array(X), np.array(y)
```

image_app/train_model.py PYTHON

```python
"""Train a small Keras CNN that recognises digits 0-9 and save it as .keras.

The training images are digits rendered with several fonts plus random rotation,
scaling and noise, so the project runs offline. Run: python train_model.py
"""
import json
from pathlib import Path

import keras
from keras import layers
from preprocessing import IMG_SIZE
from synthetic_digits import CLASS_NAMES, FONT_FILES, make_dataset, render_digit, rng

ROOT = Path(__file__).parent


def build_model() -> keras.Model:
    return keras.Sequential([
        keras.Input(shape=(IMG_SIZE, IMG_SIZE, 1)),
        layers.RandomRotation(0.05), layers.RandomZoom(0.1),
        layers.Conv2D(32, 3, activation="relu"), layers.MaxPooling2D(),
        layers.Conv2D(64, 3, activation="relu"), layers.MaxPooling2D(),
        layers.Flatten(), layers.Dropout(0.3),
        layers.Dense(64, activation="relu"),
        layers.Dense(len(CLASS_NAMES), activation="softmax"),
    ])


def main() -> None:
    X, y = make_dataset()
    idx = rng.permutation(len(X))
    X, y = X[idx], y[idx]
    split = int(0.85 * len(X))
    model = build_model()
    model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])
    model.fit(X[:split], y[:split], validation_data=(X[split:], y[split:]),
              epochs=12, batch_size=64, verbose=2)
    loss, acc = model.evaluate(X[split:], y[split:], verbose=0)
    (ROOT / "models").mkdir(exist_ok=True)
    model.save(ROOT / "models" / "digit_cnn.keras")       # native Keras 3 format
    meta = {"class_names": CLASS_NAMES, "input_size": IMG_SIZE,
            "val_accuracy": round(float(acc), 4), "n_images": int(len(X)),
            "keras_version": keras.__version__}
    (ROOT / "models" / "digit_metadata.json").write_text(json.dumps(meta, indent=2))
    # sample images for testing the app
    (ROOT / "samples").mkdir(exist_ok=True)
    for d in (2, 5, 7):
        render_digit(d, FONT_FILES[0]).save(ROOT / "samples" / f"digit_{d}.png")
```

image_app/train_model.py PYTHON

```
    print(f"Validation accuracy: {acc:.3f}")


if __name__ == "__main__":
    main()
```

image_app/app.py PYTHON

```
"""Digit image classifier (TensorFlow/Keras).  Run: streamlit run app.py"""
import json
from pathlib import Path

import numpy as np
import pandas as pd
import streamlit as st
from PIL import Image, UnidentifiedImageError

from preprocessing import preprocess_image

ROOT = Path(__file__).parent
SUPPORTED = ["png", "jpg", "jpeg", "webp"]
MAX_PIXELS = 4000 * 4000          # refuse huge images (memory / decompression bombs)
st.set_page_config(page_title="Digit Classifier", page_icon=":material/image:")


@st.cache_resource(show_spinner="Loading TensorFlow model...")
def load_model():
    import keras  # imported lazily: TensorFlow start-up is slow
    model = keras.models.load_model(ROOT / "models" / "digit_cnn.keras")
    meta = json.loads((ROOT / "models" / "digit_metadata.json").read_text())
    return model, meta


def read_image(file) -> Image.Image:
    img = Image.open(file)
    if img.width * img.height > MAX_PIXELS:
        raise ValueError("Image is too large; please upload a smaller photo.")
    img.load()                        # force decoding now so errors surface here
    return img


model, meta = load_model()
st.title("Handwritten Digit Classifier")
st.caption(f"Keras CNN | validation accuracy {meta['val_accuracy']:.1%} | "
           f"input {meta['input_size']}x{meta['input_size']} grayscale")

source = st.segmented_control("Image source", ["Upload", "Camera", "Sample"],
                              default="Upload")
file = None
if source == "Upload":
    file = st.file_uploader("Upload a digit image", type=SUPPORTED)
elif source == "Camera":
    file = st.camera_input("Take a photo of one handwritten digit")
else:
    samples = sorted((ROOT / "samples").glob("*.png"))
    choice = st.selectbox("Sample image", samples, format_func=lambda p: p.name)
    file = choice

if file is None:
    st.info(f"Supported formats: {', '.join(SUPPORTED)}. One digit per image works best.")
    st.stop()

try:
    image = read_image(file)
except (UnidentifiedImageError, ValueError, OSError) as exc:
    st.error(f"Could not read this image: {exc}")
    st.stop()
```

image_app/app.py PYTHON

```python
x = preprocess_image(image)
probs = model.predict(x[np.newaxis, ...], verbose=0)[0]     # add batch dimension
classes = meta["class_names"]
top_k = st.slider("Show top-k classes", 1, len(classes), 3)
order = np.argsort(probs)[::-1][:top_k]

left, right = st.columns(2)
with left:
    st.image(image, caption="Uploaded image", width="stretch")
    st.image(x.squeeze(), caption="What the model sees (28x28)", width=140, clamp=True)
with right:
    best = order[0]
    st.metric("Predicted digit", classes[best], f"{probs[best]:.1%} confidence",
              delta_color="off")
    if probs[best] < 0.6:
        st.warning("Low confidence. Try a clearer image with one centred digit.")
    top = pd.DataFrame({"class": [classes[i] for i in order],
                        "probability": probs[order]})
    st.dataframe(top, hide_index=True, width="stretch",
                 column_config={"probability": st.column_config.ProgressColumn(
                     "Probability", min_value=0.0, max_value=1.0, format="%.3f")})
    st.bar_chart(pd.Series(probs, index=classes, name="probability"))
```

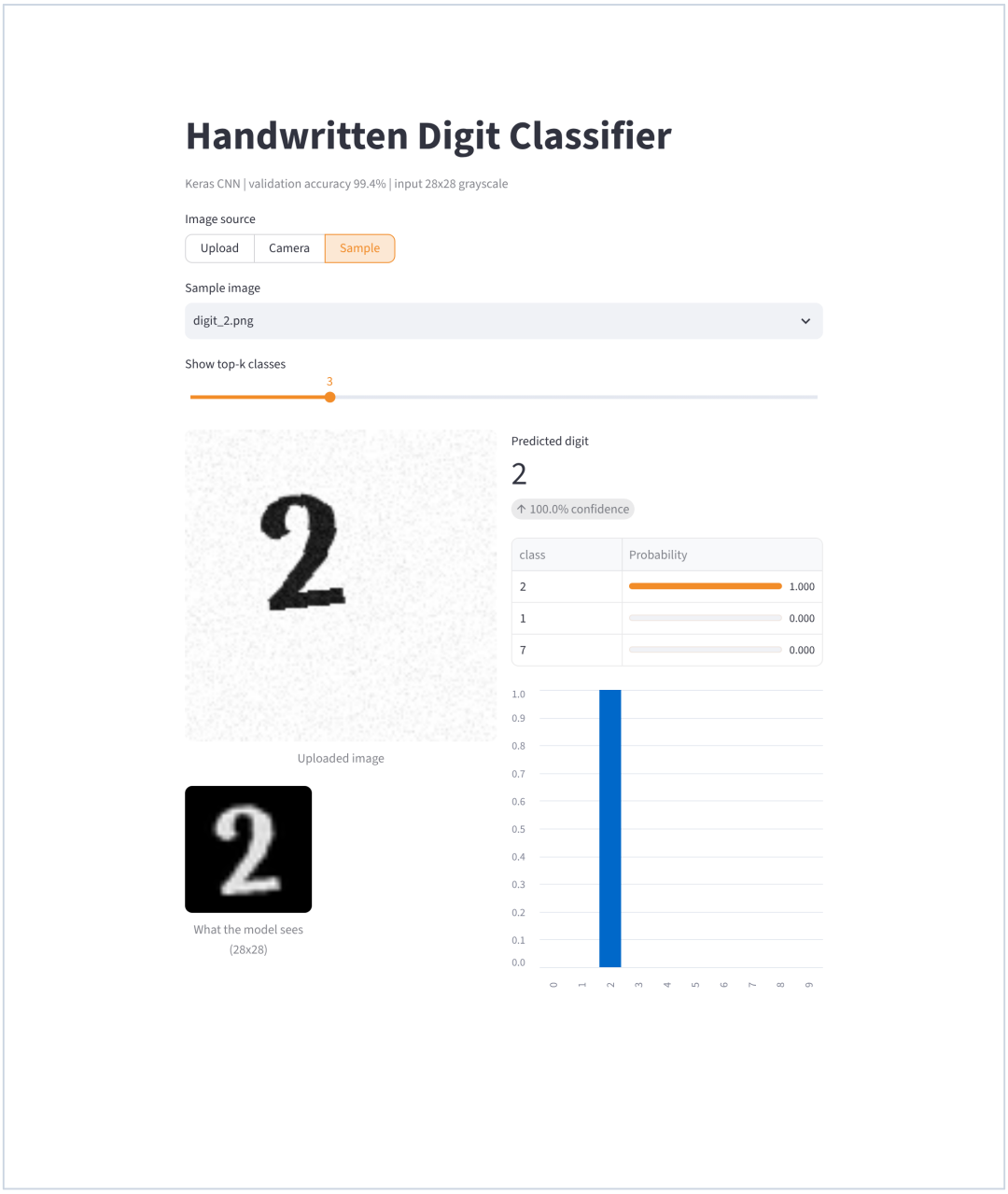


Figure 6.9: Uploaded image, the 28x28 tensor the model sees, the predicted class and top-k probabilities

Issue	How the app handles it
Supported formats	`type=["png", "jpg", "jpeg", "webp"]`; Pillow decodes them
Corrupted or fake image files	`Image.open` + `img.load()` in try/except `UnidentifiedImageError`, `OSError`
Huge images	Pixel-count limit before processing
Phone photos rotated	`ImageOps.exif_transpose`
Colour vs grayscale, scale 0-255 vs 0-1	One shared `preprocess_image()`
Batch dimension	`x[np.newaxis, ...]` -> shape (1, 28, 28, 1)
Slow TensorFlow import	Lazy import inside the cached loader

6.8.1 Adapting the inference workflow to PyTorch

PyTorch saves **weights** (a `state_dict`), not the architecture, so the model class must be defined in code that the app imports. Two other differences: PyTorch expects channels-first tensors (N, C, H, W) and the model must be put in evaluation mode (`model.eval()`) with gradients disabled.

image_app/torch_version.py PYTHON

```python
"""PyTorch version of the digit classifier: model definition, training, inference.

Train:  python torch_version.py      -> models/digit_cnn_state.pt
Use in Streamlit: see torch_app.py (same preprocessing as the Keras version).
"""
from pathlib import Path

import numpy as np
import torch
from torch import nn

ROOT = Path(__file__).parent
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")


class DigitCNN(nn.Module):
    def __init__(self, n_classes: int = 10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, 3), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3), nn.ReLU(), nn.MaxPool2d(2),
        )
        self.head = nn.Sequential(nn.Flatten(), nn.Dropout(0.3),
                                  nn.Linear(64 * 5 * 5, 64), nn.ReLU(),
                                  nn.Linear(64, n_classes))

    def forward(self, x):
        return self.head(self.features(x))


def load_torch_model(path: Path = ROOT / "models" / "digit_cnn_state.pt") -> nn.Module:
    model = DigitCNN()
    # weights_only=True refuses to unpickle arbitrary objects (safe default)
    state = torch.load(path, map_location=DEVICE, weights_only=True)
    model.load_state_dict(state)
    return model.to(DEVICE).eval()


@torch.no_grad()
def predict_proba(model: nn.Module, x_hwc: np.ndarray) -> np.ndarray:
    """x_hwc: (28, 28, 1) array from preprocess_image -> (10,) probabilities."""
    tensor = torch.from_numpy(x_hwc).permute(2, 0, 1).unsqueeze(0).to(DEVICE)  # NCHW
    return torch.softmax(model(tensor), dim=1).cpu().numpy()[0]


def train(epochs: int = 8) -> None:
    from synthetic_digits import make_dataset  # same images as the Keras model
    X, y = make_dataset()
    X_t = torch.from_numpy(X).permute(0, 3, 1, 2)
    y_t = torch.from_numpy(y).long()
    model = DigitCNN().to(DEVICE)
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    loss_fn = nn.CrossEntropyLoss()
    for epoch in range(epochs):
        perm = torch.randperm(len(X_t))
        model.train()
        for i in range(0, len(X_t), 64):
            idx = perm[i:i + 64]
            xb, yb = X_t[idx].to(DEVICE), y_t[idx].to(DEVICE)
            opt.zero_grad()
            loss = loss_fn(model(xb), yb)
```

image_app/torch_version.py PYTHON

```
            loss.backward()
            opt.step()
        print(f"epoch {epoch + 1}: loss {loss.item():.4f}")
    (ROOT / "models").mkdir(exist_ok=True)
    torch.save(model.state_dict(), ROOT / "models" / "digit_cnn_state.pt")


if __name__ == "__main__":
    train()
```

image_app/torch_app.py PYTHON

```
"""Minimal Streamlit front-end for the PyTorch model.  Run: streamlit run torch_app.py"""
import numpy as np
import streamlit as st
from PIL import Image, UnidentifiedImageError

from preprocessing import preprocess_image
from torch_version import DEVICE, load_torch_model, predict_proba

st.set_page_config(page_title="Digit Classifier (PyTorch)")


@st.cache_resource(show_spinner="Loading PyTorch model...")
def get_model():
    return load_torch_model()


model = get_model()
st.title("Digit Classifier - PyTorch")
st.caption(f"Running on: {DEVICE}")
file = st.file_uploader("Digit image", type=["png", "jpg", "jpeg", "webp"])
if file is not None:
    try:
        image = Image.open(file)
        image.load()
    except (UnidentifiedImageError, OSError) as exc:
        st.error(f"Could not read the image: {exc}")
    else:
        probs = predict_proba(model, preprocess_image(image))
        best = int(np.argmax(probs))
        st.image(image, width=200)
        st.metric("Predicted digit", str(best), f"{probs[best]:.1%}", delta_color="off")
        st.bar_chart(probs)
```

Concern	Keras	PyTorch
Load	`keras.models.load_model(path)`	build class, `load_state_dict(torch.load(path, weights_only=True))`
Layout	NHWC (28, 28, 1)	NCHW (1, 28, 28)
Inference mode	`model.predict(x)` / `model(x, training=False)`	`model.eval()` + `@torch.no_grad()`
Device	automatic	`.to(DEVICE)` for model and tensors; `map_location` when loading
Probabilities	softmax layer in model	`torch.softmax(logits, dim=1)`

WARNING

Importing TensorFlow and PyTorch in the same process can crash on some systems. Keep framework-specific code in separate modules (as above) and never import both in one app unless you have tested it.

6.9 NLP project: Sentiment Analysis

Text models need the cleaning step **and** the vectoriser vocabulary saved with the classifier. Passing `preprocessor=clean_text` to `TfidfVectorizer` puts both inside one pipeline.

sentiment_app/text_utils.py — PYTHON

```python
"""Text cleaning used INSIDE the saved pipeline.

Because the pipeline stores a reference to clean_text (not its source code),
this module must be importable wherever the model is loaded (the Streamlit app,
the API, tests). Keep it in the project and never rename it after training.
"""
import re

_URL = re.compile(r"https?://\S+|www\.\S+")
_NON_ALPHA = re.compile(r"[^a-z0-9!?' ]+")
_SPACES = re.compile(r"\s+")


def clean_text(text: str) -> str:
    text = str(text).lower()
    text = _URL.sub(" ", text)
    text = text.replace("n't", " not")          # "didn't" -> "did not"
    text = _NON_ALPHA.sub(" ", text)
    return _SPACES.sub(" ", text).strip()
```

sentiment_app/train_model.py — PYTHON

```python
"""Build a review corpus, train TF-IDF + Logistic Regression, save the pipeline.

Run: python train_model.py
"""
import json
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, f1_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline

from text_utils import clean_text

ROOT = Path(__file__).parent
rng = np.random.default_rng(8)
PRODUCTS = ["phone", "laptop", "headphones", "delivery", "app", "charger", "smartwatch",
            "service", "camera", "speaker"]
POS = ["works perfectly", "is excellent value for money", "exceeded my expectations",
       "is fast and reliable", "has amazing battery life", "was delivered quickly",
       "is really easy to use", "has superb sound quality", "is worth every rupee",
       "made my day"]
NEG = ["stopped working after a week", "is a complete waste of money",
       "was delivered late and damaged", "keeps crashing", "has terrible battery life",
       "is very slow", "did not match the description", "broke on the first day",
       "has awful customer support", "is overpriced and flimsy"]
OPEN_POS = ["Loved it!", "Highly recommend.", "Five stars.", "Very happy.", "Great buy."]
OPEN_NEG = ["Very disappointed.", "Do not buy.", "One star.", "Returning it.",
            "Not happy at all."]


def make_reviews(n: int = 2400) -> pd.DataFrame:
    rows = []
    for _ in range(n):
```

sentiment_app/train_model.py PYTHON

```python
        label = rng.choice(["positive", "negative"])
        phrases, opens = (POS, OPEN_POS) if label == "positive" else (NEG, OPEN_NEG)
        text = f"The {rng.choice(PRODUCTS)} {rng.choice(phrases)}."
        if rng.random() < 0.6:
            text = f"{rng.choice(opens)} {text}"
        if rng.random() < 0.15:  # mixed signal makes the task less trivial
            other = NEG if label == "positive" else POS
            text += f" Although the box {rng.choice(other)}."
        rows.append((text, label))
    return pd.DataFrame(rows, columns=["review", "sentiment"])


def main() -> None:
    (ROOT / "data").mkdir(exist_ok=True)
    df = make_reviews()
    df.to_csv(ROOT / "data" / "reviews.csv", index=False)
    X_train, X_test, y_train, y_test = train_test_split(
        df["review"], df["sentiment"], test_size=0.2, stratify=df["sentiment"],
        random_state=42)
    pipe = Pipeline([
        ("tfidf", TfidfVectorizer(preprocessor=clean_text, ngram_range=(1, 2),
                                  min_df=2, sublinear_tf=True)),
        ("clf", LogisticRegression(max_iter=1000, C=2.0)),
    ])
    pipe.fit(X_train, y_train)
    pred = pipe.predict(X_test)
    meta = {"classes": list(pipe.classes_),
            "accuracy": round(float(accuracy_score(y_test, pred)), 4),
            "macro_f1": round(float(f1_score(y_test, pred, average="macro")), 4),
            "report": classification_report(y_test, pred, output_dict=True),
            "vocabulary_size": len(pipe.named_steps["tfidf"].vocabulary_)}
    (ROOT / "models").mkdir(exist_ok=True)
    joblib.dump(pipe, ROOT / "models" / "sentiment_pipeline.joblib")
    (ROOT / "models" / "sentiment_metadata.json").write_text(json.dumps(meta, indent=2))
    print("accuracy", meta["accuracy"], "macro F1", meta["macro_f1"])


if __name__ == "__main__":
    main()
```

sentiment_app/app.py PYTHON

```python
"""Sentiment Analysis app.  Run: streamlit run app.py"""
import json
from pathlib import Path

import joblib
import pandas as pd
import streamlit as st

import text_utils  # noqa: F401  (must be importable so joblib can rebuild the pipeline)

ROOT = Path(__file__).parent
MAX_CHARS = 2000
MAX_BATCH_ROWS = 5000
st.set_page_config(page_title="Sentiment Analyzer", page_icon=":material/mood:")


@st.cache_resource
def load_model():
    pipe = joblib.load(ROOT / "models" / "sentiment_pipeline.joblib")
    meta = json.loads((ROOT / "models" / "sentiment_metadata.json").read_text())
    return pipe, meta


def predict(texts: list[str]) -> pd.DataFrame:
    proba = pipe.predict_proba(texts)
    classes = list(pipe.classes_)
```

sentiment_app/app.py PYTHON

```
    best = proba.argmax(axis=1)
    return pd.DataFrame({
        "text": texts,
        "sentiment": [classes[i] for i in best],
        "confidence": proba.max(axis=1).round(4),
    })


pipe, meta = load_model()
st.title("Product Review Sentiment")
st.caption(f"TF-IDF + logistic regression | test accuracy {meta['accuracy']:.1%} | "
           f"vocabulary {meta['vocabulary_size']:,} terms")

single, batch = st.tabs(["Analyse one review", "Batch analysis"])
with single:
    text = st.text_area("Review text", max_chars=MAX_CHARS, height=120,
                        placeholder="e.g. The headphones have superb sound quality.")
    if st.button("Analyse sentiment", type="primary"):
        if len(text.strip()) < 3:
            st.warning("Please enter at least a few words.")
        else:
            res = predict([text]).iloc[0]
            icon = ":material/sentiment_satisfied:" if res.sentiment == "positive" \
                else ":material/sentiment_dissatisfied:"
            c1, c2 = st.columns(2)
            c1.metric("Sentiment", res.sentiment.title())
            c2.metric("Confidence", f"{res.confidence:.1%}")
            if res.confidence < 0.65:
                st.info("Low confidence: the review may contain mixed opinions.",
                        icon=":material/help:")
            else:
                st.success(f"Predicted {res.sentiment}", icon=icon)
            with st.expander("What the model sees after cleaning"):
                st.code(text_utils.clean_text(text), language=None)

with batch:
    st.write("Upload a **.txt** file (one review per line) or a **.csv** with a "
             "`review` column.")
    up = st.file_uploader("Reviews file", type=["txt", "csv"])
    if up is not None:
        if up.name.lower().endswith(".txt"):
            lines = up.getvalue().decode("utf-8", errors="replace").splitlines()
            texts = [ln.strip() for ln in lines if ln.strip()]
        else:
            df = pd.read_csv(up)
            if "review" not in df.columns:
                st.error("The CSV must contain a column named 'review'.")
                st.stop()
            texts = df["review"].dropna().astype(str).tolist()
        if len(texts) > MAX_BATCH_ROWS:
            st.warning(f"Only the first {MAX_BATCH_ROWS:,} reviews are analysed.")
            texts = texts[:MAX_BATCH_ROWS]
        results = predict([t[:MAX_CHARS] for t in texts])
        counts = results["sentiment"].value_counts()
        c1, c2, c3 = st.columns(3)
        c1.metric("Reviews", len(results))
        c2.metric("Positive", int(counts.get("positive", 0)))
        c3.metric("Negative", int(counts.get("negative", 0)))
        st.bar_chart(counts)
        st.dataframe(results, hide_index=True, width="stretch",
                     column_config={"confidence": st.column_config.NumberColumn(
                         format="percent")})
        st.download_button("Download results", results.to_csv(index=False),
                           "sentiment_results.csv", "text/csv")
```

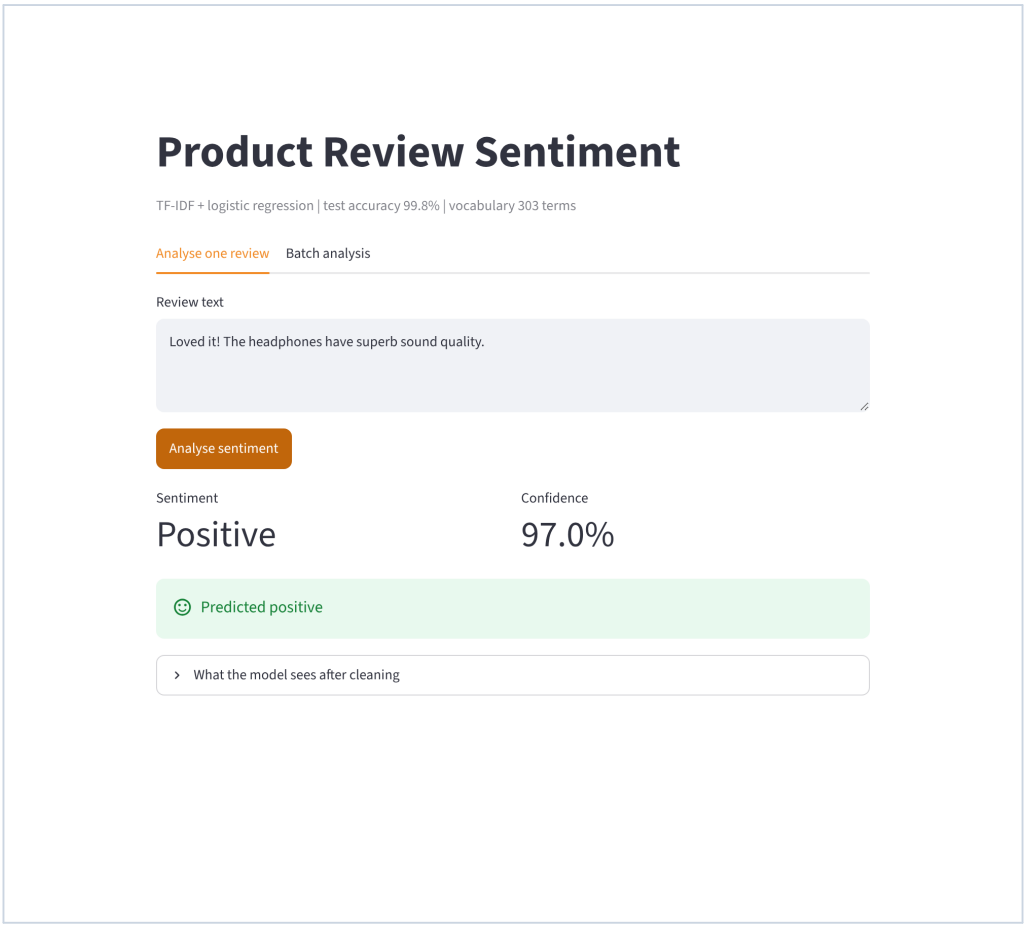


Figure 6.10: Sentiment prediction with confidence

COMMON MISTAKE

Defining `clean_text` inside the notebook and pickling the pipeline. Loading it in the app then fails with `AttributeError: Can't get attribute 'clean_text' on <module '__main__'>`. Define custom functions in an importable module (`text_utils.py`) **before** training.

NOTE

The synthetic reviews are easy, so accuracy is near 100%. On a real corpus such as IMDB or Amazon reviews expect 85-92% for TF-IDF + logistic regression; transformer models (e.g. DistilBERT via Hugging Face `pipeline("sentiment-analysis")`) do better but need more memory.

6.10 Anomaly detection project: Transaction Fraud Screening

Fraud labels are rare and delayed, so a common first system is **unsupervised**: an Isolation Forest learns what normal transactions look like and scores how unusual each new one is. The user chooses how many to flag.

fraud_app/train_model.py — PYTHON

```python
"""Unsupervised anomaly detector for card transactions (Isolation Forest).

Run: python train_model.py   -> data/transactions.csv, models/fraud_*.{joblib,json}
"""
import json
from pathlib import Path

import joblib
import numpy as np
```

fraud_app/train_model.py PYTHON

```python
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import IsolationForest
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

ROOT = Path(__file__).parent
NUMERIC = ["amount", "hour", "distance_from_home_km", "txn_count_24h"]
CATEGORICAL = ["merchant_category", "is_foreign"]
FEATURES = NUMERIC + CATEGORICAL
rng = np.random.default_rng(21)


def make_transactions(n: int = 8000, fraud_rate: float = 0.012) -> pd.DataFrame:
    cats = ["grocery", "fuel", "restaurant", "electronics", "travel", "online"]
    df = pd.DataFrame({
        "transaction_id": [f"T{i:06d}" for i in range(n)],
        "amount": rng.lognormal(6.4, 0.8, n).round(2),            # INR
        "hour": rng.choice(np.arange(24), n, p=np.r_[np.full(7, 0.01),
                                                     np.full(17, 0.93 / 17)]),
        "distance_from_home_km": rng.exponential(6, n).round(1),
        "txn_count_24h": rng.poisson(2, n),
        "merchant_category": rng.choice(cats, n, p=[0.3, 0.15, 0.2, 0.08, 0.07, 0.2]),
        "is_foreign": rng.choice(["No", "Yes"], n, p=[0.97, 0.03]),
    })
    fraud = rng.random(n) < fraud_rate
    k = int(fraud.sum())
    df.loc[fraud, "amount"] = rng.lognormal(9.6, 0.6, k).round(2)
    df.loc[fraud, "hour"] = rng.integers(0, 5, k)
    df.loc[fraud, "distance_from_home_km"] = rng.uniform(150, 2000, k).round(1)
    df.loc[fraud, "txn_count_24h"] = rng.integers(6, 15, k)
    df.loc[fraud, "is_foreign"] = rng.choice(["No", "Yes"], k, p=[0.4, 0.6])
    df["is_fraud"] = fraud.astype(int)  # known ONLY for evaluation in this demo
    return df


def main() -> None:
    (ROOT / "data").mkdir(exist_ok=True)
    df = make_transactions()
    df.to_csv(ROOT / "data" / "transactions.csv", index=False)
    pipe = Pipeline([
        ("prep", ColumnTransformer([
            ("num", StandardScaler(), NUMERIC),
            ("cat", OneHotEncoder(handle_unknown="ignore"), CATEGORICAL)])),
        ("iso", IsolationForest(n_estimators=300, contamination="auto", random_state=42)),
    ])
    pipe.fit(df[FEATURES])                      # labels are NOT used for training
    scores = -pipe.score_samples(df[FEATURES])  # higher = more anomalous
    pct = {str(p): float(np.percentile(scores, p)) for p in (90, 95, 97, 98, 99, 99.5)}
    (ROOT / "models").mkdir(exist_ok=True)
    joblib.dump(pipe, ROOT / "models" / "fraud_iforest.joblib")
    meta = {"features": FEATURES, "numeric": NUMERIC, "categorical": CATEGORICAL,
            "score_percentiles": pct, "n_train": len(df)}
    (ROOT / "models" / "fraud_metadata.json").write_text(json.dumps(meta, indent=2))
    top = df.assign(score=scores).nlargest(int(0.012 * len(df)), "score")
    print(f"Fraud captured in top 1.2% by score: {top['is_fraud'].mean():.1%}")


if __name__ == "__main__":
    main()
```

fraud_app/app.py PYTHON

```python
"""Transaction anomaly detection app.  Run: streamlit run app.py"""
import json
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import plotly.express as px
import streamlit as st
from sklearn.metrics import precision_score, recall_score

ROOT = Path(__file__).parent
st.set_page_config(page_title="Anomaly Detection", page_icon=":material/shield:",
                   layout="wide")


@st.cache_resource
def load_artifacts():
    meta = json.loads((ROOT / "models" / "fraud_metadata.json").read_text())
    return joblib.load(ROOT / "models" / "fraud_iforest.joblib"), meta


@st.cache_data(show_spinner="Scoring transactions...")
def score(df: pd.DataFrame) -> np.ndarray:
    return -model.score_samples(df[meta["features"]])


model, meta = load_artifacts()
st.title("Transaction Anomaly Detection")
st.caption("Isolation Forest flags transactions that look unusual compared with "
           "normal behaviour. Flags need human review; they are not proof of fraud.")

with st.sidebar:
    up = st.file_uploader("Transactions CSV", type=["csv"])
    df = pd.read_csv(up) if up else pd.read_csv(ROOT / "data" / "transactions.csv")
    missing = [c for c in meta["features"] if c not in df.columns]
    if missing:
        st.error(f"Missing columns: {missing}")
        st.stop()
    mode = st.radio("Threshold method", ["Flag top % most unusual", "Score cut-off"])
    scores = score(df)
    if mode == "Flag top % most unusual":
        top_pct = st.slider("Percent of transactions to flag", 0.5, 10.0, 1.5, 0.5)
        threshold = float(np.percentile(scores, 100 - top_pct))
    else:
        threshold = st.slider("Anomaly score threshold", float(scores.min()),
                              float(scores.max()), meta["score_percentiles"]["99"])

df = df.assign(anomaly_score=scores.round(4), flagged=scores >= threshold)
flagged = df[df["flagged"]].sort_values("anomaly_score", ascending=False)

c1, c2, c3, c4 = st.columns(4)
c1.metric("Transactions", f"{len(df):,}")
c2.metric("Flagged", f"{len(flagged):,}", f"{len(flagged) / len(df):.2%} of total",
          delta_color="off")
c3.metric("Threshold", f"{threshold:.3f}")
c4.metric("Flagged amount (INR)", f"{flagged['amount'].sum():,.0f}")

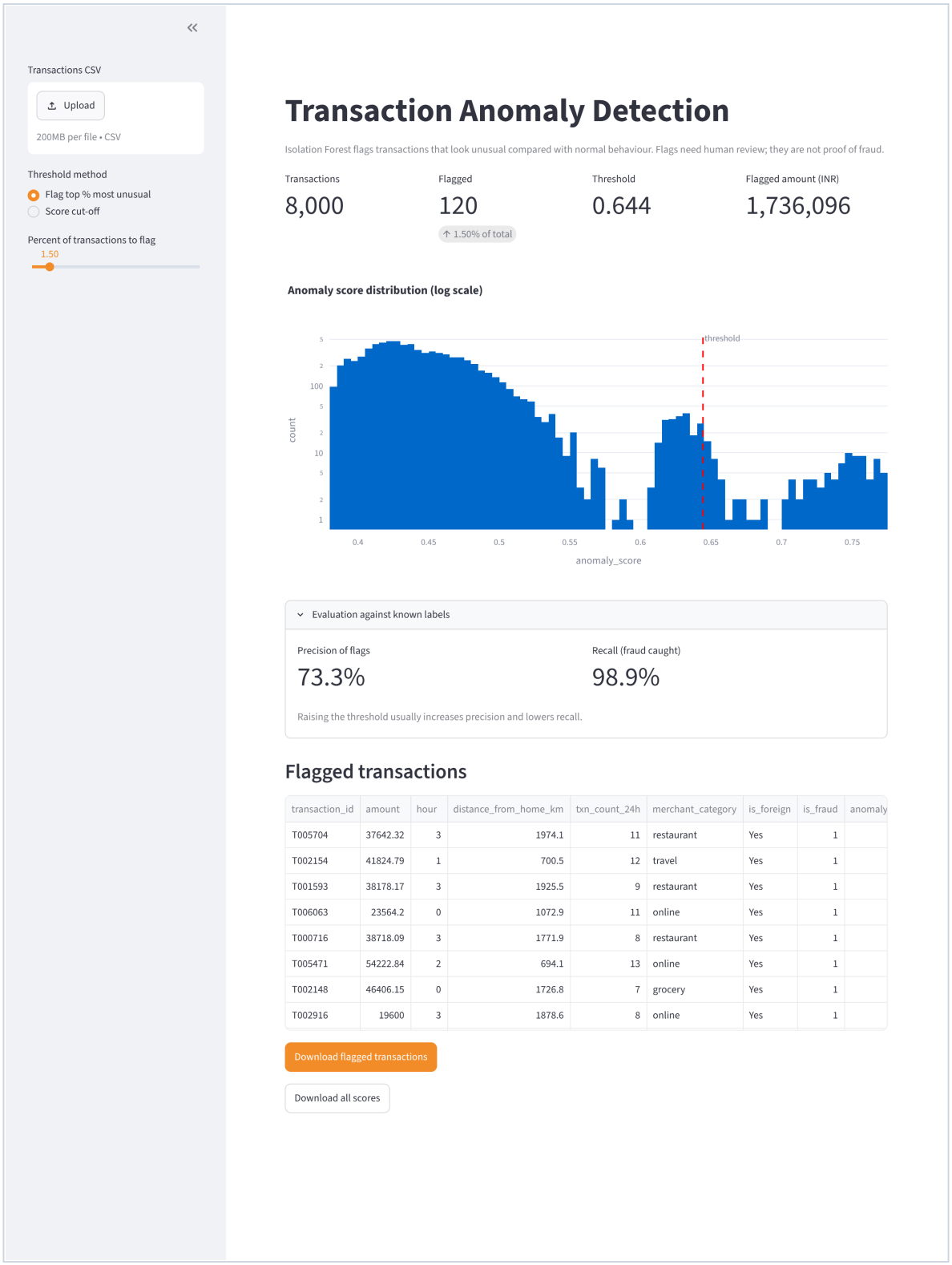
fig = px.histogram(df, x="anomaly_score", nbins=80, log_y=True,
                   title="Anomaly score distribution (log scale)")
fig.add_vline(x=threshold, line_dash="dash", line_color="red",
              annotation_text="threshold")
st.plotly_chart(fig, width="stretch")

if "is_fraud" in df.columns:  # labels available -> we can evaluate the threshold
    with st.expander("Evaluation against known labels", expanded=True):
        p = precision_score(df["is_fraud"], df["flagged"], zero_division=0)
        r = recall_score(df["is_fraud"], df["flagged"], zero_division=0)
```

fraud_app/app.py PYTHON

```
        e1, e2 = st.columns(2)
        e1.metric("Precision of flags", f"{p:.1%}")
        e2.metric("Recall (fraud caught)", f"{r:.1%}")
        st.caption("Raising the threshold usually increases precision and lowers recall.")

st.subheader("Flagged transactions")
st.dataframe(flagged, hide_index=True, width="stretch", height=320)
st.download_button("Download flagged transactions", flagged.to_csv(index=False),
                   "flagged_transactions.csv", "text/csv", type="primary")
st.download_button("Download all scores", df.to_csv(index=False), "all_scores.csv",
                   "text/csv")
```



transaction_id	amount	hour	distance_from_home_km	txn_count_24h	merchant_category	is_foreign	is_fraud	anomaly
T005704	37642.32	3	1974.1	11	restaurant	Yes	1	
T002154	41824.79	1	700.5	12	travel	Yes	1	
T001593	38178.17	3	1925.5	9	restaurant	Yes	1	
T006063	23564.2	0	1072.9	11	online	Yes	1	
T000716	38718.09	3	1771.9	8	restaurant	Yes	1	
T005471	54222.84	2	694.1	13	online	Yes	1	
T002148	46406.15	0	1726.8	7	grocery	Yes	1	
T002916	19600	3	1878.6	8	online	Yes	1	

Figure 6.11: Anomaly scores, an adjustable threshold, flagged transactions and evaluation against known labels

RESPONSIBLE ML: Model limitations

An anomaly is *unusual*, not necessarily *fraudulent*: a customer on holiday abroad looks anomalous. Flags must go to human review, never trigger automatic account blocks.

Isolation Forest scores are relative to the training data. Retrain when spending patterns shift (festivals, inflation) and monitor the flag rate.

Once enough confirmed labels exist, compare with a supervised model and measure precision and recall at the operating threshold.

6.11 Saving prediction history and preparing to deploy

The churn app keeps a per-session history in `st.session_state.history` and offers it as a CSV download. That history disappears when the browser tab is closed or the server restarts; Chapter 11 stores it in a database instead.

Pre-deployment checklist for every project

- [ ] `train_model.py` runs from a clean environment and recreates every artifact.
- [ ] `requirements.txt` pins the same library versions used for training.
- [ ] Model files are small enough for Git (< 100 MB) or fetched at start-up (Chapter 13).
- [ ] Paths are built from `Path(__file__).parent`, not the current working directory.
- [ ] The app starts with an informative error if the model is missing.
- [ ] Tests in `tests/` pass (`pytest -q`).

CHAPTER 7

Building UI for Different Types of ML Projects

07

Blueprints for 25 application types

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Map each ML problem type to the inputs, components, validation and outputs it needs.
- ✓ Choose page layouts that match how users work with each model.
- ✓ Apply extra safeguards for medical, financial and hiring use cases.

Use this chapter as a design reference before you write code. The summary table gives the essentials; the blueprint cards give all ten design decisions for each project type; the wireframes show recommended layouts for the major categories.

7.1 Project-to-UI mapping (summary)

Application type	User inputs	Key components	KPIs / charts
1. Binary classification	Feature values via form; optional CSV	form, number_input, selectbox, slider (threshold), metric, progress	ROC, PR, confusion matrix, threshold vs precision/recall
2. Multiclass classification	Features or file	form, metric, bar_chart of class probabilities, dataframe (top-k)	Per-class precision/recall, normalised confusion matrix
3. Regression	Numeric/categorical features	number_input with units, selectbox, metric	Actual vs predicted, residuals
4. Clustering	CSV/Excel upload, column mapping	file_uploader, selectbox mapping, scatter, tabs	2D/3D scatter, segment sizes, profile heatmap
5. Time-series forecasting	History CSV, date column, horizon	date_input range, slider horizon, radio method, toggle back-test	History + forecast + interval band, back-test MAPE
6. Recommendation system	User id or item selection, number of results	selectbox (searchable), slider k, columns of cards, image	Coverage, diversity, precision@k (offline)
7. NLP classification	Text area or TXT/CSV upload	text_area (max_chars), file_uploader, metric, bar_chart	Label distribution for batches
8. Image classification	Image upload or camera	file_uploader(type=images), camera_input, image, slider top-k	Probability bars
9. Object detection	Image/video upload, confidence and IoU sliders	file_uploader, slider, image with drawn boxes, dataframe	Counts per class
10. Image segmentation	Image upload, opacity slider	image (original + overlay), slider, columns	Area per class
11. Fraud detection	Transaction CSV or single form	file_uploader, slider threshold, dataframe, metric	Score histogram, precision/recall at threshold
12. Anomaly detection	CSV or streaming feed	line_chart with markers, slider, fragment(run_every)	Score distribution
13. Customer churn prediction	Customer features; batch CSV	form, tabs, ProgressColumn, download	Lift/gain chart, ROC
14. Credit risk prediction	Applicant data	form, number_input (currency), metric, expander (reasons)	Calibration curve, KS, Gini

Application type	User inputs	Key components	KPIs / charts
15. House price prediction	Property features	number_input, selectbox, toggle, map	Actual vs predicted, residuals
16. Sales prediction	History file, filters	multiselect store/product, date_input, line_chart	Forecast vs actual, MAPE by segment
17. Medical image classification (education only)	Image upload (de-identified)	file_uploader, image, bar_chart, warning banner	Per-class sensitivity/specificity
18. Document classification	PDF/DOCX/TXT upload (multiple)	file_uploader(multiple), dataframe, selectbox, text	Category distribution
19. Resume screening	Resume files	file_uploader(multiple), warning banner, dataframe	Category distribution, confidence
20. Semantic search	Query text, filters, k	text_input(type="search"), slider k, expander per result	Score distribution
21. Retrieval-augmented generation (RAG)	Chat messages; document upload	chat_input, chat_message, file_uploader, expander (sources), status	Latency, tokens, feedback
22. Generative AI application	Prompt, parameters	text_area, slider (temperature), write_stream, image	Usage/cost KPIs
23. Model comparison	Labeled data, model selection	multiselect models, dataframe (highlight), tabs	Overlaid ROC curves
24. Batch scoring	CSV/Excel upload	file_uploader, progress, dataframe, download_button	Distribution of scores
25. Data quality monitoring	Reference vs current data	file_uploader x2, dataframe, metric, fragment(run_every)	Missing %, drift per feature

Table 7.1: Summary mapping of ML project types to Streamlit UI

7.2 Blueprint cards for all 25 application types

1. Binary classification

Purpose	Predict one of two outcomes (yes/no) with a probability.
Required inputs	Feature values via form; optional CSV
UI components	form, number_input, selectbox, slider (threshold), metric, progress
Input validation	Ranges, categories, cross-field rules
Inference workflow	validate -> DataFrame[FEATURES] -> predict_proba -> apply threshold
Result presentation	Probability, label, explanation expander
Charts and KPIs	ROC, PR, confusion matrix, threshold vs precision/recall
Download options	Session history CSV
Page layout	Sidebar: threshold; main: 2-col form + 3 KPI result row
Considerations	Expose the threshold; show what it costs in false positives.

2. Multiclass classification

Purpose	Assign one of k classes.
Required inputs	Features or file
UI components	form, metric, bar_chart of class probabilities, dataframe (top-k)
Input validation	As above; unknown categories
Inference workflow	predict_proba -> argsort -> top-k
Result presentation	Top class, top-3 table, probability bars
Charts and KPIs	Per-class precision/recall, normalised confusion matrix
Download options	Predictions CSV
Page layout	Form left, probability chart right
Considerations	Macro vs weighted averages; rare classes.

3. Regression

Purpose	Predict a continuous number.
Required inputs	Numeric/categorical features
UI components	number_input with units, selectbox, metric
Input validation	Hard limits + training-range warnings
Inference workflow	predict -> format in units/currency
Result presentation	Point estimate, typical error (MAE), optional quantile interval
Charts and KPIs	Actual vs predicted, residuals
Download options	Estimate report
Page layout	Form + result card; performance section below
Considerations	Never present MAE bands as confidence intervals.

4. Clustering

Purpose	Group similar records into segments.
Required inputs	CSV/Excel upload, column mapping
UI components	file_uploader, selectbox mapping, scatter, tabs
Input validation	Required numeric columns, missing values
Inference workflow	saved scaler+model pipeline -> predict -> map names
Result presentation	Segment per row, profiles in original units
Charts and KPIs	2D/3D scatter, segment sizes, profile heatmap
Download options	Segmented CSV
Page layout	Sidebar: data + mapping; tabs: visualise / profiles / data
Considerations	Never refit in the app: cluster ids would change.

5. Time-series forecasting

Purpose	Forecast future values.
Required inputs	History CSV, date column, horizon
UI components	date_input range, slider horizon, radio method, toggle back-test
Input validation	>= 2 seasonal cycles, regular frequency, gaps
Inference workflow	prepare series -> (re)fit cached -> forecast
Result presentation	Forecast table + chart
Charts and KPIs	History + forecast + interval band, back-test MAPE
Download options	Forecast CSV
Page layout	Sidebar settings; full-width chart; table below
Considerations	Intervals only when the model provides them.

6. Recommendation system

Purpose	Suggest items for a user or item.
Required inputs	User id or item selection, number of results
UI components	selectbox (searchable), slider k, columns of cards, image
Input validation	Known user/item ids; cold-start handling
Inference workflow	lookup embeddings/similarities -> top-k -> filter seen items
Result presentation	Ranked item cards with reasons ('because you liked...')
Charts and KPIs	Coverage, diversity, precision@k (offline)
Download options	Recommendation list CSV
Page layout	Search bar top; grid of cards
Considerations	Explain why; avoid filter bubbles; respect privacy.

7. NLP classification

Purpose	Label text (sentiment, topic, intent).
Required inputs	Text area or TXT/CSV upload
UI components	text_area (max_chars), file_uploader, metric, bar_chart
Input validation	Length limits, encoding, empty text
Inference workflow	clean -> vectorise (in pipeline) -> predict_proba
Result presentation	Label, confidence, cleaned text
Charts and KPIs	Label distribution for batches
Download options	Results CSV
Page layout	Tabs: single / batch
Considerations	Confidence thresholds; language mismatch.

8. Image classification	
Purpose	Assign a label to an image.
Required inputs	Image upload or camera
UI components	file_uploader(type=images), camera_input, image, slider top-k
Input validation	Format, size, pixel count, corrupted files
Inference workflow	decode -> shared preprocess -> add batch dim -> predict
Result presentation	Image preview, top-k probabilities
Charts and KPIs	Probability bars
Download options	Results CSV
Page layout	Image left, results right
Considerations	Show what the model sees; warn on low confidence.

9. Object detection	
Purpose	Locate and label objects with boxes.
Required inputs	Image/video upload, confidence and IoU sliders
UI components	file_uploader, slider, image with drawn boxes, dataframe
Input validation	Image size, supported formats
Inference workflow	preprocess -> detector -> NMS -> draw boxes (PIL/OpenCV)
Result presentation	Annotated image, detections table
Charts and KPIs	Counts per class
Download options	Annotated image, JSON detections
Page layout	Controls sidebar; annotated image main
Considerations	GPU memory; letterbox resizing must match training.

10. Image segmentation	
Purpose	Classify every pixel.
Required inputs	Image upload, opacity slider
UI components	image (original + overlay), slider, columns
Input validation	Same as detection
Inference workflow	preprocess -> model -> argmax mask -> colourise -> blend
Result presentation	Mask overlay, class area %
Charts and KPIs	Area per class
Download options	Mask PNG
Page layout	Side-by-side original vs overlay
Considerations	Resize masks back to the original size.

11. Fraud detection

Purpose	Flag suspicious transactions.
Required inputs	Transaction CSV or single form
UI components	file_uploader, slider threshold, dataframe, metric
Input validation	Schema, amount ranges, timestamps
Inference workflow	score -> threshold -> rank
Result presentation	Flagged list, reason codes
Charts and KPIs	Score histogram, precision/recall at threshold
Download options	Flagged CSV
Page layout	KPIs top; histogram; flagged table
Considerations	Human review; never auto-block.

12. Anomaly detection

Purpose	Find unusual records or sensor readings.
Required inputs	CSV or streaming feed
UI components	line_chart with markers, slider, fragment(run_every)
Input validation	Timestamps, units
Inference workflow	score_samples -> percentile threshold
Result presentation	Anomalies on a timeline
Charts and KPIs	Score distribution
Download options	Anomaly CSV
Page layout	Timeline chart + table
Considerations	Thresholds drift with seasonality.

13. Customer churn prediction

Purpose	Rank customers by cancellation risk.
Required inputs	Customer features; batch CSV
UI components	form, tabs, ProgressColumn, download
Input validation	Ranges, business rules
Inference workflow	pipeline.predict_proba
Result presentation	Risk score, bands, explanation
Charts and KPIs	Lift/gain chart, ROC
Download options	Scored list
Page layout	Single / batch / performance tabs
Considerations	Combine risk with customer value for prioritisation.

14. Credit risk prediction

Purpose	Estimate default probability.
Required inputs	Applicant data
UI components	form, number_input (currency), metric, expander (reasons)
Input validation	Strict validation, affordability rules
Inference workflow	pipeline -> PD -> risk grade
Result presentation	Grade, PD, adverse-action reasons
Charts and KPIs	Calibration curve, KS, Gini
Download options	Decision memo PDF
Page layout	Centered form; result card
Considerations	HIGH IMPACT: regulation, fairness, explainability, human review.

15. House price prediction

Purpose	Estimate property value.
Required inputs	Property features
UI components	number_input, selectbox, toggle, map
Input validation	Hard limits + range warnings
Inference workflow	pipeline -> price -> currency format
Result presentation	Price, price per sq ft, typical error
Charts and KPIs	Actual vs predicted, residuals
Download options	Valuation report
Page layout	Form left, result right
Considerations	Location leakage; market drift.

16. Sales prediction

Purpose	Forecast sales by product/store.
Required inputs	History file, filters
UI components	multiselect store/product, date_input, line_chart
Input validation	Date coverage
Inference workflow	group -> forecast per series
Result presentation	Forecast by segment
Charts and KPIs	Forecast vs actual, MAPE by segment
Download options	Forecast CSV/Excel
Page layout	Filters sidebar; chart + table
Considerations	Holidays, promotions, stock-outs distort history.

17. Medical image classification (education only)

Purpose	Demonstrate a classifier on medical images.
Required inputs	Image upload (de-identified)
UI components	file_uploader, image, bar_chart, warning banner
Input validation	Format, DICOM handling, size
Inference workflow	preprocess -> model -> probabilities
Result presentation	Probabilities with strong disclaimer
Charts and KPIs	Per-class sensitivity/specificity
Download options	None by default
Page layout	Disclaimer at top; image + probabilities
Considerations	HIGH IMPACT: not a medical device; no diagnosis; privacy.

18. Document classification

Purpose	Route documents by type.
Required inputs	PDF/DOCX/TXT upload (multiple)
UI components	file_uploader(multiple), dataframe, selectbox, text
Input validation	Encrypted/scanned PDFs, size
Inference workflow	extract text -> clean -> classify
Result presentation	Category + confidence per file
Charts and KPIs	Category distribution
Download options	Routing CSV
Page layout	Upload top; results table; inspector
Considerations	OCR needed for scans; redact PII before display.

19. Resume screening

Purpose	Route resumes to job families.
Required inputs	Resume files
UI components	file_uploader(multiple), warning banner, dataframe
Input validation	File types, text length
Inference workflow	extract -> redact PII -> classify
Result presentation	Top categories (routing only)
Charts and KPIs	Category distribution, confidence
Download options	Results CSV
Page layout	Warning banner; results; inspector
Considerations	HIGH IMPACT: bias, legal compliance, never auto-reject.

20. Semantic search

Purpose	Find documents by meaning.
Required inputs	Query text, filters, k
UI components	text_input(type="search"), slider k, expander per result
Input validation	Query length
Inference workflow	embed query -> vector index (FAISS) -> top-k
Result presentation	Ranked passages with scores and sources
Charts and KPIs	Score distribution
Download options	Results JSON
Page layout	Search bar; result list
Considerations	Cache the embedding model with cache_resource.

21. Retrieval-augmented generation (RAG)

Purpose	Answer questions from documents.
Required inputs	Chat messages; document upload
UI components	chat_input, chat_message, file_uploader, expander (sources), status
Input validation	File types, prompt length
Inference workflow	retrieve -> build prompt -> LLM -> cite sources
Result presentation	Answer with citations
Charts and KPIs	Latency, tokens, feedback
Download options	Chat transcript
Page layout	Chat main; documents sidebar
Considerations	Hallucination; prompt injection from documents; secret API keys.

22. Generative AI application

Purpose	Generate text/images from prompts.
Required inputs	Prompt, parameters
UI components	text_area, slider (temperature), write_stream, image
Input validation	Prompt length, content policy
Inference workflow	call model API with streaming
Result presentation	Streamed output
Charts and KPIs	Usage/cost KPIs
Download options	Output text/image
Page layout	Prompt left/top; output right/below
Considerations	Rate limits, cost control, content moderation.

23. Model comparison

Purpose	Compare candidate models on the same data.
Required inputs	Labeled data, model selection
UI components	multiselect models, dataframe (highlight), tabs
Input validation	Same features/target for all
Inference workflow	evaluate each -> table
Result presentation	Metric table, best per column
Charts and KPIs	Overlaid ROC curves
Download options	Comparison CSV
Page layout	Selector top; table; charts
Considerations	Use identical held-out data; consider latency and size too.

24. Batch scoring

Purpose	Score large files.
Required inputs	CSV/Excel upload
UI components	file_uploader, progress, dataframe, download_button
Input validation	Schema, dtypes, row limits
Inference workflow	validate -> chunked predict -> append columns
Result presentation	Summary KPIs, preview
Charts and KPIs	Distribution of scores
Download options	Full scored file
Page layout	Stepper: upload / check / score / download
Considerations	Memory; offload very large jobs to a queue.

25. Data quality monitoring

Purpose	Track data health and drift.
Required inputs	Reference vs current data
UI components	file_uploader x2, dataframe, metric, fragment(run_every)
Input validation	Matching schemas
Inference workflow	profile -> compare (PSI, KS) -> alert
Result presentation	Quality scorecard
Charts and KPIs	Missing %, drift per feature
Download options	Quality report
Page layout	KPIs; drift table; charts
Considerations	Alert thresholds agreed with stakeholders.

7.3 Wireframes for the major categories

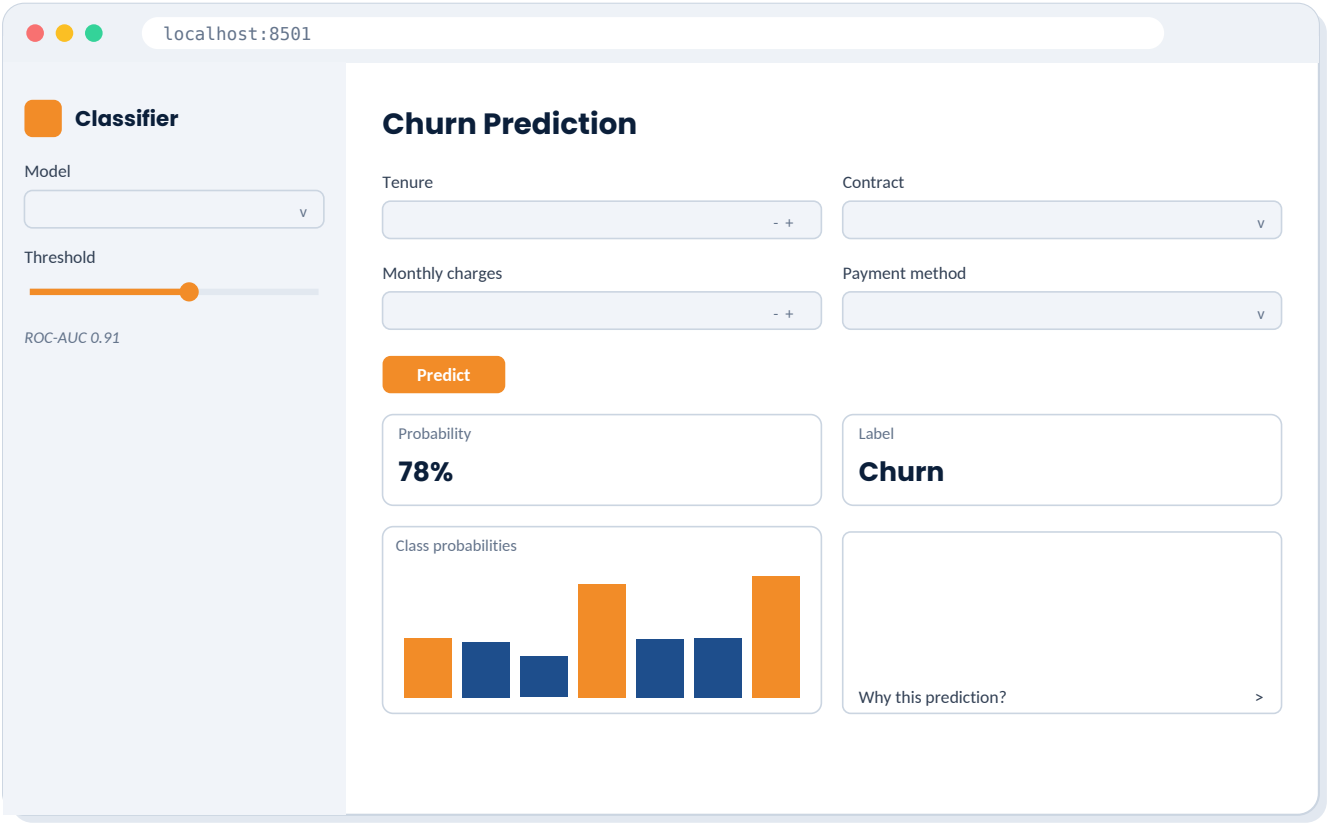


Figure 7.1: Recommended layout: Binary / multiclass classification

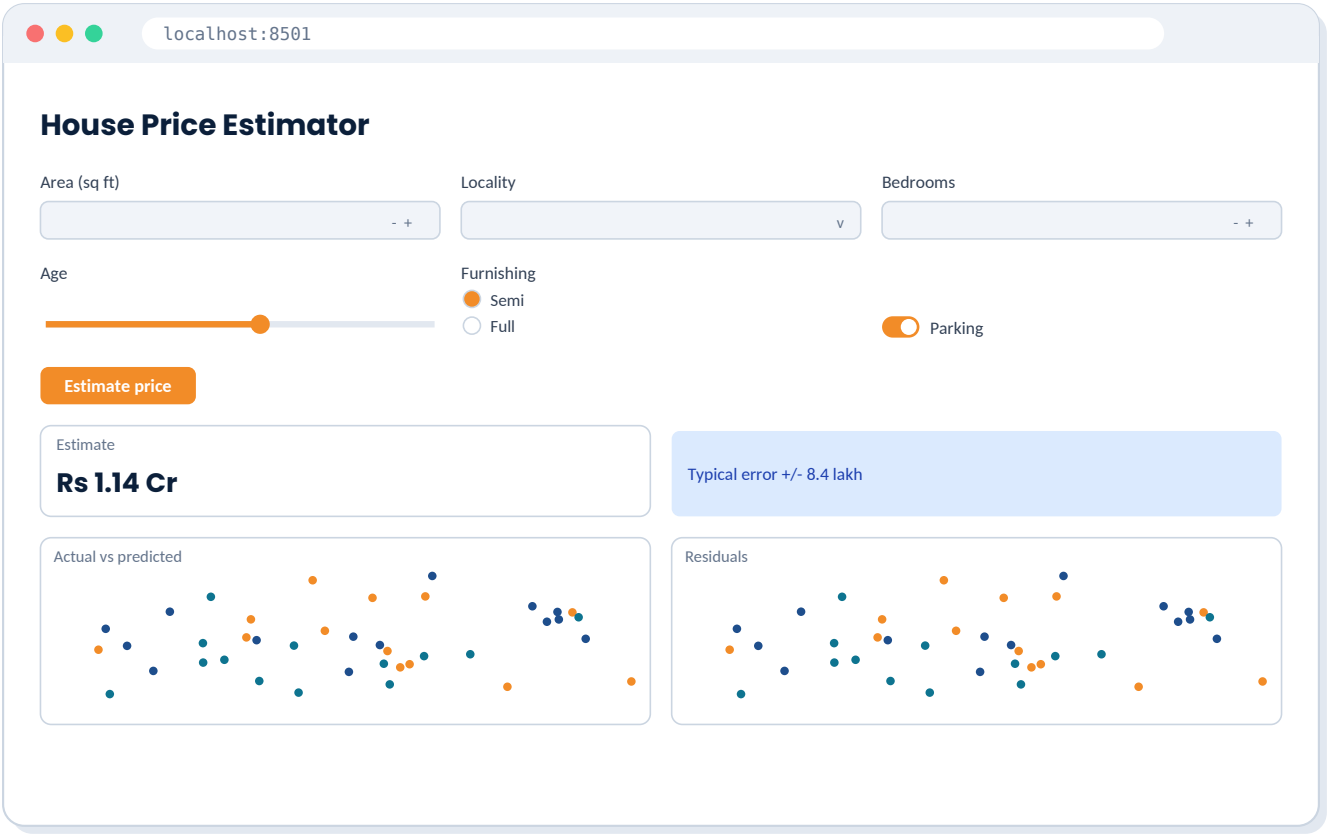


Figure 7.2: Recommended layout: Regression

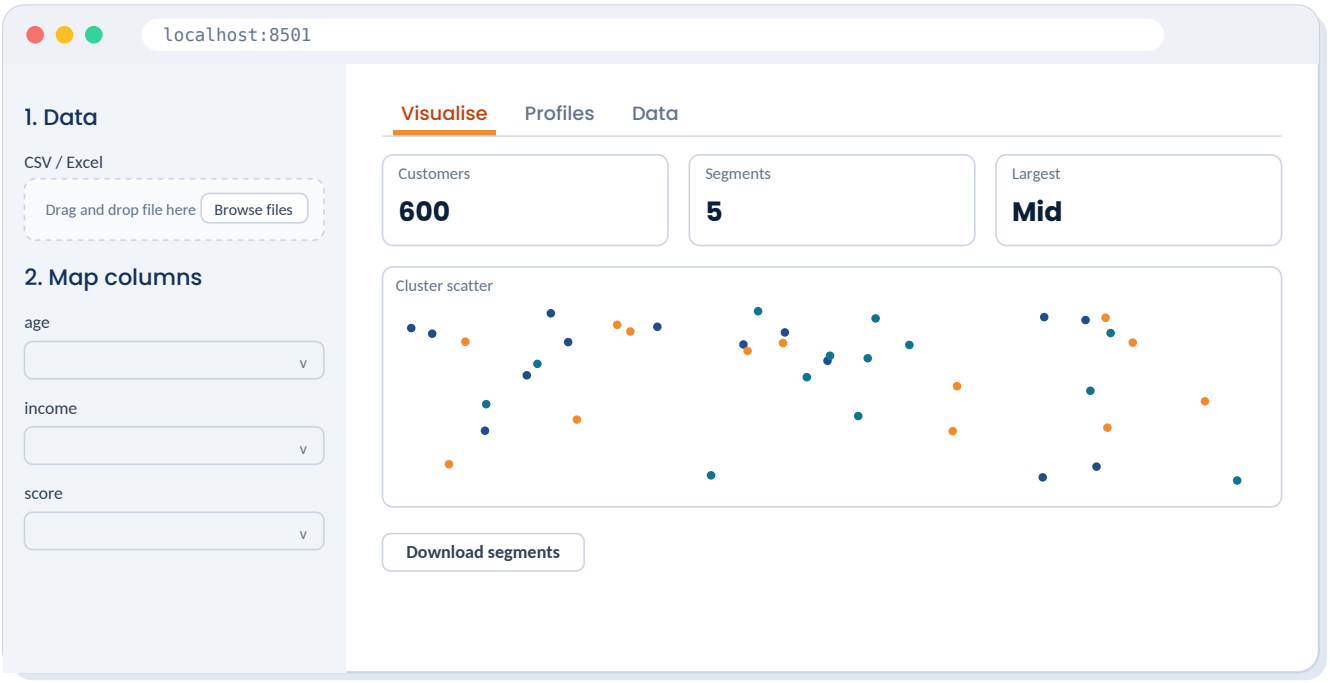


Figure 7.3: Recommended layout: Clustering / segmentation

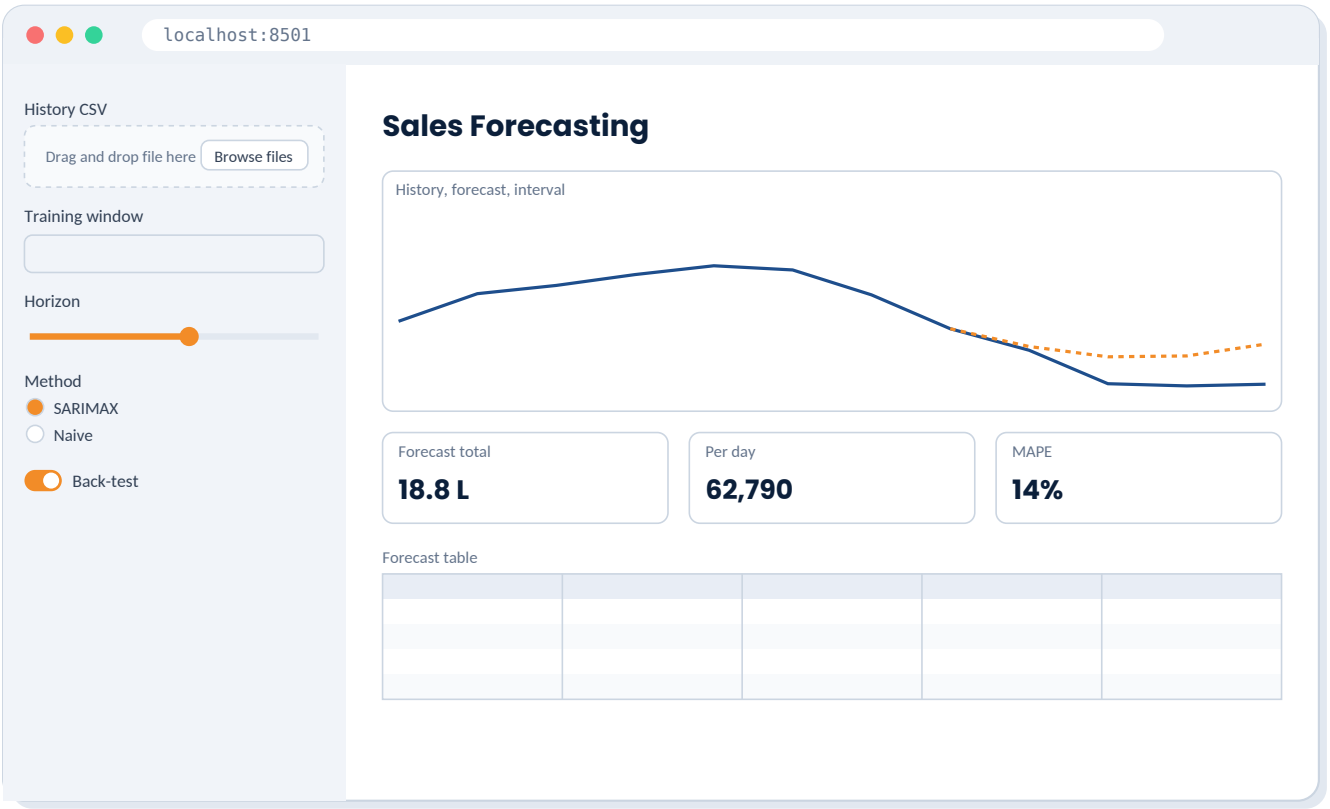


Figure 7.4: Recommended layout: Forecasting

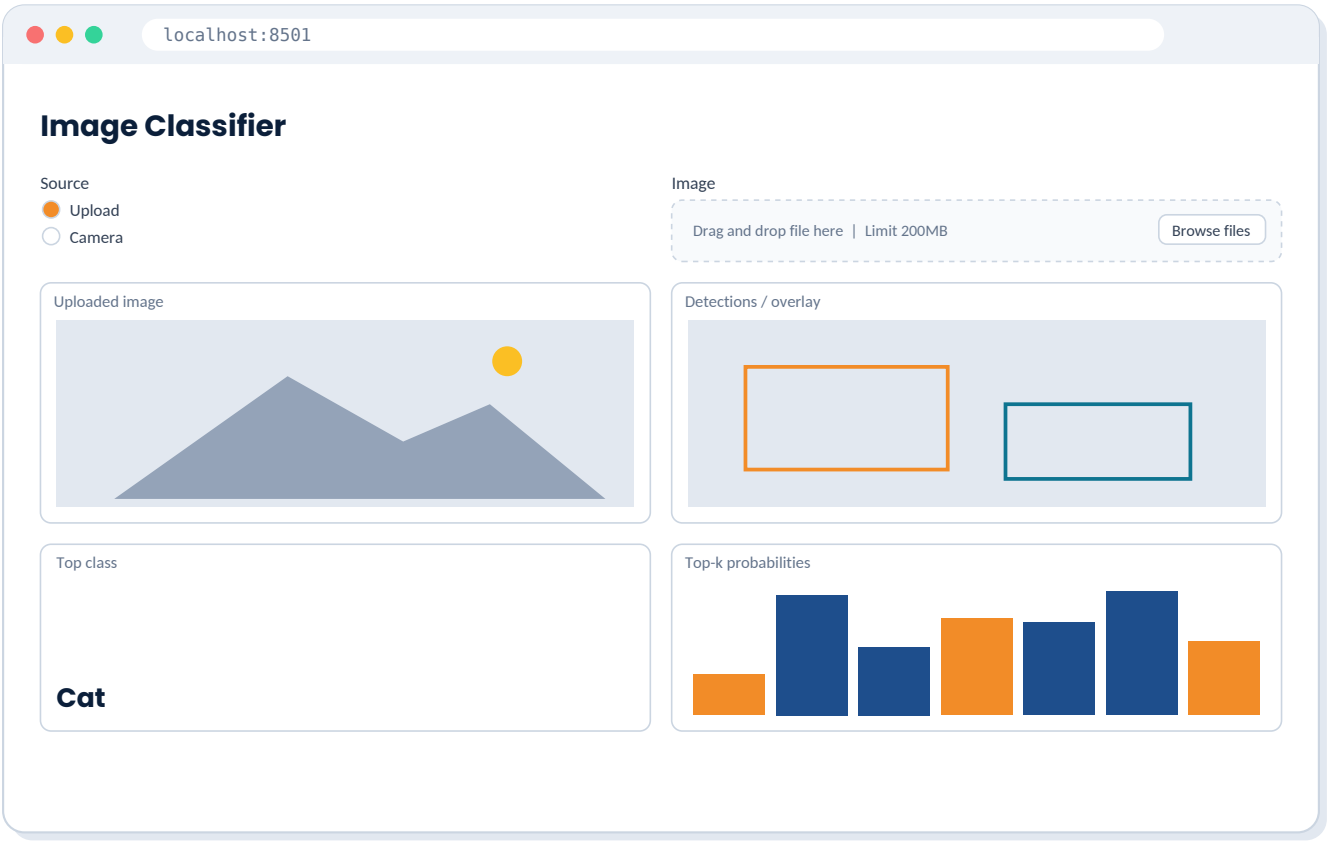


Figure 7.5: Recommended layout: Image classification / detection / segmentation

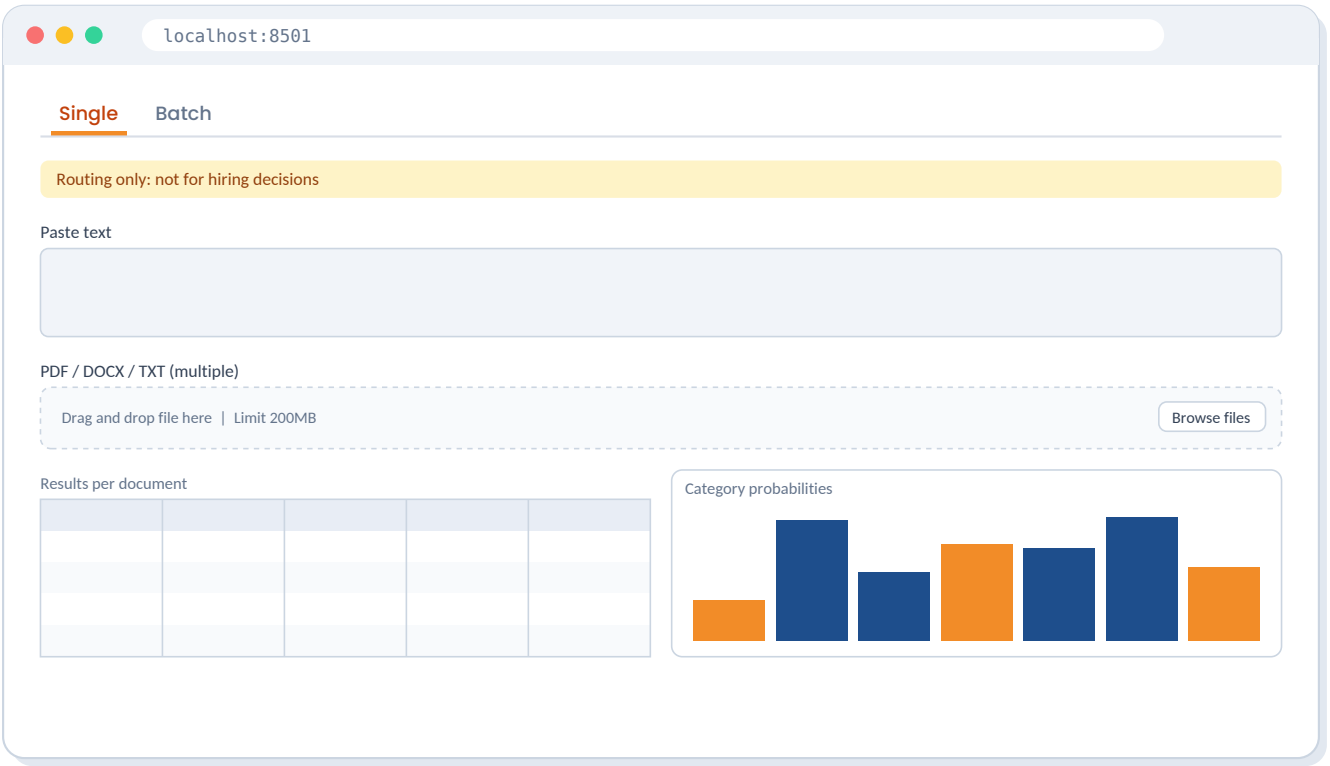


Figure 7.6: Recommended layout: NLP / document / resume

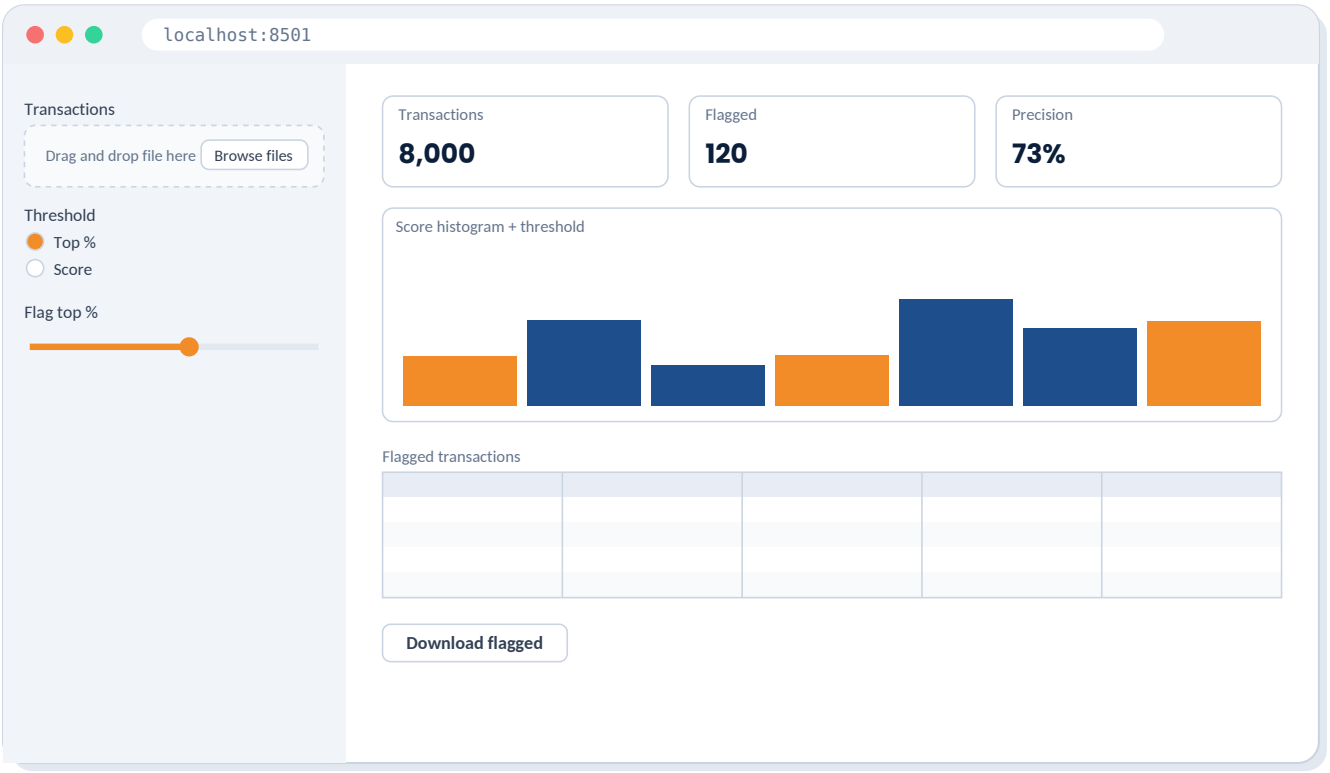


Figure 7.7: Recommended layout: Fraud / anomaly detection

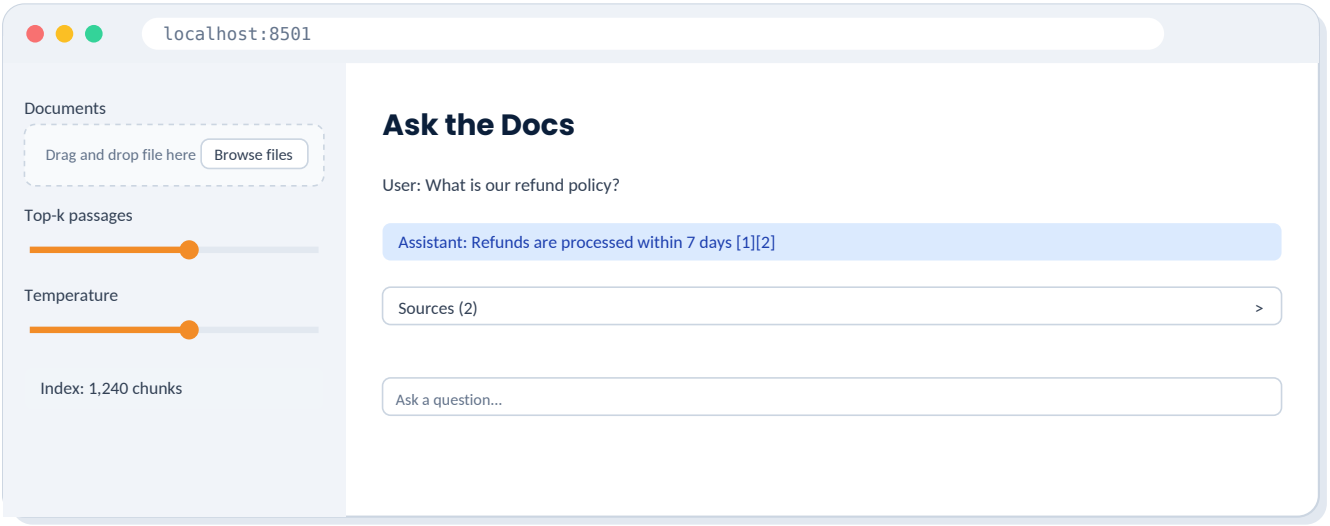


Figure 7.8: Recommended layout: Semantic search / RAG / GenAI

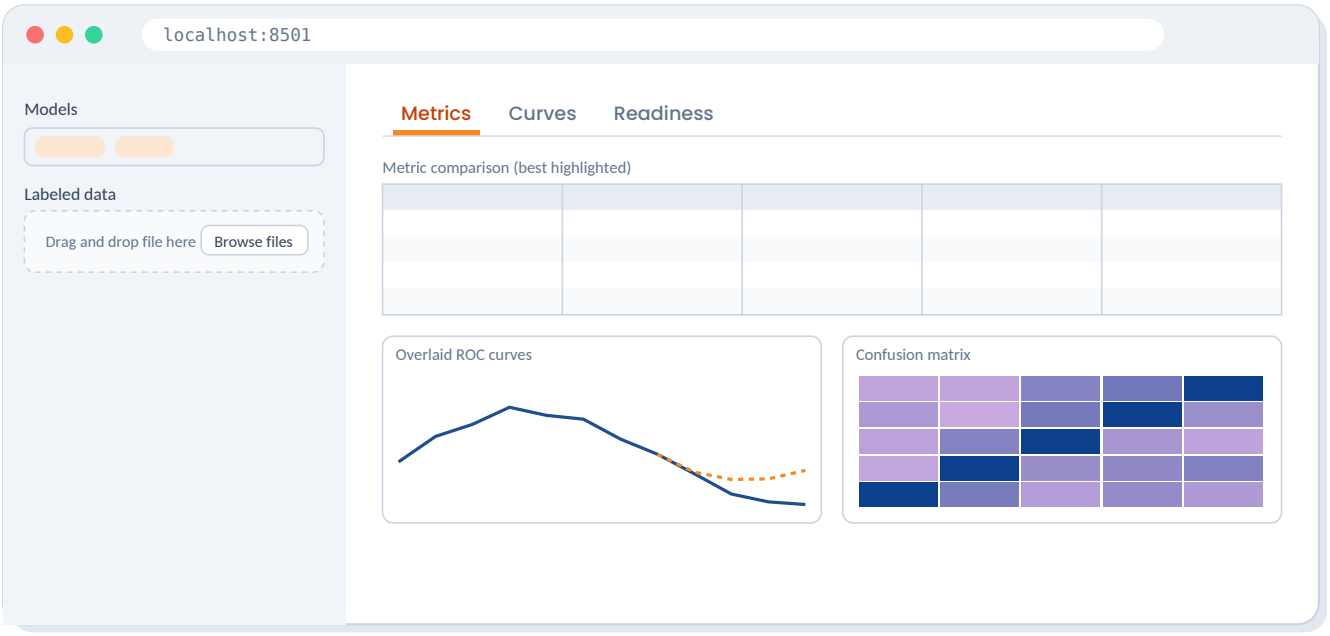


Figure 7.9: Recommended layout: Model comparison / evaluation

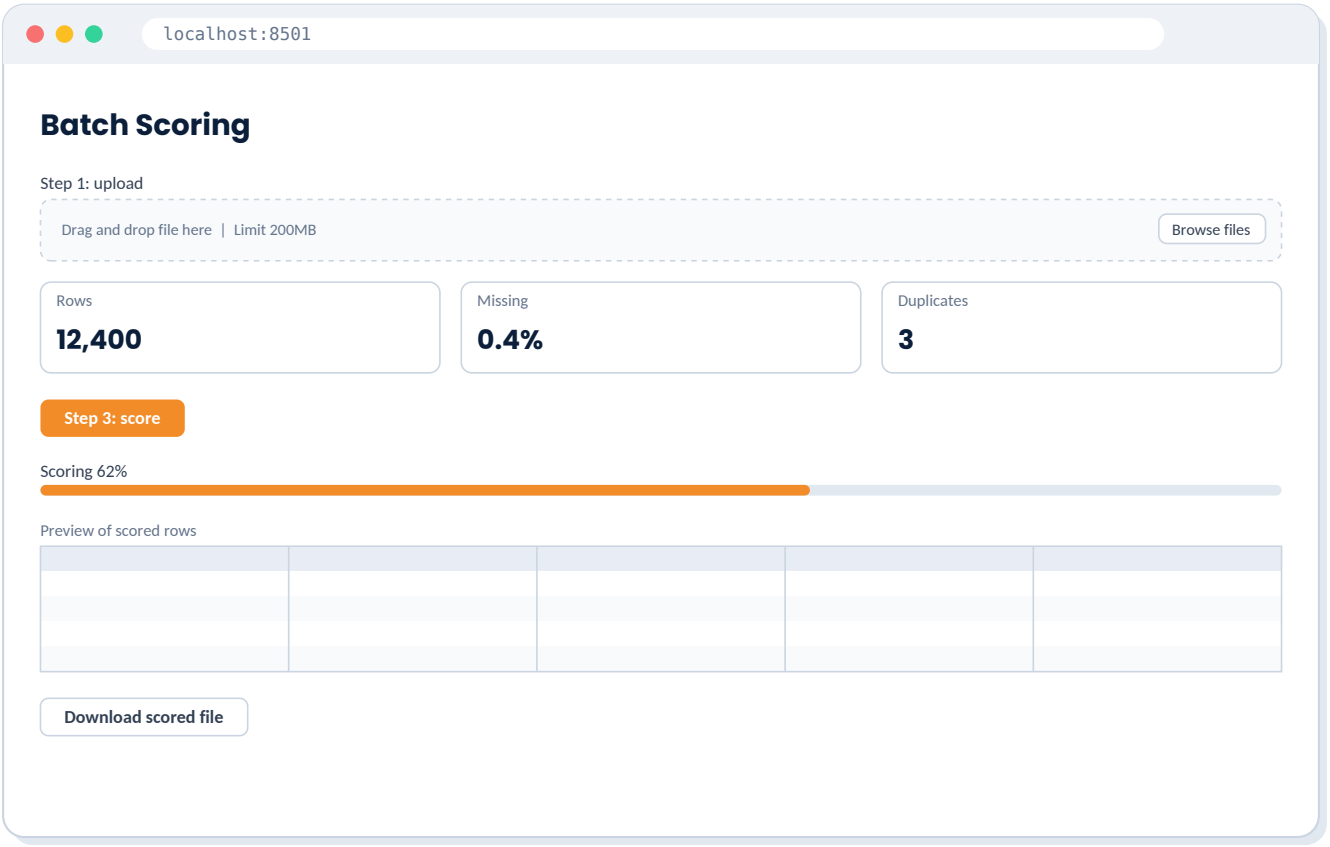


Figure 7.10: Recommended layout: Batch scoring / data quality monitoring

7.4 High-impact applications: medical, financial and hiring

RESPONSIBLE ML: Mandatory considerations

Validation: offline accuracy is not enough. High-impact models need evaluation on representative, recent data, subgroup analysis and, where required, formal regulatory approval.

Privacy: collect the minimum data, avoid logging personal data, encrypt storage, and follow applicable laws (for India, the Digital Personal Data Protection Act 2023; in Europe, GDPR).

Fairness: check error rates across gender, age, region and other protected groups. Remove or audit proxy features (postcode, college name).

Human oversight: the app should support a decision-maker, not replace them. Show reasons, allow overrides, and record who decided.

Transparency: state clearly on screen what the model does, what data it was trained on, its known limitations and who to contact.

Limits of automation: never auto-reject a loan, a candidate or a patient based only on a model score.

high_impact_banner.py — PYTHON

```python
import streamlit as st

st.warning(
    "This tool is for **education only**. It is not a medical device and must not be "
    "used for diagnosis or treatment decisions. Consult a qualified clinician.",
    icon=":material/medical_information:",
)
consent = st.checkbox("I understand that this image must be de-identified and that "
                      "results are not medical advice.")
image = st.file_uploader("De-identified image", type=["png", "jpg"], disabled=not consent)
```

CHAPTER 8

Dataset Upload and Batch Prediction

08

Score thousands of rows safely, with validation and downloads

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Accept CSV and Excel files and validate extensions, columns and types.
- ✓ Detect missing values, duplicates and out-of-range data before scoring.
- ✓ Apply the saved pipeline to many rows with progress feedback.
- ✓ Return predictions appended to the original data as a download.
- ✓ Keep memory use under control for large files.

8.1 Training data versus inference data

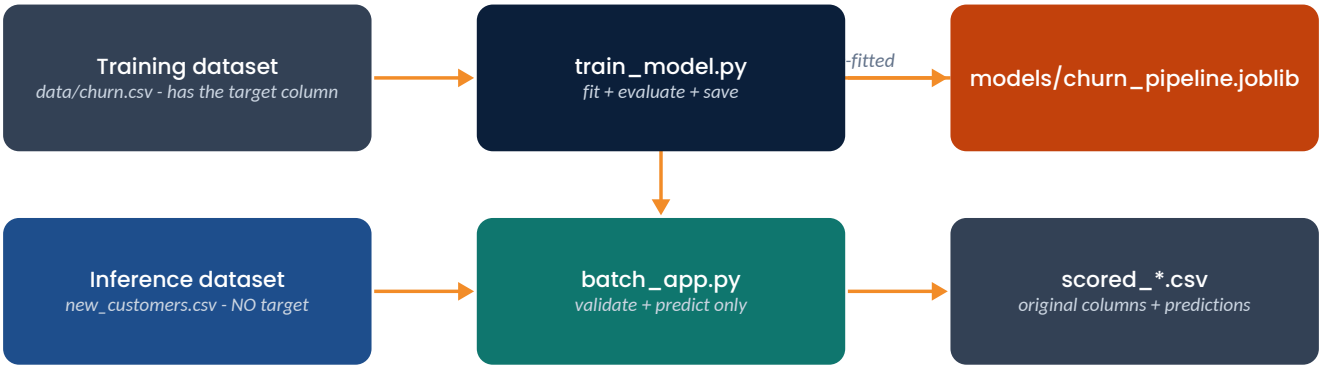


Figure 8.1: Training and inference use different files and different code paths

WARNING

A batch-scoring app must **not** retrain the model on uploaded data. Uploaded files are unlabeled, may be unrepresentative, and silently changing the model would make predictions irreproducible. Retraining is a separate, deliberate workflow with its own evaluation.

8.2 The complete batch prediction app

The app below covers all eighteen topics of this chapter. Save it as `churn_app/batch_app.py` (it reuses `schema.py`, `utils/model_io.py` and `utils/validation.py` from Chapter 6) and run `streamlit run batch_app.py`.

churn_app/batch_app.py PYTHON

```python
"""Batch scoring app for the churn model.  Run: streamlit run batch_app.py

Training data  (data/churn.csv)          -> used ONLY by train_model.py
Inference data (e.g. new_customers.csv)  -> uploaded here; no target column needed
"""
import io
from pathlib import Path

import numpy as np
import pandas as pd
import streamlit as st

from schema import FEATURES, ID_COLUMN, NUMERIC_FEATURES, POSITIVE_CLASS
from utils.model_io import ModelLoadError, load_trusted_model
from utils.validation import validate_dataframe

ROOT = Path(__file__).parent
ALLOWED = ["csv", "xlsx"]
MAX_ROWS = 200_000
CHUNK = 5_000

st.set_page_config(page_title="Batch Churn Scoring", layout="wide")


@st.cache_resource
def get_model():
    return load_trusted_model(ROOT / "models" / "churn_pipeline.joblib",
                              ROOT / "models" / "churn_metadata.json")


@st.cache_data(show_spinner="Reading file...")
def read_upload(name: str, data: bytes) -> pd.DataFrame:
    """Cached on (name, bytes) so re-runs do not re-parse the same file."""
    if name.lower().endswith(".xlsx"):
        return pd.read_excel(io.BytesIO(data))           # needs openpyxl
    for encoding in ("utf-8", "utf-8-sig", "latin-1"):
        try:
            return pd.read_csv(io.BytesIO(data), encoding=encoding)
        except UnicodeDecodeError:
            continue
    raise ValueError("Could not decode the file. Save it as UTF-8 CSV and retry.")


def score_in_chunks(model, X: pd.DataFrame, pos_idx: int) -> np.ndarray:
    """Predict in chunks so we can show progress and keep memory use flat."""
    bar = st.progress(0.0, text="Scoring...")
    out = np.empty(len(X))
    for start in range(0, len(X), CHUNK):
        stop = min(start + CHUNK, len(X))
        out[start:stop] = model.predict_proba(X.iloc[start:stop])[:, pos_idx]
        bar.progress(stop / len(X), text=f"Scored {stop:,} of {len(X):,} rows")
    bar.empty()
    return out


try:
    model, meta, _ = get_model()
except ModelLoadError as exc:
    st.error(str(exc))
    st.stop()
pos_idx = meta["classes"].index(POSITIVE_CLASS)

st.title("Batch Churn Scoring")
st.markdown("**Step 1.** Upload new customers (no `churn` column required). "
            "Required columns:")
st.code(", ".join(FEATURES), language=None)
upload = st.file_uploader("Customer file", type=ALLOWED)
if upload is None:
```

churn_app/batch_app.py PYTHON

```
    st.stop()

# ---- step 2: read + validate
try:
    df = read_upload(upload.name, upload.getvalue())
except Exception as exc:
    st.error(f"Could not read {upload.name}: {exc}")
    st.stop()
if len(df) > MAX_ROWS:
    st.error(f"File has {len(df):,} rows; the limit is {MAX_ROWS:,}. Split it first.")
    st.stop()

st.markdown("**Step 2.** Check the data")
c1, c2, c3, c4 = st.columns(4)
c1.metric("Rows", f"{len(df):,}")
c2.metric("Columns", df.shape[1])
c3.metric("Missing cells", int(df.isna().sum().sum()))
c4.metric("Duplicate rows", int(df.duplicated().sum()))
st.dataframe(df.head(10), width="stretch")

with st.expander("Column-level quality report"):
    report = pd.DataFrame({"dtype": df.dtypes.astype(str), "missing": df.isna().sum(),
                           "unique": df.nunique()})
    report["required"] = report.index.isin(FEATURES)
    st.dataframe(report, width="stretch")

errors, warns = validate_dataframe(df)
if "churn" in df.columns:
    warns.append("A 'churn' column is present. It is ignored here; to measure accuracy "
                 "use the evaluation dashboard instead.")
if ID_COLUMN in df.columns and df[ID_COLUMN].duplicated().any():
    warns.append(f"Duplicate {ID_COLUMN} values found; check for repeated customers.")
for w in warns:
    st.warning(w)
for e in errors:
    st.error(e)
if errors:
    st.stop()

# ---- step 3: predict
st.markdown("**Step 3.** Score")
threshold = st.slider("Decision threshold", 0.05, 0.95, 0.5, 0.05)
if not st.button("Run batch prediction", type="primary"):
    st.stop()

X = df[FEATURES].copy()
for col in NUMERIC_FEATURES:
    X[col] = pd.to_numeric(X[col], errors="coerce")       # pipeline imputes NaNs
proba = score_in_chunks(model, X, pos_idx)
result = df.copy()
result["churn_probability"] = proba.round(4)
result["churn_prediction"] = np.where(proba >= threshold, "Yes", "No")
result["risk_band"] = pd.cut(proba, [0, 0.3, 0.6, 1.0], labels=["Low", "Medium", "High"],
                             include_lowest=True)

# ---- step 4: summarise + download
st.markdown("**Step 4.** Results")
k1, k2, k3 = st.columns(3)
k1.metric("Predicted churners", int((result["churn_prediction"] == "Yes").sum()))
k2.metric("Churn rate (predicted)", f"{(proba >= threshold).mean():.1%}")
k3.metric("Mean probability", f"{proba.mean():.3f}")
left, right = st.columns(2)
left.bar_chart(result["risk_band"].value_counts().sort_index())
right.dataframe(result.groupby("contract", observed=True)["churn_probability"]
                .describe()[["count", "mean", "min", "max"]].round(3), width="stretch")
st.dataframe(result.head(500), width="stretch")
st.download_button("Download predictions (CSV)", result.to_csv(index=False).encode("utf-8"),
                   file_name=f"scored_{Path(upload.name).stem}.csv", mime="text/csv",
```

churn_app/batch_app.py PYTHON

```
                type="primary", on_click="ignore")
```

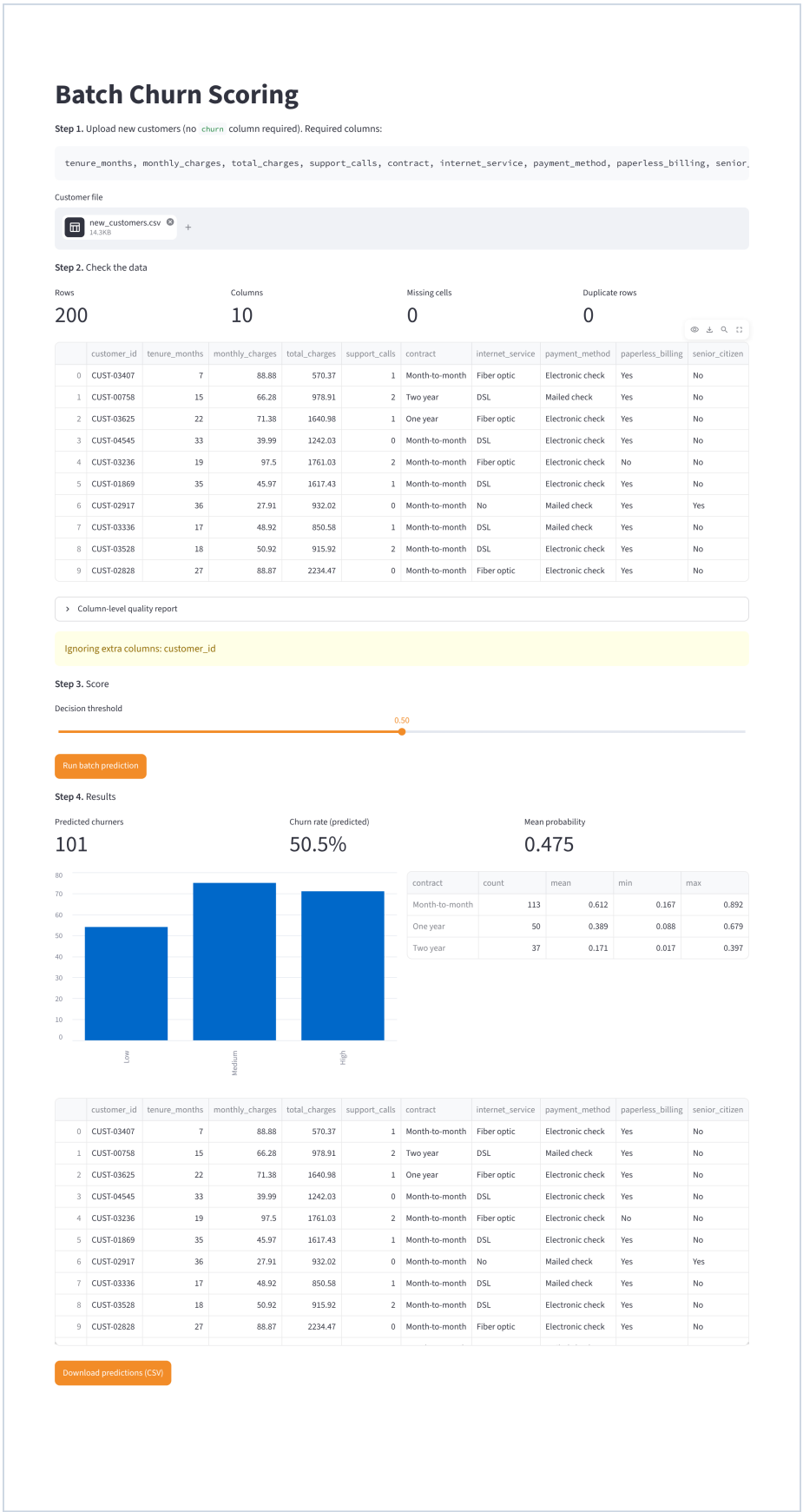


Figure 8.2: Batch scoring: data checks, risk bands, summary by contract type and the download button

8.3 How each step works

#	Topic	Where in batch_app.py / how
1	File uploader	`st.file_uploader("Customer file", type=["csv", "xlsx"])`
2	Extension validation	`type=` filters in the browser and on the server; `read_upload` branches on `.xlsx`
3	Reading CSV	`pd.read_csv(io.BytesIO(data), encoding=...)` with a fallback chain utf-8 -> utf-8-sig (Excel BOM) -> latin-1
4	Reading Excel	`pd.read_excel(...)` (needs `openpyxl`); use `sheet_name=` for workbooks with several sheets
5	Dataset preview	`st.dataframe(df.head(10))` + four KPI metrics
6	Column validation	`validate_dataframe` reports missing required columns as errors
7	Missing values	Counted and reported; the pipeline's imputers handle them
8	Duplicates	`df.duplicated().sum()` and duplicate customer ids
9	Data types	`pd.to_numeric(errors="coerce")` finds non-numeric entries
10	Feature selection	`df[FEATURES]` sends only model columns, in training order; extra columns are kept for the output but ignored
11	Saved preprocessing	Inside the pipeline: no manual scaling or encoding in the app
12	Batch prediction	`score_in_chunks` calls `predict_proba` on 5,000-row slices
13	Result table	Original data + probability, label and risk band
14	Download	`st.download_button(... on_click="ignore")` so clicking does not rerun and clear the results
15	Large datasets	`MAX_ROWS` guard, chunking, show only 500 rows
16	Progress	`st.progress` updated after each chunk
17	Memory	Cached parse, no copies of the full frame beyond one result
18	Error reporting	Errors stop the flow with `st.error`; warnings let the user continue

8.4 Handling large files and memory

- **Upload limit** — The default `server.maxUploadSize` is 200 MB. Lower it in `.streamlit/config.toml` for public apps, or set `max_upload_size=` on the widget.
- **Memory rule of thumb** — A CSV in pandas often needs 3-10x its file size in RAM. A 100 MB file can need 1 GB; free hosting tiers may have less.
- **Read only what you need** — `pd.read_csv(f, usecols=FEATURES + [ID_COLUMN], dtype={...})` reduces memory dramatically.
- **Chunked reading** — For very large CSVs, `pd.read_csv(f, chunksize=50_000)` yields DataFrames you can score and append to an output buffer one at a time.
- **Do not cache huge objects per user** — `st.cache_data` stores a copy per unique input; set `max_entries` or `ttl`.
- **Offload** — For millions of rows, run scoring as a background job (a scheduled script, Airflow, a cloud batch service) and let Streamlit show the status and results.

churn_app/utils/large_files.py PYTHON

```python
import io

import pandas as pd

from schema import FEATURES


def score_large_csv(file_bytes: bytes, model, pos_idx: int, chunksize: int = 50_000):
    """Stream a large CSV through the model without loading it all at once."""
    out = io.StringIO()
    reader = pd.read_csv(io.BytesIO(file_bytes), chunksize=chunksize)
    for i, chunk in enumerate(reader):
        chunk["churn_probability"] = model.predict_proba(chunk[FEATURES])[:, pos_idx]
        chunk.to_csv(out, index=False, header=(i == 0))
    return out.getvalue().encode("utf-8")
```

8.5 Error reporting that users understand

Technical problem	Message to show
`KeyError: 'contract'`	Missing required columns: contract
`ValueError: could not convert string to float: 'N/A'`	Column 'monthly_charges' has 12 non-numeric value(s)
`UnicodeDecodeError`	Could not decode the file. Save it as UTF-8 CSV
`ParserError: Error tokenizing data`	The CSV has inconsistent columns (check separators and quotes)
`MemoryError`	The file is too large to process here. Split it or use the batch service

BEST PRACTICE

Collect **all** validation problems and show them together. Users fix their file once instead of uploading it five times to discover five problems.

CHAPTER 9

State Management and Performance

09

Session state, caching and fast, reliable apps

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Predict exactly what reruns and when.
- ✓ Use widget keys and st.session_state to remember selections and history.
- ✓ Choose correctly between st.cache_data and st.cache_resource.
- ✓ Invalidate caches safely and control memory use.
- ✓ Handle long-running and expensive work without freezing the app.

9.1 Rerun behaviour, precisely

Event	What reruns
Widget value changes (outside a form)	The whole script (or only the fragment containing the widget)
Button click	Whole script; the button returns True for that one run
Form submit	Whole script once, with all form values updated together
Widget inside a form changes	Nothing until submit
`st.rerun()` called	Whole script immediately (or `scope="fragment"`)
Source file saved	Whole script (if run-on-save is enabled or user clicks Rerun)
`@st.fragment(run_every=...)` timer	Only that fragment
Download button with `on_click="ignore"`	Nothing

Callbacks (`on_click`, `on_change`) execute **before** the rerun starts, which is why they are the right place to modify `st.session_state` in response to an interaction.

9.2 Widget keys and st.session_state

`st.session_state` is a dictionary-like object that lives on the server, one per browser tab. Use attribute or item syntax: `st.session_state.history` or `st.session_state["history"]`.

state_demos/remember_inputs.py — PYTHON

```python
"""Remember user selections across reruns and pages.  Run: streamlit run remember_inputs.py"""
import streamlit as st

# 1) give keyed widgets their starting value through session state ONLY.
#    Passing value=... as well would trigger a "set via Session State API" warning
#    once a callback (like reset below) changes the key.
st.session_state.setdefault("model", "Random forest")
st.session_state.setdefault("threshold", 0.5)
model = st.selectbox("Model", ["Random forest", "Logistic regression"], key="model")
threshold = st.slider("Threshold", 0.0, 1.0, key="threshold")
```

state_demos/remember_inputs.py PYTHON

```python
# 2) widget state is deleted when the widget is not drawn on a run (e.g. another page).
#    To keep a value beyond that, copy it to a separate, non-widget key:
st.session_state["saved_threshold"] = threshold

# 3) initialise keys once with setdefault (or `if key not in st.session_state`)
st.session_state.setdefault("runs", 0)
st.session_state.runs += 1

st.write(f"Model: {st.session_state.model} | threshold: {st.session_state.threshold}")
st.caption(f"This session has rerun {st.session_state.runs} times.")


def reset():
    """Callbacks run BEFORE the rerun, so widgets pick up the new values."""
    st.session_state.model = "Random forest"
    st.session_state.threshold = 0.5


st.button("Reset to defaults", on_click=reset)
```

Rule	Consequence
A widget with `key="k"` stores its value at `st.session_state["k"]`	Read it anywhere in the script, or in callbacks
Set a key either with `value=` **or** via session state, not both	Otherwise Streamlit warns that the widget was set both ways
Widget keys are deleted when the widget is not rendered in a run	Copy values you need later (other pages, other steps) to a non-widget key
You cannot modify a widget's key after the widget is drawn in that run	Use a callback or `st.rerun()` pattern
Session state is per tab, in memory	Lost on refresh? No. Lost on server restart or new tab? Yes

9.3 Session-specific prediction history

state_demos/session_history.py PYTHON

```python
"""Session-specific prediction history.  Run: streamlit run session_history.py"""
from datetime import datetime

import joblib
import pandas as pd
import streamlit as st


@st.cache_resource
def load_model():
    return joblib.load("models/iris_model.joblib")


model = load_model()
st.session_state.setdefault("history", [])     # a list that lives for this tab only

with st.form("measure"):
    c1, c2 = st.columns(2)
    values = {
        "sepal length (cm)": c1.number_input("Sepal length", 4.0, 8.0, 5.8),
        "sepal width (cm)": c2.number_input("Sepal width", 2.0, 4.5, 3.0),
        "petal length (cm)": c1.number_input("Petal length", 1.0, 7.0, 4.3),
        "petal width (cm)": c2.number_input("Petal width", 0.1, 2.5, 1.3),
    }
    if st.form_submit_button("Predict", type="primary"):
        pred = int(model.predict(pd.DataFrame([values]))[0])
```

state_demos/session_history.py PYTHON

```python
        st.session_state.history.append(
            {"time": datetime.now().strftime("%H:%M:%S"), **values, "class": pred})

hist = pd.DataFrame(st.session_state.history)
st.subheader(f"History ({len(hist)} predictions in this browser tab)")
if hist.empty:
    st.info("No predictions yet.")
else:
    st.dataframe(hist, hide_index=True, width="stretch")
    c1, c2 = st.columns(2)
    c1.download_button("Download", hist.to_csv(index=False), "history.csv")
    if c2.button("Clear history"):
        st.session_state.history = []
        st.rerun()
```

WARNING

Session state is **not** a database. It is lost when the server restarts, it is invisible to other users, and every active session holds its own copy in server memory. Persist important history in a database (Chapter 11).

9.4 Multi-step workflows

Store the current step as an integer in session state and draw only that step. Chapter 10 builds a complete five-step prediction wizard on this idea.

steps_skeleton.py PYTHON

```python
import streamlit as st

st.session_state.setdefault("step", 1)


def next_step():
    st.session_state.step += 1


if st.session_state.step == 1:
    st.text_input("Customer name", key="name")
    st.button("Next", on_click=next_step)
elif st.session_state.step == 2:
    st.write(f"Step 2 for {st.session_state.get('name', '?')}")
```

9.5 st.cache_data vs st.cache_resource

	`@st.cache_data`	`@st.cache_resource`
Caches	The **return value** of a function as data	A **shared object** such as a model, DB engine, tokenizer
What callers receive	A fresh **copy** each time (safe to mutate)	The **same object** for every caller and session
Typical uses	Reading CSV/Parquet, API responses, query results, computed DataFrames, predictions for identical inputs	`joblib.load`, `keras.models.load_model`, SQLAlchemy engines, HTTP sessions, embedding models
Must be	Pickle-able (serialisable)	Thread-safe and not mutated per user
Options	`ttl`, `max_entries`, `persist="disk"`, `show_spinner`, `hash_funcs`, `scope`	`ttl`, `max_entries`, `validate`, `on_release`, `scope`
Cleared by	`func.clear()`, `st.cache_data.clear()`	`func.clear()`, `st.cache_resource.clear()`

The cache key is built from the function's source code and the values of its arguments. Parameters whose names start with an underscore are **not** hashed, which is how you pass unhashable objects such as models.

9.6 Caching ML models and avoiding repeated loading

state_demos/lazy_model.py PYTHON

```python
"""Load a heavy model only when it is actually needed.  Run: streamlit run lazy_model.py"""
import time

import joblib
import streamlit as st


@st.cache_resource(show_spinner="Loading the large model (first time only)...")
def load_big_model(path: str):
    time.sleep(2)                       # simulate a slow 500 MB load
    return joblib.load(path)


st.title("Lazy model loading")
st.write("The page opens instantly. The model loads the first time anyone asks for an "
         "advanced prediction, and is then shared by every session.")
use_advanced = st.toggle("Use the advanced model")
if use_advanced:
    model = load_big_model("models/iris_model.joblib")
    st.success(f"Model ready: {type(model).__name__}")
else:
    st.info("Advanced model not loaded yet.")
```

COMMON MISTAKE

Using `@st.cache_data` for a model. It works for small models but pickles and copies the object on **every call**, which is slow and wastes memory. Use `@st.cache_resource`.

SECURITY WARNING

Because `cache_resource` objects are shared, never store user-specific data on them (for example `model.last_user = ...`). One user's data would leak to every other user.

9.7 Caching datasets and cache invalidation

state_demos/cache_dataset.py PYTHON

```python
"""Caching datasets with st.cache_data, TTLs and invalidation."""
import time
from pathlib import Path

import pandas as pd
import streamlit as st

DATA = Path("data/sales.csv")
if not DATA.exists():
    DATA.parent.mkdir(exist_ok=True)
    pd.DataFrame({"day": range(1, 366), "sales": range(1000, 1365)}).to_csv(DATA,
                                                                            index=False)


@st.cache_data(ttl="10m", max_entries=20, show_spinner="Reading data...")
def load_sales(path: str, modified: float) -> pd.DataFrame:
    """`modified` is part of the cache key: editing the file invalidates the cache."""
    time.sleep(1)                       # pretend the file is big
```

state_demos/cache_dataset.py PYTHON

```
    return pd.read_csv(path)


df = load_sales(str(DATA), DATA.stat().st_mtime)
st.metric("Rows", len(df))
st.line_chart(df, x="day", y="sales")

c1, c2 = st.columns(2)
if c1.button("Clear this function's cache"):
    load_sales.clear()                  # next call re-reads the file
if c2.button("Clear ALL cached data"):
    st.cache_data.clear()
```

- **Time-based** — `ttl="10m"` re-runs the function after ten minutes, ideal for data that changes (database queries, APIs).
- **Input-based** — Include a version argument (file modification time, model hash, date) so changes automatically produce a new cache key.
- **Manual** — `load_sales.clear()` for one function, `st.cache_data.clear()` for all, or the **Clear cache** item in the app menu during development.
- **Size-based** — `max_entries=20` evicts the oldest entries.
- **Code changes** — Editing the function body invalidates its cache automatically.

9.8 Preventing duplicate work and handling expensive predictions

state_demos/expensive_predictions.py PYTHON

```
"""Avoid duplicate work for expensive predictions."""
import hashlib
import time

import joblib
import numpy as np
import pandas as pd
import streamlit as st


@st.cache_resource
def load_model():
    return joblib.load("models/iris_model.joblib")


@st.cache_data(max_entries=100)
def predict_cached(_model, model_version: str, rows: pd.DataFrame) -> np.ndarray:
    """Leading underscore: Streamlit does not hash `_model` (unhashable / expensive).
    `model_version` IS hashed, so a new model never reuses old cached results."""
    time.sleep(1.5)                     # pretend inference is slow
    return _model.predict_proba(rows)


model = load_model()
version = hashlib.sha256(open("models/iris_model.joblib", "rb").read()).hexdigest()[:8]

st.title("Expensive predictions")
rows = pd.DataFrame({"sepal length (cm)": [5.1, 6.7], "sepal width (cm)": [3.5, 3.0],
                     "petal length (cm)": [1.4, 5.2], "petal width (cm)": [0.2, 2.3]})
st.dataframe(rows, hide_index=True)
if st.button("Score rows", type="primary"):
    t0 = time.perf_counter()
    with st.spinner("Scoring...", show_time=True):
        proba = predict_cached(model, version, rows)
```

state_demos/expensive_predictions.py PYTHON

```
    st.dataframe(pd.DataFrame(proba, columns=["setosa", "versicolor", "virginica"])
                 .round(3), hide_index=True)
    st.caption(f"Took {time.perf_counter() - t0:.2f}s. Click again: identical inputs "
               "are answered from the cache almost instantly.")
```

- Cache prediction functions on (model version, inputs) so repeated requests are instant and a new model never returns stale results.
- Use forms so a prediction runs once per submit, not once per keystroke.
- Use fragments so heavy charts are not recomputed when a small widget changes.
- Hash uploaded file **bytes** (not the UploadedFile object) when caching parsing functions: `read_upload(f.name, f.getvalue())`.
- Store results of a button-triggered job in `st.session_state` so later reruns can display them without recomputing.

9.9 Memory usage and session-state limitations

Consumer	Scales with	Control
`cache_resource` models	Number of distinct models	Load once; choose compact models; `max_entries`
`cache_data` entries	Distinct inputs x result size	`max_entries`, `ttl`
`session_state`	Active users x what you store	Store ids and small results, not whole datasets; clear on reset
Uploaded files	Active users x file size	`maxUploadSize`, row limits
Matplotlib figures	Every rerun if not closed	`plt.close(fig)`

9.10 Long-running tasks and progress

long_task.py PYTHON

```
import time

import streamlit as st

if st.button("Retrain model (demo)"):
    with st.status("Retraining...", expanded=True) as status:
        for step in ["Loading data", "Fitting pipeline", "Evaluating", "Saving"]:
            st.write(step)
            time.sleep(0.5)
        status.update(label="Retraining complete", state="complete")
```

A script run blocks its own session while it works (other users are unaffected because each session runs in its own thread). For tasks longer than about a minute, start a background job (a separate process, a task queue such as Celery/RQ, or a cloud job) and let the app poll its status with `@st.fragment(run_every="5s")`.

9.11 Error handling and performance checklist

- [ ] Every expensive function is cached with the right decorator.
- [ ] Model loading is wrapped in try/except with a friendly `st.error` + `st.stop()`.
- [ ] Inputs are collected with forms where several fields feed one prediction.
- [ ] Heavy, independent sections are fragments.
- [ ] Large tables show a preview (`head(500)`); full data goes to downloads.
- [ ] No secrets, models or DataFrames are written into shared module-level globals.
- [ ] `st.cache_data` functions have `max_entries` or `ttl` when inputs vary a lot.

CHAPTER 10

Forms, Callbacks and Interactive Workflows

10

Controlled, multi-step user journeys

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Use forms and submit buttons to batch inputs.
- ✓ Write on_click and on_change callbacks that update session state.
- ✓ Build dependent widgets, conditional UI and dynamic input fields.
- ✓ Implement a five-step prediction wizard with validation and a report.

10.1 Forms and submit buttons

form_basics.py — PYTHON

```python
import streamlit as st

with st.form("loan_form", clear_on_submit=False, enter_to_submit=True):
    income = st.number_input("Monthly income (INR)", 0, 1_000_000, 60_000, step=5_000)
    loan = st.number_input("Loan amount (INR)", 10_000, 5_000_000, 500_000)
    submitted = st.form_submit_button("Check", type="primary")

if submitted:                        # True only on the run triggered by the submit
    st.write(f"Loan-to-income ratio: {loan / max(income, 1):.1f}x")
```

Form rule	Why it matters
Widgets inside a form do not trigger reruns	Users can fill ten fields without ten predictions
Every form needs exactly one `st.form_submit_button`	Otherwise an error is shown
Only the submit button may have a callback	Other widgets' `on_change` is not allowed inside forms
`st.button` cannot be inside a form	Use `form_submit_button` (you may have several)
`clear_on_submit=True` resets inputs after submit	Good for data entry, bad for what-if analysis
Forms can contain columns, expanders, containers	Layout freely inside

10.2 Callbacks: on_click and on_change

callbacks.py — PYTHON

```python
import streamlit as st

st.session_state.setdefault("units", "metric")


def to_imperial():
    st.session_state.units = "imperial"
```

callbacks.py PYTHON

```python
def area_changed():
    # runs BEFORE the rerun; st.session_state.area already holds the new value
    st.session_state.price_per_unit = 9500 if st.session_state.area < 1500 else 9000


st.button("Use square feet", on_click=to_imperial)
st.number_input("Area", 100, 10_000, 1050, key="area", on_change=area_changed)
st.write("Units:", st.session_state.units,
         "| rate:", st.session_state.get("price_per_unit", "n/a"))
```

Pass arguments with `args=(...)` and `kwargs={...}`: `st.button("Back", on_click=go, args=(0,))` calls `go(0)` before the rerun. This is cleaner than checking `if st.button(...)` and then calling `st.rerun()`.

10.3 Dependent widgets and conditional UI

dependent_widgets.py PYTHON

```python
import streamlit as st

OPTIONS = {
    "Classification": ["Logistic regression", "Random forest", "XGBoost"],
    "Regression": ["Linear regression", "Gradient boosting"],
}
task = st.radio("Task", list(OPTIONS), horizontal=True)
model = st.selectbox("Algorithm", OPTIONS[task])          # options depend on task

if task == "Classification":                              # conditional UI
    st.slider("Decision threshold", 0.05, 0.95, 0.5)
else:
    st.number_input("Prediction cap", value=1_000_000)

n_features = st.number_input("Number of custom features", 0, 5, 1)
custom = [st.text_input(f"Feature {i + 1} name", key=f"feat_{i}")    # dynamic fields
          for i in range(n_features)]
st.write(model, custom)
```

TIP

Dynamic widgets need **unique, stable keys** (`key=f"feat_{i}"`). Without keys, removing field 2 of 3 can shift values between fields.

10.4 Input validation and reset buttons

validate_reset.py PYTHON

```python
import streamlit as st

DEFAULTS = {"age": 35, "salary": 50_000}
for k, v in DEFAULTS.items():
    st.session_state.setdefault(k, v)


def reset_form():
    for k, v in DEFAULTS.items():
        st.session_state[k] = v


with st.form("emp"):
    st.number_input("Age", 18, 70, key="age")
    st.number_input("Salary", 0, 10_000_000, key="salary", step=1000)
    ok = st.form_submit_button("Validate")
st.button("Reset", on_click=reset_form)
```

validate_reset.py PYTHON

```python
if ok:
    errors = []
    if st.session_state.age < 21 and st.session_state.salary > 2_000_000:
        errors.append("Unusual combination: please double-check age and salary.")
    st.error("\n".join(errors)) if errors else st.success("Looks good.")
```

10.5 Project: a five-step prediction wizard



Figure 10.1: Wizard steps; Back buttons return to earlier steps

churn_app/multistep_app.py PYTHON

```python
"""Five-step prediction wizard with callbacks.  Run: streamlit run multistep_app.py

Step 1 enter -> Step 2 validate -> Step 3 predict -> Step 4 result -> Step 5 report
The current step and all answers live in st.session_state, so they survive reruns.
"""
import json
from datetime import datetime
from pathlib import Path

import pandas as pd
import streamlit as st

from schema import CATEGORY_OPTIONS, FEATURE_LABELS, FEATURES, NUMERIC_RANGES
from schema import POSITIVE_CLASS
from utils.model_io import load_trusted_model
from utils.validation import validate_record

ROOT = Path(__file__).parent
STEPS = ["Customer info", "Validate", "Predict", "Result", "Report"]
DEFAULTS = {"tenure_months": 12, "monthly_charges": 70.0, "total_charges": 840.0,
            "support_calls": 0, "contract": "Month-to-month",
            "internet_service": "Fiber optic", "payment_method": "Electronic check",
            "paperless_billing": "Yes", "senior_citizen": "No"}

st.set_page_config(page_title="Churn Wizard")


@st.cache_resource
def get_model():
    return load_trusted_model(ROOT / "models" / "churn_pipeline.joblib",
                              ROOT / "models" / "churn_metadata.json")


# ---------------------------------------------------------- state + callbacks
def init_state():
    st.session_state.setdefault("step", 0)
    st.session_state.setdefault("record", dict(DEFAULTS))
    st.session_state.setdefault("result", None)


def go(step: int):
    st.session_state.step = step
```

churn_app/multistep_app.py PYTHON

```python
def save_step1():
    """on_click callback: copy widget values into the record BEFORE the rerun."""
    rec = st.session_state.record
    for key in FEATURES:
        rec[key] = st.session_state[f"w_{key}"]
    rec["call_reason"] = st.session_state.get("w_call_reason")
    st.session_state.result = None          # inputs changed -> old result is stale
    go(1)


def run_prediction():
    model, meta, _ = get_model()
    X = pd.DataFrame([st.session_state.record])[FEATURES]
    pos = meta["classes"].index(POSITIVE_CLASS)
    proba = float(model.predict_proba(X)[0, pos])
    st.session_state.result = {"probability": proba,
                               "label": "Likely to churn" if proba >= 0.5 else
                               "Likely to stay",
                               "model": meta["model_type"],
                               "timestamp": datetime.now().isoformat(timespec="seconds")}
    go(3)


def reset():
    for key in list(st.session_state.keys()):
        del st.session_state[key]


def sync_total():
    """Dependent widget: suggest total charges = tenure x monthly charge."""
    t = st.session_state.w_tenure_months
    m = st.session_state.w_monthly_charges
    st.session_state.w_total_charges = round(min(t * m, 9000.0), 2)


init_state()
rec = st.session_state.record
st.title("Churn Prediction Wizard")
st.progress((st.session_state.step + 1) / len(STEPS),
            text=f"Step {st.session_state.step + 1} of {len(STEPS)}: "
                 f"{STEPS[st.session_state.step]}")
st.button("Start over", on_click=reset, icon=":material/restart_alt:")

step = st.session_state.step
if step == 0:
    # Seed widget keys from the saved record. Widget state is discarded when a widget
    # is not drawn (steps 2-5), so we restore it here. Do NOT also pass value=/index=:
    # setting a widget both ways triggers a Streamlit warning.
    for key in FEATURES:
        st.session_state.setdefault(f"w_{key}", rec[key])
    c1, c2 = st.columns(2)
    c1.number_input(FEATURE_LABELS["tenure_months"], *NUMERIC_RANGES["tenure_months"],
                    key="w_tenure_months", on_change=sync_total)
    c1.number_input(FEATURE_LABELS["monthly_charges"], *NUMERIC_RANGES["monthly_charges"],
                    key="w_monthly_charges", on_change=sync_total)
    c1.number_input(FEATURE_LABELS["total_charges"], *NUMERIC_RANGES["total_charges"],
                    key="w_total_charges",
                    help="Auto-filled from tenure x monthly charge; you can edit it.")
    c1.number_input(FEATURE_LABELS["support_calls"], *NUMERIC_RANGES["support_calls"],
                    key="w_support_calls")
    for col in ("contract", "internet_service", "payment_method", "paperless_billing",
                "senior_citizen"):
        c2.selectbox(FEATURE_LABELS[col], CATEGORY_OPTIONS[col], key=f"w_{col}")
    # conditional UI: only ask for a reason when the customer called support
    if st.session_state.w_support_calls > 0:
        st.selectbox("Main reason for the last support call",
                     ["Billing", "Network quality", "Device", "Other"],
                     key="w_call_reason")
```

churn_app/multistep_app.py PYTHON

```
    st.button("Next: validate", type="primary", on_click=save_step1)

elif step == 1:
    problems = validate_record(rec)
    st.subheader("Check the details")
    st.table(pd.DataFrame({"value": {FEATURE_LABELS[k]: str(rec[k]) for k in FEATURES}}))
    if problems:
        for p in problems:
            st.error(p)
        st.button("Back to edit", on_click=go, args=(0,))
    else:
        st.success("All inputs are valid.")
        b1, b2 = st.columns(2)
        b1.button("Back", on_click=go, args=(0,))
        b2.button("Next: predict", type="primary", on_click=go, args=(2,))

elif step == 2:
    st.write("Ready to score this customer with the trained model.")
    b1, b2 = st.columns(2)
    b1.button("Back", on_click=go, args=(1,))
    b2.button("Generate prediction", type="primary", on_click=run_prediction)

elif step == 3:
    res = st.session_state.result
    st.metric("Churn probability", f"{res['probability']:.1%}", res["label"],
              delta_color="inverse" if res["probability"] >= 0.5 else "normal")
    st.caption(f"Model: {res['model']} | scored at {res['timestamp']}")
    b1, b2 = st.columns(2)
    b1.button("Change inputs", on_click=go, args=(0,))
    b2.button("Next: report", type="primary", on_click=go, args=(4,))

else:
    res = st.session_state.result
    report = {"inputs": {k: rec[k] for k in FEATURES}, "result": res,
              "notes": "Risk estimate for retention prioritisation; not a decision."}
    st.json(report)
    lines = [f"# Churn prediction report ({res['timestamp']})", "",
             f"**Probability:** {res['probability']:.1%} | **Result:** {res['label']}", "",
             "| Feature | Value |", "|---|---|"]
    lines += [f"| {FEATURE_LABELS[k]} | {rec[k]} |" for k in FEATURES]
    c1, c2 = st.columns(2)
    c1.download_button("Download JSON", json.dumps(report, indent=2),
                       "churn_report.json", "application/json", on_click="ignore")
    c2.download_button("Download Markdown", "\n".join(lines), "churn_report.md",
                       "text/markdown", on_click="ignore")
    st.button("Score another customer", on_click=reset, type="primary")
```

Execution flow

1. First run: `init_state()` sets `step = 0` and a default record. Step 0 seeds widget keys from the record and draws the form.
2. Changing tenure or monthly charges fires `sync_total` **before** the rerun, which updates the dependent total-charges widget.
3. Setting support calls above 0 makes a new 'reason' selectbox appear (conditional UI).
4. 'Next: validate' runs `save_step1`, which copies widget values into `st.session_state.record`, clears any stale result and moves to step 1.
5. Step 1 runs `validate_record`. Errors block progress; otherwise 'Next' moves to step 2.
6. 'Generate prediction' runs `run_prediction` as a callback: the model is called once and the result is stored; the step becomes 3.

7. Step 4 builds JSON and Markdown reports from the stored record and result; downloads use `on_click="ignore"` so the page does not rerun.
8. 'Start over' deletes every session-state key, returning to a clean step 0.

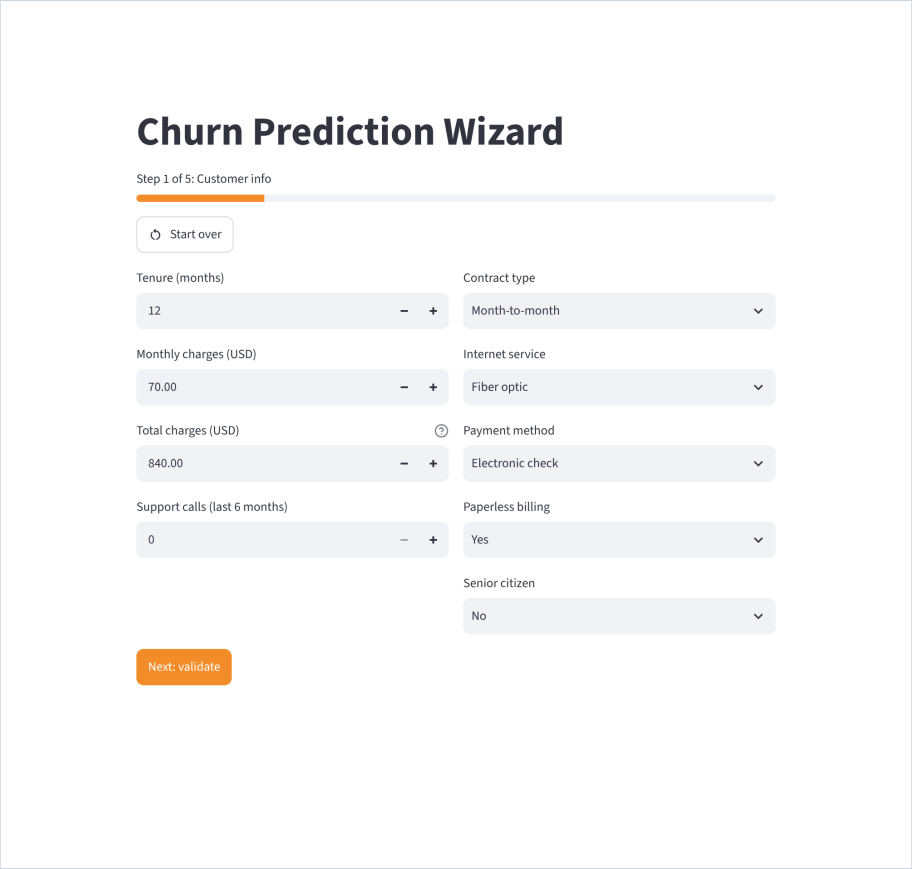


Figure 10.2: Step 1: entering customer information

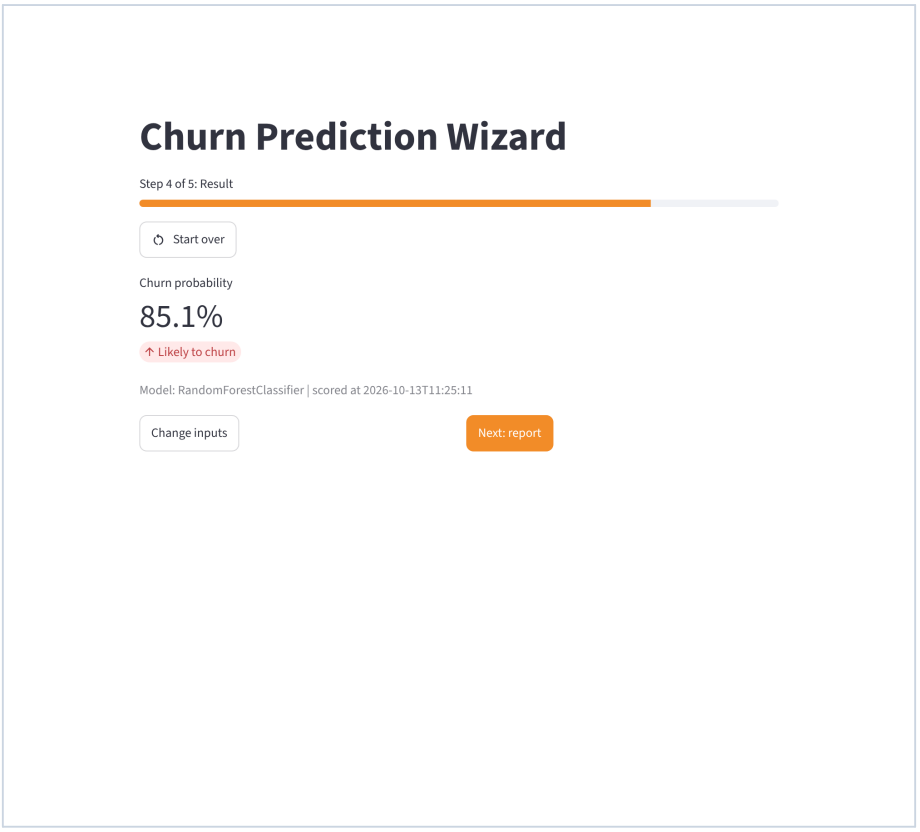


Figure 10.3: Step 4: the stored prediction result

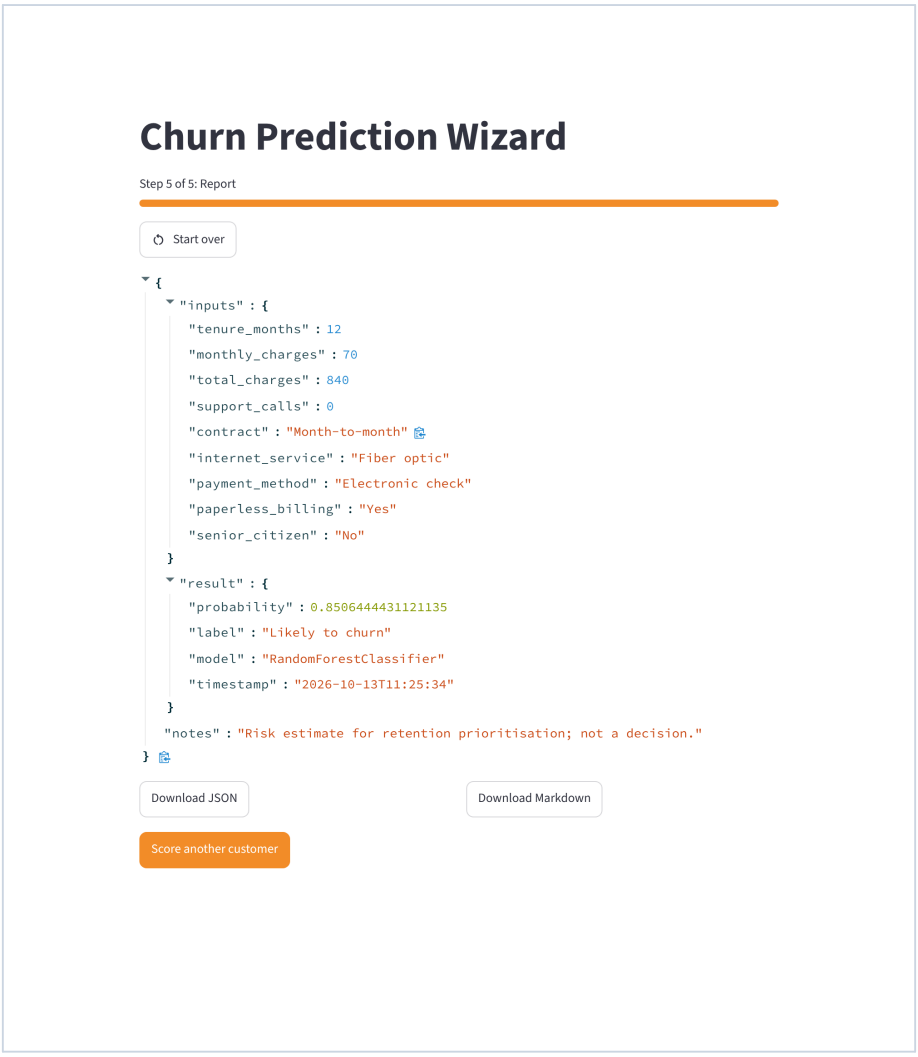


Figure 10.4: Step 5: report preview and downloads

BEST PRACTICE

Run expensive actions (predictions, API calls, database writes) in **callbacks** triggered by a button, store the result in session state, and let the normal script body only *display* state. This keeps each action running exactly once per click.

CHAPTER 11

Database and API Integration

11

Persist predictions, collect feedback and call model services

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Store and query prediction history in SQLite, PostgreSQL or MySQL.
- ✓ Use parameterised SQL and connection pooling correctly.
- ✓ Manage credentials with environment variables and st.secrets.
- ✓ Serve a model with FastAPI and call it from Streamlit.
- ✓ Decide when a separate API service is better than in-app inference.

Before you start: `pip install sqlalchemy fastapi uvicorn requests`; for PostgreSQL `pip install "psycopg[binary]"`, for MySQL `pip install pymysql`.

11.1 Architecture



Figure 11.1: Every prediction is validated, scored and then stored

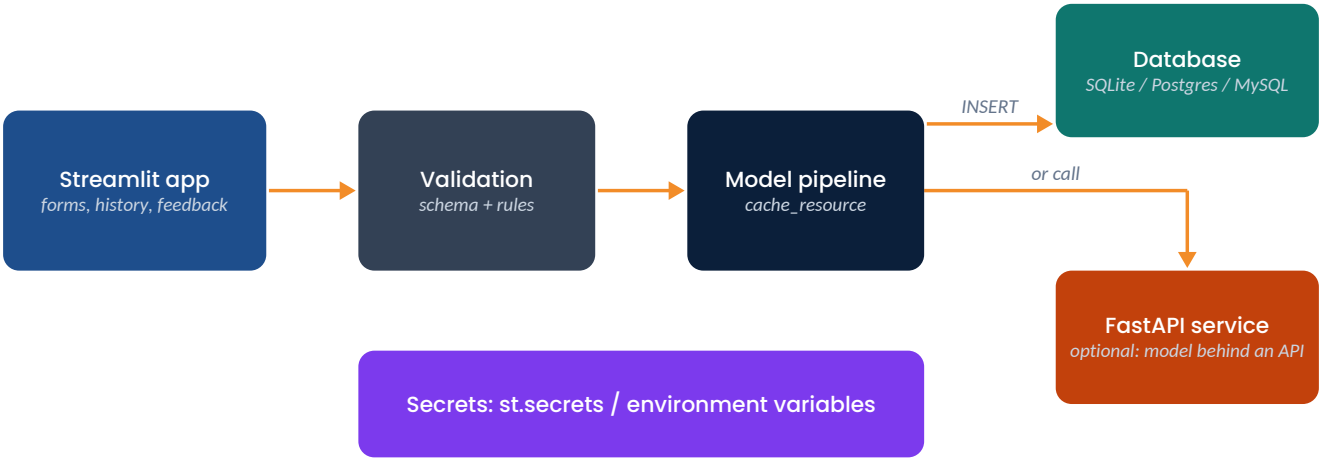


Figure 11.2: Components of a database-backed ML app

11.2 Database helper module (SQLite, PostgreSQL, MySQL)

SQLAlchemy lets the same code talk to SQLite locally and PostgreSQL or MySQL in production. Only the connection URL changes.

churn_app/utils/db.py PYTHON

```python
"""Prediction-history storage with SQLAlchemy Core.

The same code works with SQLite, PostgreSQL and MySQL; only the URL changes:
    sqlite:///predictions.db
    postgresql+psycopg://user:password@host:5432/mlapp      (pip install psycopg[binary])
    mysql+pymysql://user:password@host:3306/mlapp            (pip install pymysql)
Every query uses bound parameters (:name), never string formatting, which prevents
SQL injection.
"""
import json
from datetime import datetime, timezone

import pandas as pd
from sqlalchemy import create_engine, text
from sqlalchemy.engine import Engine

SCHEMA = [
    """CREATE TABLE IF NOT EXISTS predictions (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        created_at VARCHAR(32) NOT NULL,
        model_version VARCHAR(64) NOT NULL,
        inputs TEXT NOT NULL,
        probability FLOAT NOT NULL,
        label VARCHAR(32) NOT NULL
    )""",
    """CREATE TABLE IF NOT EXISTS feedback (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        prediction_id INTEGER NOT NULL REFERENCES predictions(id),
        created_at VARCHAR(32) NOT NULL,
        helpful INTEGER NOT NULL,
        comment TEXT
    )""",
]


def make_engine(url: str) -> Engine:
    # pool_pre_ping re-checks pooled connections so dropped DB connections recover
    engine = create_engine(url, pool_pre_ping=True)
    with engine.begin() as conn:
        for ddl in SCHEMA:
            if not url.startswith("sqlite"):
                ddl = ddl.replace("INTEGER PRIMARY KEY AUTOINCREMENT",
                                  "SERIAL PRIMARY KEY" if url.startswith("postgres")
                                  else "INTEGER PRIMARY KEY AUTO_INCREMENT")
            conn.execute(text(ddl))
    return engine


def _now() -> str:
    return datetime.now(timezone.utc).isoformat(timespec="seconds")


def save_prediction(engine: Engine, inputs: dict, probability: float, label: str,
                    model_version: str) -> int:
    with engine.begin() as conn:   # begin() commits on success, rolls back on error
        result = conn.execute(
            text("INSERT INTO predictions (created_at, model_version, inputs, "
                 "probability, label) VALUES (:ts, :ver, :inputs, :p, :label)"),
            {"ts": _now(), "ver": model_version, "inputs": json.dumps(inputs),
             "p": probability, "label": label},
        )
```

churn_app/utils/db.py PYTHON

```python
        return int(result.lastrowid) if result.lastrowid else int(conn.execute(
            text("SELECT MAX(id) FROM predictions")).scalar())


def save_feedback(engine: Engine, prediction_id: int, helpful: bool,
                  comment: str = "") -> None:
    with engine.begin() as conn:
        conn.execute(
            text("INSERT INTO feedback (prediction_id, created_at, helpful, comment) "
                 "VALUES (:pid, :ts, :h, :c)"),
            {"pid": prediction_id, "ts": _now(), "h": int(helpful), "c": comment[:500]},
        )


def load_history(engine: Engine, label: str | None = None, limit: int = 200) -> pd.DataFrame:
    sql = ("SELECT p.id, p.created_at, p.model_version, p.probability, p.label, "
           "p.inputs, f.helpful FROM predictions p "
           "LEFT JOIN feedback f ON f.prediction_id = p.id ")
    params = {"limit": int(limit)}
    if label:
        sql += "WHERE p.label = :label "
        params["label"] = label
    sql += "ORDER BY p.id DESC LIMIT :limit"
    with engine.connect() as conn:
        df = pd.read_sql(text(sql), conn, params=params)
    if not df.empty:
        inputs = pd.json_normalize(df.pop("inputs").map(json.loads))
        df = pd.concat([df, inputs], axis=1)
    return df
```

Database	URL format	Driver to install
SQLite (file)	`sqlite:///predictions.db`	built in
PostgreSQL	`postgresql+psycopg://user:pass@host:5432/db`	`psycopg[binary]`
MySQL / MariaDB	`mysql+pymysql://user:pass@host:3306/db`	`pymysql`
SQL Server	`mssql+pyodbc://user:pass@dsn`	`pyodbc` + ODBC driver

SECURITY WARNING

Always use bound parameters: `text("... WHERE label = :label")` with `{"label": value}`. Never build SQL with f-strings such as `f"... WHERE label = '{value}'"` - a value like `x' OR '1'='1` would change the query (SQL injection).

Never hard-code passwords in code or commit them to Git.

11.3 Secrets and environment variables

.streamlit/secrets.toml TOML

```toml
# .streamlit/secrets.toml  -- add this file to .gitignore
[database]
url = "postgresql+psycopg://ml_app:CHANGE_ME@db.example.com:5432/mlapp"

[api]
url = "http://localhost:8000"
key = "CHANGE_ME"
```

reading_secrets.py (requires the secrets file above) PYTHON

```python
import os

import streamlit as st

db_url = st.secrets["database"]["url"]           # nested TOML tables
api_key = st.secrets.api.key                     # attribute access works too
fallback = os.environ.get("DATABASE_URL")        # environment variables
```

- Locally, secrets come from `.streamlit/secrets.toml` (project) or `~/.streamlit/secrets.toml` (user).
- On Streamlit Community Cloud, paste the same TOML into **App settings -> Secrets**.
- On Docker, VMs and other clouds, prefer environment variables or the platform's secret manager (AWS Secrets Manager, Azure Key Vault, GCP Secret Manager).
- `st.connection("name", type="sql")` reads `[connections.name]` from secrets and returns a cached SQLAlchemy-backed connection with a `query()` helper.

st_connection_demo.py (requires the connection secret) PYTHON

```python
# secrets.toml
# [connections.history]
# url = "sqlite:///predictions.db"

import streamlit as st

conn = st.connection("history", type="sql")          # cached, pooled
df = conn.query("SELECT label, COUNT(*) AS n FROM predictions GROUP BY label",
                ttl="1m")                             # result cached for 1 minute
st.dataframe(df)
```

11.4 A Streamlit app that stores predictions and feedback

churn_app/db_app.py PYTHON

```python
"""Churn predictions stored in a database.  Run: streamlit run db_app.py

Database URL resolution order (never hard-code credentials):
  1. st.secrets["database"]["url"]   (.streamlit/secrets.toml or cloud secrets UI)
  2. DATABASE_URL environment variable
  3. local SQLite file for development
"""
import os
from pathlib import Path

import pandas as pd
import streamlit as st
from sqlalchemy.exc import SQLAlchemyError

from schema import CATEGORICAL_FEATURES, CATEGORY_OPTIONS, FEATURE_LABELS, FEATURES
from schema import POSITIVE_CLASS
from utils.db import load_history, make_engine, save_feedback, save_prediction
from utils.model_io import load_trusted_model
from utils.validation import validate_record

ROOT = Path(__file__).parent
st.set_page_config(page_title="Churn + Database", layout="wide")


def database_url() -> str:
    try:
        return st.secrets["database"]["url"]
    except (KeyError, FileNotFoundError):
        return os.environ.get("DATABASE_URL", f"sqlite:///{ROOT / 'predictions.db'}")
```

churn_app/db_app.py PYTHON

```python
@st.cache_resource
def get_engine():
    return make_engine(database_url())   # one connection pool per server process


@st.cache_resource
def get_model():
    return load_trusted_model(ROOT / "models" / "churn_pipeline.joblib",
                              ROOT / "models" / "churn_metadata.json")


model, meta, _ = get_model()
try:
    engine = get_engine()
except SQLAlchemyError as exc:
    st.error(f"Database unavailable: {exc.__class__.__name__}. Check the URL/secrets.")
    st.stop()
version = f"{meta['model_type']}-{meta['sha256'][:8]}"

st.title("Churn Predictions with History")
predict_tab, history_tab = st.tabs(["Predict", "History & feedback"])

with predict_tab:
    with st.form("predict"):
        c1, c2 = st.columns(2)
        rec = {
            "tenure_months": c1.number_input("Tenure (months)", 0, 72, 6),
            "monthly_charges": c1.number_input("Monthly charges", 18.0, 120.0, 89.0),
            "total_charges": c1.number_input("Total charges", 0.0, 9000.0, 530.0),
            "support_calls": c1.number_input("Support calls", 0, 10, 3),
        }
        for col in CATEGORICAL_FEATURES:
            rec[col] = c2.selectbox(FEATURE_LABELS[col], CATEGORY_OPTIONS[col])
        go = st.form_submit_button("Predict and save", type="primary")
    if go:
        problems = validate_record(rec)
        if problems:
            st.error(" ".join(problems))
        else:
            pos = meta["classes"].index(POSITIVE_CLASS)
            p = float(model.predict_proba(pd.DataFrame([rec])[FEATURES])[0, pos])
            label = "churn" if p >= 0.5 else "stay"
            try:
                pid = save_prediction(engine, rec, p, label, version)
                st.session_state.last_id = pid
                st.success(f"Prediction #{pid} saved: {label} ({p:.1%})")
            except SQLAlchemyError:
                st.warning("Prediction made but could not be saved to the database.")
                st.metric("Churn probability", f"{p:.1%}")
    if "last_id" in st.session_state:
        st.write(f"Was prediction #{st.session_state.last_id} useful?")
        rating = st.feedback("thumbs", key=f"fb_{st.session_state.last_id}")
        if rating is not None:
            save_feedback(engine, st.session_state.last_id, helpful=bool(rating))
            st.toast("Thanks for the feedback!")
            del st.session_state["last_id"]

with history_tab:
    f1, f2 = st.columns(2)
    label_filter = f1.selectbox("Filter by result", ["All", "churn", "stay"])
    limit = f2.number_input("Rows", 10, 1000, 100, step=10)
    try:
        hist = load_history(engine, None if label_filter == "All" else label_filter,
                            limit)
    except SQLAlchemyError as exc:
        st.error(f"Could not read history: {exc.__class__.__name__}")
```

churn_app/db_app.py PYTHON

```
        hist = pd.DataFrame()
    if hist.empty:
        st.info("No predictions stored yet.")
    else:
        k1, k2, k3 = st.columns(3)
        k1.metric("Stored predictions", len(hist))
        k2.metric("Predicted churn", int((hist["label"] == "churn").sum()))
        rated = hist["helpful"].dropna()
        k3.metric("Helpful rating", f"{rated.mean():.0%}" if len(rated) else "n/a")
        st.dataframe(hist, hide_index=True, width="stretch")
        st.download_button("Export history", hist.to_csv(index=False), "history.csv")
```

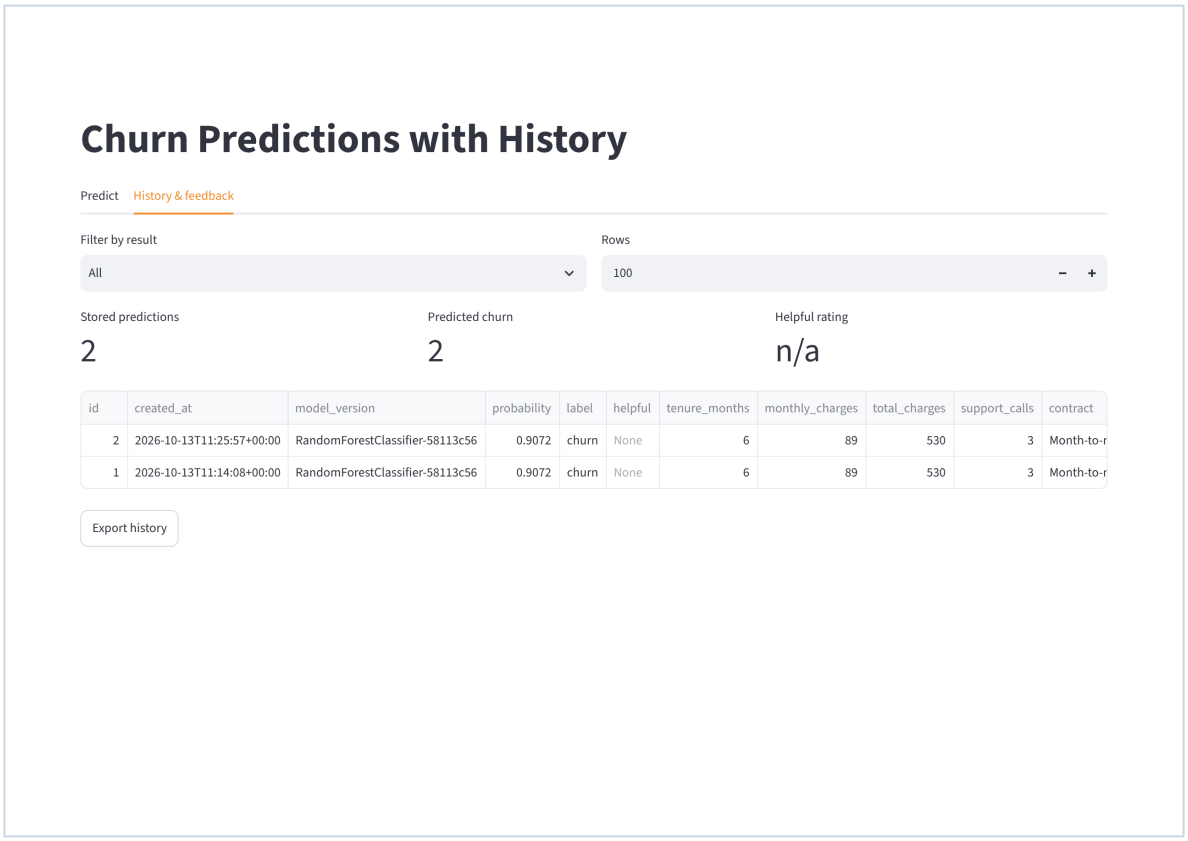


Figure 11.3: Stored history with feedback ratings read back from the database

Concern	How the app handles it
Connection management	`make_engine` inside `@st.cache_resource`: one pool per process, reused by all sessions
Dropped connections	`pool_pre_ping=True` tests connections before use
Transactions	`engine.begin()` commits on success, rolls back on error
Errors	`SQLAlchemyError` is caught: the prediction is still shown, with a warning that saving failed
Feedback	`st.feedback("thumbs")` stored in a `feedback` table linked by prediction id
Reading history	Parameterised filter and `LIMIT`, JSON inputs expanded to columns

NOTE

SQLite is perfect for learning and single-server apps, but each container on platforms like Streamlit Community Cloud has its own temporary disk: the file is lost on redeploy and is not shared between replicas. Use a hosted PostgreSQL (Supabase, Neon, AWS RDS, Azure Database) for persistent history.

11.5 Serving the model with FastAPI

churn_app/api/main.py PYTHON

```python
"""FastAPI model service for the churn pipeline.

Run from the project root:
    export CHURN_API_KEY="change-me"            # Windows: set CHURN_API_KEY=change-me
    uvicorn api.main:app --host 0.0.0.0 --port 8000
Docs (interactive): http://localhost:8000/docs
"""
import os
import sys
from pathlib import Path
from typing import Literal

import pandas as pd
from fastapi import Depends, FastAPI, Header, HTTPException
from pydantic import BaseModel, Field

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))                    # so `schema` and `utils` import
from schema import FEATURES, POSITIVE_CLASS  # noqa: E402
from utils.model_io import load_trusted_model  # noqa: E402

model, meta, _ = load_trusted_model(ROOT / "models" / "churn_pipeline.joblib",
                                    ROOT / "models" / "churn_metadata.json")
POS_IDX = meta["classes"].index(POSITIVE_CLASS)
app = FastAPI(title="Churn model API", version=meta["sha256"][:8])


class Customer(BaseModel):
    """Pydantic validates types and ranges before the model ever sees the data."""
    tenure_months: int = Field(ge=0, le=72)
    monthly_charges: float = Field(ge=18, le=120)
    total_charges: float = Field(ge=0, le=9000)
    support_calls: int = Field(ge=0, le=10)
    contract: Literal["Month-to-month", "One year", "Two year"]
    internet_service: Literal["Fiber optic", "DSL", "No"]
    payment_method: Literal["Electronic check", "Mailed check", "Bank transfer",
                            "Credit card"]
    paperless_billing: Literal["Yes", "No"]
    senior_citizen: Literal["Yes", "No"]


class Prediction(BaseModel):
    churn_probability: float
    label: str
    model_version: str


def require_api_key(x_api_key: str = Header(default="")) -> None:
    expected = os.environ.get("CHURN_API_KEY")
    if not expected or x_api_key != expected:
        raise HTTPException(status_code=401, detail="Invalid or missing API key")


@app.get("/health")
def health() -> dict:
    return {"status": "ok", "model": meta["model_type"]}


@app.post("/predict", response_model=list[Prediction],
          dependencies=[Depends(require_api_key)])
def predict(customers: list[Customer]) -> list[Prediction]:
    if not 1 <= len(customers) <= 1000:
        raise HTTPException(status_code=422, detail="Send between 1 and 1000 records")
```

churn_app/api/main.py PYTHON

```python
    X = pd.DataFrame([c.model_dump() for c in customers])[FEATURES]
    probs = model.predict_proba(X)[:, POS_IDX]
    return [Prediction(churn_probability=round(float(p), 4),
                       label="churn" if p >= 0.5 else "stay",
                       model_version=app.version) for p in probs]
```

Run and call the API TERMINAL

```
export CHURN_API_KEY="change-me"        # Windows CMD: set CHURN_API_KEY=change-me
uvicorn api.main:app --host 0.0.0.0 --port 8000

curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" -H "X-API-Key: change-me" \
  -d '[{"tenure_months": 6, "monthly_charges": 89, "total_charges": 530,
        "support_calls": 3, "contract": "Month-to-month",
        "internet_service": "Fiber optic", "payment_method": "Electronic check",
        "paperless_billing": "Yes", "senior_citizen": "No"}]'
```

Response OUTPUT

```
[{"churn_probability":0.9072,"label":"churn","model_version":"58113c56"}]
```

11.6 Calling a REST API from Streamlit

churn_app/api_client_app.py PYTHON

```python
"""Streamlit front-end that calls the FastAPI service.  Run: streamlit run api_client_app.py

.streamlit/secrets.toml (never commit this file):
    [api]
    url = "http://localhost:8000"
    key = "change-me"
"""
import requests
import streamlit as st

from schema import CATEGORY_OPTIONS

st.set_page_config(page_title="Churn (API client)")
API_URL = st.secrets["api"]["url"].rstrip("/")
HEADERS = {"X-API-Key": st.secrets["api"]["key"]}


@st.cache_data(ttl=30)
def api_health() -> dict | None:
    try:
        r = requests.get(f"{API_URL}/health", timeout=3)
        r.raise_for_status()
        return r.json()
    except requests.RequestException:
        return None


st.title("Churn Prediction via REST API")
health = api_health()
if health is None:
    st.error(f"Model service at {API_URL} is not reachable.")
    st.stop()
st.caption(f"Connected to {API_URL} | model {health['model']}")

with st.form("api_form"):
    payload = {
        "tenure_months": st.number_input("Tenure (months)", 0, 72, 6),
        "monthly_charges": st.number_input("Monthly charges", 18.0, 120.0, 89.0),
```

churn_app/api_client_app.py PYTHON

```python
        "total_charges": st.number_input("Total charges", 0.0, 9000.0, 530.0),
        "support_calls": st.number_input("Support calls", 0, 10, 3),
        **{c: st.selectbox(c, opts) for c, opts in CATEGORY_OPTIONS.items()},
    }
    send = st.form_submit_button("Predict", type="primary")

if send:
    try:
        r = requests.post(f"{API_URL}/predict", json=[payload], headers=HEADERS,
                          timeout=10)
    except requests.RequestException as exc:
        st.error(f"Request failed: {exc}")
    else:
        if r.status_code == 200:
            res = r.json()[0]
            st.metric("Churn probability", f"{res['churn_probability']:.1%}", res["label"])
            st.caption(f"Model version {res['model_version']}")
        elif r.status_code == 401:
            st.error("The API rejected the key. Check st.secrets['api']['key'].")
        elif r.status_code == 422:
            st.error(f"Validation error from API: {r.json()['detail']}")
        else:
            st.error(f"API error {r.status_code}")
```

- **Timeouts** — Always pass `timeout=`; otherwise a hung API freezes the session.
- **Status codes** — Handle 401 (auth), 422 (validation) and 5xx (server) separately with clear messages.
- **Authentication** — Send the key in a header (`X-API-Key` or `Authorization: Bearer ...`), never in the URL query string where it ends up in logs.
- **Caching** — Cache idempotent GET calls (`@st.cache_data(ttl=30)`); never cache POST calls that change data.
- **Pooling** — For many calls, reuse a `requests.Session()` stored with `@st.cache_resource` to keep connections alive.

11.7 When should the model live behind an API?

Run the model inside Streamlit when...	Use a separate FastAPI service when...
Only the Streamlit app uses the model	Other systems (CRM, mobile app, batch jobs) also need predictions
The model is small and loads in seconds	The model is large or needs a GPU and should scale separately
One team owns everything	ML engineers and app developers deploy independently
Traffic is low to moderate	You need autoscaling, request logging, rate limits and an SLA
Fast prototyping matters most	Strict API contracts and versioning matter

BEST PRACTICE

Even when the model starts inside Streamlit, keep prediction logic in a plain-Python module (no `st.` calls). Moving it behind FastAPI later is then a matter of importing the same function.

CHAPTER 12

Report Generation and Downloads

12

CSV, Excel, PDF and JSON reports built in memory

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Generate CSV, Excel, PDF and JSON outputs from predictions.
- ✓ Embed charts in reports without temporary files.
- ✓ Cache report generation so downloads are instant.
- ✓ Validate that generated files open correctly.

Before you start: `pip install openpyxl reportlab matplotlib`

12.1 Choosing a report format

Format	Best for	Library	MIME type
CSV	Full prediction tables for further analysis	pandas	`text/csv`
Excel (.xlsx)	Business users; several sheets (summary, detail)	pandas + openpyxl	`application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`
PDF	Fixed-layout summaries to email or archive	ReportLab (or fpdf2)	`application/pdf`
JSON	Machine-readable results, model cards, API payloads	json	`application/json`
PNG	A single chart	matplotlib / plotly+kaleido	`image/png`

12.2 Report-building helpers

churn_app/utils/reports.py — PYTHON

```python
"""Build downloadable CSV / Excel / PDF / JSON reports entirely in memory.

Using io.BytesIO instead of temporary files means nothing is left on the server's
disk, and concurrent users never overwrite each other's files.
Requires: openpyxl (Excel), reportlab (PDF), matplotlib (chart image).
"""
import io
import json
from datetime import datetime

import matplotlib
import pandas as pd

matplotlib.use("Agg")                    # headless backend for servers
import matplotlib.pyplot as plt  # noqa: E402
from reportlab.lib import colors  # noqa: E402
from reportlab.lib.pagesizes import A4  # noqa: E402
from reportlab.lib.styles import getSampleStyleSheet  # noqa: E402
```

churn_app/utils/reports.py PYTHON

```python
from reportlab.lib.units import cm  # noqa: E402
from reportlab.platypus import (Image, Paragraph, SimpleDocTemplate, Spacer,  # noqa: E402
                                Table, TableStyle)


def to_csv_bytes(df: pd.DataFrame) -> bytes:
    return df.to_csv(index=False).encode("utf-8")


def to_json_bytes(obj: dict) -> bytes:
    return json.dumps(obj, indent=2, default=str).encode("utf-8")


def to_excel_bytes(sheets: dict[str, pd.DataFrame]) -> bytes:
    """One worksheet per DataFrame; sheet names are truncated to Excel's 31 chars."""
    buffer = io.BytesIO()
    with pd.ExcelWriter(buffer, engine="openpyxl") as writer:
        for name, frame in sheets.items():
            frame.to_excel(writer, sheet_name=name[:31], index=True)
            ws = writer.sheets[name[:31]]
            for col_cells in ws.columns:          # simple auto-width
                width = max(len(str(c.value or "")) for c in col_cells) + 2
                ws.column_dimensions[col_cells[0].column_letter].width = min(width, 40)
    return buffer.getvalue()


def risk_chart_png(probabilities: pd.Series) -> bytes:
    fig, ax = plt.subplots(figsize=(6, 2.6), dpi=150)
    ax.hist(probabilities, bins=20, color="#1f4e8c", edgecolor="white")
    ax.axvline(0.5, color="#f28c28", linestyle="--", label="threshold 0.5")
    ax.set_xlabel("Churn probability")
    ax.set_ylabel("Customers")
    ax.legend()
    fig.tight_layout()
    buf = io.BytesIO()
    fig.savefig(buf, format="png")
    plt.close(fig)                              # free memory on long-running servers
    return buf.getvalue()


def _table(df: pd.DataFrame) -> Table:
    data = [list(df.columns)] + df.astype(str).values.tolist()
    t = Table(data, hAlign="LEFT")
    t.setStyle(TableStyle([
        ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#0b1f3a")),
        ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
        ("GRID", (0, 0), (-1, -1), 0.4, colors.HexColor("#cbd5e1")),
        ("FONTSIZE", (0, 0), (-1, -1), 8.5),
        ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#f1f5f9")]),
    ]))
    return t


def to_pdf_bytes(title: str, summary: pd.DataFrame, metrics: dict,
                 chart_png: bytes, top_risk: pd.DataFrame) -> bytes:
    buffer = io.BytesIO()
    doc = SimpleDocTemplate(buffer, pagesize=A4, leftMargin=2 * cm, rightMargin=2 * cm,
                            title=title)
    styles = getSampleStyleSheet()
    story = [
        Paragraph(title, styles["Title"]),
        Paragraph(f"Generated {datetime.now():%d %b %Y %H:%M}", styles["Normal"]),
        Spacer(1, 12),
        Paragraph("Prediction summary", styles["Heading2"]), _table(summary),
        Spacer(1, 10),
        Paragraph("Model evaluation (held-out test set)", styles["Heading2"]),
        _table(pd.DataFrame({"metric": list(metrics),
                             "value": [f"{v:.3f}" for v in metrics.values()]})),
```

churn_app/utils/reports.py PYTHON

```
        Spacer(1, 10),
        Paragraph("Risk distribution", styles["Heading2"]),
        Image(io.BytesIO(chart_png), width=16 * cm, height=6.9 * cm),
        Paragraph("Highest-risk customers", styles["Heading2"]), _table(top_risk),
        Spacer(1, 10),
        Paragraph("Predictions are risk estimates to prioritise outreach; they are not "
                  "decisions about individual customers.", styles["Italic"]),
    ]
    doc.build(story)
    return buffer.getvalue()
```

- **In-memory files** — `io.BytesIO()` acts like a file. Writers save into it and `getvalue()` returns the bytes for `st.download_button`.
- **No temporary files** — Nothing is written to disk, so concurrent users cannot overwrite each other and no clean-up is needed. If a library insists on a path, use `tempfile.TemporaryDirectory()` in a `with` block so it is deleted automatically.
- **Headless charts** — `matplotlib.use("Agg")` avoids GUI back-ends on servers; `plt.close(fig)` frees memory.
- **Excel formatting** — openpyxl worksheets are available via `writer.sheets[name]` for column widths and number formats.

12.3 The report app

churn_app/report_app.py PYTHON

```
"""Generate CSV, Excel, PDF and JSON reports from batch predictions.

Run: streamlit run report_app.py   (extra deps: openpyxl reportlab matplotlib)
"""
from pathlib import Path

import pandas as pd
import streamlit as st

from schema import FEATURES, ID_COLUMN, POSITIVE_CLASS
from utils.model_io import load_trusted_model
from utils.reports import (risk_chart_png, to_csv_bytes, to_excel_bytes, to_json_bytes,
                           to_pdf_bytes)
from utils.validation import validate_dataframe

ROOT = Path(__file__).parent
st.set_page_config(page_title="Churn Reports", layout="wide")


@st.cache_resource
def get_model():
    return load_trusted_model(ROOT / "models" / "churn_pipeline.joblib",
                              ROOT / "models" / "churn_metadata.json")


@st.cache_data(show_spinner="Building reports...")
def build_reports(scored: pd.DataFrame, metrics: dict) -> dict:
    """Cached: reruns caused by clicking a download button do not rebuild files."""
    summary = (scored.groupby("contract")["churn_probability"]
               .agg(customers="count", mean_probability="mean")
               .assign(predicted_churners=scored.groupby("contract")["prediction"]
                       .apply(lambda s: (s == "Yes").sum()))
               .round(3))
    overall = pd.DataFrame({"value": {
        "customers": len(scored),
        "predicted_churners": int((scored["prediction"] == "Yes").sum()),
        "mean_probability": round(float(scored["churn_probability"].mean()), 4)}})
    top = (scored.nlargest(10, "churn_probability")
```

churn_app/report_app.py PYTHON

```
            [[ID_COLUMN, "contract", "tenure_months", "churn_probability"]])
    pdf_summary = summary.reset_index()
    chart = risk_chart_png(scored["churn_probability"])
    return {
        "csv": to_csv_bytes(scored),
        "xlsx": to_excel_bytes({"Overview": overall, "By contract": summary,
                                "Top risk": top.set_index(ID_COLUMN)}),
        "pdf": to_pdf_bytes("Churn Prediction Report", pdf_summary, metrics, chart, top),
        "json": to_json_bytes({"overview": overall["value"].to_dict(),
                               "by_contract": summary.reset_index().to_dict("records"),
                               "model_metrics": metrics}),
        "chart": chart,
    }


model, meta, _ = get_model()
pos = meta["classes"].index(POSITIVE_CLASS)
st.title("Churn Prediction Reports")
up = st.file_uploader("Customers to score (CSV)", type=["csv"])
df = pd.read_csv(up) if up else pd.read_csv(ROOT / "data" / "new_customers.csv")
errors, _ = validate_dataframe(df)
if errors:
    st.error("; ".join(errors))
    st.stop()

scored = df.copy()
scored["churn_probability"] = model.predict_proba(df[FEATURES])[:, pos].round(4)
scored["prediction"] = (scored["churn_probability"] >= 0.5).map({True: "Yes", False: "No"})
files = build_reports(scored, meta["metrics"])

c1, c2, c3 = st.columns(3)
c1.metric("Customers", len(scored))
c2.metric("Predicted churners", int((scored["prediction"] == "Yes").sum()))
c3.metric("Model ROC-AUC (test)", f"{meta['metrics']['roc_auc']:.3f}")
st.image(files["chart"], caption="Risk distribution (also embedded in the PDF)")

st.subheader("Downloads")
d1, d2, d3, d4 = st.columns(4)
d1.download_button("Predictions (CSV)", files["csv"], "churn_predictions.csv", "text/csv",
                   on_click="ignore", icon=":material/table:")
d2.download_button("Summary (Excel)", files["xlsx"], "churn_summary.xlsx",
                   "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
                   on_click="ignore", icon=":material/grid_on:")
d3.download_button("Report (PDF)", files["pdf"], "churn_report.pdf", "application/pdf",
                   on_click="ignore", icon=":material/picture_as_pdf:", type="primary")
d4.download_button("Results (JSON)", files["json"], "churn_results.json",
                   "application/json", on_click="ignore", icon=":material/data_object:")
```

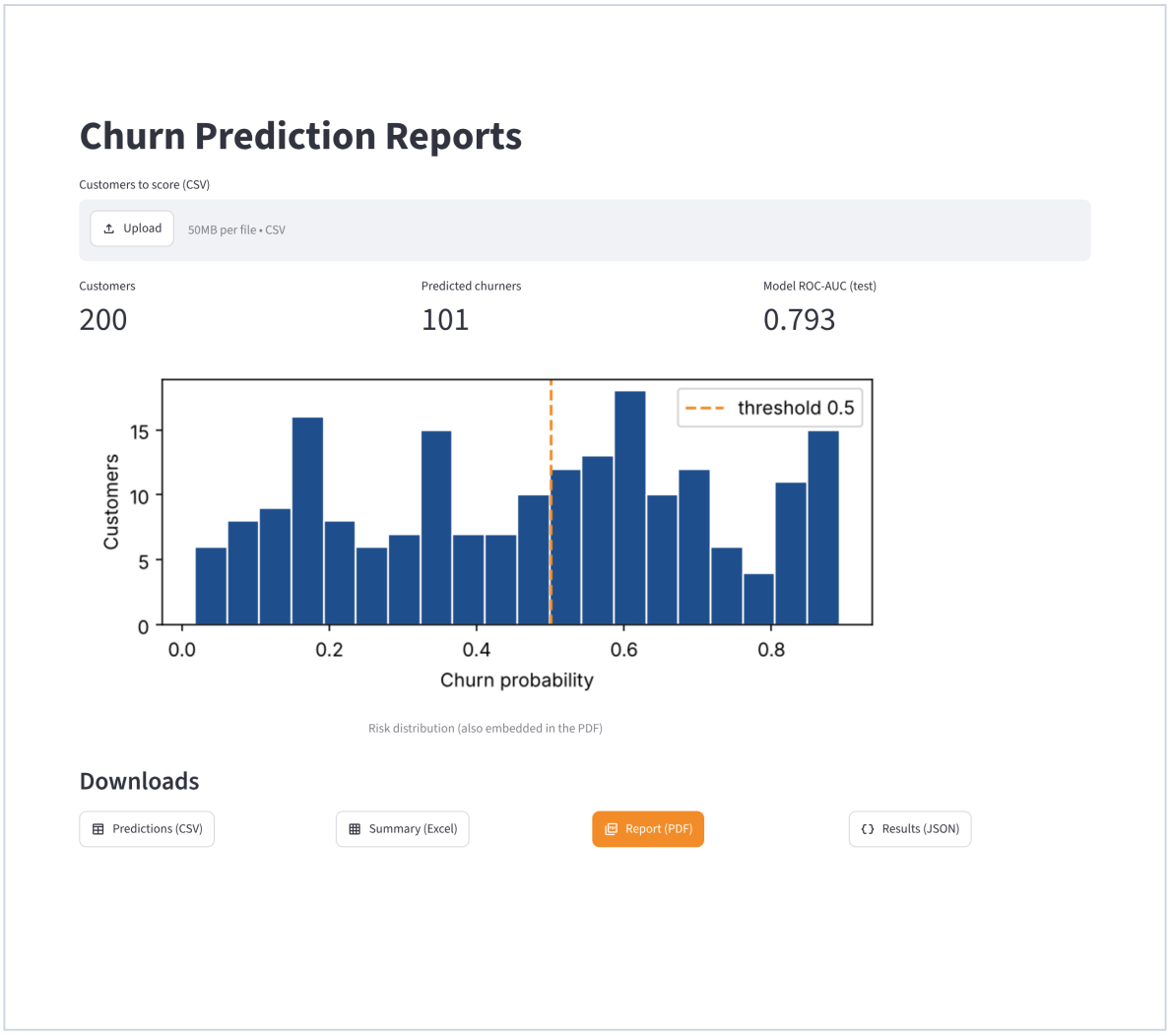


Figure 12.1: Report app: KPIs, the embedded risk chart and four download buttons

`build_reports` is decorated with `@st.cache_data`, so clicking one download does not regenerate the other files. The `on_click="ignore"` setting on each download button prevents a rerun entirely.

12.4 Verifying generated files

churn_app/tests/test_reports.py (pip install pypdf pytest) PYTHON

```python
import io

import pandas as pd
from pypdf import PdfReader

from utils.reports import risk_chart_png, to_excel_bytes, to_pdf_bytes


def test_excel_roundtrip():
    data = to_excel_bytes({"Summary": pd.DataFrame({"value": [1, 2]})})
    assert pd.read_excel(io.BytesIO(data)).shape == (2, 2)


def test_pdf_is_valid():
    pdf = to_pdf_bytes("T", pd.DataFrame({"a": [1]}), {"auc": 0.8},
                       risk_chart_png(pd.Series([0.1, 0.6, 0.9])),
                       pd.DataFrame({"b": [2]}))
    assert pdf.startswith(b"%PDF") and len(PdfReader(io.BytesIO(pdf)).pages) >= 1
```

TIP

Open each generated format at least once by hand during development: Excel may warn about invalid sheet names (> 31 characters or containing `/`), and PDFs may silently drop characters that the chosen font cannot render.

12.5 Model evaluation reports

The same helpers produce evaluation reports: pass the metrics dictionary from the metadata file (or from the evaluation dashboard in Chapter 18) and a chart such as the confusion matrix rendered to PNG.

evaluation_chart.py PYTHON

```python
import io

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402
from sklearn.metrics import ConfusionMatrixDisplay  # noqa: E402


def confusion_png(y_true, y_pred) -> bytes:
    fig, ax = plt.subplots(figsize=(4, 3.4), dpi=150)
    ConfusionMatrixDisplay.from_predictions(y_true, y_pred, ax=ax, colorbar=False,
                                            cmap="Blues")
    buf = io.BytesIO()
    fig.tight_layout()
    fig.savefig(buf, format="png")
    plt.close(fig)
    return buf.getvalue()
```

CHAPTER 13

Deployment to the Cloud

13

From localhost to a public URL

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Prepare an ML app for deployment: dependencies, models, secrets.
- ✓ Deploy from GitHub to Streamlit Community Cloud step by step.
- ✓ Deploy to a virtual machine and to AWS, Azure or Google Cloud.
- ✓ Set up basic CI/CD, logging and an update workflow.
- ✓ Manage model artifacts, large models and app visibility.

VERSION NOTE

Hosting platforms change their interfaces, quotas and pricing often. The steps below reflect the platforms as of October 2026. Always check the official pages (docs.streamlit.io/deploy and your cloud provider's docs) before a real deployment. No hosting option here should be assumed to be permanently free or unlimited.

13.1 The deployment lifecycle

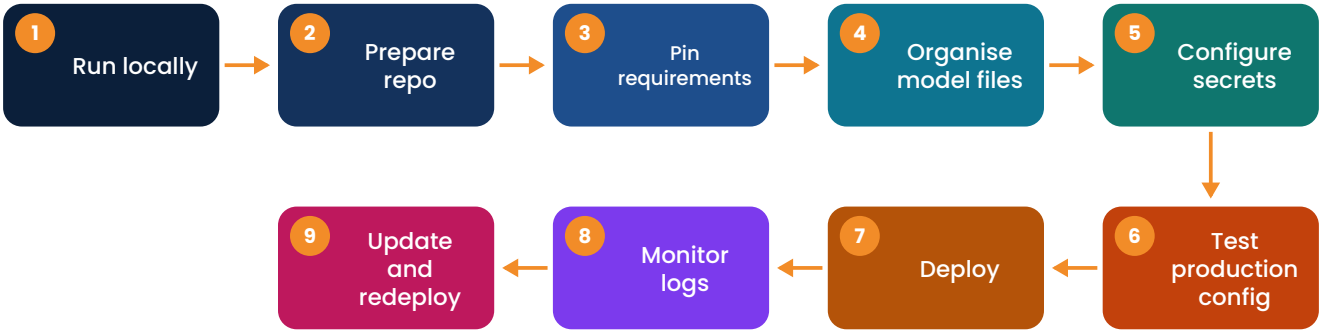


Figure 13.1: Deployment lifecycle used in this chapter

13.2 Preparing the application

Readiness checklist

- ☐ The app starts with `streamlit run app.py` from a **fresh clone** after `pip install -r requirements.txt`.
- ☐ All paths use `Path(__file__).parent`; nothing depends on your laptop's folders.
- ☐ No secrets in code or Git: `.streamlit/secrets.toml` is in `.gitignore`.
- ☐ Model artifacts are committed (if small) or downloaded at start-up (if large).
- ☐ Sample data needed by the app is in the repo; private data is not.

- [ ] `README.md` explains purpose, setup and limitations.
- [ ] Tests pass: `pytest -q`.

churn_app/.gitignore TEXT

```
.venv/
__pycache__/
.streamlit/secrets.toml
predictions.db
*.db
```

13.3 requirements.txt for deployment

churn_app/requirements.txt REQUIREMENTS

```
# Core app
streamlit==1.65.0
pandas==3.0.5
numpy==2.5.3
scikit-learn==1.9.1
plotly==7.1.0
joblib==1.6.0
# Batch Excel uploads and reports (Chapters 8 and 12)
openpyxl==3.1.5
reportlab==5.0.1
matplotlib==3.11.2
# Database (Chapter 11)
SQLAlchemy==2.1.4
psycopg[binary]>=3.2      # only needed for PostgreSQL
# API (Chapter 11)
fastapi==0.143.0
uvicorn>=0.30
requests==2.34.2
```

- Pin the libraries that affect model loading exactly (`scikit-learn==1.9.1`).
- List only what the deployed app imports; heavy training-only packages (Jupyter, xgboost if unused) slow builds and can exceed memory.
- Use `tensorflow-cpu` instead of `tensorflow` on CPU-only servers to save hundreds of MB.
- System libraries (for example `libgl1` for OpenCV) go in `packages.txt` on Community Cloud (one apt package per line) or in the Dockerfile elsewhere.

13.4 Organising model files

Model size	Recommended storage
< 25 MB	Commit to Git in `models/` with its metadata JSON
25-100 MB	Git LFS, or a release asset downloaded at start-up
> 100 MB	Object storage (S3, Azure Blob, GCS) or a model registry (MLflow, Hugging Face Hub); download once with `@st.cache_resource`

remote_model.py (requires [model] url and sha256 in secrets) PYTHON

```python
from pathlib import Path

import joblib
import requests
import streamlit as st

MODEL_URL = st.secrets["model"]["url"]          # e.g. a pre-signed S3 URL
EXPECTED_SHA256 = st.secrets["model"]["sha256"]
LOCAL = Path("/tmp/churn_pipeline.joblib")


@st.cache_resource(show_spinner="Downloading model...")
def load_remote_model():
    import hashlib

    if not LOCAL.exists():
        with requests.get(MODEL_URL, stream=True, timeout=60) as r:
            r.raise_for_status()
            with open(LOCAL, "wb") as fh:
                for chunk in r.iter_content(1 << 20):
                    fh.write(chunk)
    digest = hashlib.sha256(LOCAL.read_bytes()).hexdigest()
    if digest != EXPECTED_SHA256:                # integrity check before unpickling
        LOCAL.unlink()
        raise RuntimeError("Downloaded model failed its integrity check")
    return joblib.load(LOCAL)
```

13.5 Environment variables and secrets

Where you run	How to provide secrets
Local development	`.streamlit/secrets.toml` (git-ignored) or a `.env` file loaded by your shell
Streamlit Community Cloud	App settings -> Secrets (TOML), available as `st.secrets`
Docker	`docker run -e KEY=value` or `--env-file .env`; never `COPY` secrets into the image
VM	systemd `Environment=` / `EnvironmentFile=`, or a secret manager
AWS / Azure / GCP	Secrets Manager / Key Vault / Secret Manager injected as environment variables

13.6 Testing the production configuration locally

Simulate a fresh server TERMINAL

```
python -m venv /tmp/clean && source /tmp/clean/bin/activate
git clone https://github.com/<you>/churn-app.git /tmp/churn && cd /tmp/churn
pip install -r requirements.txt
streamlit run app.py --server.headless true --server.port 8501
```

This catches the most frequent deployment failures: a package missing from requirements, a file not committed, or a path that only exists on your laptop.

13.7 Step by step: deploy to Streamlit Community Cloud

1. **Create a GitHub repository** — Push the project folder (app.py, requirements.txt, models/, data/ samples, .streamlit/config.toml).
2. **Sign in** — Go to share.streamlit.io and sign in with GitHub; authorise access to the repository.
3. **Create app** — Choose **Create app** -> deploy from GitHub. Select the repository, the branch (`main`) and the **main file path** (`app.py`, or `churn_app/app.py` if the app is in a subfolder).
4. **Choose a URL** — Pick a custom subdomain such as `churn-predictor`.
5. **Advanced settings** — Select the Python version (match your local one) and paste the contents of your `secrets.toml` into the Secrets box.
6. **Deploy** — The platform clones the repo, installs `requirements.txt` (and `packages.txt` if present) and starts the app. Watch the build log.
7. **Share** — Open the URL, test every page, then share the link or embed it in your portfolio/README.

Item	What to know
Dependency files	The platform looks for a dependency file next to the entrypoint or at the repo root: `requirements.txt` is the usual choice; `uv.lock`, `Pipfile`, `environment.yml` and `pyproject.toml` are also recognised. Keep only one to avoid confusion.
Resources	Apps run with limited CPU, memory and storage. Large models or big uploads can exceed limits; check the 'Status and limitations' page.
Sleeping	Apps without traffic for a period are put to sleep and wake on the next visit (with a short delay).
Storage	The container's disk is temporary. Files written at run time (SQLite, uploads) are lost on reboot/redeploy.
Updates	Every push to the deployed branch redeploys automatically.
Visibility	Apps can be public or restricted to invited viewers (subject to the plan's current rules).

Diagnosing deployment failures

Log message	Cause	Fix
`ModuleNotFoundError: No module named 'plotly'`	Missing from requirements	Add it, push
`ERROR: Could not find a version that satisfies ...`	Pinned version not available for the selected Python	Change the Python version or the pin
`FileNotFoundError: models/churn_pipeline.joblib`	File not committed or wrong relative path	Commit it; build paths from `__file__`
`StreamlitSecretNotFoundError` / `KeyError` on secrets	Secrets not configured	Add them in App settings -> Secrets
App killed / 'over resource limits'	Too much memory	Smaller model, `tensorflow-cpu`, fewer cached objects, sample data
`InconsistentVersionWarning` when unpickling	scikit-learn version differs	Pin the training version

13.8 Deploying to a virtual machine

Server setup — TERMINAL

```
# on an Ubuntu VM (AWS EC2, Azure VM, GCP Compute Engine, DigitalOcean, ...)
sudo apt update && sudo apt install -y python3-venv git nginx
git clone https://github.com/<you>/churn-app.git /opt/churn && cd /opt/churn
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
```

/etc/systemd/system/churn.service — INI

```
[Unit]
Description=Churn Streamlit app
After=network.target

[Service]
User=www-data
WorkingDirectory=/opt/churn
EnvironmentFile=/etc/churn.env
ExecStart=/opt/churn/.venv/bin/streamlit run app.py --server.port 8501 \
          --server.address 127.0.0.1 --server.headless true
Restart=always

[Install]
WantedBy=multi-user.target
```

/etc/nginx/sites-available/churn (reverse proxy) — TEXT

```
server {
    listen 80;
    server_name churn.example.com;
    location / {
        proxy_pass http://127.0.0.1:8501;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;      # Streamlit needs WebSockets
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_read_timeout 86400;
    }
}
```

Start, enable HTTPS, follow logs — TERMINAL

```
sudo systemctl daemon-reload && sudo systemctl enable --now churn
sudo ln -s /etc/nginx/sites-available/churn /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
sudo apt install -y certbot python3-certbot-nginx && sudo certbot --nginx   # HTTPS
journalctl -u churn -f                                                       # logs
```

WARNING

Without the `Upgrade`/`Connection` headers the page loads but stays on 'Connecting…', because Streamlit's WebSocket cannot pass through the proxy.

13.9 Deploying with Docker and managed cloud platforms

Chapter 14 builds the Docker image. Once you have an image, every major cloud can run it:

Platform	Service for a containerised Streamlit app	Notes
AWS	App Runner, ECS on Fargate, Elastic Beanstalk, or EC2	Push the image to ECR; enable WebSocket support / sticky sessions behind a load balancer
Azure	Azure App Service (containers) or Azure Container Apps	Turn on WebSockets in App Service configuration; set `WEBSITES_PORT=8501`
Google Cloud	Cloud Run	`gcloud run deploy --port 8501`; enable session affinity for multi-instance deployments
Hugging Face Spaces	Docker Space	Good for ML portfolio demos
Snowflake	Streamlit in Snowflake	Runs next to data inside Snowflake

● Example: Google Cloud Run TERMINAL

```
gcloud builds submit --tag gcr.io/PROJECT_ID/churn-app
gcloud run deploy churn-app --image gcr.io/PROJECT_ID/churn-app \
    --port 8501 --region asia-south1 --memory 2Gi --session-affinity \
    --set-secrets DATABASE_URL=churn-db-url:latest
```

NOTE

Streamlit keeps each user's session in the memory of one server instance. When a platform runs several instances behind a load balancer, enable **session affinity** (sticky sessions) so a user's WebSocket stays on the same instance.

13.10 Git, GitHub and CI/CD basics

● First push TERMINAL

```
git init && git add . && git commit -m "Churn app v1"
git branch -M main
git remote add origin https://github.com/<you>/churn-app.git
git push -u origin main
```

● churn_app/.github/workflows/ci.yml YAML

```yaml
# Run tests on every push and pull request.  Saved as .github/workflows/ci.yml
name: ci

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.13"
          cache: pip
      - name: Install dependencies
        run: pip install -r requirements.txt pytest pypdf
      - name: Recreate data and model artifacts
        run: |
          python data/make_dataset.py
```

churn_app/.github/workflows/ci.yml YAML

```
          python train_model.py
      - name: Run tests
        run: pytest -q
```

With this workflow, every push runs the tests on GitHub's servers. Protect the `main` branch so only passing pull requests merge, and Community Cloud (or your CD pipeline) deploys only tested code.

13.11 Logs and error monitoring

logging_pattern.py PYTHON

```python
import logging

import streamlit as st

log = logging.getLogger("churn_app")
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")

try:
    risky = 1 / 0
except ZeroDivisionError:
    log.exception("prediction failed")          # full traceback in server logs
    st.error("Prediction failed. The team has been notified.")
```

- Community Cloud: open the app's **Manage app** panel to view logs.
- VM: `journalctl -u churn -f`; Docker: `docker logs -f <container>`.
- Cloud: CloudWatch (AWS), Azure Monitor, Cloud Logging (GCP).
- Add an error tracker (e.g. Sentry) for alerts with stack traces.
- Log request counts and latency, not raw personal data (Chapter 15).

13.12 Updating deployed apps and managing artifacts

Change	Safe process
UI or code fix	Branch -> PR -> CI passes -> merge -> automatic redeploy
New model version	Train -> evaluate vs current -> save with new version in metadata -> deploy to staging -> compare -> promote
Dependency upgrade	Upgrade in a branch, retrain or re-validate the model with the new versions, then deploy
Rollback	Redeploy the previous Git tag / image tag; keep old model files

BEST PRACTICE

Version models explicitly (`churn_rf_v3.joblib` + metadata with metrics and training date, or an MLflow registry). Never overwrite the only copy of a production model.

13.13 Large models and public versus private apps

- **Large models** — Prefer smaller architectures, quantisation, ONNX Runtime, or moving inference to a GPU-backed API service that Streamlit calls.
- **Start-up time** — Load lazily and cache; show a spinner on first load.
- **Public apps** — Anyone can use them: rate-limit expensive actions, cap upload sizes, never expose internal data, hide error details.

- **Private apps** — Use platform viewer restrictions, `st.login()` with an OIDC provider, or put the app behind a company SSO proxy (Chapter 15).

CHAPTER 14

Docker Deployment

14

Package the app, its dependencies and its model into one image

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Explain what Docker images and containers are and why they help ML apps.
- ✓ Write a production-minded Dockerfile and .dockerignore.
- ✓ Configure ports, environment variables and model files.
- ✓ Build, run, inspect, log and stop containers.
- ✓ Run the app with a database using Docker Compose.

Before you start: Docker Desktop (Windows/macOS) or Docker Engine (Linux).

14.1 What Docker is and why it helps

An **image** is a read-only package containing an operating-system layer, Python, your libraries, your code and your model. A **container** is a running instance of an image. Because the image contains everything, the app behaves the same on your laptop, a colleague's machine and any cloud.

Problem	How Docker solves it
'Works on my machine'	Identical environment everywhere
Library versions drift between training and serving	Versions frozen in the image
System packages (libgomp, libgl) missing on servers	Installed in the Dockerfile
Different clouds need different setups	Every cloud runs containers
Rollback is hard	Redeploy the previous image tag

14.2 The Dockerfile

churn_app/Dockerfile — DOCKERFILE

```
# ---- Streamlit ML platform image ------------------------------------
FROM python:3.13-slim

# Python writes no .pyc files and logs straight to the console
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

# install dependencies first: this layer is cached until requirements change
COPY requirements.txt .
RUN pip install --upgrade pip && pip install -r requirements.txt

# copy the application code and model artifacts
COPY . .
```

churn_app/Dockerfile DOCKERFILE

```
# run as an unprivileged user
RUN useradd --create-home appuser && chown -R appuser /app
USER appuser

EXPOSE 8501

HEALTHCHECK --interval=30s --timeout=5s --start-period=20s \
  CMD python -c "import urllib.request as u; u.urlopen('http://localhost:8501/_stcore/health')"

ENTRYPOINT ["streamlit", "run", "app.py", \
            "--server.port=8501", "--server.address=0.0.0.0"]
```

Instruction	Purpose
`FROM python:3.13-slim`	Base image: Debian with Python, without build tools (small). Match the Python version you trained with.
`ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1`	No .pyc files; logs appear immediately in `docker logs`.
`ENV PIP_NO_CACHE_DIR=1`	Do not keep pip's download cache in the image.
`WORKDIR /app`	All following commands run in /app (created if needed).
`COPY requirements.txt .` then `RUN pip install ...`	Installing dependencies **before** copying code means Docker reuses this cached layer when only code changes, making rebuilds fast.
`COPY . .`	Copies code, models and sample data (filtered by .dockerignore).
`RUN useradd ... && chown`, `USER appuser`	Run as a non-root user to limit damage if the app is compromised.
`EXPOSE 8501`	Documents the port the app listens on.
`HEALTHCHECK ... /_stcore/health`	Docker (and orchestrators) can detect a hung app; Streamlit returns `ok` at this endpoint.
`ENTRYPOINT ["streamlit", "run", "app.py", ...]`	Starts the server on all interfaces (`0.0.0.0`) so it is reachable from outside the container.

churn_app/.dockerignore TEXT

```
.git
.venv
__pycache__/
*.pyc
.pytest_cache/
.streamlit/secrets.toml
predictions.db
tests/
*.ipynb
```

SECURITY WARNING

`.dockerignore` must exclude `.streamlit/secrets.toml`, `.env` files and local databases. Anything copied into an image can be extracted by anyone who can pull the image.

14.3 Build, run, inspect and stop

Essential Docker commands TERMINAL

```
# build an image named churn-app with tag 1.0 from the current folder
docker build -t churn-app:1.0 .

# run it: map host port 8501 to container port 8501, detached, with a name
docker run -d --name churn -p 8501:8501 churn-app:1.0
# open http://localhost:8501

# pass configuration and secrets as environment variables
docker run -d --name churn -p 8501:8501 \
    -e DATABASE_URL="sqlite:////app/predictions.db" \
    --env-file .env churn-app:1.0

# inspect
docker ps                       # running containers
docker logs -f churn            # follow logs (Ctrl+C to stop following)
docker inspect --format '{{.State.Health.Status}}' churn   # healthy / unhealthy
docker exec -it churn bash      # shell inside the container
docker stats churn              # live CPU / memory use

# stop and clean up
docker stop churn && docker rm churn
docker image ls && docker image rm churn-app:1.0
```

Flag	Meaning
`-p 8501:8501`	host_port:container_port; use `-p 80:8501` to serve on port 80
`-d`	Detached (background)
`--name churn`	A memorable name for logs/stop commands
`-e KEY=value`, `--env-file`	Environment variables (configuration, secrets)
`-v $(pwd)/models:/app/models:ro`	Mount models from the host read-only instead of baking them in
`--memory 2g --cpus 1`	Resource limits that mimic production
`--restart unless-stopped`	Restart automatically after crashes/reboots

14.4 Model files: bake in or mount?

Approach	Pros	Cons
Bake into the image (`COPY models/`)	Self-contained; image tag = model version; easy rollback	Rebuild for every new model; big images
Mount a volume (`-v ...:/app/models:ro`)	Swap models without rebuilding	Image and model can drift apart; manage files on hosts
Download at start-up (object storage)	Small image; central model store	Needs network and credentials; verify integrity (Chapter 13)

14.5 Docker Compose: app + PostgreSQL

churn_app/docker-compose.yml YAML

```yaml
# Streamlit app + PostgreSQL for prediction history.  Run: docker compose up --build
services:
  app:
    build: .
    # run the database-backed app instead of the image's default app.py
    entrypoint: ["streamlit", "run", "db_app.py", "--server.port=8501",
                 "--server.address=0.0.0.0"]
    ports:
      - "8501:8501"
    environment:
      DATABASE_URL: postgresql+psycopg://mlapp:${POSTGRES_PASSWORD}@db:5432/mlapp
    depends_on:
      - db
    restart: unless-stopped

  db:
    image: postgres:17
    environment:
      POSTGRES_USER: mlapp
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}     # read from a local .env file
      POSTGRES_DB: mlapp
    volumes:
      - pgdata:/var/lib/postgresql/data
    restart: unless-stopped

volumes:
  pgdata:
```

Run the stack TERMINAL

```
echo "POSTGRES_PASSWORD=$(openssl rand -hex 16)" > .env   # git-ignored
docker compose up --build -d
docker compose logs -f app
docker compose down            # add -v to also delete the database volume
```

14.6 Production considerations

- [ ] Pin the base image (`python:3.13-slim`) and rebuild regularly for security patches; scan images (`docker scout cves` or Trivy).
- [ ] Run as non-root; give the container only the permissions it needs.
- [ ] Set memory/CPU limits and monitor them; large models need more RAM than the file size.
- [ ] Put HTTPS and authentication in front (reverse proxy, cloud load balancer).
- [ ] Enable WebSockets and session affinity on load balancers.
- [ ] Log to stdout; ship logs to a central system.
- [ ] Tag images with versions (`churn-app:1.4.0`) rather than relying on `latest`.
- [ ] Use multi-stage builds when you need compilers only at build time.

● Multi-stage Dockerfile (optional) DOCKERFILE

```dockerfile
# ---- stage 1: build wheels with compilers available
FROM python:3.13 AS builder
WORKDIR /wheels
COPY requirements.txt .
RUN pip wheel --no-cache-dir -r requirements.txt -w /wheels

# ---- stage 2: slim runtime without compilers
FROM python:3.13-slim
WORKDIR /app
COPY --from=builder /wheels /wheels
RUN pip install --no-cache-dir /wheels/* && rm -rf /wheels
COPY . .
EXPOSE 8501
ENTRYPOINT ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

CHAPTER 15

Security and Production Best Practices

15

From classroom demo to a responsible production service

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Validate inputs and restrict uploads.
- ✓ Prevent unsafe deserialisation and protect secrets.
- ✓ Add authentication and user-specific data access.
- ✓ Log safely, manage dependencies and version models.
- ✓ Distinguish a demo from a production-ready application.

15.1 Classroom demo versus production application

Aspect	Classroom demo	Production-ready application
Users	You and your classmates	Real people relying on the output
Data	Public or synthetic	Personal or confidential; legal obligations
Model	Trained once, metrics on one split	Versioned, validated on recent data, monitored for drift, retraining plan
Inputs	Trusted	Untrusted: validated, size-limited, sanitised
Secrets	Often hard-coded	Secret manager / env vars, rotated
Access	Public link	Authentication, authorisation, audit trail
Errors	Tracebacks on screen	Friendly messages; details in logs; alerts
Testing	Manual clicks	Automated tests in CI; staging environment
Operations	Laptop or free tier	Monitoring, backups, capacity planning, incident process
Responsibility	None	Documented intended use, limitations, owner

WARNING

Running in Streamlit does not make a model production-ready. Production readiness is about validation, security, monitoring and accountability, not about the user interface.

15.2 Input validation

- Bound every widget (`min_value`, `max_value`, `max_chars`, fixed `options`).
- Re-validate on the server side in Python: callbacks and session state can be manipulated, and APIs can be called directly.
- Check business rules and training ranges; refuse or warn on out-of-domain input.
- Use Pydantic models for API payloads (Chapter 11).

- Escape any user text you render as HTML (`html.escape`).

15.3 File upload restrictions and malicious files

Control	How
Allowed types	`type=["csv"]` on the widget **and** content parsing that fails safely
Size limits	`server.maxUploadSize` (global, MB) and `max_upload_size=` per widget; row-count limits after parsing
Images	Pixel-count limit; `Image.MAX_IMAGE_PIXELS`; decode in try/except
Spreadsheets	Read values only (`pd.read_excel`); never execute macros; beware CSV formula injection (`=HYPERLINK(...)`) in files you export
Archives	Avoid accepting zips; if needed, check total uncompressed size (zip bombs) and reject paths with `..`
Storage	Process in memory; if you must save, generate your own random file names in a private directory
Pickles	Never load uploaded `.pkl`, `.joblib`, `.pt` files

safe_export.py PYTHON

```python
import pandas as pd

DANGEROUS_PREFIXES = ("=", "+", "-", "@", "\t", "\r")


def neutralise_formulas(df: pd.DataFrame) -> pd.DataFrame:
    """Prevent CSV/Excel formula injection when users open exported files."""
    def fix(value):
        if isinstance(value, str) and value.startswith(DANGEROUS_PREFIXES):
            return "'" + value
        return value
    return df.map(fix)
```

15.4 Preventing unsafe model deserialisation

- Load only artifacts you produced, from storage you control.
- Verify a SHA-256 fingerprint recorded at training time before loading (`utils/model_io.py`, Chapter 6).
- Prefer formats that do not execute code where possible: Keras `.keras`, `safetensors`, ONNX, XGBoost/LightGBM native JSON/text formats.
- For PyTorch: `torch.load(path, weights_only=True)`.
- For scikit-learn, the `skops` library offers a safer persistence format that lets you audit types before loading.

15.5 Protecting secrets

- [ ] No credentials in source code, notebooks, screenshots or Docker images.
- [ ] `.streamlit/secrets.toml` and `.env` in `.gitignore` and `.dockerignore`.
- [ ] If a secret was ever committed, **rotate it**: deleting the commit is not enough.
- [ ] Least privilege: the app's database user can only insert/select its own tables.
- [ ] Separate secrets per environment (dev, staging, production).
- [ ] Never display secrets in the UI, logs or error messages.

15.6 Authentication and authorisation

Current Streamlit supports **OpenID Connect (OIDC) login** natively with `st.login()`, `st.logout()` and `st.user`. Any OIDC provider works (Google, Microsoft Entra ID, Okta, Auth0). It requires the `Authlib` package.

● .streamlit/secrets.toml (OIDC login) TOML

```toml
[auth]
redirect_uri = "http://localhost:8501/oauth2callback"   # must end in /oauth2callback
cookie_secret = "a-long-random-string-from-secrets-manager"
client_id = "your-client-id.apps.googleusercontent.com"
client_secret = "your-client-secret"
server_metadata_url = "https://accounts.google.com/.well-known/openid-configuration"
```

● auth_app.py (pip install Authlib; needs the [auth] secrets) PYTHON

```python
import streamlit as st

ADMINS = {"ashwin@example.com"}                # authorisation list (example)

if not st.user.is_logged_in:
    st.title("Churn platform")
    st.button("Log in with Google", on_click=st.login, type="primary")
    st.stop()

st.sidebar.write(f"Signed in as {st.user.name}")
st.sidebar.button("Log out", on_click=st.logout)

if st.user.email in ADMINS:
    st.success("Admin tools enabled")          # e.g. model upload, retraining
else:
    st.info("Analyst view")
```

Option	When to use
`st.login()` + OIDC provider	Most apps: company Google/Microsoft accounts
Platform viewer restrictions	Community Cloud private apps for small teams
Reverse-proxy SSO (oauth2-proxy, cloud IAP)	Enterprise: authentication handled before traffic reaches Streamlit
Hand-rolled password check	Avoid: easy to get wrong (hashing, brute force, session handling)

User-specific data access

- Filter every query by the authenticated user or their role: `WHERE owner_email = :email` with the value from `st.user.email`, never from a widget.
- Do not store one user's data in `st.cache_resource` or module globals.
- Include the user identity in cache keys for per-user cached data.
- Record who made which prediction or decision (audit trail).

15.7 Avoiding exposure of sensitive data and logging safely

Production config.toml settings TOML

```toml
[client]
showErrorDetails = "none"     # hide tracebacks from viewers in production
toolbarMode = "minimal"

[server]
enableXsrfProtection = true   # default; keep it on
```

safe_logging.py PYTHON

```python
import hashlib
import logging

log = logging.getLogger("predictions")


def log_prediction(customer_id: str, probability: float, model_version: str) -> None:
    """Log what operations need - not names, emails or raw feature values."""
    pseudo_id = hashlib.sha256(customer_id.encode()).hexdigest()[:12]
    log.info("prediction id=%s p=%.3f model=%s", pseudo_id, probability, model_version)
```

- Log counts, latencies, model versions, pseudonymous ids and error types.
- Do not log full input rows, uploaded files, free text, tokens or passwords.
- Mask identifiers in the UI (`XXXX-1234`) unless the user needs them.
- Define retention periods and delete logs and history accordingly.

15.8 Dependency management and model versioning

- Pin direct dependencies; regenerate lock files deliberately; run `pip-audit` (or GitHub Dependabot) to find known vulnerabilities.
- Version models with semantic names and metadata: training data snapshot, code commit, library versions, metrics, approval date.
- Keep a model registry (even a simple `registry.json` like Chapters 18 and 22) and display the active version in the app.
- Log the model version with every stored prediction so results are traceable.

15.9 Error handling and resource management

- Wrap model loading, file parsing, database calls and API calls in try/except with specific exception types.
- Show users what they can do next; log the technical details.
- Use an error boundary around page execution in multipage apps (Chapter 22).
- Limit rows, upload size, text length and batch sizes to protect memory.
- Close figures and connections; use context managers (`with engine.begin()`).
- Rate-limit expensive actions per session (e.g. at most N batch jobs per hour).

15.10 Data privacy and responsible ML predictions

RESPONSIBLE ML

Collect and keep only the data the model needs; anonymise when possible.

Tell users how their inputs are used and stored.

Communicate uncertainty: show probabilities and typical errors, not just labels.

State the intended use and the populations/data the model was not tested on.

Evaluate performance by subgroup and monitor for drift and bias over time.

Keep humans responsible for consequential decisions; provide a way to contest or correct outcomes.

15.11 Production monitoring

Monitor	Examples	Typical action
Service health	Uptime, error rate, latency, memory	Alert on-call, scale, fix
Data quality	Missing values, schema changes, out-of-range inputs	Fix upstream data; block bad batches
Data drift	PSI/KS per feature vs training	Investigate; retrain
Prediction drift	Distribution of scores/labels	Check for upstream changes
Model performance	Metrics on delayed labels	Retrain or roll back
Usage and feedback	Active users, thumbs up/down	Prioritise improvements

CHAPTER 16

Streamlit Project Architecture

16

Organising code so it stays maintainable as the app grows

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Choose between single-file, multipage and modular architectures.
- ✓ Separate UI code from business logic.
- ✓ Import project modules correctly from pages, tests and APIs.
- ✓ Add configuration, logging, tests and model version management.

16.1 Principles

- **Separation of concerns** — UI files (`app.py`, `app_pages/`, `ui/`) call functions; logic files (`services/`, `utils/`) contain **no** `st.` calls.
- **Single source of truth** — Feature lists, categories and ranges live in one schema/metadata place used by training, the app, tests and the API.
- **Pure functions are testable** — `predict_frame(model, X)` can be unit-tested in milliseconds without starting Streamlit.
- **Configuration is data** — Settings in TOML/YAML/env, not scattered constants.
- **Complexity on demand** — A two-hour prototype does not need ten folders. Grow the structure when the app grows.

16.2 Architecture 1: beginner (single file)

Beginner layout (Project 1) TEXT

```
iris_app/
├── app.py              # UI + loading + prediction in one file (~60 lines)
├── train_model.py
├── requirements.txt
└── models/
    ├── iris_model.joblib
    └── iris_metadata.json
```

Good for: learning, quick demos, Kaggle-style portfolio apps. Move on when app.py passes ~250 lines, when you need tests, or when a second page appears.

16.3 Architecture 2: intermediate (modular single app)

Intermediate layout (Chapter 6) TEXT

```
churn_app/
├── app.py                  # UI only
├── schema.py               # shared feature definitions
├── train_model.py
├── utils/
│   ├── __init__.py
│   ├── model_io.py         # save/load + fingerprint
│   ├── validation.py
│   ├── explain.py
│   └── charts.py
├── data/   models/   tests/
└── requirements.txt  README.md  .streamlit/config.toml
```

Good for: a serious single-purpose app with tests and a team of one or two.

16.4 Architecture 3: advanced production-style

Advanced layout (Chapter 22 capstone) TEXT

```
capstone/
├── app.py                  # router: page config, session init, navigation, error boundary
├── app_pages/              # one file per page (thin: layout + calls into services)
│   ├── home.py  upload.py  predict.py  batch.py
│   └── history.py  evaluate.py  model_info.py
├── services/               # business logic, no Streamlit imports
│   ├── config.py           # loads config/settings.toml, resolves secrets
│   ├── registry.py         # which models exist, how to load them
│   ├── model_io.py         # fingerprinted loading
│   ├── validation.py       # schema-driven validation
│   ├── predictor.py        # task-aware prediction
│   ├── evaluation.py       # metrics with safeguards
│   ├── quality.py          # data profiling
│   ├── history.py          # database persistence
│   └── reports.py          # Excel / PDF builders
├── ui/                     # reusable Streamlit components and state helpers
│   ├── components.py
│   └── state.py
├── config/settings.toml    # non-secret settings
├── models/                 # artifacts + registry.json
├── scripts/build_models.py # training / registration
├── tests/                  # unit + AppTest page tests
├── assets/  data/
├── Dockerfile  .dockerignore  requirements.txt  pytest.ini  README.md
└── .streamlit/config.toml  (+ secrets.toml, git-ignored)
```

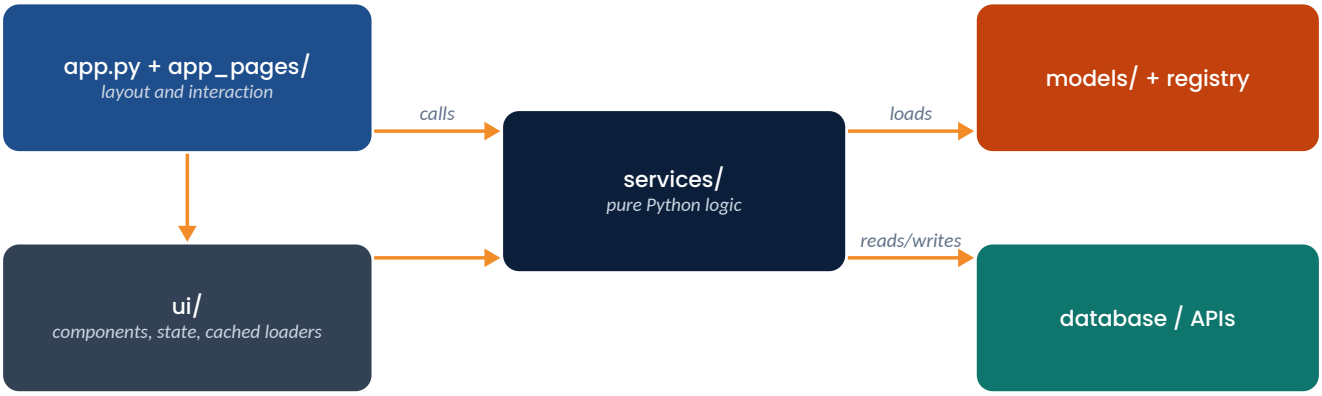


Figure 16.1: Dependencies point one way: UI depends on services, never the reverse

16.5 Importing functions correctly

When you run `streamlit run app.py`, Streamlit puts the folder containing `app.py` on `sys.path`. Every page and module can therefore use **absolute imports from the project root**:

app_pages/predict.py (excerpt) — PSEUDOCODE / FRAGMENT

```
# in app_pages/predict.py (a subfolder) - works because the project root is on sys.path
from services.predictor import predict_one
from ui.components import page_header, schema_form
```

Situation	Make imports work by...
`streamlit run app.py` from the project root	Nothing extra: root is on the path
Running a script inside a subfolder	`python -m scripts.build_models` from the root (module mode)
pytest	`pytest.ini` with `pythonpath = .`
FastAPI app in `api/`	Run `uvicorn api.main:app` from the root, or insert the root into `sys.path` (Chapter 11)
Folders as packages	Add `__init__.py` to `services/`, `ui/`, `utils/`

COMMON MISTAKE

Naming a module after a library (`streamlit.py`, `pandas.py`, `typing.py`) or a project folder `test/`. Python imports your file instead of the real package and fails with confusing errors.

16.6 Configuration files

capstone/config/settings.toml — TOML

```
# Application settings. Safe to commit: contains NO secrets.
# Secrets (database passwords, API keys) go in .streamlit/secrets.toml or env vars.

[app]
name = "ML Deployment Platform"
environment = "development"        # development | staging | production

[models]
registry = "models/registry.json"
```

capstone/config/settings.toml TOML

```toml
[uploads]
max_rows = 100000
allowed_types = ["csv", "xlsx"]

[history]
# overridden by st.secrets["database"]["url"] or the DATABASE_URL env variable
default_url = "sqlite:///history.db"

[thresholds]
classification_default = 0.5
```

capstone/services/config.py PYTHON

```python
"""Load config/settings.toml once and resolve secrets from the environment."""
import os
import tomllib
from functools import lru_cache
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]


@lru_cache
def settings() -> dict:
    with open(ROOT / "config" / "settings.toml", "rb") as fh:
        return tomllib.load(fh)


def database_url(secrets=None) -> str:
    """Priority: Streamlit secrets -> DATABASE_URL env var -> settings default."""
    if secrets is not None:
        try:
            return secrets["database"]["url"]
        except (KeyError, FileNotFoundError):
            pass
    url = os.environ.get("DATABASE_URL") or settings()["history"]["default_url"]
    if url.startswith("sqlite:///") and not url.startswith("sqlite:////"):
        url = f"sqlite:///{ROOT / url.removeprefix('sqlite:///')}"   # project-relative
    return url
```

16.7 Logging

logging_setup.py PYTHON

```python
import logging

logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(name)s: %(message)s")
log = logging.getLogger("platform.predict")
log.info("model=%s rows=%d latency_ms=%.1f", "churn_rf", 200, 38.2)
```

Configure logging once in `app.py`; modules call `logging.getLogger(__name__)`. Streamlit's own log level is set with `logger.level = "info"` in config.toml.

16.8 Testing

Test type	Tool	Example
Unit tests of logic	pytest	`validate_record` rejects tenure 999
Model contract tests	pytest	Every registered model loads and predicts on sample rows with the right shape
App tests	`streamlit.testing.v1.AppTest`	Every page renders for every model; clicking Predict logs history
End-to-end browser tests	Playwright	Upload a file and download results

capstone/tests/test_services.py PYTHON

```python
import pandas as pd

from services import registry
from services.predictor import predict_frame
from services.validation import validate_frame, validate_record


def _churn():
    entry = registry.get_entry("churn_rf")
    model, meta, _ = registry.load(entry)
    return entry, model, meta


def test_registry_loads_every_model():
    for e in registry.list_models():
        model, meta, _ = registry.load(e)
        assert meta["features"]


def test_validate_record_rejects_out_of_range():
    _, _, meta = _churn()
    rec = {f: s["default"] for f, s in meta["input_schema"].items()}
    rec["tenure_months"] = 999
    assert validate_record(rec, meta["input_schema"])


def test_batch_prediction_shapes():
    entry, model, meta = _churn()
    df = pd.read_csv("data/new_customers.csv").head(25)
    X, errors, _ = validate_frame(df, meta["input_schema"], 1000)
    assert not errors
    out = predict_frame(model, entry, meta, X)
    assert len(out) == 25 and out["probability"].between(0, 1).all()


def test_missing_column_is_reported():
    _, _, meta = _churn()
    df = pd.read_csv("data/new_customers.csv").drop(columns=["contract"])
    _, errors, _ = validate_frame(df, meta["input_schema"], 1000)
    assert errors and "contract" in errors[0]
```

capstone/tests/test_pages.py PYTHON

```python
"""Every page renders for every registered model.  Run: pytest -q"""
import pytest
from streamlit.testing.v1 import AppTest

from services import registry

PAGES = ["app_pages/home.py", "app_pages/upload.py", "app_pages/predict.py",
         "app_pages/batch.py", "app_pages/history.py", "app_pages/evaluate.py",
```

capstone/tests/test_pages.py PYTHON

```python
        "app_pages/model_info.py"]


@pytest.fixture()
def app():
    return AppTest.from_file("../app.py", default_timeout=90).run()


@pytest.mark.parametrize("model_id", [e.id for e in registry.list_models()])
def test_all_pages_render(app, model_id):
    app.selectbox(key="model_id").set_value(model_id).run()
    for page in PAGES:
        app.switch_page(page).run()
        assert not app.exception, (page, app.exception)
        assert not app.error, (page, [e.value for e in app.error])


def test_single_prediction_logs_history(app):
    app.switch_page("app_pages/predict.py").run()
    app.button[0].click().run()
    assert not app.exception
    app.switch_page("app_pages/history.py").run()
    assert app.dataframe, "history table should be shown after one prediction"


def test_batch_flow_with_sample_data(app):
    app.switch_page("app_pages/upload.py").run()
    app.button[0].click().run()                    # "Load sample customer file"
    assert app.session_state.dataset is not None
    app.switch_page("app_pages/batch.py").run()
    [b for b in app.button if b.label == "Score dataset"][0].click().run()
    assert not app.exception
    assert app.session_state.batch_result["result"].shape[0] == 200
```

pytest -q (capstone) OUTPUT

```
.........                                                                [100%]
9 passed in 7.02s
```

16.9 Model version management

capstone/services/registry.py PYTHON

```python
"""Model registry: which models exist, what they expect, and how to load them."""
import json
from dataclasses import dataclass
from pathlib import Path

from services.config import ROOT, settings
from services.model_io import load_trusted_model


@dataclass(frozen=True)
class ModelEntry:
    id: str
    name: str
    task: str                # "classification" | "regression"
    model_file: str
    meta_file: str
    target: str
    positive_class: str | None = None
    description: str = ""


def registry_path() -> Path:
```

capstone/services/registry.py PYTHON

```python
    return ROOT / settings()["models"]["registry"]


def list_models() -> list[ModelEntry]:
    raw = json.loads(registry_path().read_text())
    return [ModelEntry(**item) for item in raw]


def get_entry(model_id: str) -> ModelEntry:
    for entry in list_models():
        if entry.id == model_id:
            return entry
    raise KeyError(f"Unknown model id: {model_id}")


def load(entry: ModelEntry):
    """Return (model, metadata, warnings); raises ModelLoadError."""
    folder = registry_path().parent
    return load_trusted_model(folder / entry.model_file, folder / entry.meta_file)
```

- The registry lists every model id, task, files and target; the UI offers only registered models.
- Each metadata file records metrics, input schema, training time, library versions and a fingerprint.
- Promote a model by adding a registry entry; roll back by removing it. Old artifacts stay available.
- For teams, MLflow's model registry adds stages (Staging, Production), lineage and an API; the app can load `models:/churn/Production`.

CHAPTER 17

Complete Hands-On Projects

17

Ten portfolio projects of increasing difficulty

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Plan an ML app from business problem to deployment.
- ✓ Implement, run and test ten complete projects.
- ✓ Extend projects into strong portfolio pieces.

#	Project	ML task	Level	New skills
1	Iris Flower Classification	Multiclass classification	Beginner	First app, sliders, cached model
2	House Price Prediction	Regression	Beginner+	Forms, range checks, INR formatting
3	Customer Churn Prediction	Binary classification	Intermediate	Tabs, explanation, batch, metrics
4	Customer Segmentation Dashboard	Clustering	Intermediate	Upload, column mapping, profiles
5	Sales Forecasting Application	Time series	Intermediate	Date ranges, intervals, back-test
6	Sentiment Analysis Application	NLP classification	Intermediate	Text pipelines, batch text
7	Image Classification Application	Deep learning (CNN)	Advanced	Keras/PyTorch, image preprocessing
8	Fraud Detection Application	Anomaly detection	Advanced	Unsupervised scores, thresholds
9	Resume Classification Application	Document classification	Advanced	PDF/DOCX extraction, PII, fairness
10	Model Evaluation & Deployment Dashboard	MLOps	Advanced	Registry, safeguards, release gate

NOTE

Projects 2-8 share their core code with the worked examples in Chapter 6, so the book contains one tested version of each file. Each project below lists every file with the place its full listing appears, and adds the project's remaining files (requirements, tests).

Project 1: Iris Flower Classification

Level: Beginner | **Folder:** `iris_app/`

Item	Details
Business problem	A botany club wants a simple tool that identifies the iris species from four flower measurements.
Project objective	Deploy a first ML app: load a saved model once, collect four inputs, show the predicted species and class probabilities.
Dataset requirements	Fisher's Iris dataset (150 rows, 4 numeric features, 3 classes), loaded from scikit-learn: no download needed.
Model requirements	StandardScaler + LogisticRegression pipeline; accuracy above 90% on a 25% stratified test split.

Item	Details
Streamlit components	`set_page_config`, `sidebar`, `slider`, `columns`, `metric`, `bar_chart`, `expander`, `dataframe`, `cache_resource`.

UI design

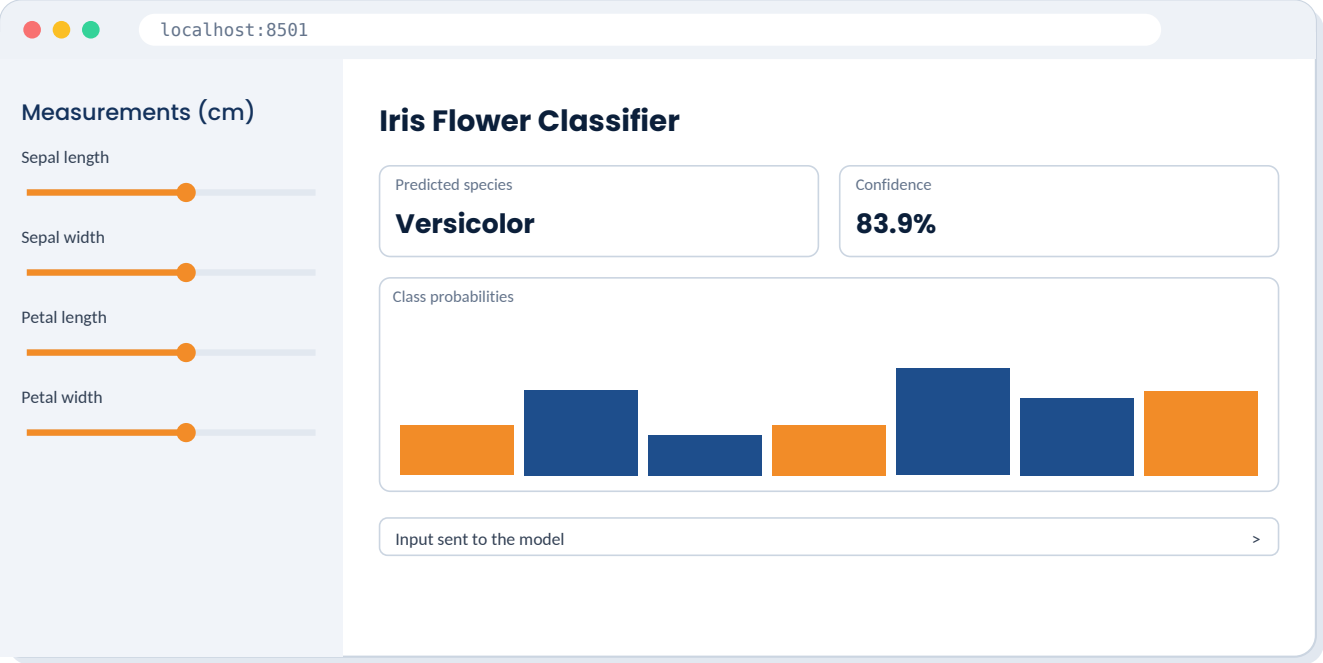


Figure 17.1: Project 1 wireframe

Application architecture



Implementation (file by file)

iris_app/train_model.py PYTHON

```python
"""Train an Iris classifier and save it.  Run: python train_model.py"""
import json

import joblib
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

iris = load_iris(as_frame=True)
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify=y,
                                                    random_state=42)

model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=500))
model.fit(X_train, y_train)
accuracy = accuracy_score(y_test, model.predict(X_test))
```

iris_app/train_model.py PYTHON

```python
joblib.dump(model, "models/iris_model.joblib")
with open("models/iris_metadata.json", "w") as f:
    json.dump({
        "feature_names": list(X.columns),
        "target_names": list(iris.target_names),
        "feature_ranges": {c: [float(X[c].min()), float(X[c].max())] for c in X.columns},
        "accuracy": round(accuracy, 4),
    }, f, indent=2)
print(f"Test accuracy: {accuracy:.3f}")
```

iris_app/app.py PYTHON

```python
"""Iris flower classifier - a first ML app.  Run: streamlit run app.py"""
import json

import joblib
import pandas as pd
import streamlit as st

st.set_page_config(page_title="Iris Classifier", page_icon=":material/local_florist:")


@st.cache_resource
def load_model():
    model = joblib.load("models/iris_model.joblib")
    with open("models/iris_metadata.json") as f:
        meta = json.load(f)
    return model, meta


model, meta = load_model()

st.title("Iris Flower Classifier")
st.write("Move the sliders to describe a flower. The model predicts its species.")
st.caption(f"Logistic regression | test accuracy {meta['accuracy']:.1%}")

with st.sidebar:
    st.header("Flower measurements (cm)")
    values = {}
    for feature in meta["feature_names"]:
        lo, hi = meta["feature_ranges"][feature]
        values[feature] = st.slider(feature.replace(" (cm)", "").title(),
                                    min_value=lo, max_value=hi,
                                    value=round((lo + hi) / 2, 1), step=0.1)

X = pd.DataFrame([values])                    # same column names as training
probabilities = model.predict_proba(X)[0]
predicted = meta["target_names"][probabilities.argmax()]

col1, col2 = st.columns(2)
col1.metric("Predicted species", predicted.title())
col2.metric("Confidence", f"{probabilities.max():.1%}")

st.subheader("Class probabilities")
prob_df = pd.DataFrame({"species": meta["target_names"], "probability": probabilities})
st.bar_chart(prob_df, x="species", y="probability")

with st.expander("Input sent to the model"):
    st.dataframe(X, hide_index=True)
```

iris_app/requirements.txt REQUIREMENTS

```
streamlit==1.65.0
pandas>=2.2
scikit-learn==1.9.1
joblib>=1.4
```

iris_app/tests/test_app.py PYTHON

```python
from streamlit.testing.v1 import AppTest


def test_prediction_shown():
    at = AppTest.from_file("../app.py", default_timeout=60).run()
    assert not at.exception
    assert at.metric[0].value in {"Setosa", "Versicolor", "Virginica"}


def test_small_petals_are_setosa():
    at = AppTest.from_file("../app.py", default_timeout=60).run()
    at.slider[2].set_value(1.2)          # petal length
    at.slider[3].set_value(0.2).run()    # petal width
    assert at.metric[0].value == "Setosa"
```

Installation and execution

Run Project 1 TERMINAL

```
cd iris_app
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
mkdir -p models
python train_model.py
streamlit run app.py
pytest -q
```

Expected output

`python train_model.py` prints `Test accuracy: 0.921`. The app shows a predicted species and a probability bar chart that updates instantly as you move the sliders.

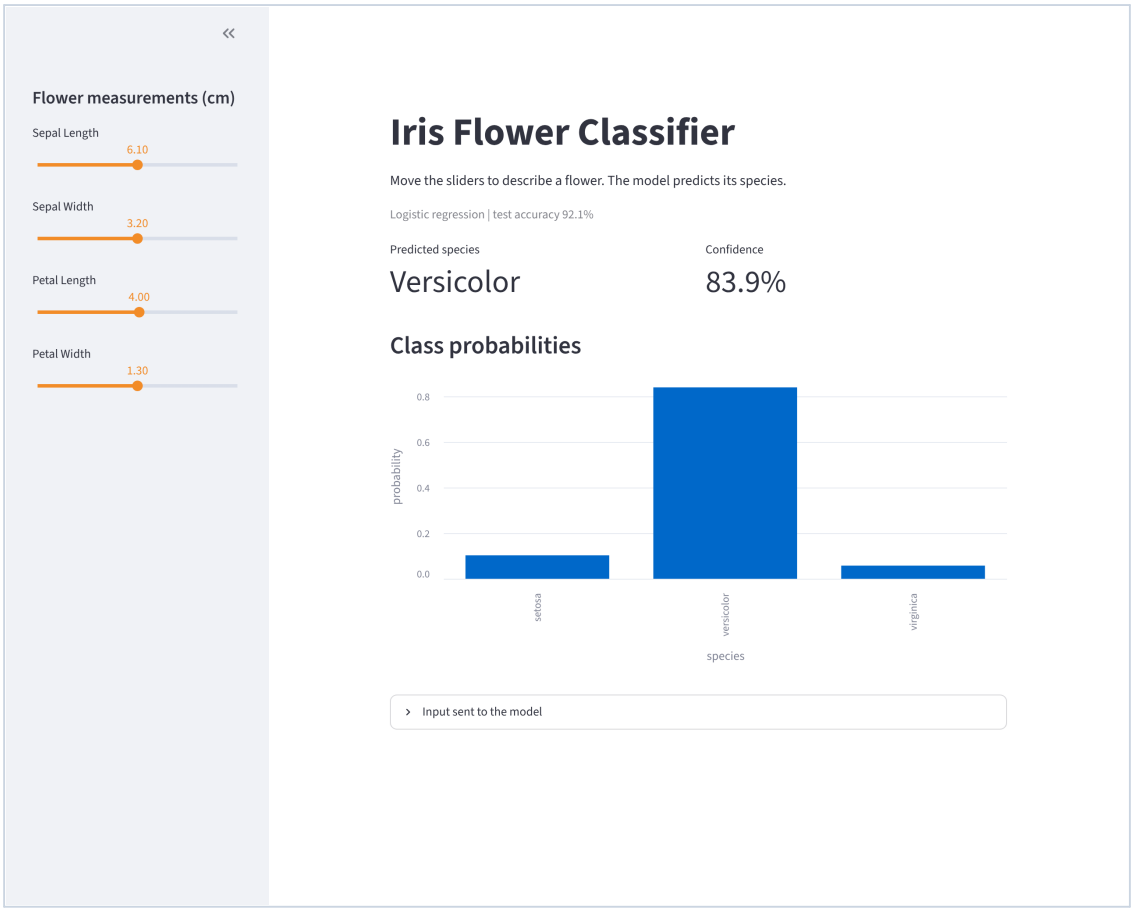


Figure 17.2: Project 1 running

Testing checklist

- [ ] App loads without exceptions.
- [ ] Small petals (length 1.2, width 0.2) predict Setosa.
- [ ] Probabilities sum to 1.
- [ ] Slider ranges match the training data.

Deployment

Push the folder to GitHub with `models/` committed, then deploy on Community Cloud with main file `app.py` (Chapter 13) or build the Docker image from Chapter 14.

Possible improvements

- Add a scatter plot of the training data with the user's flower highlighted.
- Compare two models in tabs.
- Show a confusion matrix on an About page.

Project 2: House Price Prediction

Level: Beginner+ | **Folder:** `house_app/`

Item	Details
Business problem	A real-estate portal needs instant indicative prices for Pune apartments while agents talk to customers.
Project objective	Estimate price from eight property features with input validation, training-range warnings and INR formatting.
Dataset requirements	`houses.csv` (3,000 rows): area, bedrooms, bathrooms, age, floor, locality, furnishing, parking, price. Swap in a real listings file with the same columns.
Model requirements	OneHotEncoder + HistGradientBoostingRegressor pipeline; R-squared about 0.92, MAPE about 7% on the test split.
Streamlit components	`form`, `columns`, `number_input`, `selectbox`, `slider`, `radio`, `toggle`, `metric(border=True)`, `warning`, Plotly scatter.

UI design

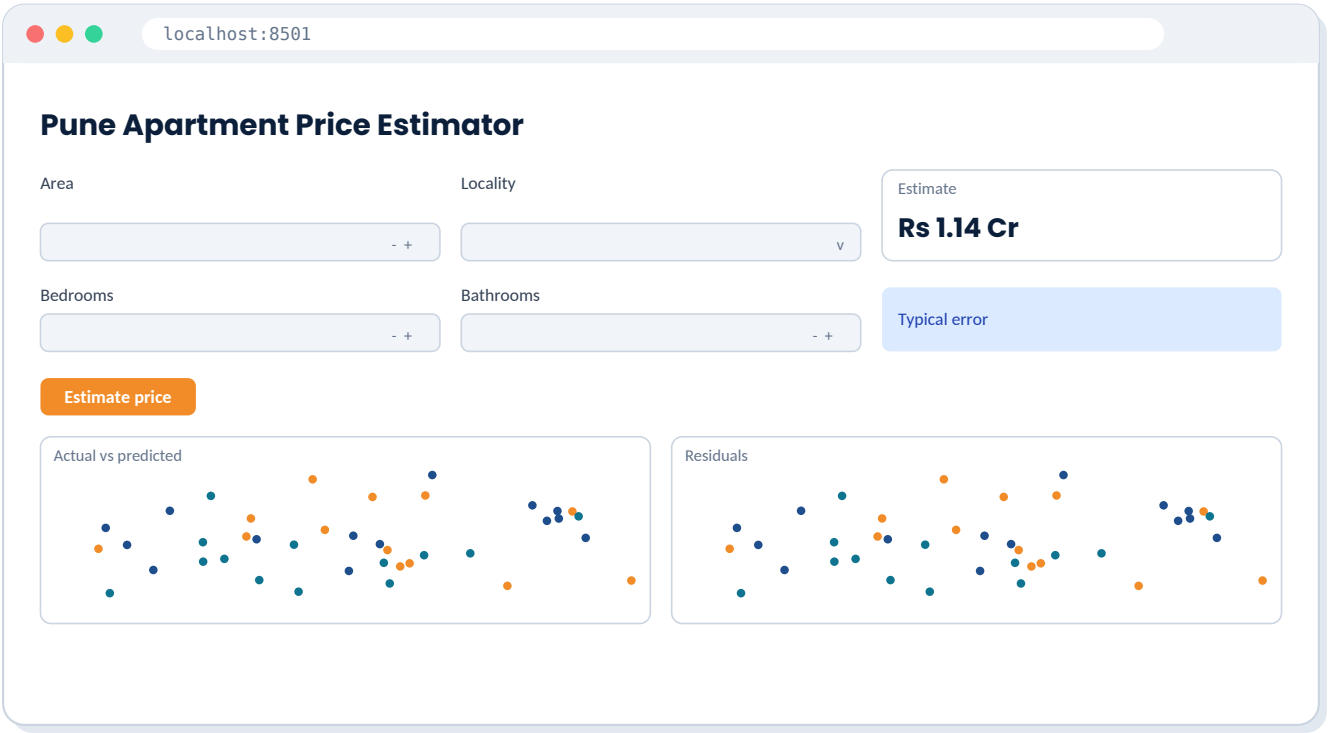


Figure 17.3: Project 2 wireframe

Application architecture



Implementation (file by file)

File	Purpose	Listing
schema.py	Feature lists, options, hard limits	Section 6.5
data/make_dataset.py	Synthetic Pune listings	Section 6.5
train_model.py	Train, evaluate, save with ranges	Section 6.5
utils/formatting.py	Lakh/crore formatting	Section 6.5
utils/model_io.py	Fingerprinted save/load	Section 6.3
app.py	Streamlit UI	Section 6.5
requirements.txt, tests/test_app.py	Dependencies, tests	below

house_app/requirements.txt REQUIREMENTS

```
streamlit==1.65.0
pandas==3.0.5
numpy==2.5.3
scikit-learn==1.9.1
plotly==7.1.0
joblib==1.6.0
```

house_app/tests/test_app.py PYTHON

```python
from streamlit.testing.v1 import AppTest

from utils.formatting import format_inr, format_lakh


def test_indian_grouping():
    assert format_inr(8740000) == "₹87,40,000"
    assert format_lakh(8740000) == "₹87.4 lakh"


def test_estimate_and_range_warning():
    at = AppTest.from_file("../app.py", default_timeout=60).run()
    # widgets inside a form: set values, then submit in the SAME run
    at.number_input[0].set_value(5800)              # far above training range
    at.button[0].click().run()
    assert not at.exception
    assert any("outside the typical training range" in w.value for w in at.warning)
```

Installation and execution

Run Project 2 TERMINAL

```
cd house_app && pip install -r requirements.txt
python data/make_dataset.py && python train_model.py
streamlit run app.py
pytest -q
```

Expected output

A 1,050 sq ft 2BHK in Baner returns roughly 1.1 crore with a typical error of about 8 lakh; entering 5,800 sq ft shows a range warning.

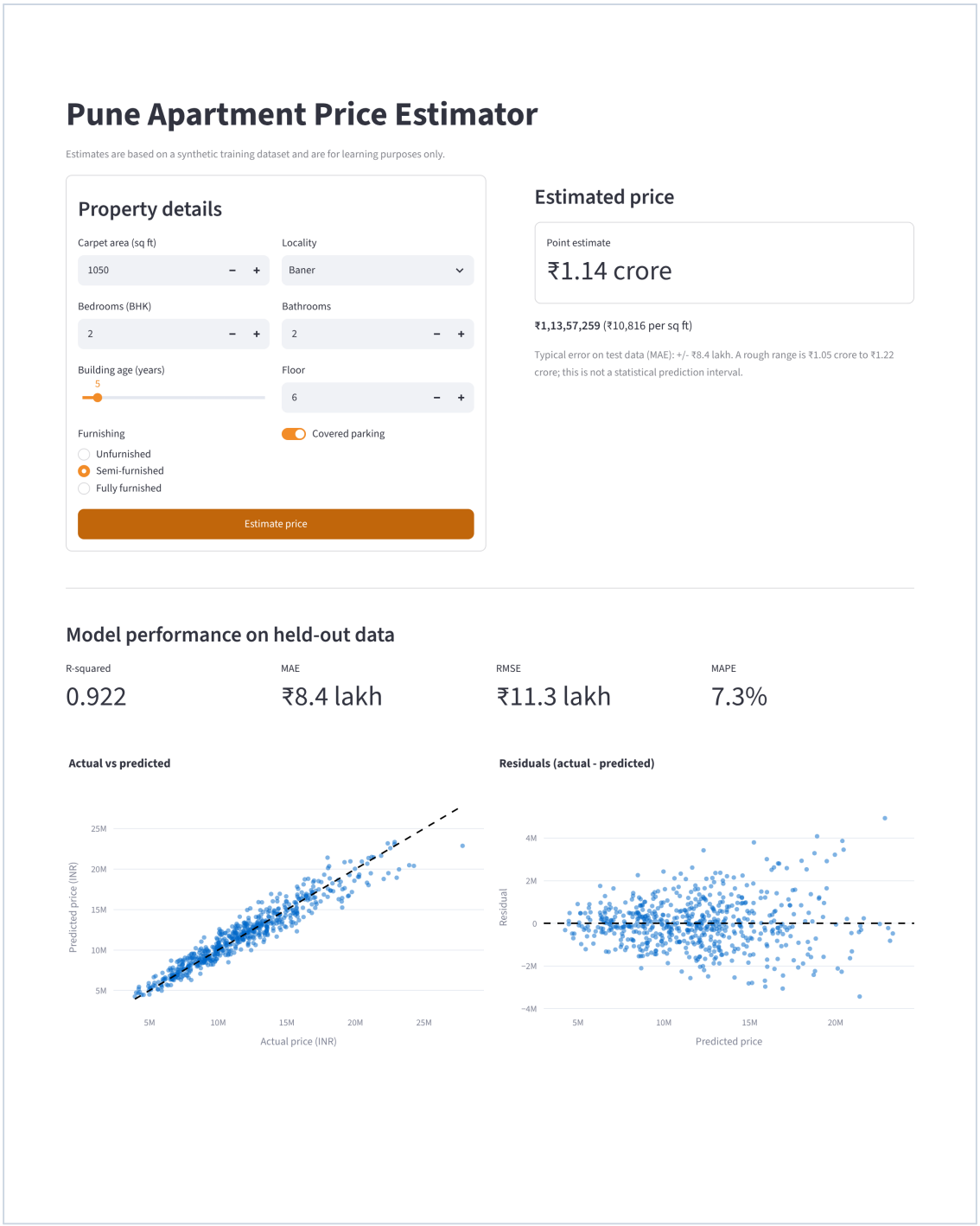


Figure 17.4: Project 2 running

Testing checklist

- [ ] Form validation blocks bathrooms > bedrooms + 1.
- [ ] Out-of-range inputs show a warning.
- [ ] `format_inr(8740000)` returns Rs 87,40,000 grouping.
- [ ] Charts render on the performance section.

Deployment

Push the folder to GitHub with `models/` committed, then deploy on Community Cloud with main file `app.py` (Chapter 13) or build the Docker image from Chapter 14.

Possible improvements

- Quantile models for a genuine 80% prediction interval.
- A map of localities with median price.
- SHAP explanation per estimate.

Project 3: Customer Churn Prediction

Level: Intermediate | **Folder:** `churn_app/`

Item	Details
Business problem	A telecom operator loses revenue to churn and wants to prioritise retention calls.
Project objective	Score single customers and files, explain risk, and show model performance to stakeholders.
Dataset requirements	`churn.csv`: 5,000 labeled customers with 9 features; `new_customers.csv` unlabeled for scoring. Compatible with the IBM Telco schema after renaming columns.
Model requirements	Imputer + scaler + one-hot + RandomForest (balanced) pipeline; ROC-AUC about 0.79, recall about 0.74.
Streamlit components	`tabs`, `form`, `slider` (threshold), `metric`, `progress`, `expander`, `file_uploader`, `column_config.ProgressColumn`, `download_button`, Plotly charts, `session_state`.

UI design

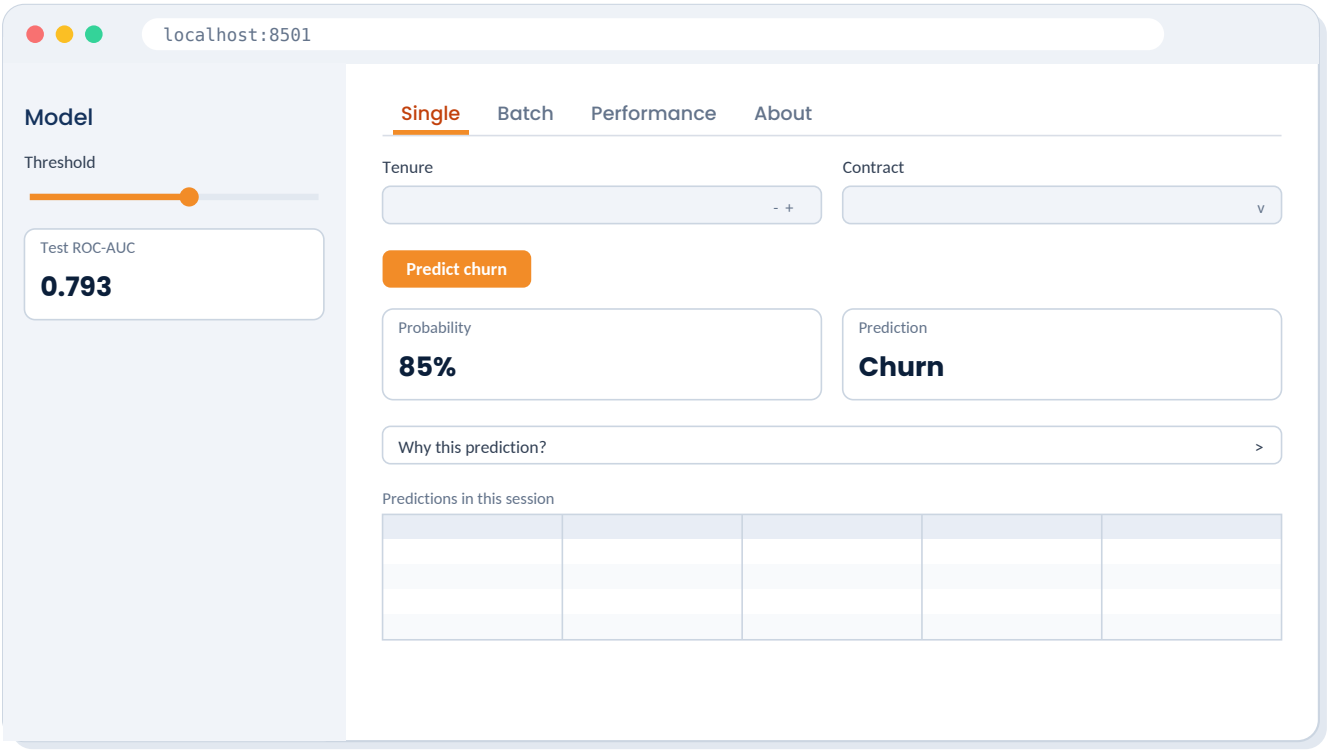


Figure 17.5: Project 3 wireframe

Application architecture



Implementation (file by file)

File	Purpose	Listing
schema.py, data/make_dataset.py	Schema and data	Section 6.4
train_model.py	Training and metadata	Section 6.4
utils/model_io.py	Integrity-checked loading	Section 6.3
utils/validation.py, explain.py, charts.py	Helpers	Section 6.4
app.py, tests/test_app.py	UI and tests	Section 6.4
batch_app.py	Batch scoring	Chapter 8
multistep_app.py	Wizard	Chapter 10
db_app.py, utils/db.py, api/	Database and API	Chapter 11
report_app.py, utils/reports.py	Reports	Chapter 12
Dockerfile, docker-compose.yml, ci.yml	Deployment	Chapters 13-14
README.md	Project documentation	below

churn_app/README.md — TEXT

```
# Customer Churn Prediction (Streamlit)

Predicts the probability that a telecom customer cancels, with single and batch
scoring, explanations and a model performance dashboard.

## Run locally
```bash
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python data/make_dataset.py      # creates data/churn.csv and data/new_customers.csv
python train_model.py            # creates models/
streamlit run app.py
```

Other apps in this folder
File	Purpose
`batch_app.py`	Batch scoring of CSV/Excel files (Chapter 8)
`multistep_app.py`	Five-step prediction wizard (Chapter 10)
`db_app.py`	Predictions stored in a database (Chapter 11)
`api/main.py` + `api_client_app.py`	FastAPI model service and client (Chapter 11)
`report_app.py`	CSV / Excel / PDF / JSON reports (Chapter 12)

Tests
```bash
pytest -q
```

Limitations
Trained on synthetic data for teaching. Predictions are risk estimates to prioritise
retention outreach, not decisions about individual customers.
```

Installation and execution

Run Project 3 — TERMINAL

```
cd churn_app && pip install -r requirements.txt
python data/make_dataset.py && python train_model.py
streamlit run app.py
pytest -q
```

Expected output

Single prediction shows probability, decision, explanation and history; the batch tab scores 200 customers; the performance tab shows the confusion matrix, ROC and PR curves.

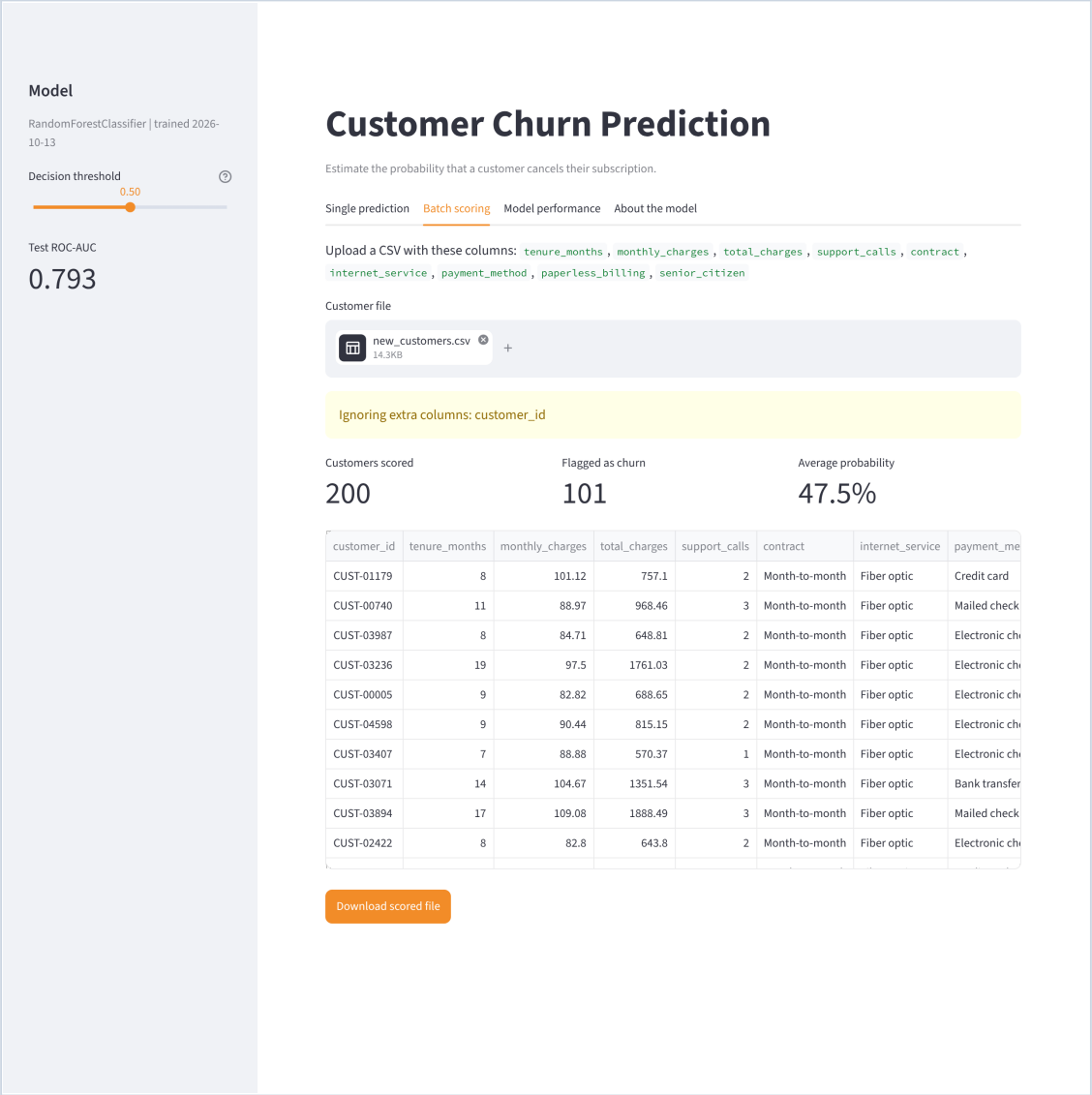


Figure 17.6: Project 3 running

Testing checklist

- [ ] Invalid totals (below half a monthly charge) are rejected.
- [ ] Missing columns in uploads produce errors, unseen categories warnings.
- [ ] Changing the threshold changes flagged counts.
- [ ] Fingerprint mismatch blocks loading (edit the JSON hash to test).

Deployment

Push the folder to GitHub with `models/` committed, then deploy on Community Cloud with main file `app.py` (Chapter 13) or build the Docker image from Chapter 14.

Possible improvements

- Expected-value prioritisation: risk x customer lifetime value.
- SHAP waterfall plots.

- Store predictions with db_app.py.

Project 4: Customer Segmentation Dashboard

Level: Intermediate | **Folder:** `segment_app/`

Item	Details
Business problem	A mall's marketing team wants to understand customer groups for targeted campaigns.
Project objective	Assign uploaded customers to saved segments and describe each segment in business terms.
Dataset requirements	Columns age, annual_income_k, spending_score (any names; mapped in the UI). A sample `mall_customers.csv` is generated.
Model requirements	StandardScaler + KMeans(k=5) in one pipeline; silhouette about 0.5; segment names derived from centroids.
Streamlit components	`radio`, `file_uploader` (CSV/XLSX), `selectbox` mapping, `tabs`, Plotly scatter, `bar_chart(horizontal=True)`, download.

UI design

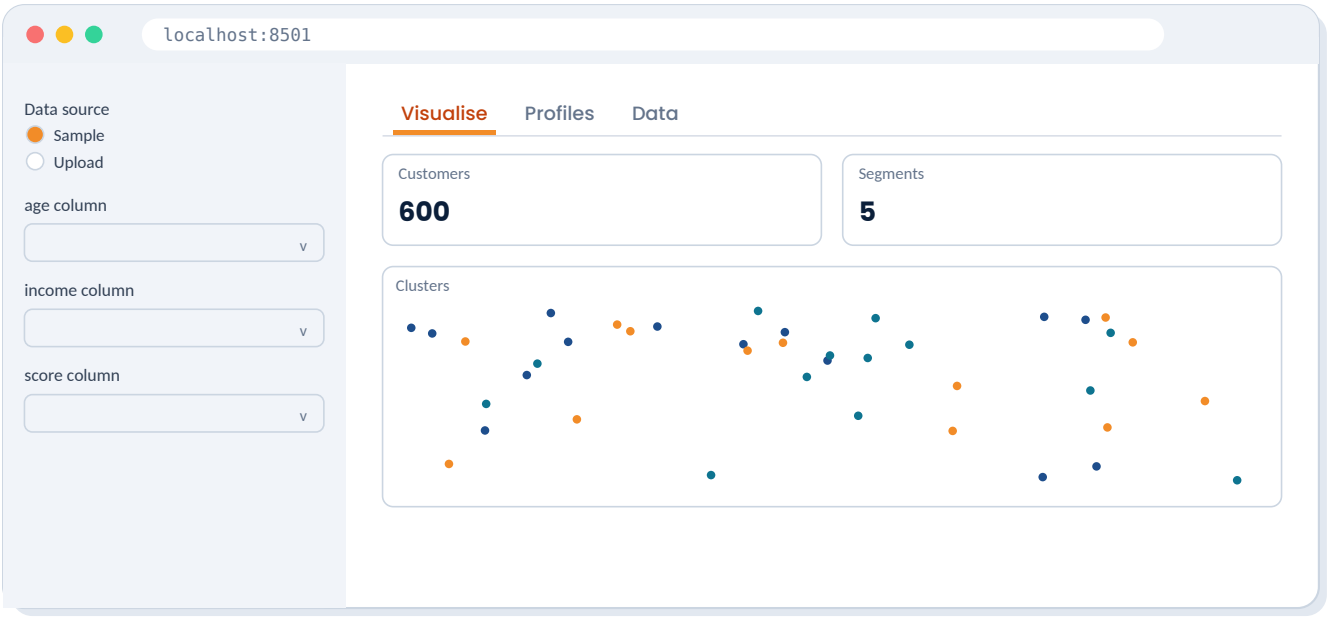


Figure 17.7: Project 4 wireframe

Application architecture



Implementation (file by file)

File	Purpose	Listing
train_model.py	Data + pipeline + names	Section 6.6
app.py	Dashboard	Section 6.6
requirements.txt, tests/test_app.py	Deps, tests	below

segment_app/tests/test_app.py PYTHON

```python
import json
from pathlib import Path

import joblib
import pandas as pd
from streamlit.testing.v1 import AppTest

ROOT = Path(__file__).parents[1]


def test_saved_pipeline_assigns_known_clusters():
    pipe = joblib.load(ROOT / "models" / "segment_pipeline.joblib")
    meta = json.loads((ROOT / "models" / "segment_metadata.json").read_text())
    X = pd.DataFrame([{"age": 25, "annual_income_k": 25, "spending_score": 80}])
    assert str(int(pipe.predict(X)[0])) in meta["segment_names"]


def test_dashboard_renders():
    at = AppTest.from_file("../app.py", default_timeout=60).run()
    assert not at.exception and at.metric[0].value == "600"
```

Installation and execution

Run Project 4 TERMINAL

```
cd segment_app && pip install -r requirements.txt
python train_model.py && streamlit run app.py
pytest -q
```

Expected output

Five named segments, an interactive scatter coloured by segment and a profile table in original units.

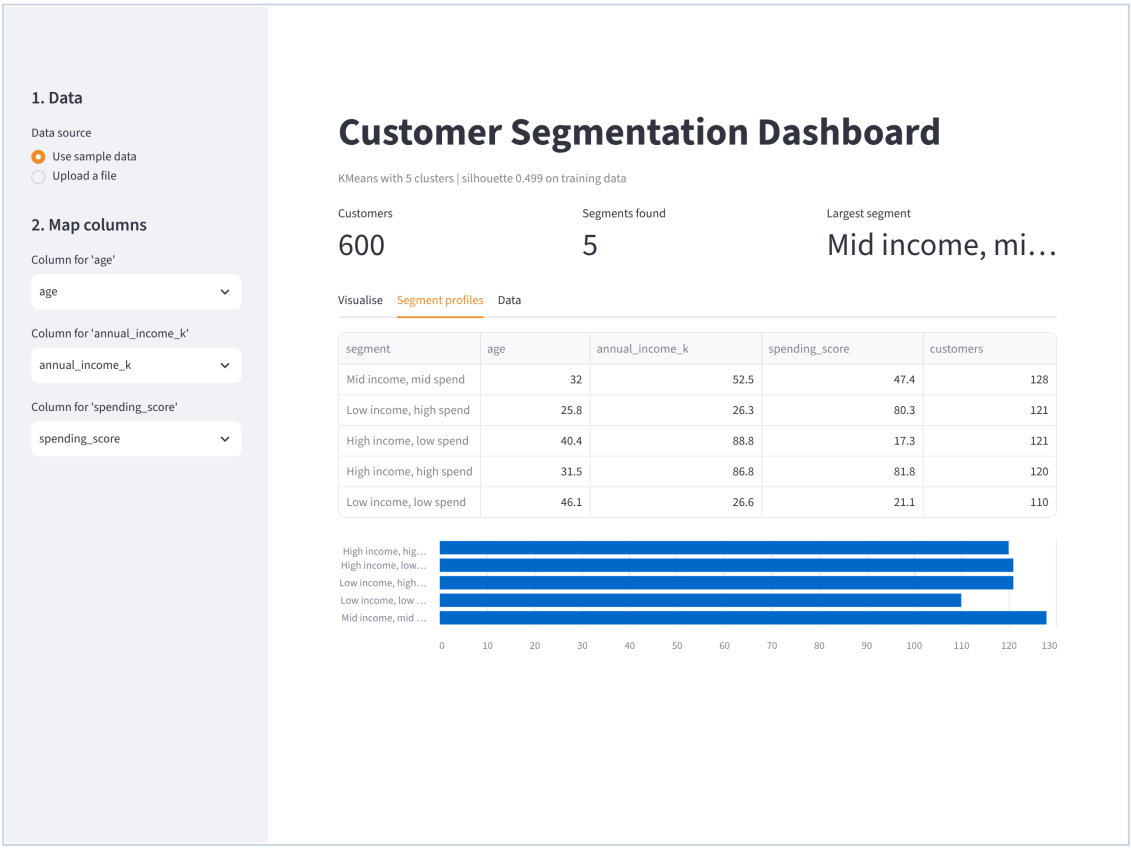


Figure 17.8: Project 4 running

Testing checklist

- [ ] Mapping the same column twice shows an error.
- [ ] Rows with missing values are reported and excluded.
- [ ] A known profile (young, low income, high spend) gets the expected segment.

Deployment

Push the folder to GitHub with `models/` committed, then deploy on Community Cloud with main file `app.py` (Chapter 13) or build the Docker image from Chapter 14.

Possible improvements

- Elbow/silhouette analysis page for choosing k.
- Radar charts per segment.
- Campaign recommendations per segment.

Project 5: Sales Forecasting Application

Level: Intermediate | **Folder:** `forecast_app/`

Item	Details
Business problem	A retailer needs daily sales forecasts for staffing and inventory.
Project objective	Forecast 7-90 days ahead with honest uncertainty and an accuracy back-test.
Dataset requirements	Daily date and sales columns; at least 60 days (ideally 2+ years for yearly seasonality).
Model requirements	SARIMAX(1,1,1)(1,0,1,7) on log sales with prediction intervals; seasonal-naive baseline without intervals.
Streamlit components	`date_input` range, `slider`, `radio`, `select_slider`, `toggle`, Plotly with interval band, `cache_data`, download.

UI design

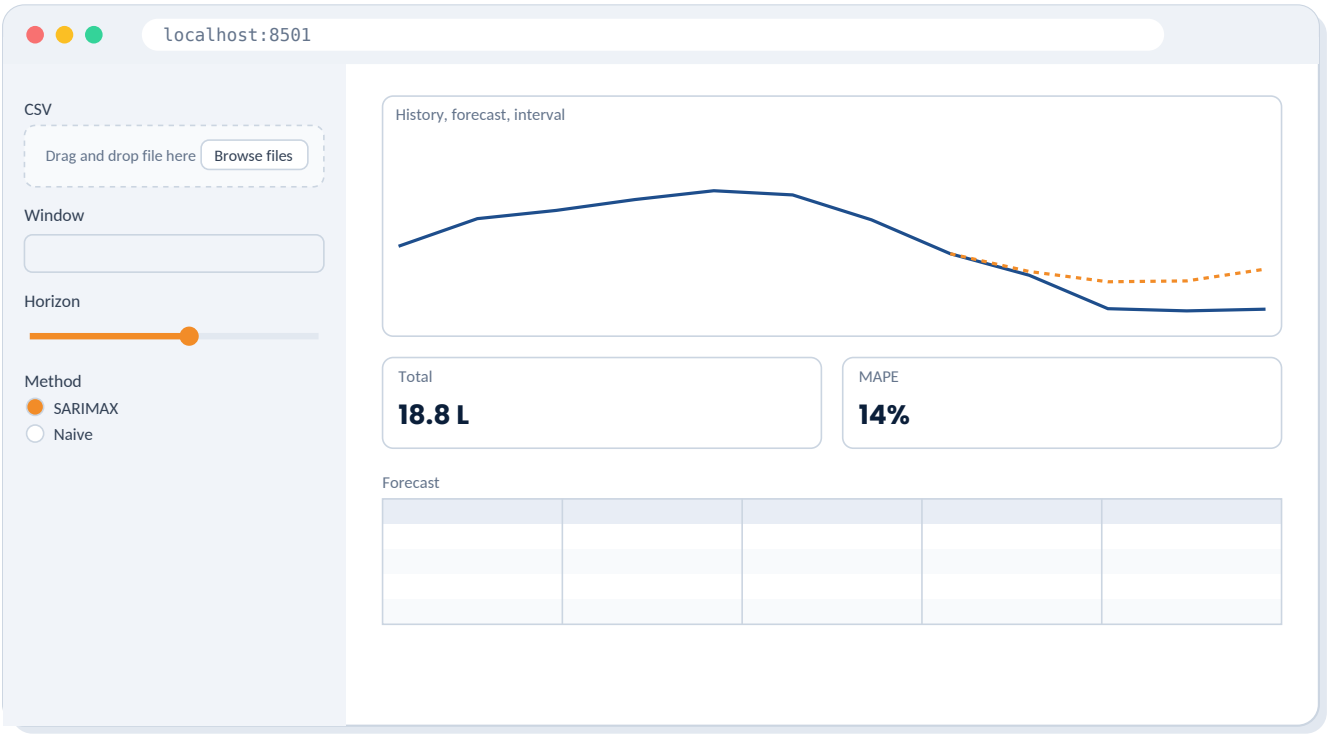


Figure 17.9: Project 5 wireframe

Application architecture



Implementation (file by file)

File	Purpose	Listing
data/make_dataset.py	Synthetic sales	Section 6.7
forecasting.py	Model functions	Section 6.7
app.py	UI	Section 6.7
tests/test_forecasting.py	Tests	below

forecast_app/requirements.txt — REQUIREMENTS

```
streamlit==1.65.0
pandas==3.0.5
numpy==2.5.3
scikit-learn==1.9.1
plotly==7.1.0
joblib==1.6.0
statsmodels==0.15.0
```

forecast_app/tests/test_forecasting.py PYTHON

```python
import pandas as pd

from forecasting import prepare_series, sarimax_forecast, seasonal_naive_forecast


def _series():
    df = pd.read_csv("data/daily_sales.csv")
    return prepare_series(df, "date", "sales").iloc[-200:]


def test_sarimax_returns_ordered_intervals():
    fc = sarimax_forecast(_series(), 14)
    assert len(fc) == 14
    assert (fc["lower"] <= fc["forecast"]).all() and (fc["forecast"] <= fc["upper"]).all()


def test_naive_has_no_interval_columns():
    fc = seasonal_naive_forecast(_series(), 10)
    assert list(fc.columns) == ["forecast"]
```

Installation and execution

Run Project 5 TERMINAL

```
cd forecast_app && pip install -r requirements.txt
python data/make_dataset.py && streamlit run app.py
pytest -q
```

Expected output

A 30-day forecast with a shaded 95% interval and a back-test MAPE of roughly 10-20%.

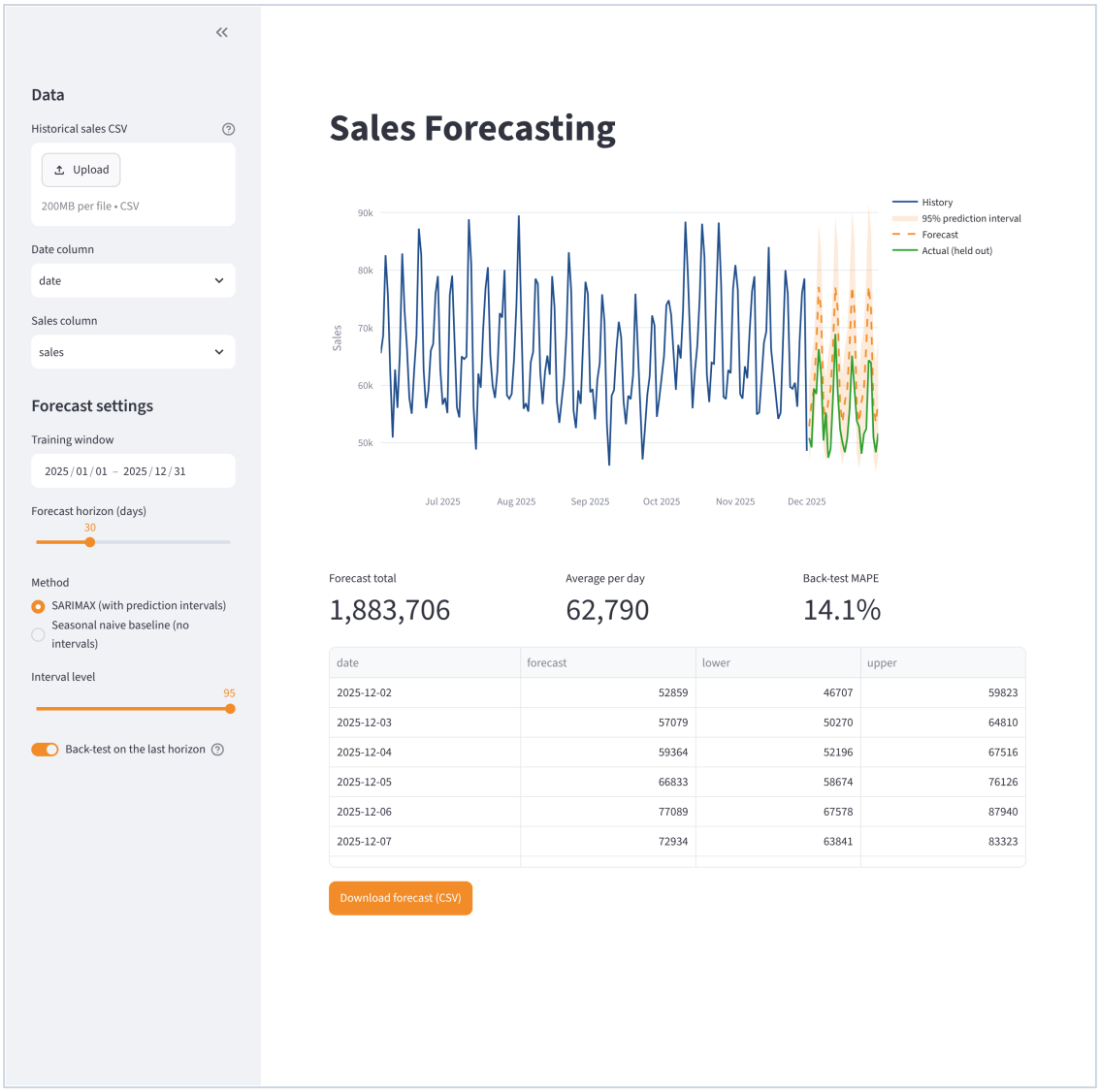


Figure 17.10: Project 5 running

Testing checklist

- [ ] Fewer than 60 days shows an error.
- [ ] Intervals satisfy lower <= forecast <= upper.
- [ ] The naive method shows no interval and explains why.

Deployment

Push the folder to GitHub with `models/` committed, then deploy on Community Cloud with main file `app.py` (Chapter 13) or build the Docker image from Chapter 14.

Possible improvements

- Holiday regressors (Diwali, Christmas).
- Forecast several stores with a selector.
- Prophet or LightGBM with conformal intervals.

Project 6: Sentiment Analysis Application

Level: Intermediate | **Folder:** `sentiment_app/`

Item	Details
Business problem	An e-commerce team wants to monitor product review sentiment.
Project objective	Classify single reviews and uploaded files with confidence scores.
Dataset requirements	Columns review, sentiment (positive/negative). Generated sample; replace with Amazon/IMDB reviews.
Model requirements	TF-IDF (1-2 grams, custom cleaner) + LogisticRegression pipeline.
Streamlit components	`tabs`, `text_area(max_chars)`, `button`, `metric`, `success`/`info` with icons, `file_uploader(txt,csv)`, `bar_chart`, `column_config.NumberColumn(format="percent")`.

UI design

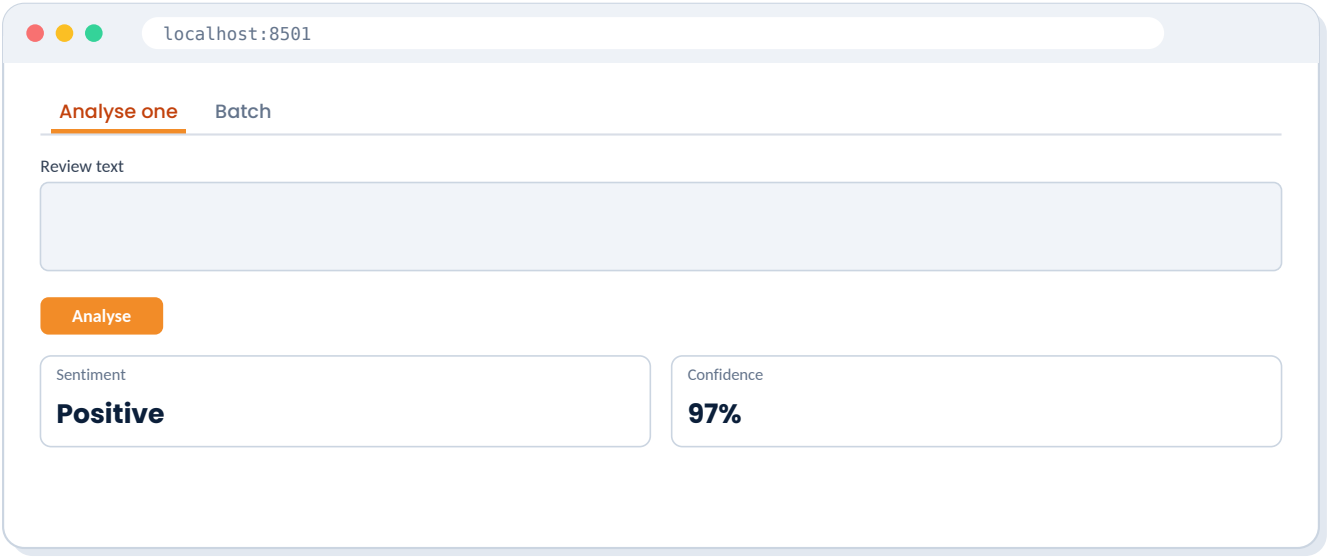


Figure 17.11: Project 6 wireframe

Application architecture



Implementation (file by file)

File	Purpose	Listing
text_utils.py	Cleaner used in the pipeline	Section 6.9
train_model.py	Corpus + training	Section 6.9
app.py	UI	Section 6.9
tests/test_model.py	Tests	below

sentiment_app/tests/test_model.py PYTHON

```python
import joblib

import text_utils  # noqa: F401


def test_clean_text_expands_negation():
    assert text_utils.clean_text("Didn't WORK!!") == "did not work!!"


def test_pipeline_predicts_both_classes():
    pipe = joblib.load("models/sentiment_pipeline.joblib")
    pred = pipe.predict(["Superb sound quality, loved it", "Stopped working, awful"])
    assert list(pred) == ["positive", "negative"]
```

Installation and execution

Run Project 6 TERMINAL

```
cd sentiment_app && pip install -r requirements.txt
python train_model.py && streamlit run app.py
pytest -q
```

Expected output

'Loved it! The headphones have superb sound quality.' is positive with high confidence; mixed reviews show a low-confidence note.

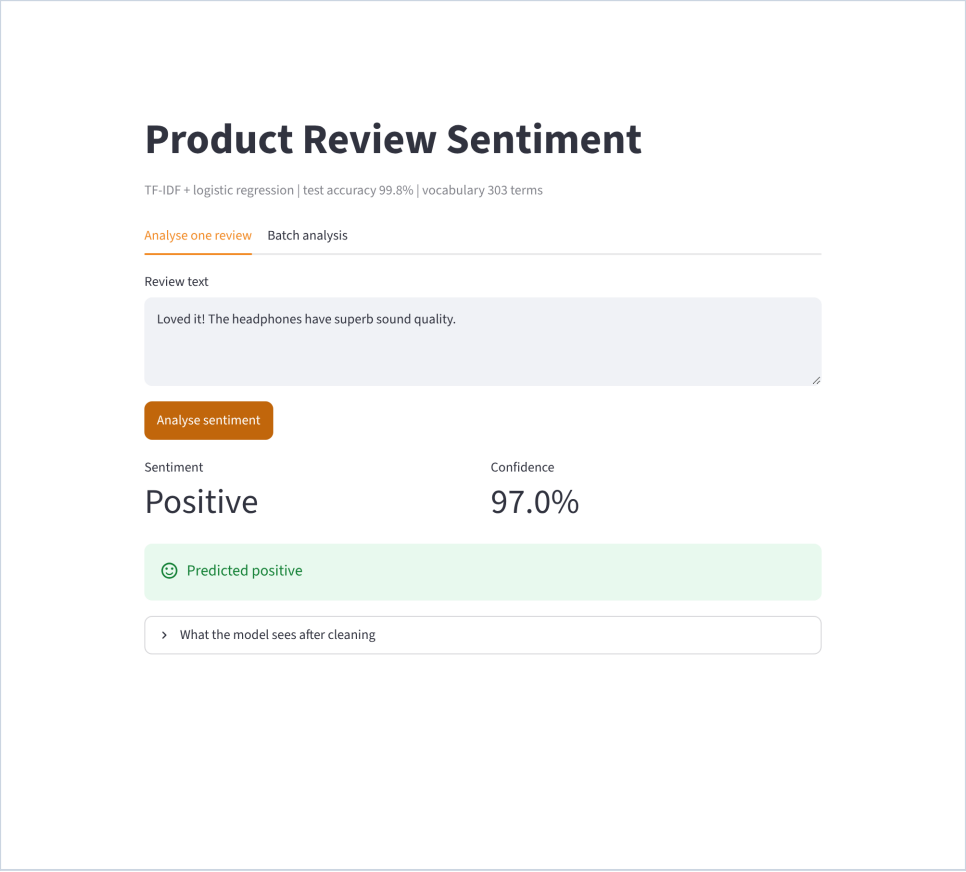


Figure 17.12: Project 6 running

Testing checklist

- [ ] Text shorter than three characters is rejected.
- [ ] TXT and CSV uploads both work; a CSV without `review` errors clearly.
- [ ] Batch limit of 5,000 rows is enforced.

Deployment

Push the folder to GitHub with `models/` committed, then deploy on Community Cloud with main file `app.py` (Chapter 13) or build the Docker image from Chapter 14.

Possible improvements

- Neutral class.
- Aspect-based sentiment (battery, delivery…).
- A Hugging Face transformer model behind cache_resource.

Project 7: Image Classification Application

Level: Advanced | **Folder:** `image_app/`

Item	Details
Business problem	A digitisation team needs to read handwritten digits from scanned forms.
Project objective	Classify uploaded or camera images with top-k probabilities in Keras, and provide a PyTorch version.
Dataset requirements	Synthetic digit images rendered from system fonts with augmentation; replace with MNIST or your own labelled crops.
Model requirements	Small CNN (2 conv blocks) saved as `.keras`; PyTorch twin saved as a `state_dict`.
Streamlit components	`segmented_control`, `file_uploader(images)`, `camera_input`, `image`, `slider` (top-k), `metric`, `ProgressColumn`, `bar_chart`.

UI design

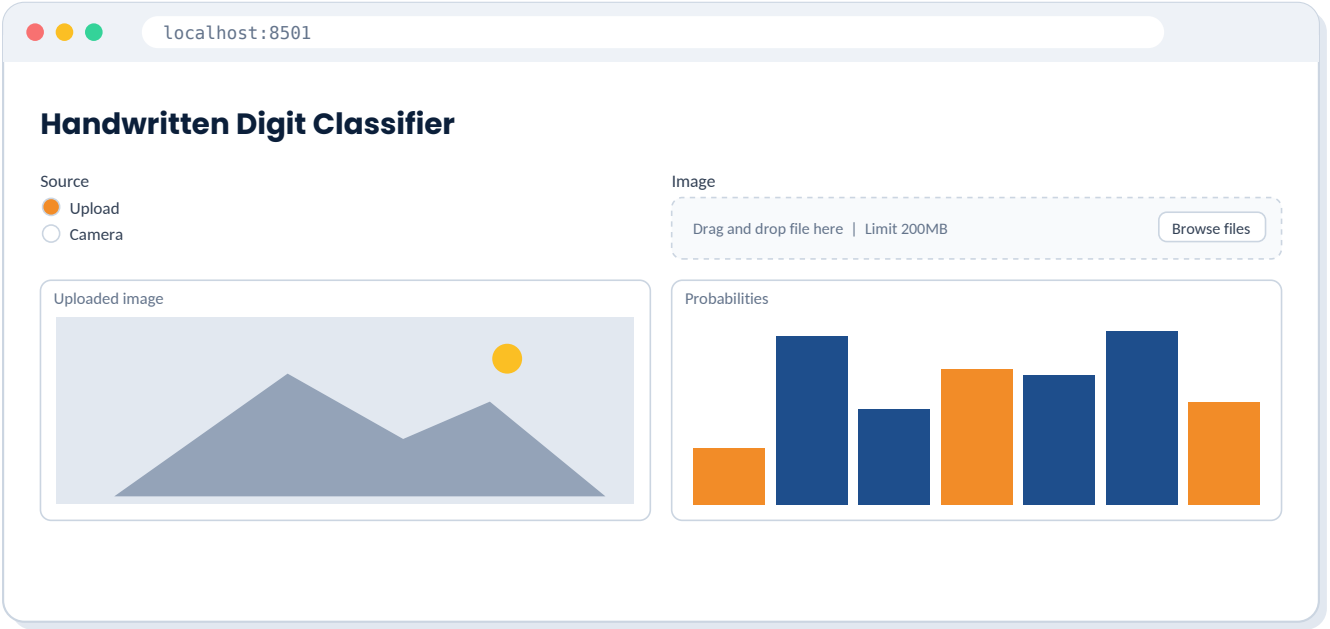


Figure 17.13: Project 7 wireframe

Application architecture



Implementation (file by file)

File	Purpose	Listing
preprocessing.py	Shared preprocessing	Section 6.8
synthetic_digits.py	Training images	Section 6.8
train_model.py, app.py	Keras training + UI	Section 6.8
torch_version.py, torch_app.py	PyTorch	Section 6.8.1
requirements.txt, tests/	Deps, tests	below

image_app/requirements.txt — REQUIREMENTS

```
streamlit==1.65.0
pandas==3.0.5
numpy==2.5.3
tensorflow-cpu==2.21.0     # or tensorflow==2.21.0 on a GPU machine
keras==3.15.1
pillow==12.3.0
# PyTorch variant (torch_app.py) only:
# torch==2.14.1
```

image_app/tests/test_preprocessing.py — PYTHON

```python
import numpy as np
from PIL import Image

from preprocessing import preprocess_image


def test_output_shape_and_range():
    x = preprocess_image(Image.open("samples/digit_7.png"))
    assert x.shape == (28, 28, 1) and x.dtype == np.float32
    assert 0.0 <= x.min() and x.max() <= 1.0


def test_dark_on_light_is_inverted():
    white = Image.new("L", (50, 50), 255)
    assert preprocess_image(white).sum() == 0          # background becomes 0
```

Installation and execution

Run Project 7 — TERMINAL

```
cd image_app && pip install -r requirements.txt
python train_model.py          # Keras model + samples/
streamlit run app.py
pip install torch && python torch_version.py && streamlit run torch_app.py
```

Expected output

Validation accuracy above 0.99 on synthetic digits; the three sample images are classified correctly by both frameworks.

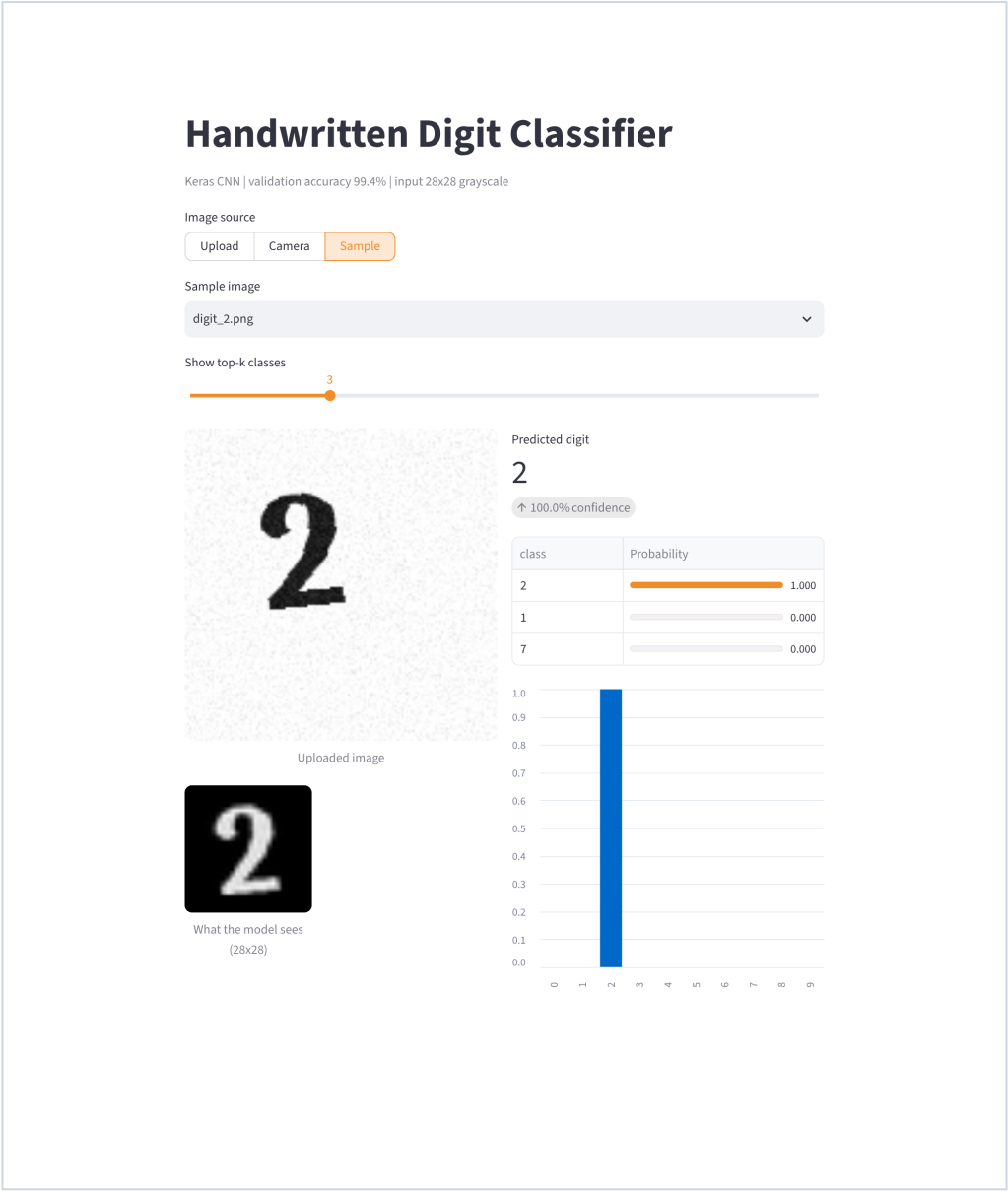


Figure 17.14: Project 7 running

Testing checklist

- [ ] Corrupt or non-image files show an error, not a crash.
- [ ] Images above the pixel limit are rejected.
- [ ] The 28x28 preview matches the training preprocessing.
- [ ] Low confidence shows a warning.

Deployment

Push the folder to GitHub with `models/` committed, then deploy on Community Cloud with main file `app.py` (Chapter 13) or build the Docker image from Chapter 14.

Possible improvements

- Transfer learning with MobileNetV2 for real photos.
- Grad-CAM heatmaps.
- Multi-digit segmentation.

Project 8: Fraud Detection Application

Level: Advanced | **Folder:** `fraud_app/`

Item	Details
Business problem	A card issuer wants analysts to review the most unusual transactions each day.
Project objective	Score transactions, flag the top x% and support evaluation when labels exist.
Dataset requirements	amount, hour, distance_from_home_km, txn_count_24h, merchant_category, is_foreign (+ optional is_fraud for evaluation).
Model requirements	One-hot + scaling + IsolationForest pipeline; score = -score_samples().
Streamlit components	`radio` threshold mode, `slider`, `metric`, Plotly histogram with threshold line, `expander`, `dataframe`, two downloads.

UI design

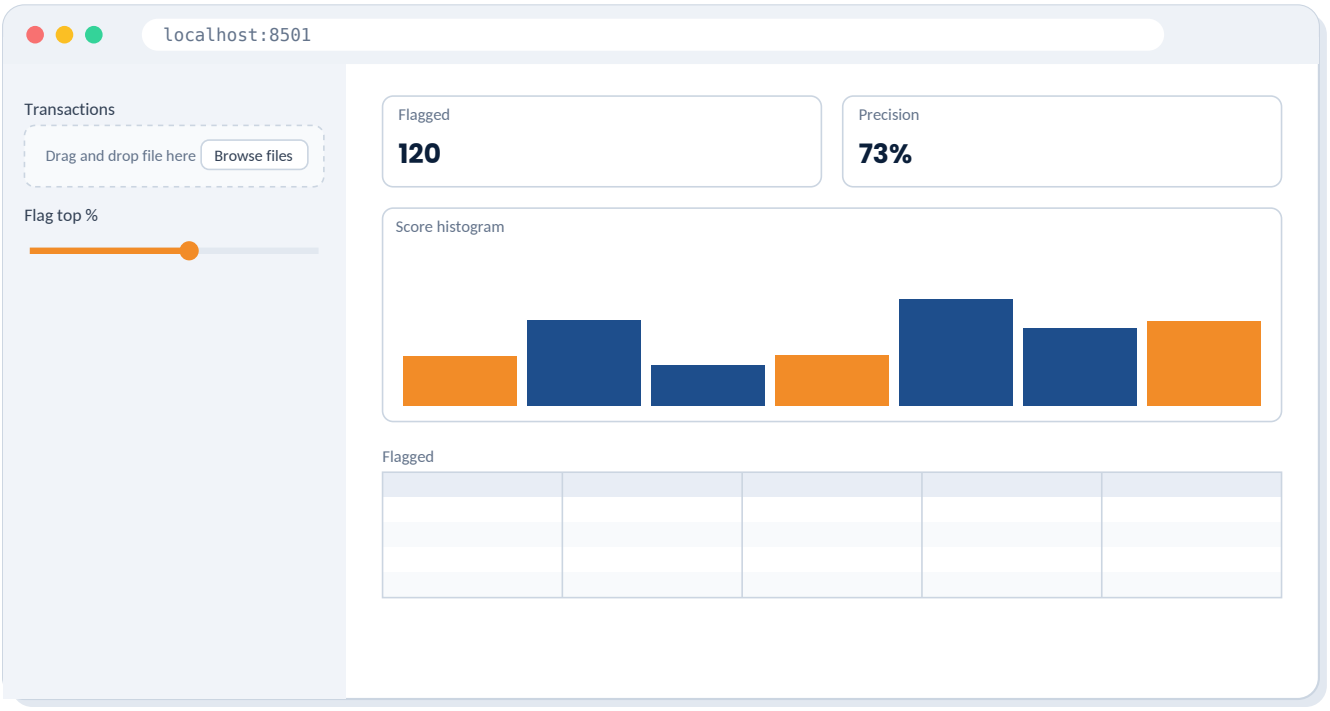


Figure 17.15: Project 8 wireframe

Application architecture



Implementation (file by file)

File	Purpose	Listing
train_model.py	Data + model + score percentiles	Section 6.10
app.py	UI	Section 6.10
tests/test_app.py	Test	below

fraud_app/tests/test_app.py PYTHON

```python
from streamlit.testing.v1 import AppTest


def test_flags_requested_share():
    at = AppTest.from_file("../app.py", default_timeout=60).run()
    assert not at.exception
    flagged = int(at.metric[1].value.replace(",", ""))
    assert 100 <= flagged <= 140                    # ~1.5% of 8,000 by default
```

Installation and execution

Run Project 8 TERMINAL

```
cd fraud_app && pip install -r requirements.txt
python train_model.py && streamlit run app.py
pytest -q
```

Expected output

About 1.5% of 8,000 transactions flagged; with labels, precision around 70% and recall near 99% on the synthetic data.

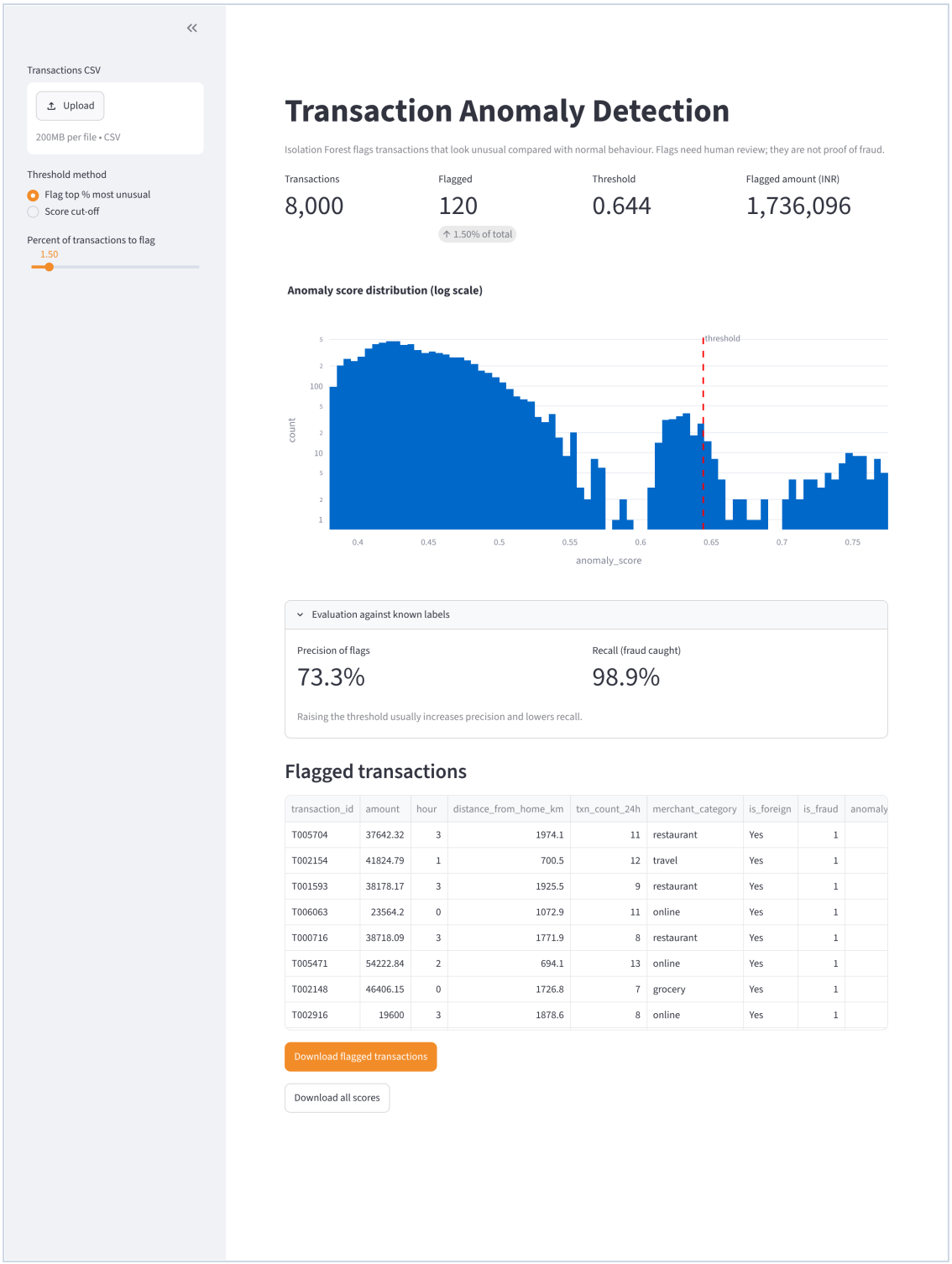


Figure 17.16: Project 8 running

Testing checklist

- [ ] Missing columns stop the app with a list.
- [ ] Raising the percentage increases flagged count.
- [ ] Downloads contain the score column.

Deployment

Push the folder to GitHub with `models/` committed, then deploy on Community Cloud with main file `app.py` (Chapter 13) or build the Docker image from Chapter 14.

Possible improvements

- Reason codes (which features are extreme).
- A supervised model once labels accumulate.
- Analyst feedback stored in a database.

Project 9: Resume Classification Application

Level: Advanced | **Folder:** `resume_app/`

Item	Details
Business problem	A recruitment team receives thousands of resumes and must route each to the right recruiter (data, web, DevOps, HR, sales).
Project objective	Extract text from PDF/DOCX/TXT files, suggest the job family with confidence, redact contact details, and keep humans in control.
Dataset requirements	Columns resume_text, category. A synthetic set (1,500 resumes, 6 categories) is generated; replace with a consented, anonymised dataset.
Model requirements	TF-IDF (with PII-redacting cleaner) + LogisticRegression; top-3 categories.
Streamlit components	`warning` banner, `file_uploader(accept_multiple_files=True)`, `text_area`, `dataframe` with ProgressColumn, `selectbox`, `text`, `bar_chart(horizontal=True)`, download.

UI design

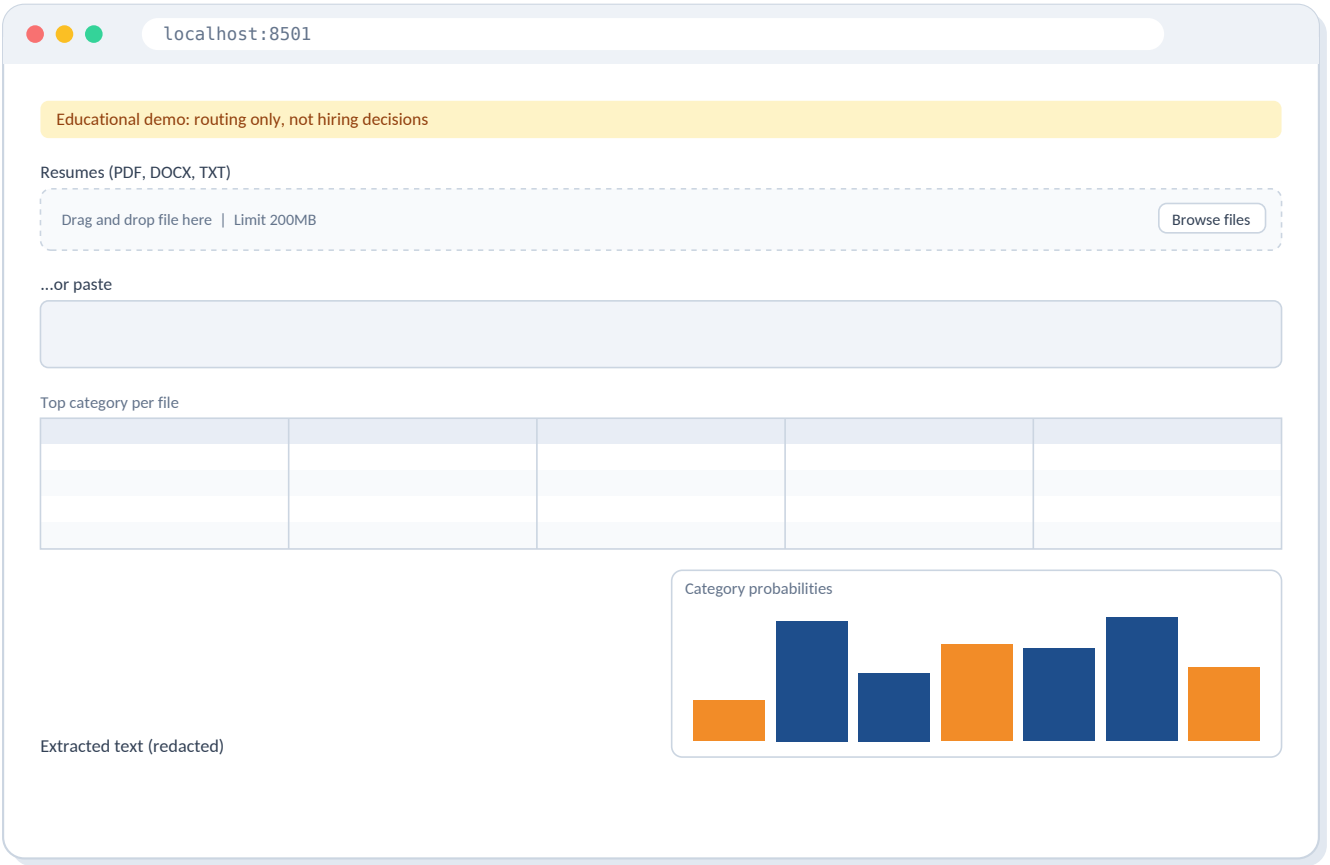


Figure 17.17: Project 9 wireframe

Application architecture



Implementation (file by file)

resume_app/resume_utils.py — PYTHON

```python
"""Text extraction, cleaning and PII redaction for resume files."""
import io
import re

from docx import Document
from pypdf import PdfReader

EMAIL = re.compile(r"[\w.+-]+@[\w-]+\.[\w.]+")
PHONE = re.compile(r"(?:\+?\d{1,3}[\s-]?)?(?:\d[\s-]?){9,11}\d")


def extract_text(name: str, data: bytes) -> str:
    """Return plain text from a .pdf, .docx or .txt file's bytes."""
    lower = name.lower()
    if lower.endswith(".pdf"):
        reader = PdfReader(io.BytesIO(data))
        return "\n".join(page.extract_text() or "" for page in reader.pages)
    if lower.endswith(".docx"):
        doc = Document(io.BytesIO(data))
        return "\n".join(p.text for p in doc.paragraphs)
    if lower.endswith(".txt"):
        return data.decode("utf-8", errors="replace")
    raise ValueError(f"Unsupported file type: {name}")


def redact_pii(text: str) -> str:
    """Hide e-mail addresses and phone numbers before display or logging."""
    return PHONE.sub("[PHONE]", EMAIL.sub("[EMAIL]", text))


def clean_resume(text: str) -> str:
    text = redact_pii(text).lower()
    text = re.sub(r"[^a-z0-9+#. ]+", " ", text)
    return re.sub(r"\s+", " ", text).strip()
```

resume_app/train_model.py — PYTHON

```python
"""Generate synthetic resumes, train a role classifier, create sample files.

Run: python train_model.py
"""
import json
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from docx import Document
from reportlab.lib.pagesizes import A4
from reportlab.pdfgen import canvas
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
```

resume_app/train_model.py PYTHON

```python
from resume_utils import clean_resume

ROOT = Path(__file__).parent
rng = np.random.default_rng(4)
SKILLS = {
    "Data Science": ["python", "machine learning", "scikit-learn", "pandas", "deep learning",
                     "tensorflow", "statistics", "nlp", "feature engineering", "pytorch"],
    "Data Analyst": ["excel", "sql", "power bi", "tableau", "dashboards", "pivot tables",
                     "data cleaning", "reporting", "kpi", "python"],
    "Web Development": ["javascript", "react", "html", "css", "node.js", "rest api",
                        "typescript", "mongodb", "frontend", "git"],
    "DevOps": ["docker", "kubernetes", "aws", "ci/cd", "terraform", "linux", "jenkins",
               "monitoring", "bash", "git"],
    "Human Resources": ["recruitment", "onboarding", "payroll", "employee engagement",
                        "hr policies", "talent acquisition", "performance appraisal",
                        "excel", "labour law", "training"],
    "Sales": ["lead generation", "crm", "negotiation", "b2b sales", "client relationship",
              "target achievement", "salesforce", "cold calling", "pipeline", "excel"],
}
VERBS = ["Built", "Led", "Developed", "Managed", "Improved", "Designed", "Automated"]


def fake_resume(role: str) -> str:
    own = list(rng.choice(SKILLS[role], 6, replace=False))
    other_role = rng.choice([r for r in SKILLS if r != role])
    noise = list(rng.choice(SKILLS[other_role], 2, replace=False))
    years = int(rng.integers(1, 12))
    lines = [f"Professional with {years} years of experience.",
             "Skills: " + ", ".join(own + noise)]
    for skill in own[:3]:
        lines.append(f"{rng.choice(VERBS)} projects using {skill} for business teams.")
    lines.append("Email: candidate@example.com  Phone: +91 98765 43210")
    return "\n".join(lines)


def write_samples(folder: Path) -> None:
    folder.mkdir(exist_ok=True)
    text = fake_resume("Data Science")
    pdf = canvas.Canvas(str(folder / "resume_data_science.pdf"), pagesize=A4)
    y = 800
    for line in ["Priya Sharma - Resume"] + text.split("\n"):
        pdf.drawString(60, y, line)
        y -= 18
    pdf.save()
    doc = Document()
    doc.add_heading("Rahul Mehta - Resume", 1)
    for line in fake_resume("DevOps").split("\n"):
        doc.add_paragraph(line)
    doc.save(folder / "resume_devops.docx")
    (folder / "resume_sales.txt").write_text(fake_resume("Sales"))


def main() -> None:
    rows = [(fake_resume(role), role) for role in SKILLS for _ in range(250)]
    df = pd.DataFrame(rows, columns=["resume_text", "category"])
    (ROOT / "data").mkdir(exist_ok=True)
    df.to_csv(ROOT / "data" / "resumes.csv", index=False)
    X_tr, X_te, y_tr, y_te = train_test_split(df["resume_text"], df["category"],
                                              test_size=0.2, stratify=df["category"],
                                              random_state=42)
    pipe = Pipeline([
        ("tfidf", TfidfVectorizer(preprocessor=clean_resume, ngram_range=(1, 2),
                                  min_df=2)),
        ("clf", LogisticRegression(max_iter=2000, C=3.0)),
    ]).fit(X_tr, y_tr)
    pred = pipe.predict(X_te)
    meta = {"classes": list(pipe.classes_),
            "accuracy": round(float(accuracy_score(y_te, pred)), 4),
```

resume_app/train_model.py PYTHON

```
            "macro_f1": round(float(f1_score(y_te, pred, average="macro")), 4)}
    (ROOT / "models").mkdir(exist_ok=True)
    joblib.dump(pipe, ROOT / "models" / "resume_classifier.joblib")
    (ROOT / "models" / "resume_metadata.json").write_text(json.dumps(meta, indent=2))
    write_samples(ROOT / "samples")
    print(meta)


if __name__ == "__main__":
    main()
```

resume_app/app.py PYTHON

```
"""Resume category classifier.  Run: streamlit run app.py"""
import json
from pathlib import Path

import joblib
import pandas as pd
import streamlit as st

import resume_utils
from resume_utils import extract_text, redact_pii

ROOT = Path(__file__).parent
MAX_FILES = 20
st.set_page_config(page_title="Resume Classifier", page_icon=":material/description:",
                   layout="wide")


@st.cache_resource
def load_model():
    meta = json.loads((ROOT / "models" / "resume_metadata.json").read_text())
    return joblib.load(ROOT / "models" / "resume_classifier.joblib"), meta


model, meta = load_model()
st.title("Resume Category Classifier")
st.warning(
    "Educational demo. This model only suggests which *job family* a resume "
    "describes so it can be routed to the right recruiter. It must not be used to "
    "accept, reject or rank candidates. Hiring decisions require human review, "
    "validated tools and checks for bias.", icon=":material/gavel:")
st.caption(f"TF-IDF + logistic regression | {len(meta['classes'])} categories | "
           f"synthetic test accuracy {meta['accuracy']:.1%}")

files = st.file_uploader("Upload resumes (PDF, DOCX or TXT)", type=["pdf", "docx", "txt"],
                         accept_multiple_files=True)
pasted = st.text_area("...or paste resume text", height=120)

docs = []
for f in (files or [])[:MAX_FILES]:
    try:
        docs.append((f.name, extract_text(f.name, f.getvalue())))
    except Exception as exc:  # encrypted PDFs, corrupt DOCX, wrong extension
        st.error(f"{f.name}: could not extract text ({exc})")
if files and len(files) > MAX_FILES:
    st.warning(f"Only the first {MAX_FILES} files were processed.")
if pasted.strip():
    docs.append(("pasted text", pasted))

docs = [(n, t) for n, t in docs if len(t.split()) >= 15] or docs
if not docs:
    st.info("Upload at least one resume or paste text to classify.")
    st.stop()

texts = [t for _, t in docs]
proba = model.predict_proba(texts)
```

resume_app/app.py PYTHON

```python
classes = list(model.classes_)
rows = []
for (name, text), p in zip(docs, proba):
    order = p.argsort()[::-1][:3]
    rows.append({"file": name, "top_category": classes[order[0]],
                 "confidence": round(float(p[order[0]]), 3),
                 "second_choice": classes[order[1]], "third_choice": classes[order[2]],
                 "words": len(text.split())})
results = pd.DataFrame(rows)
st.dataframe(results, hide_index=True, width="stretch",
             column_config={"confidence": st.column_config.ProgressColumn(
                 min_value=0.0, max_value=1.0, format="%.2f")})
st.download_button("Download results (CSV)", results.to_csv(index=False),
                   "resume_categories.csv", "text/csv")

name = st.selectbox("Inspect a document", [n for n, _ in docs])
idx = [n for n, _ in docs].index(name)
left, right = st.columns([1, 1])
with left:
    st.subheader("Extracted text (contact details redacted)")
    st.text(redact_pii(docs[idx][1])[:3000])
with right:
    st.subheader("Category probabilities")
    st.bar_chart(pd.Series(proba[idx], index=classes, name="probability"),
                 horizontal=True)
    if proba[idx].max() < 0.5:
        st.info("No category is clearly dominant; route this resume for manual review.")
    st.caption(f"Cleaned length: {len(resume_utils.clean_resume(docs[idx][1]))} chars")
```

resume_app/requirements.txt REQUIREMENTS

```
streamlit==1.65.0
pandas==3.0.5
numpy==2.5.3
scikit-learn==1.9.1
joblib==1.6.0
pypdf==5.9.0
python-docx==1.2.0
```

resume_app/tests/test_utils.py PYTHON

```python
from pathlib import Path

from resume_utils import extract_text, redact_pii

SAMPLES = Path(__file__).parents[1] / "samples"


def test_extracts_pdf_docx_txt():
    for f in SAMPLES.iterdir():
        assert len(extract_text(f.name, f.read_bytes()).split()) > 15


def test_redaction():
    out = redact_pii("Mail a.b@x.com or call +91 98765 43210")
    assert "[EMAIL]" in out and "[PHONE]" in out and "98765" not in out
```

Installation and execution

Run Project 9 TERMINAL

```
cd resume_app && pip install -r requirements.txt
python train_model.py      # data, model and samples/
streamlit run app.py       # upload files from samples/
pytest -q
```

Expected output

The three sample files are classified as Data Science, DevOps and Sales; extracted text shows [EMAIL] and [PHONE] placeholders.

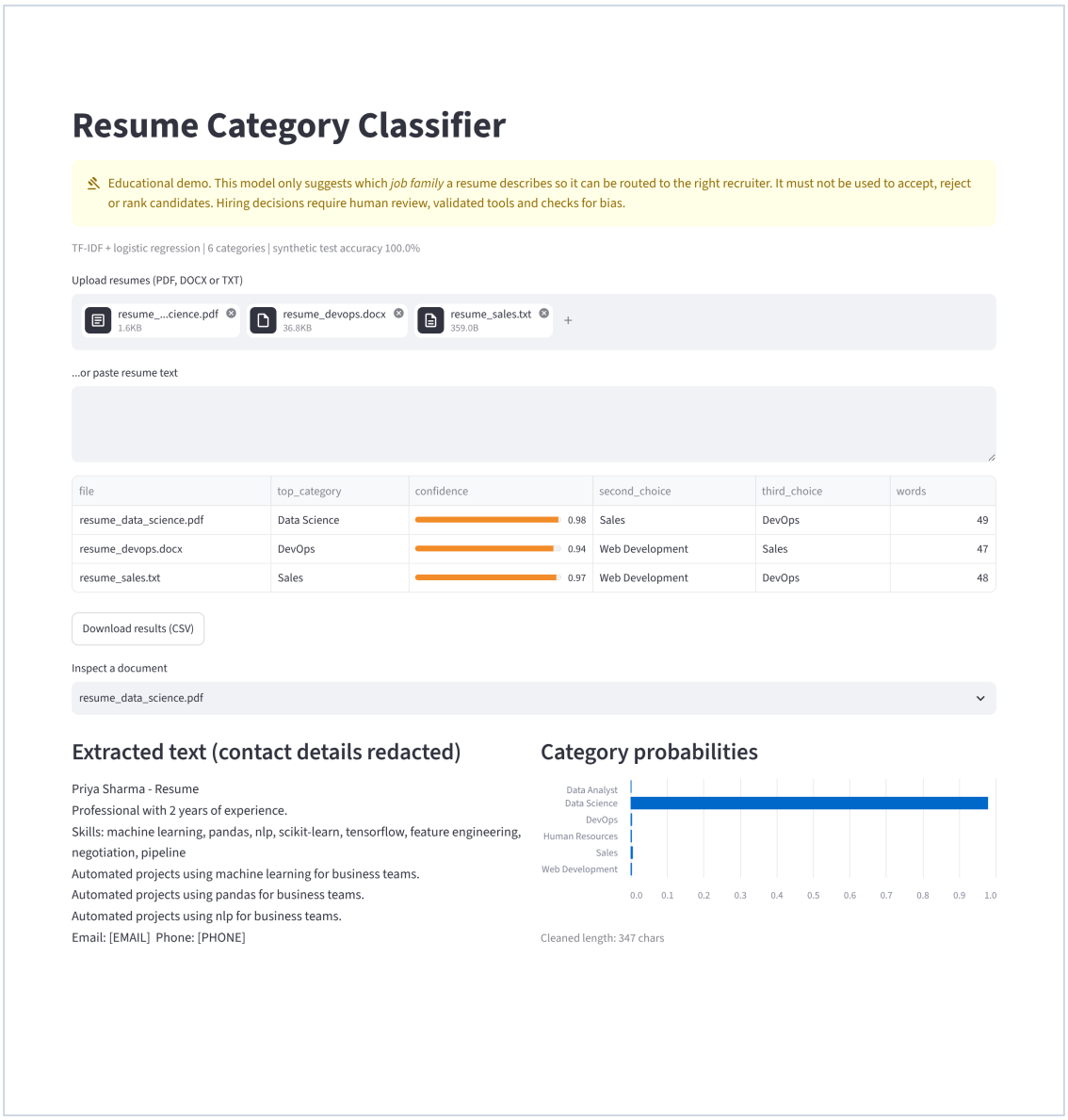


Figure 17.18: Project 9 running

Testing checklist

- [ ] Encrypted or corrupt PDFs show a per-file error.
- [ ] More than 20 files are truncated with a warning.
- [ ] Emails and phone numbers never appear in the displayed text.
- [ ] Low-confidence documents are routed to manual review.

Deployment

Push the folder to GitHub with `models/` committed, then deploy on Community Cloud with main file `app.py` (Chapter 13) or build the Docker image from Chapter 14.

Possible improvements

- OCR for scanned resumes (pytesseract).
- Skill extraction with spaCy.
- A fairness audit comparing outcomes across groups using a properly governed, consented dataset.

> **RESPONSIBLE ML: Hiring is a high-impact use case**
>
> Automated resume screening can encode historical bias (gender, caste, college, gaps in employment). This project classifies job **family** for routing only.
>
> Do not rank, score or reject candidates with it. Any real deployment needs legal review, bias testing, candidate notice and human decision-making.

Project 10: End-to-End ML Model Evaluation and Deployment Dashboard

Level: Advanced | **Folder:** `eval_app/`

Item	Details
Business problem	An ML team has several candidate models and needs a consistent, auditable way to evaluate them before release.
Project objective	Evaluate any registered classification or regression model on labeled data, compare candidates and apply a release gate.
Dataset requirements	Held-out labeled CSVs with the model's features and target (`churn_eval.csv`, `house_eval.csv` created by build_registry.py).
Model requirements	Any scikit-learn pipeline in `models/registry.json` (two churn classifiers and one house regressor).
Streamlit components	`selectbox`, `file_uploader`, `tabs`, `metric`, `dataframe` (Styler highlight), Plotly, `markdown` icons, JSON download.

UI design

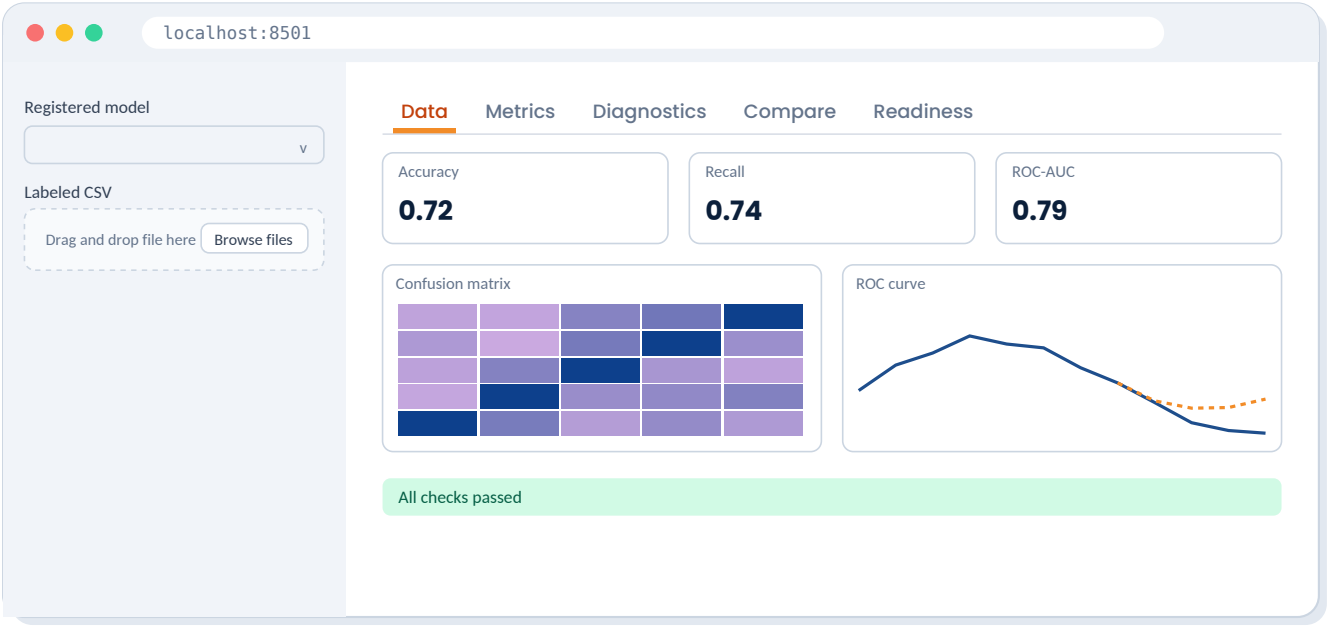


Figure 17.19: Project 10 wireframe

Application architecture



Implementation (file by file)

File	Purpose	Listing
build_registry.py	Collect models, create eval files	Chapter 18
evaluation.py	Metrics with safeguards	Chapter 18
model_io.py	Same as churn_app/utils/model_io.py	Section 6.3
app.py	Dashboard incl. readiness tab	Chapter 18
tests/	Unit + app tests	Chapter 18
requirements.txt, Dockerfile	Packaging	below

eval_app/requirements.txt — REQUIREMENTS

```
streamlit==1.65.0
pandas==3.0.5
numpy==2.5.3
scikit-learn==1.9.1
plotly==7.1.0
joblib==1.6.0
```

Installation and execution

Run Project 10 — TERMINAL

```
# train Projects 2 and 3 first, then:
cd eval_app && pip install -r requirements.txt
python build_registry.py
streamlit run app.py
pytest -q
```

Expected output

The readiness tab shows the release checks; for the random forest all pass, while stricter thresholds can make the baseline fail.

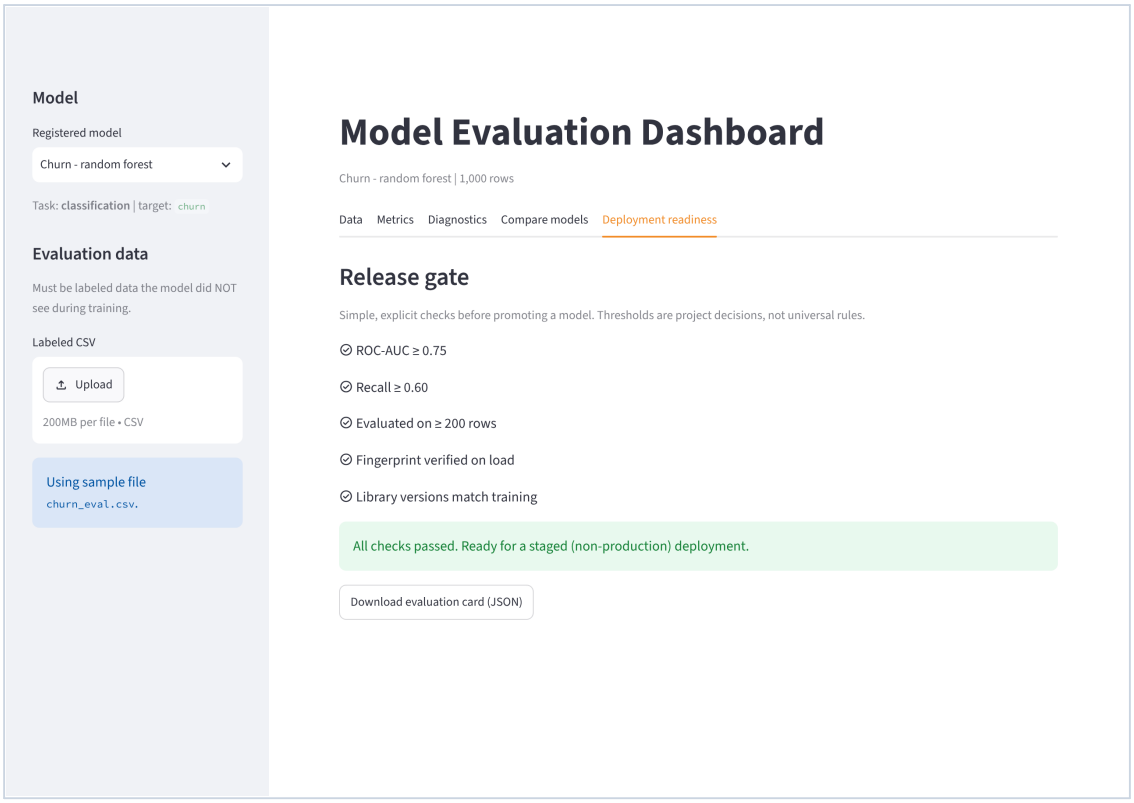


Figure 17.20: Project 10 running

Testing checklist

- [ ] A file without the target column shows an evaluation error.
- [ ] A single-class file reports ROC-AUC as n/a with a warning.
- [ ] Every registered model renders all tabs.
- [ ] The evaluation card JSON contains metrics and checks.

Deployment

Copy `churn_app/Dockerfile` into `eval_app/` (the entrypoint is already `app.py`), build with `docker build -t eval-dashboard .`, and restrict access with `st.login()` or a private app (Chapter 15).

Possible improvements

- MLflow registry integration.
- Subgroup (fairness) metrics.
- Scheduled evaluation on new labels with alerts.

CHAPTER 18

ML Model Evaluation Dashboard

18

Measure saved models on labeled data, with safeguards

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Distinguish evaluating a model (labeled data) from using it (unlabeled).
- ✓ Select the right metrics for classification and regression.
- ✓ Guard against missing targets, wrong target types, invalid outputs, missing classes, tiny datasets and undefined metrics.
- ✓ Compare models and apply an explicit release gate.

18.1 Evaluation versus prediction

	Evaluation	Prediction / scoring
Input data	Labeled: features + true target	Unlabeled: features only
Question answered	How good is the model?	What does the model say about these rows?
Output	Metrics, confusion matrix, residuals	Predictions per row
Data requirement	Must NOT overlap the training data	Any new data
Where in this book	This chapter, capstone 'Model evaluation' page	Chapters 6, 8, capstone 'Batch prediction' page

18.2 Which metrics apply to which task

Metric	Task	Definition / reading	Watch out for
Accuracy	Classification	Share of correct predictions	Misleading with imbalanced classes
Precision	Classification	TP / (TP + FP): of flagged, how many are right	Undefined if nothing is predicted positive
Recall	Classification	TP / (TP + FN): of actual positives, how many found	Undefined if the class is absent
F1-score	Classification	Harmonic mean of precision and recall	Hides which of the two is weak
ROC-AUC	Binary classification (scores)	Probability a random positive ranks above a random negative	Needs both classes present
Average precision / PR curve	Binary classification	Precision-recall trade-off	Better than ROC for rare positives
Confusion matrix	Classification	Counts per (actual, predicted)	Normalise for imbalanced classes
MAE	Regression	Mean |actual - predicted| in target units	Treats all errors linearly
MSE	Regression	Mean squared error (units squared)	Dominated by outliers
RMSE	Regression	Square root of MSE, back in target units	Sensitive to large errors

Metric	Task	Definition / reading	Watch out for
R-squared	Regression	1 - SSE/SST; share of variance explained	Undefined if target is constant; can be negative
MAPE	Regression	Mean |error| / |actual|	Undefined when actual = 0

Key formulas: Precision = TP/(TP+FP); Recall = TP/(TP+FN); F1 = 2 x Precision x Recall / (Precision + Recall); RMSE = sqrt(mean((y - y_hat)^2)); R-squared = 1 - sum((y - y_hat)^2) / sum((y - mean(y))^2).

18.3 Building the model registry

eval_app/build_registry.py PYTHON

```python
"""Collect trained models into this dashboard's registry and create evaluation files.

Prerequisite: run train_model.py in ../churn_app and ../house_app first.
Run from this folder:  python build_registry.py
"""
import json
import shutil
from pathlib import Path

import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from model_io import save_model

ROOT = Path(__file__).parent
CHURN, HOUSE = ROOT.parent / "churn_app", ROOT.parent / "house_app"
MODELS, DATA = ROOT / "models", ROOT / "data"


def copy_artifacts(src: Path, model_file: str, meta_file: str) -> None:
    for name in (model_file, meta_file):
        shutil.copy2(src / "models" / name, MODELS / name)


def main() -> None:
    MODELS.mkdir(exist_ok=True)
    DATA.mkdir(exist_ok=True)
    copy_artifacts(CHURN, "churn_pipeline.joblib", "churn_metadata.json")
    copy_artifacts(HOUSE, "house_pipeline.joblib", "house_metadata.json")

    # --- held-out evaluation files: same split (random_state=42) as training
    churn = pd.read_csv(CHURN / "data" / "churn.csv")
    churn_meta = json.loads((MODELS / "churn_metadata.json").read_text())
    feats = churn_meta["features"]
    tr, te = train_test_split(churn, test_size=0.2, stratify=churn["churn"],
                              random_state=42)
    te.to_csv(DATA / "churn_eval.csv", index=False)

    # a simple baseline model, so the dashboard has something to compare against
    # is_numeric_dtype works for both pandas 2 (object) and pandas 3 (str) text columns
    num = [c for c in feats if pd.api.types.is_numeric_dtype(churn[c])]
    cat = [c for c in feats if c not in num]
    baseline = Pipeline([
        ("prep", ColumnTransformer([
            ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                              ("sc", StandardScaler())]), num),
            ("cat", OneHotEncoder(handle_unknown="ignore"), cat)])),
        ("model", LogisticRegression(max_iter=1000)),
```

eval_app/build_registry.py PYTHON

```
    ]).fit(tr[feats], tr["churn"])
    save_model(baseline, MODELS / "churn_logreg.joblib", MODELS / "churn_logreg.json",
               {"model_name": "Churn baseline (logistic regression)",
                "task": "binary_classification", "features": feats, "target": "churn",
                "classes": list(baseline.classes_), "positive_class": "Yes"})

    house = pd.read_csv(HOUSE / "data" / "houses.csv")
    _, house_te = train_test_split(house, test_size=0.2, random_state=42)
    house_te.to_csv(DATA / "house_eval.csv", index=False)

    registry = [
        {"id": "churn_rf", "name": "Churn - random forest", "task": "classification",
         "model": "churn_pipeline.joblib", "meta": "churn_metadata.json",
         "target": "churn", "positive_class": "Yes", "sample_data": "churn_eval.csv"},
        {"id": "churn_lr", "name": "Churn - logistic regression baseline",
         "task": "classification", "model": "churn_logreg.joblib",
         "meta": "churn_logreg.json", "target": "churn", "positive_class": "Yes",
         "sample_data": "churn_eval.csv"},
        {"id": "house_gbr", "name": "House price - gradient boosting",
         "task": "regression", "model": "house_pipeline.joblib",
         "meta": "house_metadata.json", "target": "price_inr",
         "sample_data": "house_eval.csv"},
    ]
    (MODELS / "registry.json").write_text(json.dumps(registry, indent=2))
    print(f"Registered {len(registry)} models; evaluation files written to {DATA}")


if __name__ == "__main__":
    main()
```

eval_app/models/registry.json (excerpt) JSON

```
[
  {"id": "churn_rf", "name": "Churn - random forest", "task": "classification",
   "model": "churn_pipeline.joblib", "meta": "churn_metadata.json",
   "target": "churn", "positive_class": "Yes", "sample_data": "churn_eval.csv"},
  ...
]
```

18.4 Metric functions with safeguards

eval_app/evaluation.py PYTHON

```
"""Metric computation with safeguards. No Streamlit code here, so it is unit-testable."""
from dataclasses import dataclass, field

import numpy as np
import pandas as pd
from sklearn import metrics as skm

MIN_ROWS_ERROR = 2
MIN_ROWS_WARNING = 30


@dataclass
class EvalResult:
    metrics: dict = field(default_factory=dict)
    warnings: list = field(default_factory=list)
    errors: list = field(default_factory=list)
    extras: dict = field(default_factory=dict)

    @property
    def ok(self) -> bool:
        return not self.errors
```

eval_app/evaluation.py PYTHON

```python
def data_quality(df: pd.DataFrame) -> pd.DataFrame:
    """One row per column: dtype, missing count/%, unique values, example."""
    return pd.DataFrame({
        "dtype": df.dtypes.astype(str),
        "missing": df.isna().sum(),
        "missing_pct": (df.isna().mean() * 100).round(2),
        "unique": df.nunique(),
        # cast to str: a column mixing numbers and text cannot be shown by Arrow
        "example": df.apply(lambda s: str(s.dropna().iloc[0]) if s.notna().any() else ""),
    })


def check_inputs(df: pd.DataFrame, features: list, target: str, task: str) -> EvalResult:
    res = EvalResult()
    if target not in df.columns:
        res.errors.append(f"Target column '{target}' not found. Evaluation needs labels; "
                          "for unlabeled data use the prediction page instead.")
    missing = [c for c in features if c not in df.columns]
    if missing:
        res.errors.append(f"Missing feature columns: {', '.join(missing)}")
    if res.errors:
        return res
    y = df[target]
    if y.isna().any():
        res.warnings.append(f"{int(y.isna().sum())} row(s) with missing target were "
                            "dropped.")
    n = int(y.notna().sum())
    if n < MIN_ROWS_ERROR:
        res.errors.append("At least two labeled rows are required.")
    elif n < MIN_ROWS_WARNING:
        res.warnings.append(f"Only {n} labeled rows: metrics will be very noisy.")
    if task == "regression" and not pd.api.types.is_numeric_dtype(y):
        res.errors.append("Regression models need a numeric target column.")
    if task == "classification" and pd.api.types.is_float_dtype(y) and y.nunique() > 20:
        res.errors.append("Target looks continuous; a classification model cannot be "
                          "evaluated against it. Did you pick a regression target?")
    return res


def classification_report_metrics(y_true, y_pred, y_score=None, classes=None,
                                  positive_class=None) -> EvalResult:
    res = EvalResult()
    y_true = pd.Series(y_true).astype(str).reset_index(drop=True)
    y_pred = pd.Series(y_pred).astype(str).reset_index(drop=True)
    classes = [str(c) for c in (classes or sorted(set(y_true) | set(y_pred)))]
    unknown = set(y_pred) - set(classes)
    if unknown:
        res.errors.append(f"Model produced unexpected labels: {sorted(unknown)}")
        return res
    absent = [c for c in classes if c not in set(y_true)]
    if absent:
        res.warnings.append(f"Class(es) {absent} do not appear in the labels; their "
                            "recall is undefined and reported as 0.")
    unseen = set(y_true) - set(classes)
    if unseen:
        res.warnings.append(f"Labels {sorted(unseen)} were never seen by the model.")
    binary = len(classes) == 2
    avg = "binary" if binary and positive_class else "macro"
    kw = {"pos_label": positive_class} if avg == "binary" else {}
    res.metrics = {
        "accuracy": skm.accuracy_score(y_true, y_pred),
        "precision": skm.precision_score(y_true, y_pred, average=avg, zero_division=0, **kw),
        "recall": skm.recall_score(y_true, y_pred, average=avg, zero_division=0, **kw),
        "f1": skm.f1_score(y_true, y_pred, average=avg, zero_division=0, **kw),
    }
    res.extras["confusion_matrix"] = skm.confusion_matrix(y_true, y_pred, labels=classes)
    res.extras["labels"] = classes
```

eval_app/evaluation.py PYTHON

```python
    res.extras["report"] = pd.DataFrame(skm.classification_report(
        y_true, y_pred, labels=classes, output_dict=True, zero_division=0)).T
    if y_score is not None and binary and positive_class:
        y_bin = (y_true == str(positive_class)).astype(int)
        if y_bin.nunique() < 2:
            res.metrics["roc_auc"] = np.nan
            res.warnings.append("ROC-AUC is undefined because only one class is present.")
        else:
            res.metrics["roc_auc"] = skm.roc_auc_score(y_bin, y_score)
            res.metrics["average_precision"] = skm.average_precision_score(y_bin, y_score)
            res.extras["roc"] = skm.roc_curve(y_bin, y_score)
            res.extras["pr"] = skm.precision_recall_curve(y_bin, y_score)
    return res


def regression_metrics(y_true, y_pred) -> EvalResult:
    res = EvalResult()
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    if not np.isfinite(y_pred).all():
        res.errors.append("Model returned NaN or infinite predictions.")
        return res
    mse = skm.mean_squared_error(y_true, y_pred)
    res.metrics = {"mae": skm.mean_absolute_error(y_true, y_pred), "mse": mse,
                   "rmse": float(np.sqrt(mse))}
    if len(y_true) >= 2 and np.var(y_true) > 0:
        res.metrics["r2"] = skm.r2_score(y_true, y_pred)
    else:
        res.metrics["r2"] = np.nan
        res.warnings.append("R-squared is undefined when the target has no variance.")
    if (y_true == 0).any():
        res.warnings.append("MAPE skipped because some actual values are zero.")
    else:
        res.metrics["mape"] = float(np.mean(np.abs((y_true - y_pred) / y_true)))
    res.extras["residuals"] = y_true - y_pred
    return res


METRIC_GUIDE = pd.DataFrame([
    ("Accuracy", "Classification", "Share of correct predictions; misleading if imbalanced"),
    ("Precision", "Classification", "Of the rows predicted positive, how many were positive"),
    ("Recall", "Classification", "Of the actual positives, how many were found"),
    ("F1-score", "Classification", "Harmonic mean of precision and recall"),
    ("ROC-AUC", "Binary classification", "Ranking quality across all thresholds"),
    ("MAE", "Regression", "Average absolute error in target units"),
    ("MSE / RMSE", "Regression", "Squared error; RMSE is back in target units"),
    ("R-squared", "Regression", "Share of target variance explained (can be negative)"),
], columns=["metric", "applies to", "meaning"])
```

Safeguard	Implementation	User sees
Missing target column	`check_inputs` tests `target in df.columns`	Error: evaluation needs labels; use the prediction page for unlabeled data
Missing feature columns	List difference with the model's features	Error listing the columns
Unsupported target type	Numeric check for regression; 'continuous' check for classification	Error explaining the mismatch
Invalid model outputs	Unknown labels; NaN/inf regression predictions	Error; metrics not shown
Missing classes	Classes absent from labels or unseen by the model	Warning; recall reported as 0 with `zero_division=0`
Small datasets	< 2 rows error, < 30 rows warning	Warning about noisy metrics
Undefined metrics	ROC-AUC with one class; R-squared with zero variance; MAPE with zeros	'n/a' plus a warning, never a misleading number

18.5 The dashboard app

eval_app/app.py PYTHON

```python
"""ML Model Evaluation & Deployment Dashboard.  Run: streamlit run app.py"""
import json
from pathlib import Path

import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import streamlit as st

from evaluation import (METRIC_GUIDE, check_inputs, classification_report_metrics,
                        data_quality, regression_metrics)
from model_io import ModelLoadError, load_trusted_model

ROOT = Path(__file__).parent
MODELS = ROOT / "models"
st.set_page_config(page_title="Model Evaluation", page_icon=":material/analytics:",
                   layout="wide")


@st.cache_data
def load_registry() -> list[dict]:
    return json.loads((MODELS / "registry.json").read_text())


@st.cache_resource(show_spinner="Loading model...")
def load_entry(model_file: str, meta_file: str):
    return load_trusted_model(MODELS / model_file, MODELS / meta_file)


def evaluate(entry: dict, df: pd.DataFrame):
    """Run one registered model on a labeled DataFrame -> (EvalResult, predictions)."""
    model, meta, _ = load_entry(entry["model"], entry["meta"])
    target, task = entry["target"], entry["task"]
    res = check_inputs(df, meta["features"], target, task)
    if not res.ok:
        return res, None
    data = df.dropna(subset=[target])
    X, y = data[meta["features"]], data[target]
    try:
        y_pred = model.predict(X)
    except Exception as exc:
        res.errors.append(f"Prediction failed: {exc}")
        return res, None
    if task == "regression":
        out = regression_metrics(y, y_pred)
        preds = pd.DataFrame({"actual": y.values, "predicted": y_pred})
    else:
        score = None
        if hasattr(model, "predict_proba") and entry.get("positive_class"):
            pos = list(model.classes_).index(entry["positive_class"])
            score = model.predict_proba(X)[:, pos]
        out = classification_report_metrics(y, y_pred, score, list(model.classes_),
                                            entry.get("positive_class"))
        preds = pd.DataFrame({"actual": y.values, "predicted": y_pred})
        if score is not None:
            preds["score"] = score
    out.warnings = res.warnings + out.warnings
    return out, preds


registry = load_registry()
by_name = {e["name"]: e for e in registry}
```

eval_app/app.py PYTHON

```python
with st.sidebar:
    st.header("Model")
    entry = by_name[st.selectbox("Registered model", list(by_name))]
    try:
        _, meta, load_warnings = load_entry(entry["model"], entry["meta"])
    except ModelLoadError as exc:
        st.error(str(exc))
        st.stop()
    st.caption(f"Task: **{entry['task']}** | target: `{entry['target']}`")
    for w in load_warnings:
        st.warning(w)
    st.header("Evaluation data")
    st.caption("Must be labeled data the model did NOT see during training.")
    upload = st.file_uploader("Labeled CSV", type=["csv"])
    df = pd.read_csv(upload) if upload else pd.read_csv(ROOT / "data" / entry["sample_data"])
    if upload is None:
        st.info(f"Using sample file `{entry['sample_data']}`.")

st.title("Model Evaluation Dashboard")
st.caption(f"{entry['name']} | {len(df):,} rows")
result, preds = evaluate(entry, df)
for e in result.errors:
    st.error(e)
for w in result.warnings:
    st.warning(w)

t_data, t_metrics, t_plots, t_compare, t_ready = st.tabs(
    ["Data", "Metrics", "Diagnostics", "Compare models", "Deployment readiness"])

with t_data:
    st.subheader("Preview")
    st.dataframe(df.head(50), width="stretch", height=260)
    st.subheader("Data quality")
    q = data_quality(df)
    c1, c2, c3 = st.columns(3)
    c1.metric("Rows", f"{len(df):,}")
    c2.metric("Duplicate rows", int(df.duplicated().sum()))
    c3.metric("Cells missing", f"{df.isna().mean().mean():.2%}")
    st.dataframe(q, width="stretch",
                 column_config={"missing_pct": st.column_config.ProgressColumn(
                     "Missing %", min_value=0, max_value=100, format="%.1f%%")})
    if entry["task"] == "classification" and entry["target"] in df:
        st.subheader("Class distribution")
        st.bar_chart(df[entry["target"]].value_counts())

if not result.ok:
    st.stop()

with t_metrics:
    cols = st.columns(len(result.metrics))
    for col, (name, value) in zip(cols, result.metrics.items()):
        if value is None or np.isnan(value):
            shown = "n/a"
        elif name == "mse":
            shown = f"{value:.3e}"              # squared units get very large
        elif name in ("mae", "rmse"):
            shown = f"{value:,.0f}"
        else:
            shown = f"{value:.3f}"
        col.metric(name.upper().replace("_", " "), shown)
    if entry["task"] == "classification":
        st.subheader("Classification report")
        st.dataframe(result.extras["report"].round(3), width="stretch")
    with st.expander("Which metrics apply to which task?"):
        st.dataframe(METRIC_GUIDE, hide_index=True, width="stretch")

with t_plots:
    if entry["task"] == "classification":
```

eval_app/app.py PYTHON

```
        left, right = st.columns(2)
        labels = result.extras["labels"]
        left.plotly_chart(px.imshow(result.extras["confusion_matrix"], text_auto=True,
                                    x=labels, y=labels, color_continuous_scale="Blues",
                                    labels=dict(x="Predicted", y="Actual"),
                                    title="Confusion matrix"), width="stretch")
        if "roc" in result.extras:
            fpr, tpr, _ = result.extras["roc"]
            fig = go.Figure(go.Scatter(x=fpr, y=tpr, name="ROC"))
            fig.add_scatter(x=[0, 1], y=[0, 1], line=dict(dash="dash"), name="Random")
            fig.update_layout(title="ROC curve", xaxis_title="FPR", yaxis_title="TPR")
            right.plotly_chart(fig, width="stretch")
            prec, rec, _ = result.extras["pr"]
            st.plotly_chart(px.line(x=rec, y=prec, title="Precision-recall curve",
                                    labels={"x": "Recall", "y": "Precision"}),
                            width="stretch")
    else:
        left, right = st.columns(2)
        fig = px.scatter(preds, x="actual", y="predicted", opacity=0.5,
                         title="Actual vs predicted")
        lo, hi = preds["actual"].min(), preds["actual"].max()
        fig.add_scatter(x=[lo, hi], y=[lo, hi], mode="lines", name="perfect",
                        line=dict(dash="dash"))
        left.plotly_chart(fig, width="stretch")
        preds["residual"] = result.extras["residuals"]
        fig = px.scatter(preds, x="predicted", y="residual", opacity=0.5,
                         title="Residuals vs predicted")
        fig.add_hline(y=0, line_dash="dash")
        right.plotly_chart(fig, width="stretch")
        st.plotly_chart(px.histogram(preds, x="residual", nbins=50,
                                     title="Residual distribution"), width="stretch")
    st.download_button("Download predictions", preds.to_csv(index=False),
                       "evaluation_predictions.csv", "text/csv")

with t_compare:
    same_task = [e for e in registry if e["task"] == entry["task"]
                 and e["target"] == entry["target"]]
    if len(same_task) < 2:
        st.info("Register another model for the same target to compare.")
    else:
        rows = []
        for e in same_task:
            r, _ = evaluate(e, df)
            if r.ok:
                rows.append({"model": e["name"], **{k: round(float(v), 4)
                                                    for k, v in r.metrics.items()}})
        table = pd.DataFrame(rows).set_index("model")
        st.dataframe(table.style.highlight_max(axis=0, color="#d1fae5"), width="stretch")
        st.caption("Green = best value per column (for error metrics, lower is better).")

with t_ready:
    st.subheader("Release gate")
    st.caption("Simple, explicit checks before promoting a model. Thresholds are "
               "project decisions, not universal rules.")
    m = result.metrics
    if entry["task"] == "classification":
        checks = [("ROC-AUC >= 0.75", m.get("roc_auc", 0) >= 0.75),
                  ("Recall >= 0.60", m["recall"] >= 0.60),
                  ("Evaluated on >= 200 rows", len(preds) >= 200)]
    else:
        checks = [("R-squared >= 0.80", m["r2"] >= 0.80),
                  ("MAPE <= 15%", m.get("mape", 1) <= 0.15),
                  ("Evaluated on >= 200 rows", len(preds) >= 200)]
    checks.append(("Fingerprint verified on load", True))
    checks.append(("Library versions match training", not load_warnings))
    for label, passed in checks:
        icon = ":material/check_circle:" if passed else ":material/cancel:"
        st.markdown(f"{icon} {label}")
```

eval_app/app.py PYTHON

```
    if all(p for _, p in checks):
        st.success("All checks passed. Ready for a staged (non-production) deployment.")
    else:
        st.error("Some checks failed. Investigate before deploying.")
    card = {"model": entry["name"], "task": entry["task"], "rows_evaluated": len(preds),
            "metrics": {k: (None if np.isnan(v) else round(float(v), 4))
                        for k, v in m.items()},
            "checks": {label: bool(p) for label, p in checks}}
    st.download_button("Download evaluation card (JSON)", json.dumps(card, indent=2),
                       "evaluation_card.json", "application/json")
```

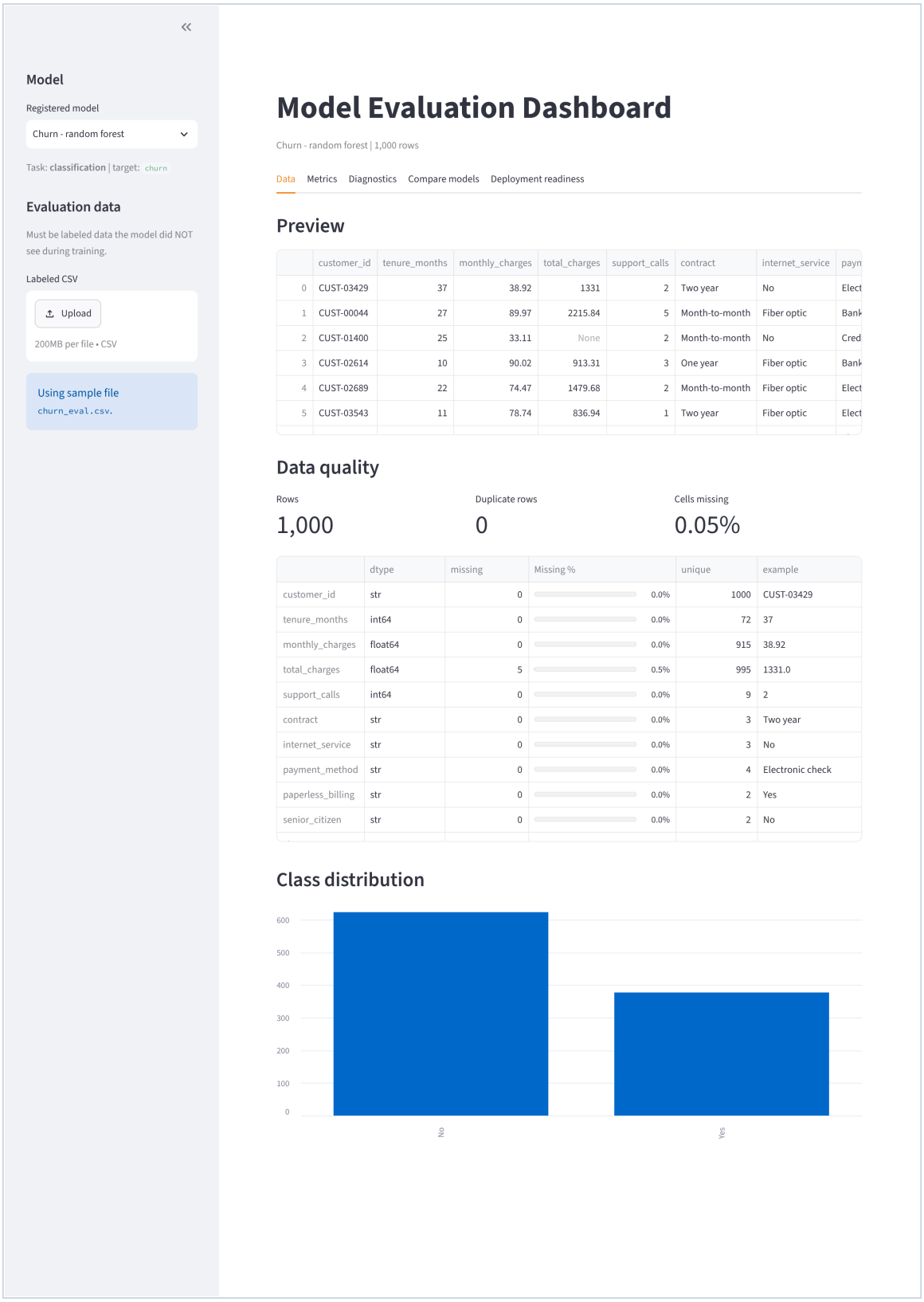


Figure 18.1: Data tab: preview, data quality and class distribution

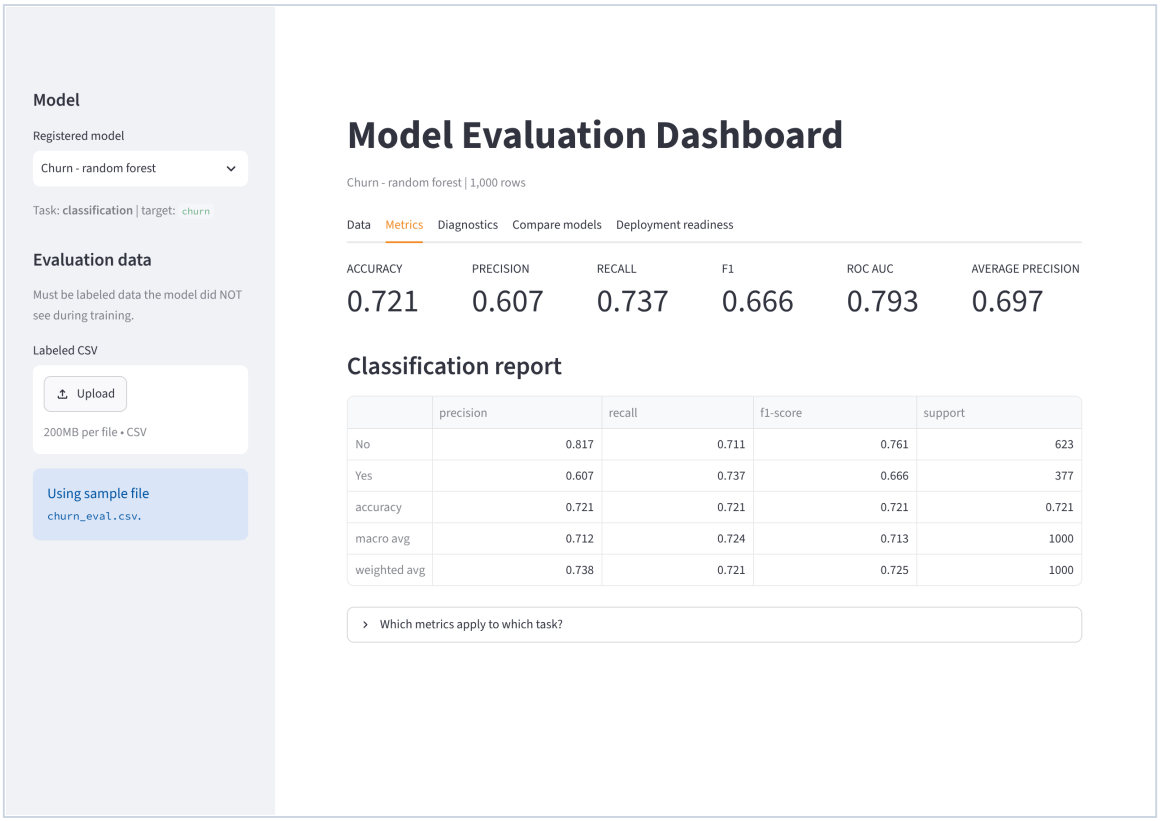


Figure 18.2: Metrics tab with the classification report

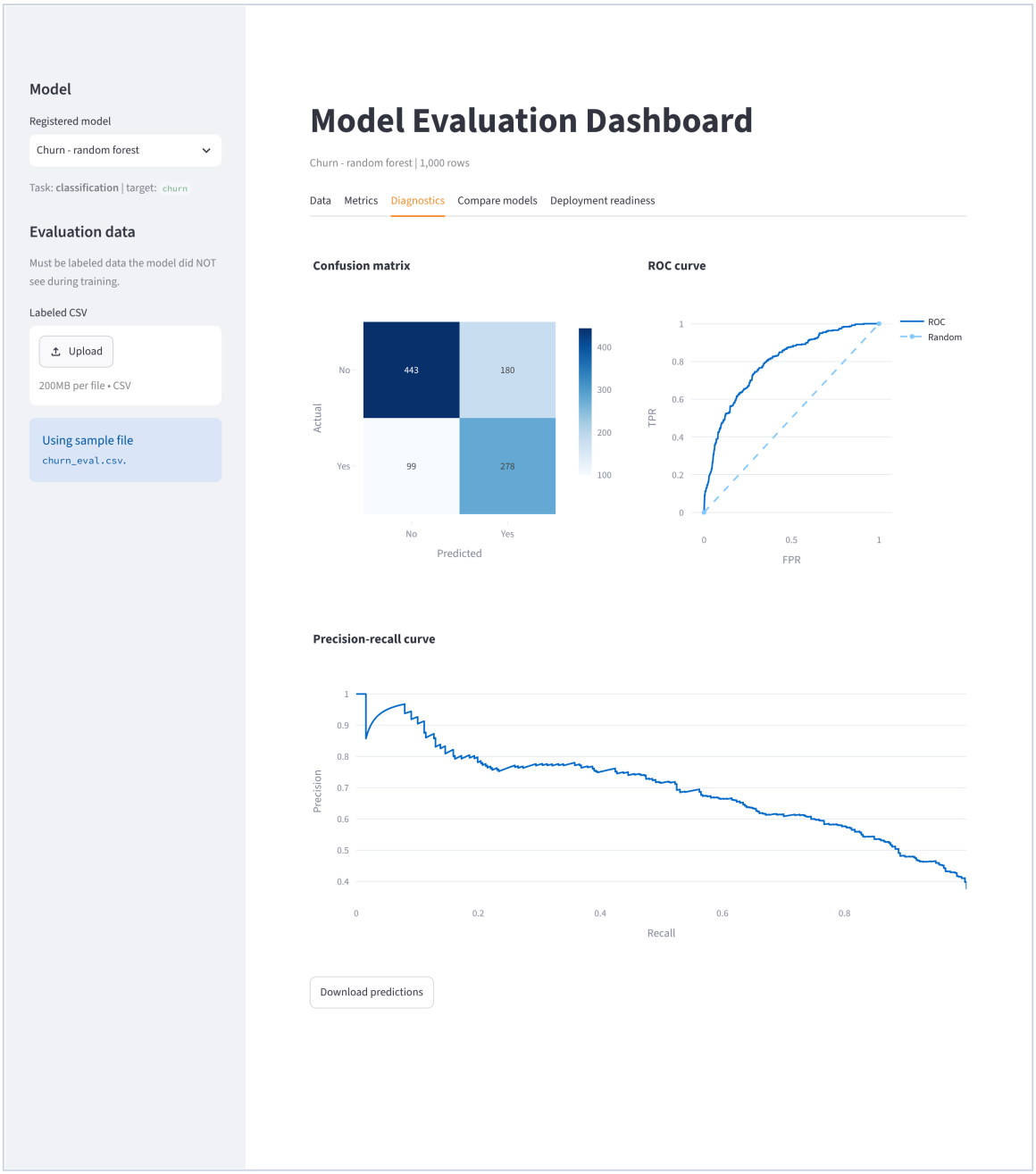


Figure 18.3: Diagnostics tab: confusion matrix, ROC and precision-recall curves

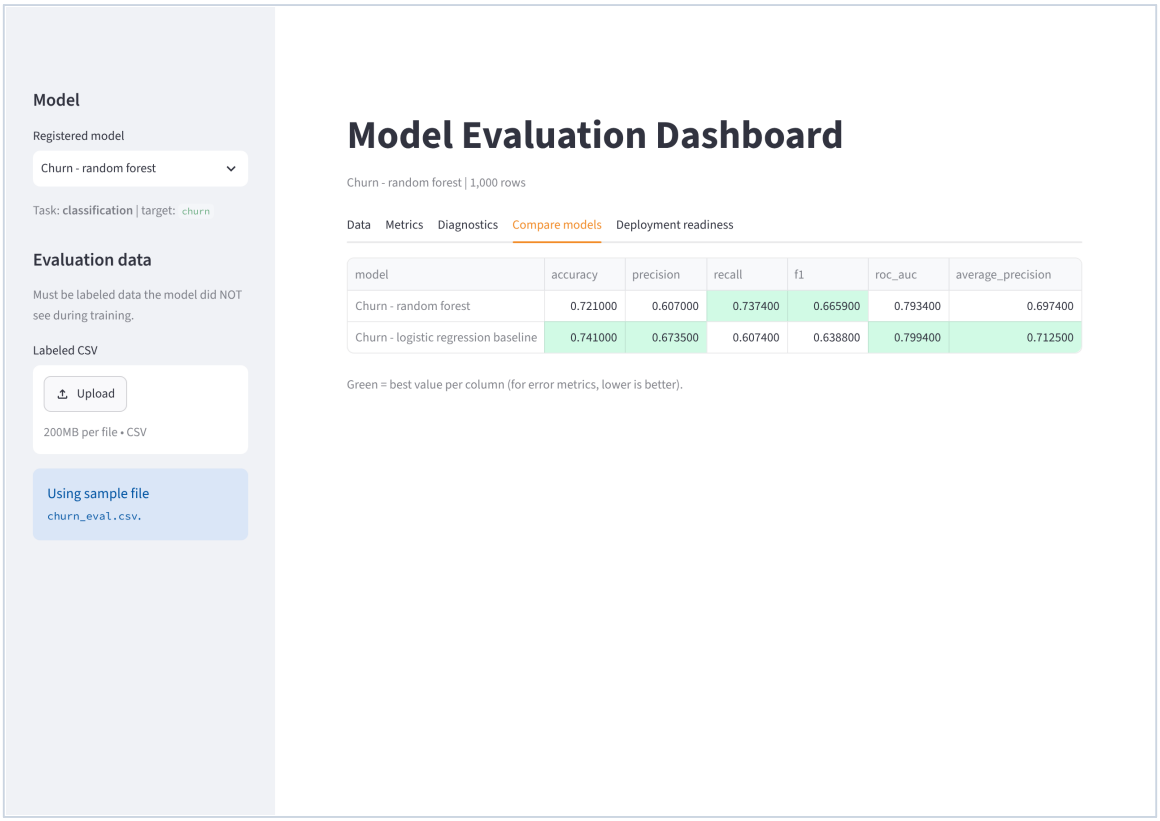


Figure 18.4: Compare tab: random forest vs logistic regression baseline on identical data

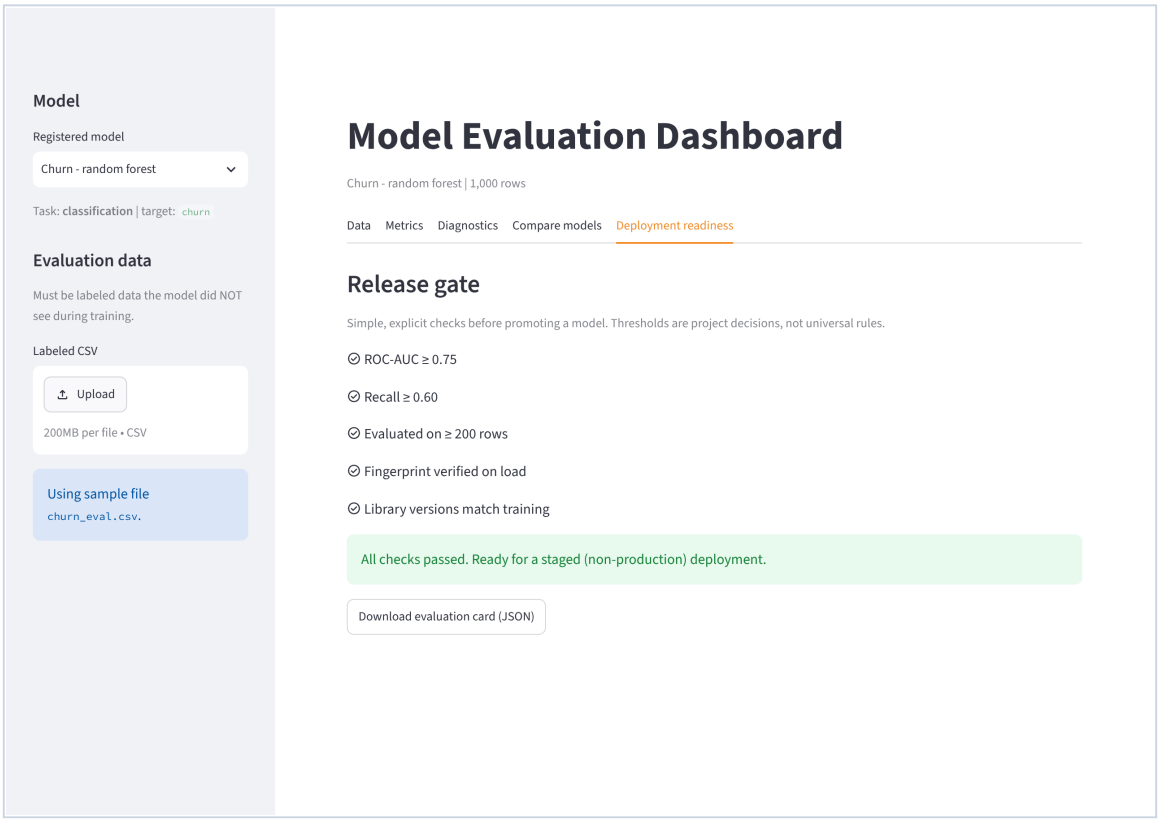


Figure 18.5: Deployment readiness: explicit release checks

18.6 Tests

eval_app/tests/test_evaluation.py PYTHON

```python
"""Unit tests for the safeguards in evaluation.py.  Run: pytest -q"""
import numpy as np
import pandas as pd

from evaluation import check_inputs, classification_report_metrics, regression_metrics


def test_missing_target_is_an_error():
    df = pd.DataFrame({"a": [1, 2, 3]})
    res = check_inputs(df, ["a"], "label", "classification")
    assert not res.ok and "not found" in res.errors[0]


def test_roc_auc_undefined_with_one_class():
    res = classification_report_metrics(["Yes", "Yes"], ["Yes", "No"], [0.9, 0.2],
                                        ["No", "Yes"], "Yes")
    assert np.isnan(res.metrics["roc_auc"])
    assert any("undefined" in w for w in res.warnings)


def test_unexpected_labels_rejected():
    res = classification_report_metrics(["a", "b"], ["a", "zzz"], classes=["a", "b"])
    assert not res.ok


def test_regression_rejects_nan_predictions():
    res = regression_metrics([1.0, 2.0], [1.0, float("nan")])
    assert not res.ok


def test_constant_target_r2_undefined():
    res = regression_metrics([5.0, 5.0, 5.0], [4.0, 5.0, 6.0])
    assert np.isnan(res.metrics["r2"])
```

eval_app/tests/test_app.py PYTHON

```python
"""App smoke test: every registered model renders without exceptions."""
import json
from pathlib import Path

from streamlit.testing.v1 import AppTest

REGISTRY = json.loads((Path(__file__).parents[1] / "models" / "registry.json").read_text())


def test_each_registered_model_evaluates():
    at = AppTest.from_file("../app.py", default_timeout=90).run()
    for entry in REGISTRY:
        at.selectbox[0].set_value(entry["name"]).run()
        assert not at.exception, entry["name"]
        assert len(at.metric) >= 4
```

pytest -q (eval_app) OUTPUT

```
......                                                   [100%]
6 passed in 5.65s
```

BEST PRACTICE

Evaluate every candidate on the **same** held-out file, with the **same** code, and store the evaluation card next to the model. When someone asks 'why did we deploy this model?', the answer is a file, not a memory.

CHAPTER 19

Troubleshooting Guide

19

Symptoms, causes, diagnostics and fixes for 25 common problems

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Recognise the most frequent Streamlit ML errors from their symptoms.
- ✓ Diagnose problems systematically instead of guessing.
- ✓ Apply proven fixes and prevention habits.

19.1 A systematic debugging routine



19.2 Quick lookup table

#	Problem	Most likely cause
1	'streamlit' is not recognized / command not found	Virtual environment not activated
2	ModuleNotFoundError	Package not installed in the active environment
3	Model file not found	Relative path resolved against the current working directory, not the script
4	Pickle or joblib loading error	Custom function defined in a notebook
5	Inconsistent feature names	Columns renamed, reordered or missing
6	Missing columns	Uploaded file lacks required columns or uses different headers.
7	Data type mismatch	Text in numeric columns
8	Shape mismatch	Passing a single sample without the batch dimension
9	Image preprocessing mismatch	RGB vs BGR, 0-255 vs 0-1, different resize/crop, missing normalisation (mean/std), grayscale vs colour, EXIF rotation.
10	TensorFlow loading errors	Saved with a different Keras major version
11	PyTorch device mismatch	Model saved on GPU and loaded on CPU
12	Streamlit reruns unexpectedly	Every widget change reruns the script
13	Session state errors	Key not initialised
14	Cache-related issues	Cache key does not include a version
15	File uploader returns unexpected data	Passing the object where a path is expected
16	CSV encoding issues	File saved as Windows-1252/Latin-1 or UTF-8 with BOM by Excel.

#	Problem	Most likely cause
17	Deployment dependency conflicts	Over-pinned transitive dependencies from `pip freeze`
18	Works locally, fails in production	Missing files or secrets
19	Memory errors	Large uploads, big models, caching many large objects, unclosed figures, storing DataFrames in session state for many users.
20	Incorrect prediction results	Training/inference preprocessing mismatch
21	Slow application performance	Uncached loading, recomputing charts, large tables rendered in full, heavy work in all tabs.
22	Blank charts	Empty or all-NaN data
23	Widget key conflicts	Two identical widgets (same label/params) or the same key used twice, often in loops or reused functions.
24	Secrets configuration errors	secrets.toml missing or in the wrong folder
25	Port conflicts	A previous Streamlit process is still running

19.3 Detailed fixes

1. 'streamlit' is not recognized / command not found

Symptoms	Terminal says the command does not exist.
Likely causes	Virtual environment not activated; Scripts/bin folder not on PATH; Streamlit installed for another Python.
Diagnostic steps	`python -m pip show streamlit`; `where python` (Windows) or `which python`.
Corrected code	`python -m streamlit run app.py`
Prevention	Always activate `.venv` first; use `python -m streamlit` in scripts.

2. ModuleNotFoundError

Symptoms	`ModuleNotFoundError: No module named 'sklearn'` (or your own `utils`).
Likely causes	Package not installed in the active environment; import name differs from pip name (sklearn vs scikit-learn, PIL vs pillow, cv2 vs opencv-python); running from the wrong folder for local modules.
Diagnostic steps	`pip list`; print `sys.executable` and `sys.path` in the app.
Corrected code	`pip install scikit-learn pillow opencv-python-headless` `# local modules: run from the project root and add utils/__init__.py`
Prevention	Keep requirements.txt complete; test in a fresh venv.

3. Model file not found

Symptoms	`FileNotFoundError: models/model.joblib`
Likely causes	Relative path resolved against the current working directory, not the script; file not committed; training not run.
Diagnostic steps	`st.write(Path.cwd(), Path(__file__).parent)`; `git ls-files models/`.

Corrected code	`from pathlib import Path`
	`MODEL = Path(__file__).parent / "models" / "model.joblib"`
	`if not MODEL.exists():`
	`  st.error("Model missing: run train_model.py")`
	`  st.stop()`

Prevention	Build every path from `__file__`; fail early with a clear message.

4. Pickle or joblib loading error

Symptoms	`AttributeError: Can't get attribute 'clean_text' on <module '__main__'>`, `ModuleNotFoundError: No module named 'numpy._core'`, `InconsistentVersionWarning`.
Likely causes	Custom function defined in a notebook; different scikit-learn/NumPy version; corrupted or Git-LFS pointer file.
Diagnostic steps	Compare versions in metadata vs `sklearn.__version__`; check file size (LFS pointers are ~130 bytes).

Corrected code	`# define helpers in a module BEFORE training`
	`from text_utils import clean_text`
	`# pin versions used for training`
	`# scikit-learn==1.9.1`

Prevention	Store versions and a fingerprint in metadata; keep custom code importable.

5. Inconsistent feature names

Symptoms	`ValueError: The feature names should match those that were passed during fit`.
Likely causes	Columns renamed, reordered or missing; DataFrame built with different names (spaces, case).
Diagnostic steps	`model.feature_names_in_` vs `X.columns`.
Corrected code	`X = pd.DataFrame([record])[list(model.feature_names_in_)]`
Prevention	Use a shared schema/FEATURES list for training and inference.

6. Missing columns

Symptoms	`KeyError: "['contract'] not in index"` in batch scoring.
Likely causes	Uploaded file lacks required columns or uses different headers.
Diagnostic steps	List `set(FEATURES) - set(df.columns)`.

Corrected code	`missing = [c for c in FEATURES if c not in df.columns]`
	`if missing:`
	`  st.error(f"Missing columns: {missing}")`
	`  st.stop()`

Prevention	Validate before predicting; publish a template CSV.

7. Data type mismatch

Symptoms	`could not convert string to float: 'N/A'`; categories become numbers.
Likely causes	Text in numeric columns; IDs read as numbers; pandas 3 string dtype differs from object.
Diagnostic steps	`df.dtypes`; `pd.to_numeric(col, errors="coerce").isna().sum()`.

Corrected code	`for c in NUMERIC:`
	`  df[c] = pd.to_numeric(df[c], errors="coerce")`
	`is_num = pd.api.types.is_numeric_dtype(df["tenure"])`

Prevention	Coerce and report; use `is_numeric_dtype` rather than `dtype == object`.

8. Shape mismatch

Symptoms	`Expected 2D array, got 1D array`; Keras: `expected shape (None, 28, 28, 1)`.
Likely causes	Passing a single sample without the batch dimension; flattening differently.
Diagnostic steps	`print(x.shape)` before predict.
Corrected code	`X = pd.DataFrame([record]) # 1 row, 2-D` `x = img_array[np.newaxis, ...] # (1, H, W, C)`
Prevention	Write a `prepare_input()` function with a shape assertion.

9. Image preprocessing mismatch

Symptoms	Model is confident and wrong on uploads, fine on the test set.
Likely causes	RGB vs BGR, 0-255 vs 0-1, different resize/crop, missing normalisation (mean/std), grayscale vs colour, EXIF rotation.
Diagnostic steps	Display the exact tensor the model receives (`st.image(x)`).
Corrected code	`from preprocessing import preprocess_image # same function as training` `x = preprocess_image(Image.open(f))`
Prevention	One preprocessing module shared by training and the app.

10. TensorFlow loading errors

Symptoms	`ValueError: File format not supported`, `Unknown layer: 'CustomLayer'`, DLL errors on Windows.
Likely causes	Saved with a different Keras major version; custom layers not registered; missing Visual C++ runtime.
Diagnostic steps	Check `keras.__version__` vs metadata.
Corrected code	`model = keras.models.load_model("m.keras",` `  custom_objects={"CustomLayer": CustomLayer})`
Prevention	Save with `.keras`; pin TensorFlow; register custom objects.

11. PyTorch device mismatch

Symptoms	`Expected all tensors to be on the same device` or CUDA errors on a CPU server.
Likely causes	Model saved on GPU and loaded on CPU; input tensor on a different device.
Diagnostic steps	Print `next(model.parameters()).device` and `x.device`.

Corrected code	`state = torch.load(path, map_location=DEVICE, weights_only=True)`
	`model.load_state_dict(state)`
	`model.to(DEVICE).eval()`
	`x = x.to(DEVICE)`

Prevention	Always use `map_location` and move inputs to the model's device.

12. Streamlit reruns unexpectedly

Symptoms	Results disappear; expensive code runs on every click.
Likely causes	Every widget change reruns the script; results not stored; no caching.
Diagnostic steps	Add `st.write(time.time())` at the top to see reruns.

Corrected code	`if st.button("Predict"):`
	`  st.session_state.result = predict(X)`
	`if "result" in st.session_state:`
	`  st.metric("Result", st.session_state.result)`

Prevention	Use forms, fragments, caching and session state.

13. Session state errors

Symptoms	`AttributeError: st.session_state has no attribute 'history'`; `StreamlitAPIException: ... cannot be modified after the widget ... is instantiated`.
Likely causes	Key not initialised; assigning to a widget key after the widget is drawn.
Diagnostic steps	Print `st.session_state.to_dict()`.

Corrected code	`st.session_state.setdefault("history", [])`
	`# change widget values in a callback (runs before widgets are drawn)`
	`st.button("Reset", on_click=lambda: st.session_state.update(threshold=0.5))`

Prevention	Initialise at the top; modify widget keys only in callbacks.

14. Cache-related issues

Symptoms	Stale data after the file changed; `UnhashableParamError`; mutated cached object affects other users.
Likely causes	Cache key does not include a version; unhashable argument; mutating a `cache_resource` object.
Diagnostic steps	Clear cache from the app menu and compare.

Corrected code	`@st.cache_data(ttl="10m")`
	`def load(path, mtime): ...`
	
	`@st.cache_data`
	`def predict(_model, version, X): ... # _model is not hashed`

Prevention	Include versions in arguments; never mutate cached resources.

15. File uploader returns unexpected data

Symptoms	`TypeError: expected str, bytes or os.PathLike object, not UploadedFile`; empty DataFrame on second read.
Likely causes	Passing the object where a path is expected; reading the buffer twice (pointer at end); multiple-file mode returns a list.
Diagnostic steps	`st.write(type(f), f.name, f.size)`.

Corrected code	`data = f.getvalue() # bytes, re-readable`
	`df = pd.read_csv(io.BytesIO(data))`
	`# or f.seek(0) before reading again`

Prevention	Read once into bytes; handle lists when `accept_multiple_files=True`.

16. CSV encoding issues

Symptoms	`UnicodeDecodeError: 'utf-8' codec can't decode byte 0xa0`; odd characters such as 'Ã©'.
Likely causes	File saved as Windows-1252/Latin-1 or UTF-8 with BOM by Excel.
Diagnostic steps	Try `encoding="utf-8-sig"`, then `latin-1`.

Corrected code	`for enc in ("utf-8", "utf-8-sig", "latin-1"):`
	`  try:`
	`  df = pd.read_csv(io.BytesIO(data), encoding=enc)`
	`  break`
	`  except UnicodeDecodeError:`
	`  continue`

Prevention	Ask for 'CSV UTF-8' exports; try a fallback chain.

17. Deployment dependency conflicts

Symptoms	Build log: `ResolutionImpossible`, or a package has no wheel for the platform's Python.
Likely causes	Over-pinned transitive dependencies from `pip freeze`; Windows-only packages; Python version mismatch.
Diagnostic steps	Read the first error in the build log; try the requirements in a fresh venv.

Corrected code	`# keep direct dependencies only, pinned`
	`streamlit==1.65.0`
	`scikit-learn==1.9.1`
	`# remove pywin32 and other OS-specific packages`

Prevention	Hand-curate requirements; match the platform's Python version.

18. Works locally, fails in production

Symptoms	Errors only after deployment.
Likely causes	Missing files or secrets; case-sensitive paths on Linux (`Models/` vs `models/`); absolute local paths; different library versions; less memory.
Diagnostic steps	Reproduce in a fresh clone + fresh venv (or the Docker image).
Corrected code	`path = Path(__file__).parent / "models" / "churn_pipeline.joblib" # exact case`
Prevention	Test the production configuration locally before deploying (Chapter 13).

19. Memory errors

Symptoms	`MemoryError`, app restarts, 'over resource limits'.
Likely causes	Large uploads, big models, caching many large objects, unclosed figures, storing DataFrames in session state for many users.
Diagnostic steps	`docker stats` or the platform metrics; log `df.memory_usage(deep=True).sum()`.

Corrected code	`df = pd.read_csv(f, usecols=FEATURES, dtype={"contract": "category"})`
	`plt.close(fig)`
	`@st.cache_data(max_entries=10)`

Prevention	Limit sizes, cache with limits, prefer compact models and dtypes.

20. Incorrect prediction results

Symptoms	No error, but predictions look implausible.
Likely causes	Training/inference preprocessing mismatch; wrong column order; wrong class index for `predict_proba`; units differ (lakh vs rupees).
Diagnostic steps	Score a known training row in the app and compare with the notebook output.
Corrected code	`pos_idx = list(model.classes_).index("Yes") # not always column 1` `proba = model.predict_proba(X)[:, pos_idx]`
Prevention	Golden-row tests: fixed inputs with expected outputs in `tests/`.

21. Slow application performance

Symptoms	Every interaction takes seconds.
Likely causes	Uncached loading, recomputing charts, large tables rendered in full, heavy work in all tabs.
Diagnostic steps	Time sections with `time.perf_counter()`; comment out blocks.

Corrected code	`@st.cache_resource`
	`def load_model(): ...`
	
	`@st.fragment`
	`def what_if(): ...`
	`st.dataframe(df.head(500))`

Prevention	Cache, fragment, paginate, and move heavy jobs offline.

22. Blank charts

Symptoms	Empty chart area, no error.
Likely causes	Empty or all-NaN data; wrong x/y column names; Matplotlib figure not passed (`st.pyplot()` without fig); Plotly figure built but not displayed.
Diagnostic steps	`st.write(df.head(), df.dtypes)` before charting.

Corrected code	`fig, ax = plt.subplots()`
	`ax.plot(x, y)`
	`st.pyplot(fig)`
	`st.plotly_chart(fig_px, width="stretch")`

Prevention	Check data before plotting; always pass the figure object.

23. Widget key conflicts

Symptoms	`StreamlitDuplicateElementId` / `DuplicateWidgetID`.
Likely causes	Two identical widgets (same label/params) or the same key used twice, often in loops or reused functions.
Diagnostic steps	Search for the label/key; print keys in the loop.
Corrected code	`for i, feat in enumerate(features):` `  st.number_input(feat, key=f"feat_{i}")`
Prevention	Give every widget in loops/reusable functions a unique key prefix.

24. Secrets configuration errors

Symptoms	`StreamlitSecretNotFoundError` or `KeyError: 'database'`.
Likely causes	secrets.toml missing or in the wrong folder; TOML syntax error; secrets not added on the cloud platform; wrong nesting.
Diagnostic steps	`st.write(list(st.secrets.keys()))` (never print values).

Corrected code	`# .streamlit/secrets.toml (next to app.py)`
	`[database]`
	`url = "sqlite:///p.db"`
	
	`# code`
	`url = st.secrets["database"]["url"]`

Prevention	Keep a `secrets.example.toml` with dummy values in Git.

25. Port conflicts

Symptoms	`Port 8501 is already in use` or the browser shows another app.
Likely causes	A previous Streamlit process is still running; another service uses the port.
Diagnostic steps	`netstat -ano | findstr 8501` (Windows), `lsof -i :8501` (macOS/Linux).
Corrected code	`streamlit run app.py --server.port 8502`
Prevention	Stop servers with Ctrl + C; set a port per project in config.toml.

CHAPTER 20

Streamlit Quick Reference

20

Commands, syntax and checklists on a few pages

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Look up commands, components and syntax quickly.
- ✓ Reuse proven patterns for caching, files, charts, navigation and errors.
- ✓ Run the deployment checklist before release.

20.1 Common commands

Command	Purpose
`pip install streamlit`	Install
`pip install --upgrade streamlit`	Upgrade to the latest release
`streamlit version`	Show installed version
`streamlit hello`	Demo gallery
`streamlit run app.py`	Run an app
`streamlit run app.py --server.port 8502`	Choose a port
`streamlit run app.py --theme.base dark`	Override a config option
`streamlit config show`	Print every config option and its value
`streamlit cache clear`	Clear persisted caches
`python -m streamlit run app.py`	Run with the current interpreter
`pytest -q`	Run AppTest and unit tests

20.2 Categorised component table

Category	Component	Purpose	Minimal syntax
Text	st.title / header / subheader	Headings	`st.header("Results")`
Text	st.markdown	Formatted text	`st.markdown("**Bold** :orange[x]")`
Text	st.caption	Small notes	`st.caption("Source: CRM")`
Text	st.code / st.latex / st.json	Code, maths, JSON	`st.json(meta)`
Status	st.success/info/warning/error	Messages	`st.error("Missing col")`
Status	st.spinner / st.progress / st.status	Long work	`with st.spinner("..."):`
Status	st.toast	Quick confirmation	`st.toast("Saved")`
Input	st.button	Action	`if st.button("Go"):`

Category	Component	Purpose	Minimal syntax
Input	st.download_button	Download	`st.download_button("CSV", csv, "p.csv")`
Input	st.selectbox / radio / multiselect	Choices	`st.selectbox("Model", names)`
Input	st.slider / select_slider	Ranges	`st.slider("t", 0.0, 1.0, 0.5)`
Input	st.number_input / text_input / text_area	Values	`st.number_input("Area", 250, 6000)`
Input	st.date_input / time_input / datetime_input	Dates	`st.date_input("Start")`
Input	st.checkbox / st.toggle	Booleans	`st.toggle("Explain")`
Input	st.file_uploader	Files	`st.file_uploader("CSV", type=["csv"])`
Input	st.camera_input / st.audio_input	Media capture	`st.camera_input("Photo")`
Input	st.segmented_control / st.pills	Compact choices	`st.pills("Tags", opts)`
Input	st.chat_input / st.feedback	Chat, ratings	`st.feedback("thumbs")`
Layout	st.sidebar	Global controls	`with st.sidebar:`
Layout	st.columns	Side by side	`a, b = st.columns(2)`
Layout	st.tabs	Modes	`t1, t2 = st.tabs(["A", "B"])`
Layout	st.container / st.empty	Groups, placeholders	`st.container(border=True)`
Layout	st.expander / st.popover	Hidden detail	`st.expander("More")`
Layout	st.form	Batch inputs	`with st.form("f"):`
Layout	@st.dialog	Modal	`@st.dialog("Confirm")`
Data	st.dataframe / st.data_editor	Tables	`st.dataframe(df, hide_index=True)`
Data	st.table / st.metric	Static table, KPI	`st.metric("AUC", 0.91, 0.01)`
Charts	st.line/area/bar/scatter_chart	Native charts	`st.line_chart(df, x="d", y="v")`
Charts	st.plotly_chart / altair_chart / pyplot	Library charts	`st.plotly_chart(fig, width="stretch")`
Charts	st.map / st.pydeck_chart	Maps	`st.map(df)`
Media	st.image / st.audio / st.video	Media	`st.image(img, width=300)`
Navigation	st.navigation / st.Page	Multipage	`st.navigation(pages).run()`
Navigation	st.page_link / st.switch_page	Links	`st.page_link("p.py", label="Go")`
Control	st.stop / st.rerun	Flow control	`st.stop()`
Control	@st.fragment	Partial reruns	`@st.fragment(run_every="10s")`
State	st.session_state	Per-session values	`st.session_state.setdefault("k", 0)`
Cache	@st.cache_data / @st.cache_resource	Caching	`@st.cache_resource`
Config	st.set_page_config / st.secrets	Setup, secrets	`st.secrets["db"]["url"]`
Auth	st.login / st.logout / st.user	OIDC login	`st.user.is_logged_in`

20.3 Widgets, layout and state

cheat_widgets_layout_state.py PYTHON

```python
import streamlit as st

model = st.selectbox("Model", ["RF", "LR"], key="model")
t = st.slider("Threshold", 0.0, 1.0, 0.5, 0.05)
left, right = st.columns([2, 1], gap="large")
with st.sidebar:
    st.header("Settings")
tab1, tab2 = st.tabs(["Predict", "Explain"])
with st.form("f"):
    area = st.number_input("Area", 250, 6000, 1000)
    go = st.form_submit_button("Predict", type="primary")
st.session_state.setdefault("history", [])
st.session_state.history.append({"area": area}) if go else None
```

20.4 Caching, files and charts

cheat_cache_files_charts.py PYTHON

```python
import io

import joblib
import pandas as pd
import plotly.express as px
import streamlit as st


@st.cache_resource
def load_model(path: str):
    return joblib.load(path)


@st.cache_data(ttl="1h")
def load_csv(data: bytes) -> pd.DataFrame:
    return pd.read_csv(io.BytesIO(data))


f = st.file_uploader("CSV", type=["csv"])
if f:
    df = load_csv(f.getvalue())
    st.dataframe(df.head(), hide_index=True)
    st.plotly_chart(px.histogram(df, x=df.columns[0]), width="stretch")
    st.download_button("Download", df.to_csv(index=False), "out.csv", "text/csv")
```

20.5 Navigation, model loading, prediction and errors

cheat_nav_predict.py (fragment) PSEUDOCODE / FRAGMENT

```python
import pandas as pd
import streamlit as st

# navigation (in app.py)
page = st.navigation([st.Page("app_pages/home.py", title="Home", default=True),
                      st.Page("app_pages/predict.py", title="Predict")])

# model loading with error handling
try:
    model = load_model("models/pipeline.joblib")
except FileNotFoundError:
    st.error("Model not found. Run train_model.py first.")
    st.stop()

# prediction
X = pd.DataFrame([record])[FEATURES]
```

cheat_nav_predict.py (fragment) PSEUDOCODE / FRAGMENT

```
pos = list(model.classes_).index("Yes")
proba = model.predict_proba(X)[0, pos]
st.metric("Probability", f"{proba:.1%}")
page.run()
```

20.6 Error-handling patterns

Situation	Pattern
Missing artifact	check `path.exists()` -> `st.error` -> `st.stop()`
Bad upload	`try: pd.read_csv(...) except (ValueError, UnicodeDecodeError)`
Validation	collect errors in a list; show all; stop if any
External call	`requests` with `timeout=`; handle 401/422/5xx
Unknown failure	`except Exception: log.exception(...); st.error(friendly)`
Optional feature fails	`st.warning` and continue with reduced output

20.7 Deployment checklist

- ☐ Fresh clone + fresh venv runs `streamlit run app.py`.
- ☐ `requirements.txt` pinned (scikit-learn exact).
- ☐ Model + metadata present (or downloaded and verified).
- ☐ Secrets configured on the platform; none in Git.
- ☐ Upload limits and row limits set.
- ☐ `client.showErrorDetails` and `toolbarMode` set for viewers.
- ☐ Tests pass in CI.
- ☐ README with purpose, usage and limitations.
- ☐ Logs visible; owner and contact documented.

CHAPTER 21

Exercises and Assessments

21

Practice from first widgets to secured, containerised apps

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Practise every major skill in the book at three levels.
- ✓ Check your work against solution outlines.

Work through the exercises in order. Each one lists a task and a hint; solution outlines are in Section 21.4. Treat the advanced exercises as small projects: write a README and tests for each.

21.1 Beginner exercises

Exercise 1: Text and headings

Create `profile.py` with a title, two headers, a Markdown paragraph with bold and a link, a caption and a divider.

Hint: Use `st.title`, `st.header`, `st.markdown`, `st.caption`, `st.divider`.

Exercise 2: Sidebar

Add a sidebar with your name, a model selectbox and a threshold slider; show the selections in the main area.

Hint: `with st.sidebar:` then read the returned values.

Exercise 3: A simple form

Build a form with name, age and city that greets the user only after Submit.

Hint: `st.form` + `st.form_submit_button`; check the returned bool.

Exercise 4: Accept user inputs

Collect area, bedrooms and locality and display them as a one-row DataFrame.

Hint: `pd.DataFrame([record])` then `st.dataframe`.

Exercise 5: Display a metric

Show three KPI cards (accuracy, precision, recall) with deltas versus last month.

Hint: `st.columns(3)` and `col.metric(label, value, delta, border=True)`.

Exercise 6: Upload a CSV

Upload any CSV and show rows, columns, missing values and the first 10 rows.

Hint: `st.file_uploader(type=["csv"])`, `pd.read_csv(f)`.

Exercise 7: A simple chart

Plot monthly sales from a small DataFrame with `st.line_chart`, then the same data with `st.bar_chart`.

Hint: Pass `x=` and `y=` column names.

Exercise 8: Status messages

Show success, info, warning and error messages for four situations in a prediction app.

Hint: One call each, with Material icons.

Exercise 9: Expander and code

Show the pipeline description inside an expander using `st.code`.

Hint: `with st.expander("Details"):`

Exercise 10: Download text

Let the user type notes and download them as a .txt file.

Hint: `st.download_button("Save", notes, "notes.txt")`.

21.2 Intermediate exercises

Exercise 11: Classification form

Build a churn form (Chapter 6 schema) that shows the predicted probability and label.

Hint: Load with `@st.cache_resource`; build X from FEATURES; choose the positive-class column.

Exercise 12: Load a saved model safely

Load a joblib model with a fingerprint check and a friendly error if the file is missing or modified.

Hint: Reuse `utils/model_io.py`.

Exercise 13: Regression application

Create a used-car price app with currency formatting and range warnings.

Hint: Store training percentiles in metadata; warn outside them.

Exercise 14: Dataset explorer

Upload a CSV, choose columns to filter, and plot any two numeric columns.

Hint: `multiselect` filters, `scatter_chart`, `data_quality` table.

Exercise 15: Multipage application

Convert the explorer and the classifier into a three-page app with `st.navigation`.

Hint: Entry `app.py` + `app_pages/`; shared `common.py`.

Exercise 16: Prediction downloads

Add CSV and Excel downloads of predictions to the classification app.

Hint: `to_csv`, `BytesIO` + `to_excel` (openpyxl).

Exercise 17: Session-state history

Keep a history of predictions in the session with a Clear button.

Hint: `setdefault("history", [])`, append on submit, `st.rerun()` after clearing.

Exercise 18: Threshold analysis

Show how precision and recall change as the threshold slider moves, using saved test predictions.

Hint: `precision_recall_curve` or compute counts at the threshold.

Exercise 19: Callbacks

Add a Reset button that restores all inputs using a callback.

Hint: Seed widget keys from defaults; reset keys inside `on_click`.

Exercise 20: Theming

Brand the app with config.toml colours and a logo; test light and dark.

Hint: `[theme]`, `[theme.dark]`, `st.logo`.

Exercise 21: Fragment

Put a what-if slider inside `@st.fragment` and confirm the rest of the page does not rerun.

Hint: Print `time.time()` outside and inside.

21.3 Advanced exercises

Exercise 22: Batch prediction system

Score uploaded files up to 100k rows in chunks with progress, validation report and download.

Hint: Follow `batch_app.py`; add risk bands.

Exercise 23: Database integration

Store every prediction and user feedback in SQLite, then switch the URL to PostgreSQL.

Hint: SQLAlchemy + bound parameters; URL from secrets.

Exercise 24: Model evaluation dashboard

Build tabs for metrics, diagnostics and model comparison with safeguards.

Hint: Start from `evaluation.py` (Chapter 18).

Exercise 25: Image classification interface

Wrap a pretrained MobileNetV2 (Keras applications) with upload, camera and top-5 output.

Hint: Use `preprocess_input` and `decode_predictions` from the same module as the model.

Exercise 26: Deployment pipeline

Create a GitHub repo with CI that trains the model and runs tests, then deploy to Community Cloud.

Hint: `.github/workflows/ci.yml` from Chapter 13.

Exercise 27: Containerise an ML app

Write a Dockerfile, build, run with a health check and environment variables.

Hint: Chapter 14 Dockerfile; `docker inspect` health.

Exercise 28: User authentication

Protect an app with `st.login()` and an OIDC provider; show admin tools only to listed emails.

Hint: `[auth]` secrets; `st.user`; `pip install Authlib`.

Exercise 29: FastAPI + Streamlit

Serve a model with FastAPI and build a Streamlit client with API-key auth and error handling.

Hint: Chapter 11 `api/main.py`.

Exercise 30: Monitoring page

Compute PSI drift between training data and an uploaded batch for every numeric feature.

Hint: Bin with training quantiles; PSI = sum((a - e) * ln(a / e)).

Exercise 31: PDF report

Generate a PDF with metrics, a chart and the top-10 riskiest customers.

Hint: `utils/reports.py`.

Exercise 32: Production hardening

Audit one of your projects against the Chapter 15 checklist and fix three issues.

Hint: Error details, upload limits, secrets, logging.

Exercise 33: Capstone extension

Add a new model type (e.g. the sentiment pipeline) to the capstone registry and make every page support it.

Hint: Extend `predictor.py` for a text input schema.

Total: 33 exercises.

21.4 Solutions and solution outlines

Exercise 1: text and headings

solution_01.py PYTHON

```python
import streamlit as st

st.title("Asha Patil - Data Scientist")
st.header("About")
st.markdown("I build **ML apps** with Streamlit. "
            "[GitHub](https://github.com/example)")
st.caption("Pune, India")
st.divider()
st.header("Projects")
```

Exercise 3: a simple form

solution_03.py PYTHON

```python
import streamlit as st

with st.form("greet"):
    name = st.text_input("Name")
    age = st.number_input("Age", 1, 120, 25)
    city = st.selectbox("City", ["Pune", "Mumbai", "Nagpur"])
    sent = st.form_submit_button("Submit")
if sent:
    if not name.strip():
        st.warning("Please enter your name.")
    else:
        st.success(f"Hello {name.strip()} ({age}) from {city}!")
```

Exercise 5: KPI cards

solution_05.py PYTHON

```python
import streamlit as st

kpis = {"Accuracy": (0.87, 0.02), "Precision": (0.81, -0.01), "Recall": (0.74, 0.05)}
for col, (name, (value, delta)) in zip(st.columns(3), kpis.items()):
    col.metric(name, f"{value:.2f}", f"{delta:+.2f}", border=True)
```

Exercise 6: upload a CSV

solution_06.py PYTHON

```python
import pandas as pd
import streamlit as st

f = st.file_uploader("CSV file", type=["csv"])
if f is not None:
    try:
        df = pd.read_csv(f)
    except Exception as exc:
        st.error(f"Could not read the file: {exc}")
        st.stop()
    c1, c2, c3 = st.columns(3)
    c1.metric("Rows", f"{len(df):,}")
    c2.metric("Columns", df.shape[1])
    c3.metric("Missing cells", int(df.isna().sum().sum()))
    st.dataframe(df.head(10))
```

Exercise 17: session-state history

solution_17.py PYTHON

```python
import streamlit as st

st.session_state.setdefault("history", [])
with st.form("f"):
    x = st.number_input("Value", 0.0, 100.0, 10.0)
    if st.form_submit_button("Predict"):
        st.session_state.history.append({"x": x, "prediction": round(x * 1.8 + 3, 2)})
if st.session_state.history:
    st.dataframe(st.session_state.history)
    if st.button("Clear"):
        st.session_state.history = []
        st.rerun()
```

Exercise 30: PSI drift monitor

solution_30_psi.py PYTHON

```python
import numpy as np
import pandas as pd


def psi(expected: pd.Series, actual: pd.Series, bins: int = 10) -> float:
    """Population Stability Index using quantile bins of the training data."""
    edges = np.unique(np.quantile(expected.dropna(), np.linspace(0, 1, bins + 1)))
    edges[0], edges[-1] = -np.inf, np.inf
    e = np.histogram(expected.dropna(), edges)[0] / expected.notna().sum()
    a = np.histogram(actual.dropna(), edges)[0] / actual.notna().sum()
    e, a = np.clip(e, 1e-6, None), np.clip(a, 1e-6, None)
    return float(np.sum((a - e) * np.log(a / e)))


rng = np.random.default_rng(0)
train = pd.Series(rng.normal(50, 10, 5000))
print(round(psi(train, pd.Series(rng.normal(50, 10, 1000))), 3))   # ~0.0  stable
print(round(psi(train, pd.Series(rng.normal(58, 10, 1000))), 3))   # > 0.25 shifted
```

Outlines for the remaining exercises

Exercise	Solution outline
2	Selectbox and slider inside `with st.sidebar:`; `st.write` the values.
4	Widgets -> dict -> `pd.DataFrame([record])` -> `st.dataframe(..., hide_index=True)`.
7	Build a DataFrame of 12 months; `st.line_chart(df, x="month", y="sales")` then `st.bar_chart`.
8	Model loaded (success), predictions are estimates (info), input out of range (warning), missing column (error).
9	`with st.expander("Pipeline"): st.code(str(pipe), language=None)`.
10	`notes = st.text_area(...)`; `st.download_button("Download", notes, "notes.txt", "text/plain")`.
11-12	See churn_app/app.py and utils/model_io.py (Chapter 6).
13	Same structure as house_app; save p01/p99 per numeric feature; `format_inr` or a USD formatter.
14	Sidebar filters build a boolean mask; `scatter_chart` with `color=` a categorical column; quality table as in Chapter 4.
15	Follow multipage_demo (Chapter 4G).
16	`df.to_csv(index=False)`; `BytesIO` + `df.to_excel(buffer, index=False)`; set MIME types.
18	Load `churn_test_predictions.csv`; for threshold t compute TP, FP, FN; show precision/recall metrics and a confusion matrix.
19-21	See Chapters 9, 5 and 3 examples (reset callback, theming, fragment).
22-24	Chapters 8, 11 and 18 provide complete reference implementations.
25	`keras.applications.MobileNetV2(weights="imagenet")` in `cache_resource`; resize to 224x224; `preprocess_input`; `decode_predictions(probs, top=5)`.
26	Push repo; add ci.yml; deploy with main file path; add secrets.
27	Dockerfile from Chapter 14; `docker run -e ...`; `docker inspect --format '{{.State.Health.Status}}'`.
28	Section 15.6 auth_app.py; register redirect URI with the provider.
29	api/main.py + api_client_app.py (Chapter 11).
31	utils/reports.py `to_pdf_bytes` (Chapter 12).
32	Set showErrorDetails="none", maxUploadSize, move secrets to st.secrets, add logging without personal data.
33	Add `input_schema` with a `text` type; branch in `schema_form` (text_area) and `predict_frame` (pass a list of strings).

CHAPTER 22

Final Capstone Project

22

Production-Style Machine Learning Deployment Platform Using Streamlit

LEARNING OBJECTIVES

By the end of this chapter you will be able to:

- ✓ Translate business and functional requirements into a modular app.
- ✓ Implement a multipage platform with a model registry, validation, single and batch prediction, history, evaluation and reports.
- ✓ Test, containerise and deploy it.
- ✓ Present it as a professional portfolio project.

22.1 Business requirements

- Analysts must score customers and properties without writing code.
- Several models (classification and regression) must be available from one place, with clear information about each.
- Uploaded data must be checked for quality before it is scored.
- Every prediction must be recorded for audit, and results must be downloadable for reporting.
- Model performance must be verifiable on held-out labeled data before use.
- The platform must run locally, in Docker and in the cloud with configuration, not code changes.

22.2 Functional requirements

#	Requirement	Implemented in
1	Home dashboard	`app_pages/home.py`
2	Dataset upload (CSV/XLSX)	`app_pages/upload.py`
3	Data quality report	`services/quality.py` + upload page tabs
4	Model selection	Sidebar selector in `app.py` + `services/registry.py`
5	Single-record prediction	`app_pages/predict.py` (form generated from the model's input schema)
6	Batch prediction	`app_pages/batch.py` + `services/predictor.py`
7	Prediction history	`services/history.py` (database) + `app_pages/history.py`
8	Model evaluation	`services/evaluation.py` + `app_pages/evaluate.py`
9	Interactive visualisations	Plotly and native charts on every page
10	Downloadable reports	`services/reports.py` (CSV, Excel, PDF)
11	Model information page	`app_pages/model_info.py`

#	Requirement	Implemented in
12	Error handling	Validation messages, `ModelLoadError`, global error boundary in `app.py`
13	Session-state management	`ui/state.py` (keys, session id, cached loaders)
14	Configuration management	`config/settings.toml`, `services/config.py`, `.streamlit/config.toml`, secrets
15	Deployment instructions	Sections 22.9-22.10, Dockerfile

22.3 UI wireframes

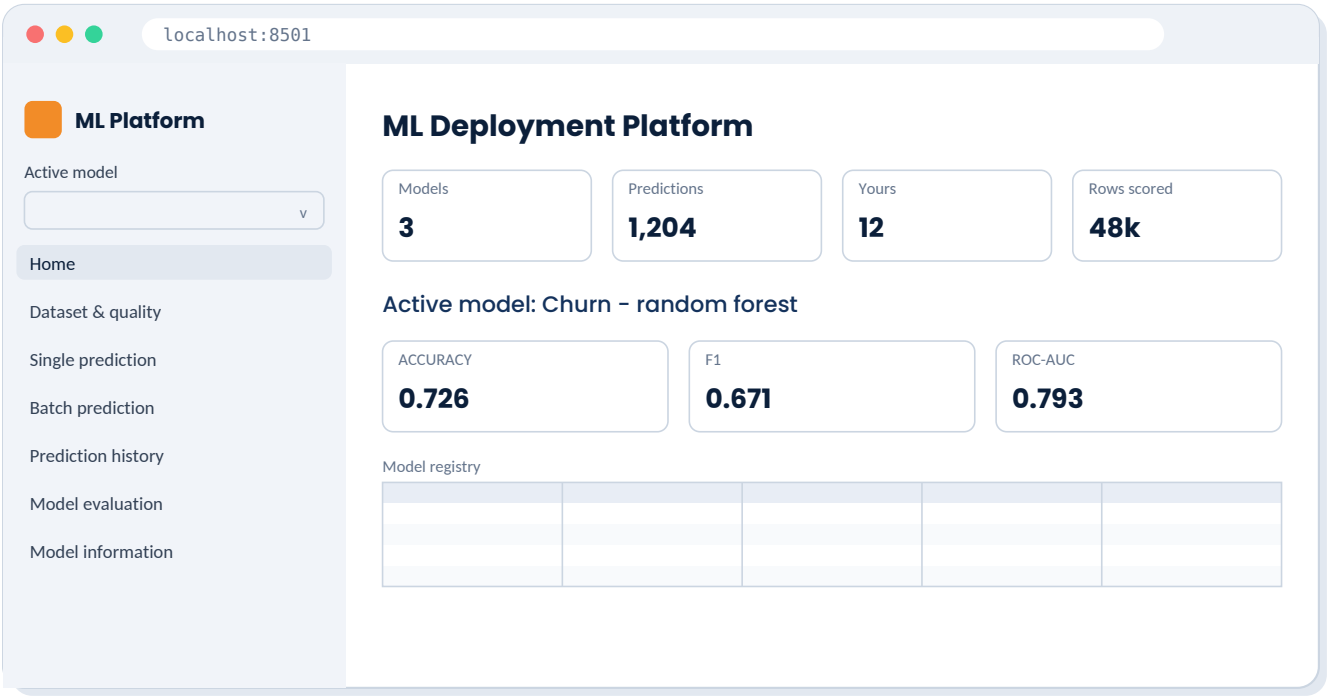


Figure 22.1: Home dashboard

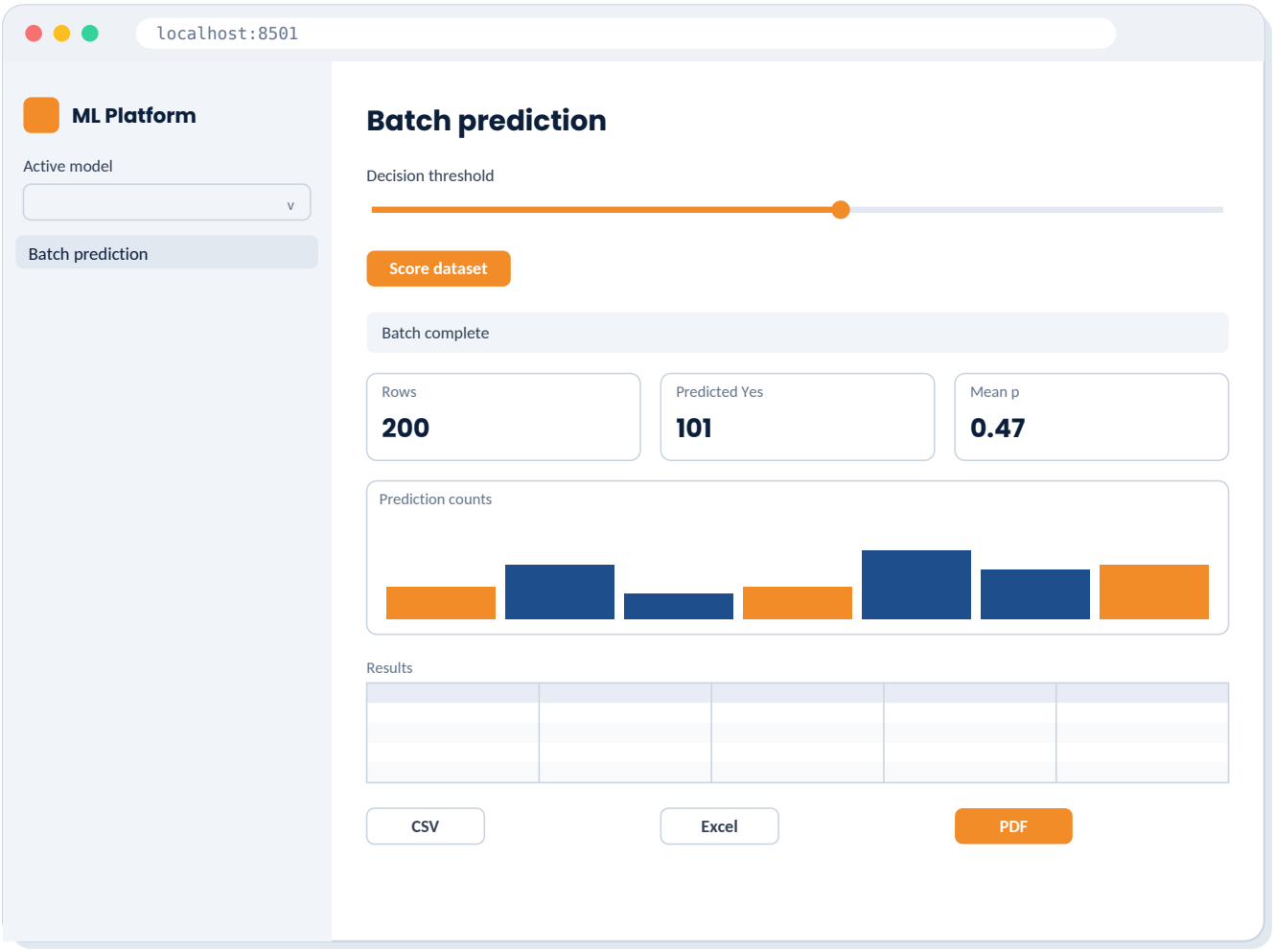


Figure 22.2: Batch prediction page

22.4 Project folder structure

capstone/ layout — TEXT

```
capstone/
├── app.py
├── app_pages/
│   ├── home.py
│   ├── upload.py
│   ├── predict.py
│   ├── batch.py
│   ├── history.py
│   ├── evaluate.py
│   └── model_info.py
├── ui/
│   ├── __init__.py
│   ├── state.py
│   └── components.py
├── scripts/build_models.py
├── tests/
│   ├── test_services.py
│   └── test_pages.py
├── services/
│   ├── __init__.py
│   ├── config.py
│   ├── registry.py
│   ├── model_io.py
│   ├── validation.py
│   ├── predictor.py
│   ├── evaluation.py
│   ├── quality.py
│   ├── history.py
│   └── reports.py
├── config/settings.toml
├── models/   (built by scripts/build_models.py)
├── data/     churn.csv houses.csv new_customers.csv
├── .streamlit/config.toml
├── Dockerfile  .dockerignore  .gitignore
└── requirements.txt  pytest.ini
```

22.5 Implementation plan

Phase	Deliverable	Done when
1. Foundations	Config, model_io, registry, build_models.py	`python -m scripts.build_models` registers three models
2. Core services	validation, predictor, quality, evaluation	`tests/test_services.py` passes
3. UI shell	app.py router, ui/state.py, ui/components.py, home page	Navigation works; model selector shared by pages
4. Prediction pages	upload, predict, batch	Sample file scored end to end
5. Persistence + reports	history service + page, reports service	History survives restart; Excel/PDF open
6. Evaluation + info	evaluate and model_info pages	Metrics match training metadata
7. Hardening	Error boundary, limits, config.toml, page tests	`pytest -q` passes for every model
8. Deployment	Dockerfile, README, cloud deploy	Public or private URL works from a fresh clone

22.6 Source files

Configuration and services

capstone/config/settings.toml TOML

```toml
# Application settings. Safe to commit: contains NO secrets.
# Secrets (database passwords, API keys) go in .streamlit/secrets.toml or env vars.

[app]
name = "ML Deployment Platform"
environment = "development"        # development | staging | production

[models]
registry = "models/registry.json"

[uploads]
max_rows = 100000
allowed_types = ["csv", "xlsx"]

[history]
# overridden by st.secrets["database"]["url"] or the DATABASE_URL env variable
default_url = "sqlite:///history.db"

[thresholds]
classification_default = 0.5
```

capstone/services/config.py PYTHON

```python
"""Load config/settings.toml once and resolve secrets from the environment."""
import os
import tomllib
from functools import lru_cache
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]


@lru_cache
def settings() -> dict:
    with open(ROOT / "config" / "settings.toml", "rb") as fh:
        return tomllib.load(fh)
```

capstone/services/config.py PYTHON

```python
def database_url(secrets=None) -> str:
    """Priority: Streamlit secrets -> DATABASE_URL env var -> settings default."""
    if secrets is not None:
        try:
            return secrets["database"]["url"]
        except (KeyError, FileNotFoundError):
            pass
    url = os.environ.get("DATABASE_URL") or settings()["history"]["default_url"]
    if url.startswith("sqlite:///") and not url.startswith("sqlite:////"):
        url = f"sqlite:///{ROOT / url.removeprefix('sqlite:///')}"   # project-relative
    return url
```

capstone/services/registry.py PYTHON

```python
"""Model registry: which models exist, what they expect, and how to load them."""
import json
from dataclasses import dataclass
from pathlib import Path

from services.config import ROOT, settings
from services.model_io import load_trusted_model


@dataclass(frozen=True)
class ModelEntry:
    id: str
    name: str
    task: str                # "classification" | "regression"
    model_file: str
    meta_file: str
    target: str
    positive_class: str | None = None
    description: str = ""


def registry_path() -> Path:
    return ROOT / settings()["models"]["registry"]


def list_models() -> list[ModelEntry]:
    raw = json.loads(registry_path().read_text())
    return [ModelEntry(**item) for item in raw]


def get_entry(model_id: str) -> ModelEntry:
    for entry in list_models():
        if entry.id == model_id:
            return entry
    raise KeyError(f"Unknown model id: {model_id}")


def load(entry: ModelEntry):
    """Return (model, metadata, warnings); raises ModelLoadError."""
    folder = registry_path().parent
    return load_trusted_model(folder / entry.model_file, folder / entry.meta_file)
```

capstone/services/validation.py PYTHON

```python
"""Validate inputs against the `input_schema` stored in each model's metadata.

input_schema = {feature: {"type": "numeric", "min": 0, "max": 72, "default": 12}
                or {"type": "categorical", "options": [...], "default": "..."}}
"""
import pandas as pd


def validate_record(record: dict, schema: dict) -> list[str]:
    errors = []
    for feat, spec in schema.items():
        val = record.get(feat)
        if val is None:
            errors.append(f"{feat} is required")
        elif spec["type"] == "numeric" and not spec["min"] <= val <= spec["max"]:
            errors.append(f"{feat} must be in [{spec['min']}, {spec['max']}]")
        elif spec["type"] == "categorical" and val not in spec["options"]:
            errors.append(f"{feat}: '{val}' is not an allowed value")
    return errors


def validate_frame(df: pd.DataFrame, schema: dict, max_rows: int):
    """Return (clean_df, errors, warnings). clean_df has model columns in order."""
    errors, warnings = [], []
    missing = [f for f in schema if f not in df.columns]
    if missing:
        return None, [f"Missing required columns: {', '.join(missing)}"], warnings
    if df.empty:
        return None, ["The file contains no rows."], warnings
    if len(df) > max_rows:
        return None, [f"{len(df):,} rows exceeds the limit of {max_rows:,}."], warnings
    X = df[list(schema)].copy()
    for feat, spec in schema.items():
        if spec["type"] == "numeric":
            converted = pd.to_numeric(X[feat], errors="coerce")
            bad = int((converted.isna() & X[feat].notna()).sum())
            if bad:
                errors.append(f"{feat}: {bad} non-numeric value(s)")
            out = int(((converted < spec["min"]) | (converted > spec["max"])).sum())
            if out:
                warnings.append(f"{feat}: {out} value(s) outside the training range")
            X[feat] = converted
        else:
            unknown = set(X[feat].dropna().astype(str)) - set(spec["options"])
            if unknown:
                warnings.append(f"{feat}: unseen categories {sorted(unknown)[:5]}")
    n_missing = int(X.isna().sum().sum())
    if n_missing:
        warnings.append(f"{n_missing} missing value(s) will be imputed or may fail "
                        "for models without imputers")
    return (None if errors else X), errors, warnings
```

capstone/services/predictor.py PYTHON

```python
"""Task-aware prediction functions. Pages call these; they never touch the model."""
import numpy as np
import pandas as pd

from services.registry import ModelEntry


def predict_frame(model, entry: ModelEntry, meta: dict, X: pd.DataFrame,
                  threshold: float = 0.5) -> pd.DataFrame:
    """Return a DataFrame with prediction columns for each row of X."""
    X = X[meta["features"]]
    if entry.task == "classification":
        classes = list(model.classes_)
```

capstone/services/predictor.py PYTHON

```
        proba = model.predict_proba(X)
        out = pd.DataFrame(index=X.index)
        if entry.positive_class and len(classes) == 2:
            pos = classes.index(entry.positive_class)
            negative = classes[1 - pos]
            out["probability"] = proba[:, pos].round(4)
            out["prediction"] = np.where(proba[:, pos] >= threshold,
                                         entry.positive_class, negative)
        else:
            out["prediction"] = [classes[i] for i in proba.argmax(axis=1)]
            out["probability"] = proba.max(axis=1).round(4)
        return out
    preds = model.predict(X)
    if not np.isfinite(preds).all():
        raise ValueError("Model returned non-finite predictions")
    return pd.DataFrame({"prediction": preds.round(2)}, index=X.index)


def predict_one(model, entry: ModelEntry, meta: dict, record: dict,
                threshold: float = 0.5) -> dict:
    row = pd.DataFrame([record])
    return predict_frame(model, entry, meta, row, threshold).iloc[0].to_dict()
```

capstone/services/quality.py PYTHON

```
"""Data quality profiling for uploaded datasets."""
import pandas as pd


def profile(df: pd.DataFrame) -> dict:
    numeric = df.select_dtypes("number")
    return {
        "rows": len(df),
        "columns": df.shape[1],
        "missing_cells": int(df.isna().sum().sum()),
        "missing_pct": float(df.isna().mean().mean()),
        "duplicate_rows": int(df.duplicated().sum()),
        "numeric_columns": numeric.shape[1],
        "text_columns": df.shape[1] - numeric.shape[1],
        "constant_columns": [c for c in df.columns if df[c].nunique(dropna=False) <= 1],
    }


def column_report(df: pd.DataFrame) -> pd.DataFrame:
    rep = pd.DataFrame({
        "dtype": df.dtypes.astype(str),
        "missing": df.isna().sum(),
        "missing_pct": (df.isna().mean() * 100).round(2),
        "unique": df.nunique(),
    })
    num = df.select_dtypes("number")
    if not num.empty:
        rep = rep.join(num.describe().T[["mean", "min", "max"]].round(3))
    return rep


def outlier_counts(df: pd.DataFrame, k: float = 1.5) -> pd.Series:
    """Values outside [Q1 - k*IQR, Q3 + k*IQR] per numeric column."""
    num = df.select_dtypes("number")
    q1, q3 = num.quantile(0.25), num.quantile(0.75)
    iqr = q3 - q1
    return ((num < q1 - k * iqr) | (num > q3 + k * iqr)).sum().rename("outliers")
```

capstone/services/history.py PYTHON

```python
"""Persistent prediction history (SQLAlchemy; SQLite locally, Postgres in production)."""
import json
from datetime import datetime, timezone

import pandas as pd
from sqlalchemy import (Column, DateTime, Float, Integer, MetaData, String, Table, Text,
                        create_engine, insert, select)
from sqlalchemy.engine import Engine

metadata_obj = MetaData()
predictions = Table(
    "predictions", metadata_obj,
    Column("id", Integer, primary_key=True, autoincrement=True),
    Column("created_at", DateTime(timezone=True), nullable=False),
    Column("session_id", String(64), nullable=False),
    Column("model_id", String(64), nullable=False),
    Column("mode", String(16), nullable=False),          # single | batch
    Column("n_rows", Integer, nullable=False),
    Column("summary", Text, nullable=False),             # JSON
    Column("score", Float),
)


def make_engine(url: str) -> Engine:
    engine = create_engine(url, pool_pre_ping=True)
    metadata_obj.create_all(engine)        # creates the table if it does not exist
    return engine


def record(engine: Engine, session_id: str, model_id: str, mode: str, n_rows: int,
           summary: dict, score: float | None = None) -> None:
    with engine.begin() as conn:
        conn.execute(insert(predictions).values(
            created_at=datetime.now(timezone.utc), session_id=session_id,
            model_id=model_id, mode=mode, n_rows=n_rows,
            summary=json.dumps(summary, default=str), score=score))


def fetch(engine: Engine, session_id: str | None = None, limit: int = 500) -> pd.DataFrame:
    """Query builder objects generate parameterised SQL automatically."""
    q = select(predictions).order_by(predictions.c.id.desc()).limit(limit)
    if session_id:
        q = q.where(predictions.c.session_id == session_id)
    with engine.connect() as conn:
        return pd.read_sql(q, conn)
```

capstone/services/reports.py PYTHON

```python
"""In-memory Excel and PDF reports for batch predictions."""
import io
from datetime import datetime

import pandas as pd
from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer, Table, TableStyle


def excel_report(predictions: pd.DataFrame, summary: pd.DataFrame) -> bytes:
    buf = io.BytesIO()
    with pd.ExcelWriter(buf, engine="openpyxl") as xw:
        summary.to_excel(xw, sheet_name="Summary")
        predictions.to_excel(xw, sheet_name="Predictions", index=False)
    return buf.getvalue()
```

capstone/services/reports.py PYTHON

```python
def pdf_report(title: str, model_name: str, summary: pd.DataFrame,
               metrics: dict | None) -> bytes:
    buf = io.BytesIO()
    styles = getSampleStyleSheet()
    story = [Paragraph(title, styles["Title"]),
             Paragraph(f"Model: {model_name} | generated {datetime.now():%Y-%m-%d %H:%M}",
                       styles["Normal"]), Spacer(1, 12),
             Paragraph("Batch summary", styles["Heading2"])]
    rows = [["statistic", "value"]] + [[str(i), str(v)] for i, v in
                                       summary.iloc[:, 0].items()]
    story.append(_styled(Table(rows, hAlign="LEFT")))
    if metrics:
        story += [Spacer(1, 10), Paragraph("Model evaluation metrics", styles["Heading2"])]
        story.append(_styled(Table([["metric", "value"]] +
                                   [[k, f"{v:.4f}"] for k, v in metrics.items()],
                                   hAlign="LEFT")))
    SimpleDocTemplate(buf, pagesize=A4, title=title).build(story)
    return buf.getvalue()


def _styled(t: Table) -> Table:
    t.setStyle(TableStyle([("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#0b1f3a")),
                           ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
                           ("GRID", (0, 0), (-1, -1), 0.4, colors.grey)]))
    return t
```

`services/model_io.py` is identical to `churn_app/utils/model_io.py` (Section 6.3) and `services/evaluation.py` is identical to `eval_app/evaluation.py` (Chapter 18). Copy them unchanged.

Model building

capstone/scripts/build_models.py PYTHON

```python
"""Train the platform's models and write models/registry.json.

Run from the capstone root:  python -m scripts.build_models
"""
import json

import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingRegressor, RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, f1_score, mean_absolute_error, r2_score,
                             roc_auc_score)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from services.config import ROOT
from services.model_io import save_model

MODELS, DATA = ROOT / "models", ROOT / "data"


def input_schema(df: pd.DataFrame, features: list[str]) -> dict:
    schema = {}
    for f in features:
        if pd.api.types.is_numeric_dtype(df[f]):
            is_int = pd.api.types.is_integer_dtype(df[f].dropna())
            cast = int if is_int else float
            schema[f] = {"type": "numeric", "min": cast(df[f].min()),
                         "max": cast(df[f].max()), "default": cast(df[f].median()),
                         "integer": bool(is_int)}
        else:
```

capstone/scripts/build_models.py PYTHON

```
            opts = sorted(df[f].dropna().astype(str).unique())
            schema[f] = {"type": "categorical", "options": opts,
                         "default": str(df[f].mode()[0])}
    return schema


def preprocessor(df: pd.DataFrame, features: list[str]) -> ColumnTransformer:
    num = [f for f in features if pd.api.types.is_numeric_dtype(df[f])]
    cat = [f for f in features if f not in num]
    return ColumnTransformer([
        ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                          ("sc", StandardScaler())]), num),
        ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                          ("oh", OneHotEncoder(handle_unknown="ignore"))]), cat)])


def train_classifiers(registry: list) -> None:
    df = pd.read_csv(DATA / "churn.csv")
    feats = [c for c in df.columns if c not in ("customer_id", "churn")]
    tr, te = train_test_split(df, test_size=0.2, stratify=df["churn"], random_state=42)
    te.to_csv(DATA / "churn_eval.csv", index=False)
    candidates = {
        "churn_rf": ("Churn - random forest",
                     RandomForestClassifier(n_estimators=300, max_depth=8,
                                            class_weight="balanced", random_state=42)),
        "churn_lr": ("Churn - logistic regression",
                     LogisticRegression(max_iter=1000, class_weight="balanced")),
    }
    for model_id, (name, est) in candidates.items():
        pipe = Pipeline([("prep", preprocessor(tr, feats)), ("model", est)])
        pipe.fit(tr[feats], tr["churn"])
        pred = pipe.predict(te[feats])
        pos = list(pipe.classes_).index("Yes")
        score = pipe.predict_proba(te[feats])[:, pos]
        metrics = {"accuracy": accuracy_score(te["churn"], pred),
                   "f1": f1_score(te["churn"], pred, pos_label="Yes"),
                   "roc_auc": roc_auc_score(te["churn"] == "Yes", score)}
        save_model(pipe, MODELS / f"{model_id}.joblib", MODELS / f"{model_id}.json", {
            "model_name": name, "task": "classification", "features": feats,
            "target": "churn", "classes": list(pipe.classes_), "positive_class": "Yes",
            "input_schema": input_schema(tr, feats), "n_train": len(tr),
            "metrics": {k: round(float(v), 4) for k, v in metrics.items()},
            "intended_use": "Prioritise retention outreach; human review required."})
        registry.append({"id": model_id, "name": name, "task": "classification",
                         "model_file": f"{model_id}.joblib",
                         "meta_file": f"{model_id}.json", "target": "churn",
                         "positive_class": "Yes",
                         "description": "Predicts subscription cancellation risk."})


def train_regressor(registry: list) -> None:
    df = pd.read_csv(DATA / "houses.csv")
    feats = [c for c in df.columns if c != "price_inr"]
    tr, te = train_test_split(df, test_size=0.2, random_state=42)
    te.to_csv(DATA / "house_eval.csv", index=False)
    pipe = Pipeline([("prep", preprocessor(tr, feats)),
                     ("model", HistGradientBoostingRegressor(max_iter=400,
                                                             learning_rate=0.05,
                                                             random_state=42))])
    pipe.fit(tr[feats], tr["price_inr"])
    pred = pipe.predict(te[feats])
    metrics = {"mae": mean_absolute_error(te["price_inr"], pred),
               "r2": r2_score(te["price_inr"], pred)}
    save_model(pipe, MODELS / "house_hgb.joblib", MODELS / "house_hgb.json", {
        "model_name": "House price - gradient boosting", "task": "regression",
        "features": feats, "target": "price_inr", "input_schema": input_schema(tr, feats),
        "n_train": len(tr), "metrics": {k: round(float(v), 4) for k, v in metrics.items()},
        "intended_use": "Indicative price estimates for synthetic Pune listings."})
```

capstone/scripts/build_models.py PYTHON

```
    registry.append({"id": "house_hgb", "name": "House price - gradient boosting",
                     "task": "regression", "model_file": "house_hgb.joblib",
                     "meta_file": "house_hgb.json", "target": "price_inr",
                     "description": "Estimates apartment prices in INR."})


if __name__ == "__main__":
    MODELS.mkdir(exist_ok=True)
    reg: list = []
    train_classifiers(reg)
    train_regressor(reg)
    (MODELS / "registry.json").write_text(json.dumps(reg, indent=2))
    print("Registered:", ", ".join(r["id"] for r in reg))
```

UI layer

capstone/app.py PYTHON

```
"""Production-style ML Deployment Platform.  Run: streamlit run app.py

app.py is the router. On every rerun it: configures the page, initialises session
state, draws the shared sidebar (model selector), then runs the selected page
inside an error boundary.
"""
import logging

import streamlit as st

from services import registry
from services.config import settings
from services.model_io import ModelLoadError
from ui.state import init_session, load_model

logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(name)s: %(message)s")
log = logging.getLogger("platform")
cfg = settings()

st.set_page_config(page_title=cfg["app"]["name"], page_icon=":material/hub:",
                   layout="wide", initial_sidebar_state="expanded")
init_session()

PAGES = {
    "Overview": [st.Page("app_pages/home.py", title="Home", icon=":material/home:",
                         default=True)],
    "Data": [st.Page("app_pages/upload.py", title="Dataset & quality",
                     icon=":material/upload_file:")],
    "Predict": [
        st.Page("app_pages/predict.py", title="Single prediction",
                icon=":material/person_search:"),
        st.Page("app_pages/batch.py", title="Batch prediction",
                icon=":material/dataset:"),
        st.Page("app_pages/history.py", title="Prediction history",
                icon=":material/history:"),
    ],
    "Model": [
        st.Page("app_pages/evaluate.py", title="Model evaluation",
                icon=":material/analytics:"),
        st.Page("app_pages/model_info.py", title="Model information",
                icon=":material/info:"),
    ],
}
page = st.navigation(PAGES)

with st.sidebar:
    st.subheader(cfg["app"]["name"])
    entries = registry.list_models()
```

capstone/app.py PYTHON

```
    ids = [e.id for e in entries]
    names = {e.id: e.name for e in entries}
    if st.session_state.model_id not in ids:
        st.session_state.model_id = ids[0]
    st.selectbox("Active model", ids, format_func=names.get, key="model_id")
    if cfg["app"]["environment"] != "production":
        st.caption(f"Environment: {cfg['app']['environment']} | "
                   f"session {st.session_state.session_id}")

try:
    load_model(st.session_state.model_id)     # fail fast with a clear message
    page.run()
except ModelLoadError as exc:
    log.error("model load failed: %s", exc)
    st.error(f"The selected model could not be loaded: {exc}")
except Exception:                             # global error boundary
    log.exception("unhandled error on page %s", page.title)
    st.error("Something went wrong on this page. The error has been logged; "
             "please try again or contact the maintainer.")
    if cfg["app"]["environment"] == "development":
        import traceback
        st.code(traceback.format_exc(), language=None)
```

capstone/ui/state.py PYTHON

```
"""Session-state keys in one place, plus cached loaders shared by all pages."""
import uuid

import streamlit as st

from services import history, registry
from services.config import database_url

KEYS = {"model_id": None, "dataset": None, "dataset_name": None, "batch_result": None}


def init_session() -> None:
    st.session_state.setdefault("session_id", uuid.uuid4().hex[:12])
    for key, default in KEYS.items():
        st.session_state.setdefault(key, default)


@st.cache_resource(show_spinner="Loading model...")
def load_model(model_id: str):
    """Cached per model id: each model is loaded once per server process."""
    entry = registry.get_entry(model_id)
    model, meta, warnings = registry.load(entry)
    return entry, model, meta, warnings


@st.cache_resource
def history_engine():
    return history.make_engine(database_url(st.secrets))


def current_model():
    return load_model(st.session_state.model_id)
```

capstone/ui/components.py PYTHON

```python
"""Reusable UI building blocks so every page looks consistent."""
import pandas as pd
import streamlit as st


def page_header(title: str, subtitle: str = "", icon: str | None = None) -> None:
    st.title(title, icon=icon)
    if subtitle:
        st.caption(subtitle)


def kpi_row(items: list[tuple[str, str]], border: bool = True) -> None:
    for col, (label, value) in zip(st.columns(len(items)), items):
        col.metric(label, value, border=border)


def show_messages(errors: list[str], warnings: list[str]) -> None:
    for w in warnings:
        st.warning(w, icon=":material/warning:")
    for e in errors:
        st.error(e, icon=":material/error:")


def format_metric(name: str, value: float) -> str:
    if value is None or pd.isna(value):
        return "n/a"
    if name in ("mae", "rmse"):
        return f"{value:,.0f}"
    if name == "mse":
        return f"{value:.2e}"
    return f"{value:.3f}"


def schema_form(schema: dict, key_prefix: str) -> dict:
    """Build an input form from a model's input_schema (two columns)."""
    cols = st.columns(2)
    record = {}
    for i, (feat, spec) in enumerate(schema.items()):
        col = cols[i % 2]
        label = feat.replace("_", " ").capitalize()
        if spec["type"] == "numeric":
            if spec.get("integer"):
                record[feat] = col.number_input(label, int(spec["min"]), int(spec["max"]),
                                                int(spec["default"]), step=1,
                                                key=f"{key_prefix}_{feat}")
            else:
                record[feat] = col.number_input(label, float(spec["min"]),
                                                float(spec["max"]), float(spec["default"]),
                                                key=f"{key_prefix}_{feat}")
        else:
            opts = spec["options"]
            record[feat] = col.selectbox(label, opts, index=opts.index(spec["default"]),
                                         key=f"{key_prefix}_{feat}")
    return record
```

Pages

capstone/app_pages/home.py PYTHON

```python
import pandas as pd
import streamlit as st

from services import history, registry
from ui.components import format_metric, kpi_row, page_header
from ui.state import current_model, history_engine

entry, model, meta, warnings = current_model()
page_header("ML Deployment Platform", "Upload data, score it with a registered model, "
            "evaluate performance and download reports.", ":material/hub:")
hist = history.fetch(history_engine(), limit=10_000)
mine = hist[hist["session_id"] == st.session_state.session_id] if len(hist) else hist
kpi_row([("Registered models", str(len(registry.list_models()))),
         ("Predictions logged (all users)", f"{len(hist):,}"),
         ("Your predictions", f"{len(mine):,}"),
         ("Rows scored (all)", f"{int(hist['n_rows'].sum()) if len(hist) else 0:,}")])

st.subheader(f"Active model: {entry.name}")
st.caption(entry.description)
kpi_row([(k.upper(), format_metric(k, v)) for k, v in meta["metrics"].items()],
        border=False)
for w in warnings:
    st.warning(w)

st.subheader("Model registry")
st.dataframe(pd.DataFrame([{"id": e.id, "name": e.name, "task": e.task,
                            "target": e.target} for e in registry.list_models()]),
             hide_index=True, width="stretch")

st.subheader("Get started")
c1, c2, c3 = st.columns(3)
c1.page_link("app_pages/upload.py", label="1. Upload a dataset",
             icon=":material/upload_file:")
c2.page_link("app_pages/batch.py", label="2. Run batch prediction",
             icon=":material/dataset:")
c3.page_link("app_pages/evaluate.py", label="3. Evaluate the model",
             icon=":material/analytics:")
```

capstone/app_pages/upload.py PYTHON

```python
import io

import pandas as pd
import streamlit as st

from services import quality
from services.config import settings
from services.validation import validate_frame
from ui.components import kpi_row, page_header, show_messages
from ui.state import current_model

cfg = settings()["uploads"]
entry, _, meta, _ = current_model()
page_header("Dataset & data quality", "Uploaded data stays in your browser session "
            "only; it is not saved on the server.", ":material/upload_file:")


@st.cache_data(show_spinner="Reading file...", max_entries=5)
def parse(name: str, data: bytes) -> pd.DataFrame:
    if name.lower().endswith(".xlsx"):
        return pd.read_excel(io.BytesIO(data))
    try:
        return pd.read_csv(io.BytesIO(data))
    except UnicodeDecodeError:
        return pd.read_csv(io.BytesIO(data), encoding="latin-1")
```

capstone/app_pages/upload.py PYTHON

```python
up = st.file_uploader("CSV or Excel file", type=cfg["allowed_types"])
if up is not None:
    try:
        st.session_state.dataset = parse(up.name, up.getvalue())
        st.session_state.dataset_name = up.name
        st.session_state.batch_result = None        # new data invalidates old results
    except Exception as exc:
        st.error(f"Could not parse {up.name}: {exc}")
if st.session_state.dataset is None and st.button("Load sample customer file"):
    st.session_state.dataset = pd.read_csv("data/new_customers.csv")
    st.session_state.dataset_name = "new_customers.csv (sample)"

df = st.session_state.dataset
if df is None:
    st.info("Upload a file or load the sample to see the quality report.")
    st.stop()

st.success(f"Loaded **{st.session_state.dataset_name}**")
p = quality.profile(df)
kpi_row([("Rows", f"{p['rows']:,}"), ("Columns", str(p["columns"])),
         ("Missing cells", f"{p['missing_cells']:,} ({p['missing_pct']:.1%})"),
         ("Duplicate rows", str(p["duplicate_rows"]))])
if p["constant_columns"]:
    st.warning(f"Constant columns (no information): {p['constant_columns']}")

tab_prev, tab_cols, tab_fit = st.tabs(["Preview", "Column report", "Model fit check"])
with tab_prev:
    st.dataframe(df.head(100), width="stretch")
with tab_cols:
    rep = quality.column_report(df)
    st.dataframe(rep, width="stretch")
    out = quality.outlier_counts(df)
    if not out.empty:
        st.bar_chart(out, horizontal=True)
        st.caption("Outliers by the 1.5 x IQR rule (a screening heuristic, not an error).")
with tab_fit:
    st.write(f"Checking against **{entry.name}** "
             f"({len(meta['features'])} required features)")
    _, errors, warnings = validate_frame(df, meta["input_schema"], cfg["max_rows"])
    show_messages(errors, warnings)
    if not errors:
        st.success("This dataset can be scored by the active model.")
        st.page_link("app_pages/batch.py", label="Continue to batch prediction",
                     icon=":material/arrow_forward:")
```

capstone/app_pages/predict.py PYTHON

```python
import streamlit as st

from services import history
from services.config import settings
from services.predictor import predict_one
from services.validation import validate_record
from ui.components import page_header, schema_form, show_messages
from ui.state import current_model, history_engine

entry, model, meta, _ = current_model()
page_header("Single prediction", f"Model: {entry.name}", ":material/person_search:")

threshold = settings()["thresholds"]["classification_default"]
if entry.task == "classification":
    threshold = st.slider("Decision threshold", 0.05, 0.95, threshold, 0.05)

with st.form(f"single_{entry.id}"):          # key per model: forms differ by schema
    record = schema_form(meta["input_schema"], key_prefix=entry.id)
    submitted = st.form_submit_button("Predict", type="primary")

if submitted:
```

capstone/app_pages/predict.py PYTHON

```
    errors = validate_record(record, meta["input_schema"])
    show_messages(errors, [])
    if not errors:
        result = predict_one(model, entry, meta, record, threshold)
        if entry.task == "classification":
            c1, c2 = st.columns(2)
            c1.metric("Prediction", str(result["prediction"]))
            c2.metric(f"P({entry.positive_class})", f"{result['probability']:.1%}")
            st.progress(float(result["probability"]))
        else:
            st.metric(f"Predicted {entry.target}", f"{result['prediction']:,.0f}")
        history.record(history_engine(), st.session_state.session_id, entry.id,
                       "single", 1, {"inputs": record, "result": result},
                       result.get("probability"))
        st.toast("Prediction logged to history", icon=":material/check:")
```

capstone/app_pages/batch.py PYTHON

```
import pandas as pd
import streamlit as st

from services import history, reports
from services.config import settings
from services.predictor import predict_frame
from services.validation import validate_frame
from ui.components import kpi_row, page_header, show_messages
from ui.state import current_model, history_engine

entry, model, meta, _ = current_model()
page_header("Batch prediction", f"Model: {entry.name}", ":material/dataset:")
df = st.session_state.dataset
if df is None:
    st.info("No dataset loaded yet.")
    st.page_link("app_pages/upload.py", label="Upload a dataset first",
                 icon=":material/upload_file:")
    st.stop()

st.caption(f"Dataset: {st.session_state.dataset_name} ({len(df):,} rows)")
X, errors, warnings = validate_frame(df, meta["input_schema"],
                                     settings()["uploads"]["max_rows"])
show_messages(errors, warnings)
if errors:
    st.stop()

threshold = 0.5
if entry.task == "classification":
    threshold = st.slider("Decision threshold", 0.05, 0.95, 0.5, 0.05)

if st.button("Score dataset", type="primary"):
    with st.status("Running batch prediction...", expanded=True) as status:
        st.write("Validating inputs... done")
        preds = predict_frame(model, entry, meta, X, threshold)
        st.write(f"Scored {len(preds):,} rows")
        result = df.join(preds)
        if entry.task == "classification":
            summary = result["prediction"].value_counts().rename("count").to_frame()
            summary.loc["mean_probability", "count"] = round(preds["probability"].mean(),
                                                             4)
        else:
            summary = preds["prediction"].describe().round(2).to_frame("value")
        st.session_state.batch_result = {"model_id": entry.id, "result": result,
                                         "summary": summary}
        history.record(history_engine(), st.session_state.session_id, entry.id,
                       "batch", len(result), {"summary": summary.to_dict()},
                       float(preds["probability"].mean()) if "probability" in preds
                       else None)
        status.update(label="Batch prediction complete", state="complete")
```

capstone/app_pages/batch.py PYTHON

```python
res = st.session_state.batch_result
if res is None or res["model_id"] != entry.id:
    st.stop()                    # nothing scored yet for the active model

result, summary = res["result"], res["summary"]
if entry.task == "classification":
    flagged = int((result["prediction"] == entry.positive_class).sum())
    kpi_row([("Rows scored", f"{len(result):,}"),
             (f"Predicted '{entry.positive_class}'", f"{flagged:,}"),
             ("Mean probability", f"{result['probability'].mean():.3f}")])
    st.bar_chart(result["prediction"].value_counts())
else:
    kpi_row([("Rows scored", f"{len(result):,}"),
             ("Mean prediction", f"{result['prediction'].mean():,.0f}"),
             ("Max prediction", f"{result['prediction'].max():,.0f}")])
    st.bar_chart(pd.cut(result["prediction"], 10).value_counts().sort_index()
                 .rename(index=str))
st.dataframe(result.head(1000), width="stretch", hide_index=True)

d1, d2, d3 = st.columns(3)
d1.download_button("Predictions (CSV)", result.to_csv(index=False), "predictions.csv",
                   "text/csv", on_click="ignore", type="primary")
d2.download_button("Summary (Excel)", reports.excel_report(result, summary),
                   "batch_summary.xlsx", on_click="ignore",
                   mime="application/vnd.openxmlformats-officedocument."
                        "spreadsheetml.sheet")
d3.download_button("Report (PDF)", reports.pdf_report("Batch prediction report",
                                                      entry.name, summary,
                                                      meta.get("metrics")),
                   "batch_report.pdf", "application/pdf", on_click="ignore")
```

capstone/app_pages/history.py PYTHON

```python
import json

import pandas as pd
import streamlit as st

from services import history
from ui.components import kpi_row, page_header
from ui.state import history_engine

page_header("Prediction history", "Stored in the history database; survives restarts "
            "(unlike session state).", ":material/history:")
scope = st.segmented_control("Show", ["This session", "All sessions"],
                             default="This session")
sid = st.session_state.session_id if scope == "This session" else None
hist = history.fetch(history_engine(), session_id=sid)
if hist.empty:
    st.info("No predictions recorded yet.")
    st.stop()

kpi_row([("Requests", str(len(hist))), ("Rows scored", f"{hist['n_rows'].sum():,}"),
         ("Models used", str(hist["model_id"].nunique()))])
daily = (hist.assign(day=pd.to_datetime(hist["created_at"]).dt.date)
         .groupby(["day", "mode"]).size().unstack(fill_value=0))
st.bar_chart(daily)
view = hist.copy()
view["summary"] = view["summary"].map(lambda s: json.dumps(json.loads(s))[:120])
st.dataframe(view, hide_index=True, width="stretch")
st.download_button("Export history (CSV)", hist.to_csv(index=False), "history.csv")
```

capstone/app_pages/evaluate.py PYTHON

```python
import numpy as np
import pandas as pd
import plotly.express as px
import streamlit as st

from services.evaluation import (check_inputs, classification_report_metrics,
                                 regression_metrics)
from ui.components import format_metric, kpi_row, page_header, show_messages
from ui.state import current_model

entry, model, meta, _ = current_model()
page_header("Model evaluation", "Evaluation needs LABELED data the model did not "
            "train on.", ":material/analytics:")
SAMPLES = {"classification": "data/churn_eval.csv", "regression": "data/house_eval.csv"}

source = st.radio("Evaluation data", ["Held-out sample file", "Uploaded dataset"],
                  horizontal=True)
if source == "Uploaded dataset":
    df = st.session_state.dataset
    if df is None:
        st.info("Upload a labeled dataset on the Dataset page first.")
        st.stop()
else:
    df = pd.read_csv(SAMPLES[entry.task])

check = check_inputs(df, meta["features"], entry.target, entry.task)
show_messages(check.errors, check.warnings)
if not check.ok:
    st.stop()
data = df.dropna(subset=[entry.target])
X, y = data[meta["features"]], data[entry.target]

if entry.task == "classification":
    pos = list(model.classes_).index(entry.positive_class)
    score = model.predict_proba(X)[:, pos]
    res = classification_report_metrics(y, model.predict(X), score,
                                        list(model.classes_), entry.positive_class)
else:
    res = regression_metrics(y, model.predict(X))
show_messages(res.errors, res.warnings)
if not res.ok:
    st.stop()

kpi_row([(k.upper(), format_metric(k, v)) for k, v in res.metrics.items()])
if entry.task == "classification":
    c1, c2 = st.columns(2)
    labels = res.extras["labels"]
    c1.plotly_chart(px.imshow(res.extras["confusion_matrix"], text_auto=True, x=labels,
                              y=labels, title="Confusion matrix",
                              labels=dict(x="Predicted", y="Actual")), width="stretch")
    if "roc" in res.extras:
        fpr, tpr, _ = res.extras["roc"]
        c2.plotly_chart(px.area(x=fpr, y=tpr, title="ROC curve",
                                labels={"x": "False positive rate",
                                        "y": "True positive rate"}), width="stretch")
    st.dataframe(res.extras["report"].round(3), width="stretch")
else:
    pred = y.to_numpy() - res.extras["residuals"]
    frame = pd.DataFrame({"actual": y.to_numpy(), "predicted": pred,
                          "residual": res.extras["residuals"]})
    c1, c2 = st.columns(2)
    c1.plotly_chart(px.scatter(frame, x="actual", y="predicted", opacity=0.5,
                               title="Actual vs predicted"), width="stretch")
    c2.plotly_chart(px.histogram(frame, x="residual", nbins=50, title="Residuals"),
                    width="stretch")
    st.caption(f"Residual mean {np.mean(frame['residual']):,.0f} (close to 0 = unbiased)")
```

capstone/app_pages/model_info.py PYTHON

```python
import pandas as pd
import streamlit as st

from ui.components import format_metric, page_header
from ui.state import current_model

entry, model, meta, warnings = current_model()
page_header("Model information", entry.name, ":material/info:")
c1, c2 = st.columns(2)
with c1:
    st.subheader("Model card")
    st.markdown(f"""
- **Task:** {entry.task}
- **Target:** `{entry.target}`
- **Algorithm:** {type(model.steps[-1][1]).__name__} (scikit-learn Pipeline)
- **Training rows:** {meta.get('n_train', 'n/a'):,}
- **Trained at (UTC):** {meta['trained_at']}
- **scikit-learn / Python:** {meta['sklearn_version']} / {meta['python_version']}
- **SHA-256:** `{meta['sha256'][:24]}...`
""")
    st.info(f"Intended use: {meta.get('intended_use', 'not documented')}")
    for w in warnings:
        st.warning(w)
with c2:
    st.subheader("Test-set metrics")
    st.dataframe(pd.DataFrame({"value": {k.upper(): format_metric(k, v)
                                         for k, v in meta["metrics"].items()}}),
                 width="stretch")
st.subheader("Input schema")
schema = pd.DataFrame(meta["input_schema"]).T
st.dataframe(schema.astype(str), width="stretch")
with st.expander("Pipeline structure"):
    st.code(str(model), language=None)
```

Configuration, dependencies and container

capstone/.streamlit/config.toml TOML

```toml
[server]
maxUploadSize = 50
headless = true

[client]
showErrorDetails = "none"     # never show raw tracebacks to end users

[theme]
base = "light"
primaryColor = "#F28C28"
```

capstone/requirements.txt REQUIREMENTS

```
# Pin exact versions in production so the server matches the training environment.
streamlit==1.65.0
pandas==3.0.5
numpy==2.5.3
scikit-learn==1.9.1
plotly==7.1.0
SQLAlchemy==2.1.4
openpyxl==3.1.5
reportlab==5.0.1
# PostgreSQL in production (optional):  psycopg[binary]>=3.2
```

capstone/Dockerfile DOCKERFILE

```
# ---- Streamlit ML platform image ------------------------------------
FROM python:3.13-slim

# Python writes no .pyc files and logs straight to the console
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

# install dependencies first: this layer is cached until requirements change
COPY requirements.txt .
RUN pip install --upgrade pip && pip install -r requirements.txt

# copy the application code and model artifacts
COPY . .

# run as an unprivileged user
RUN useradd --create-home appuser && chown -R appuser /app
USER appuser

EXPOSE 8501

HEALTHCHECK --interval=30s --timeout=5s --start-period=20s \
  CMD python -c "import urllib.request as u; u.urlopen('http://localhost:8501/_stcore/health')"

ENTRYPOINT ["streamlit", "run", "app.py", \
            "--server.port=8501", "--server.address=0.0.0.0"]
```

capstone/pytest.ini INI

```
[pytest]
pythonpath = .
testpaths = tests
```

The tests (`tests/test_services.py`, `tests/test_pages.py`) are listed in Section 16.8.

22.7 Running the capstone

Setup and run TERMINAL

```
cd capstone
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp ../churn_app/data/churn.csv ../churn_app/data/new_customers.csv data/
cp ../house_app/data/houses.csv data/
python -m scripts.build_models                        # trains + registers 3 models
streamlit run app.py
pytest -q                                             # 9 tests
```

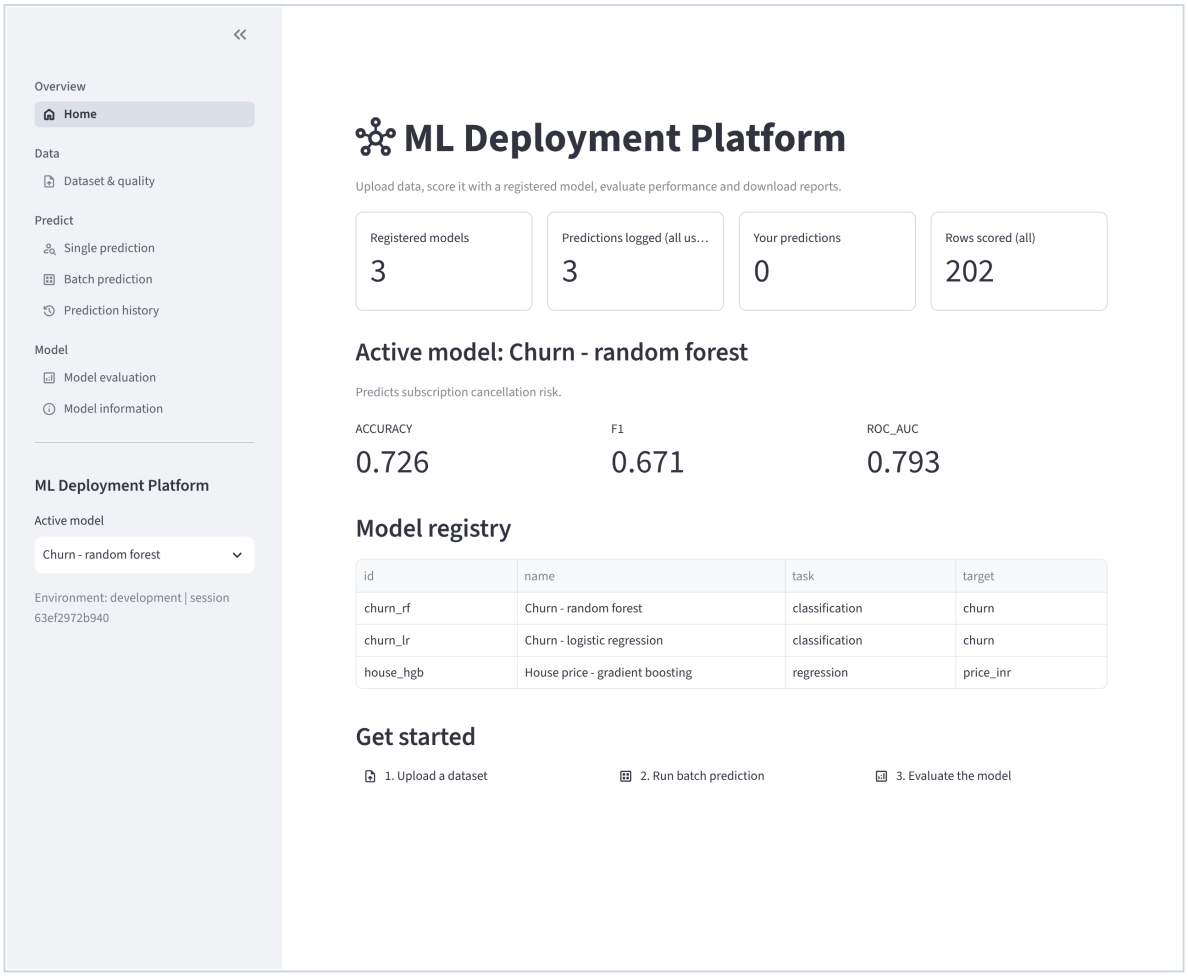


Figure 22.3: Home dashboard with registry and active-model metrics

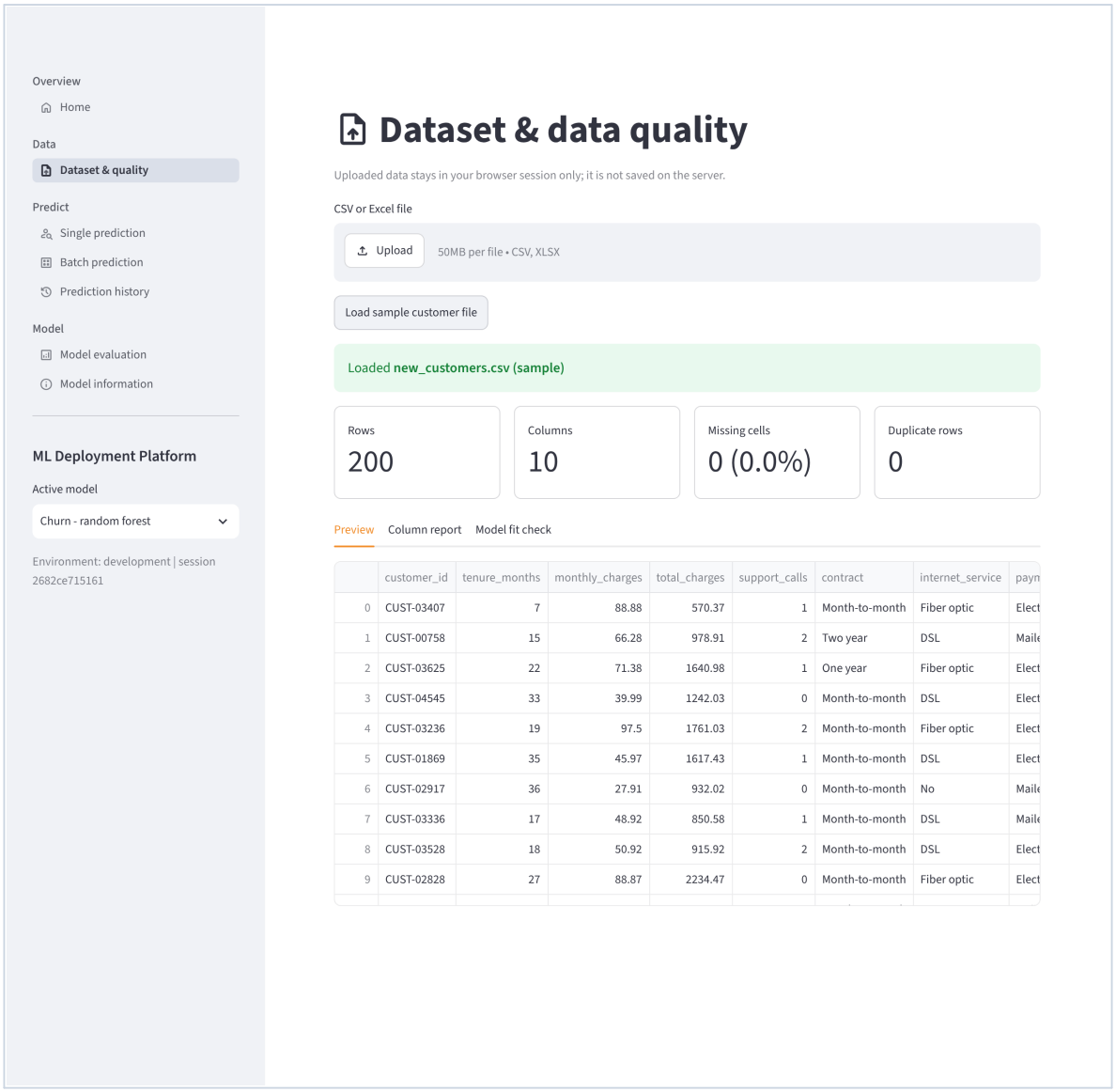


Figure 22.4: Dataset & quality page after loading the sample file

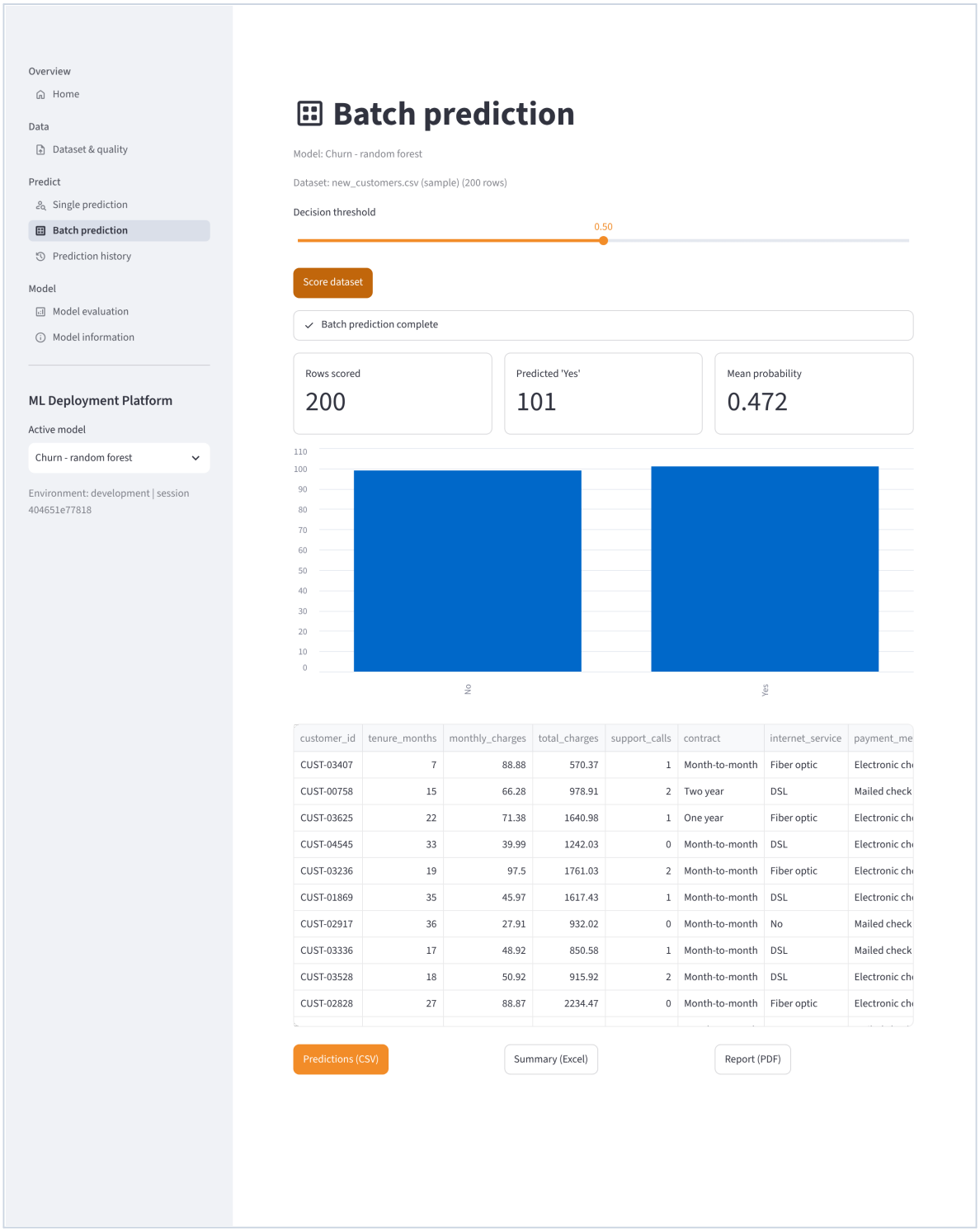


Figure 22.5: Batch prediction with status log, KPIs, chart, table and three report downloads

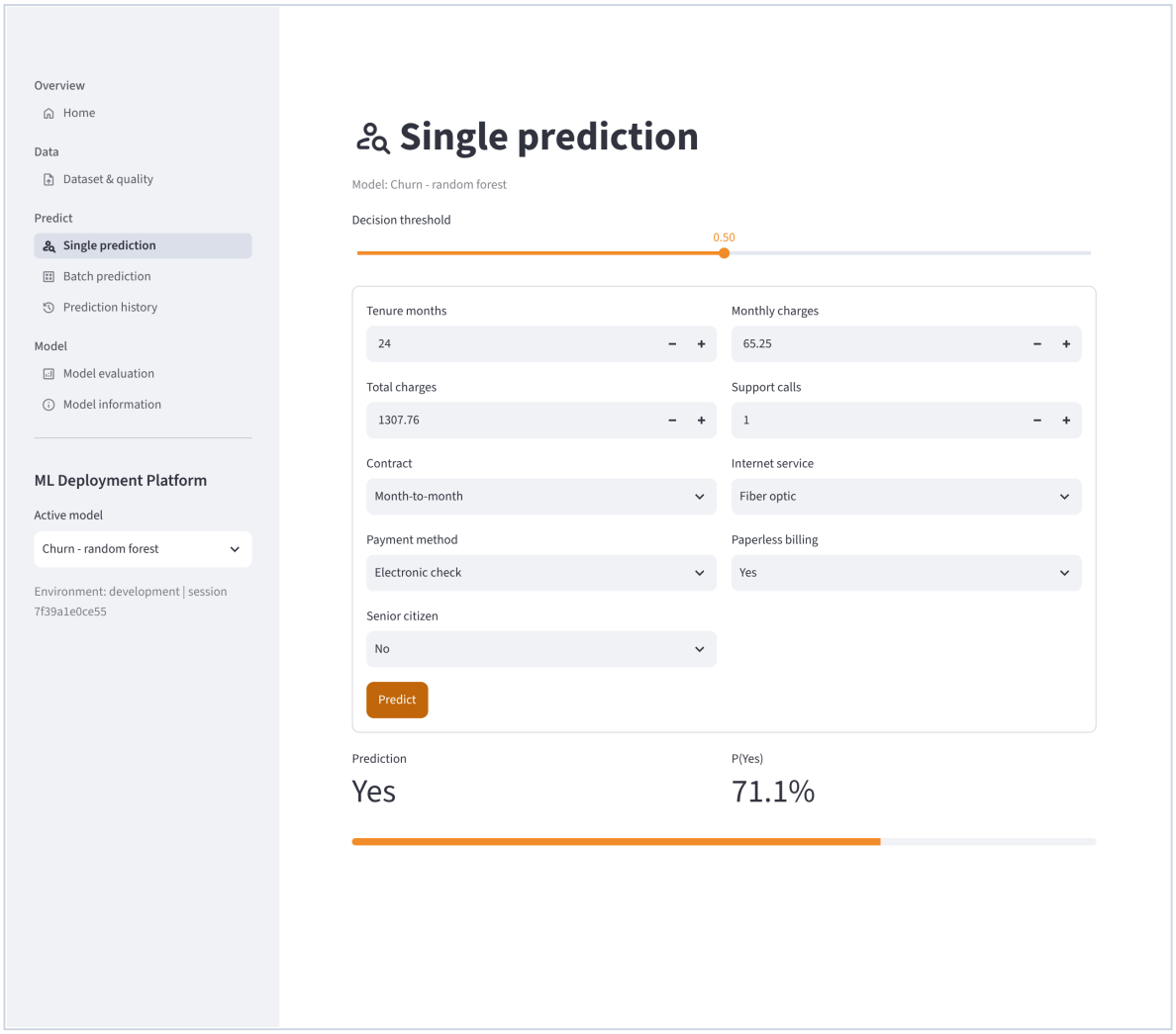


Figure 22.6: Single prediction form generated from the model's input schema

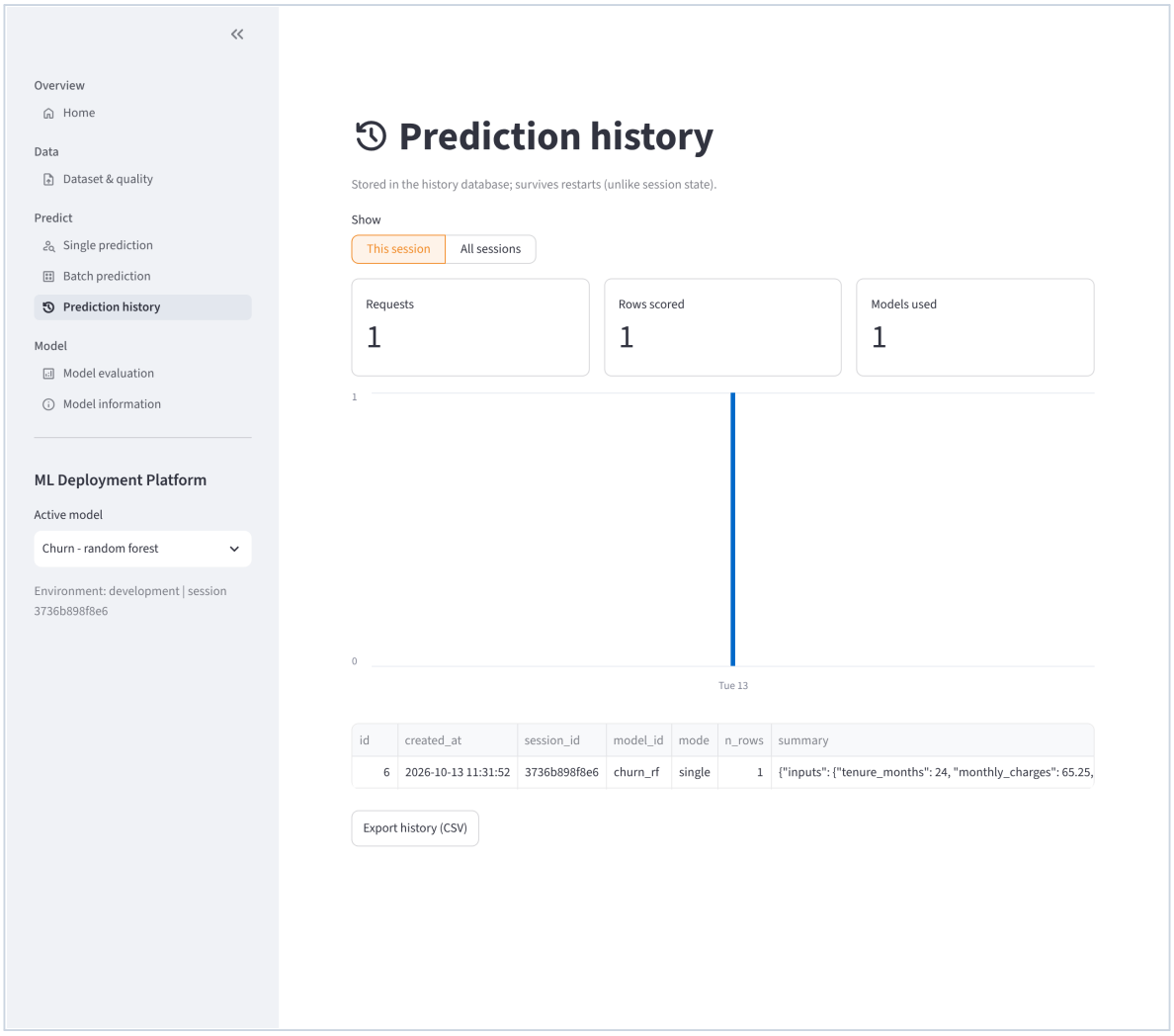


Figure 22.7: Prediction history read from the database

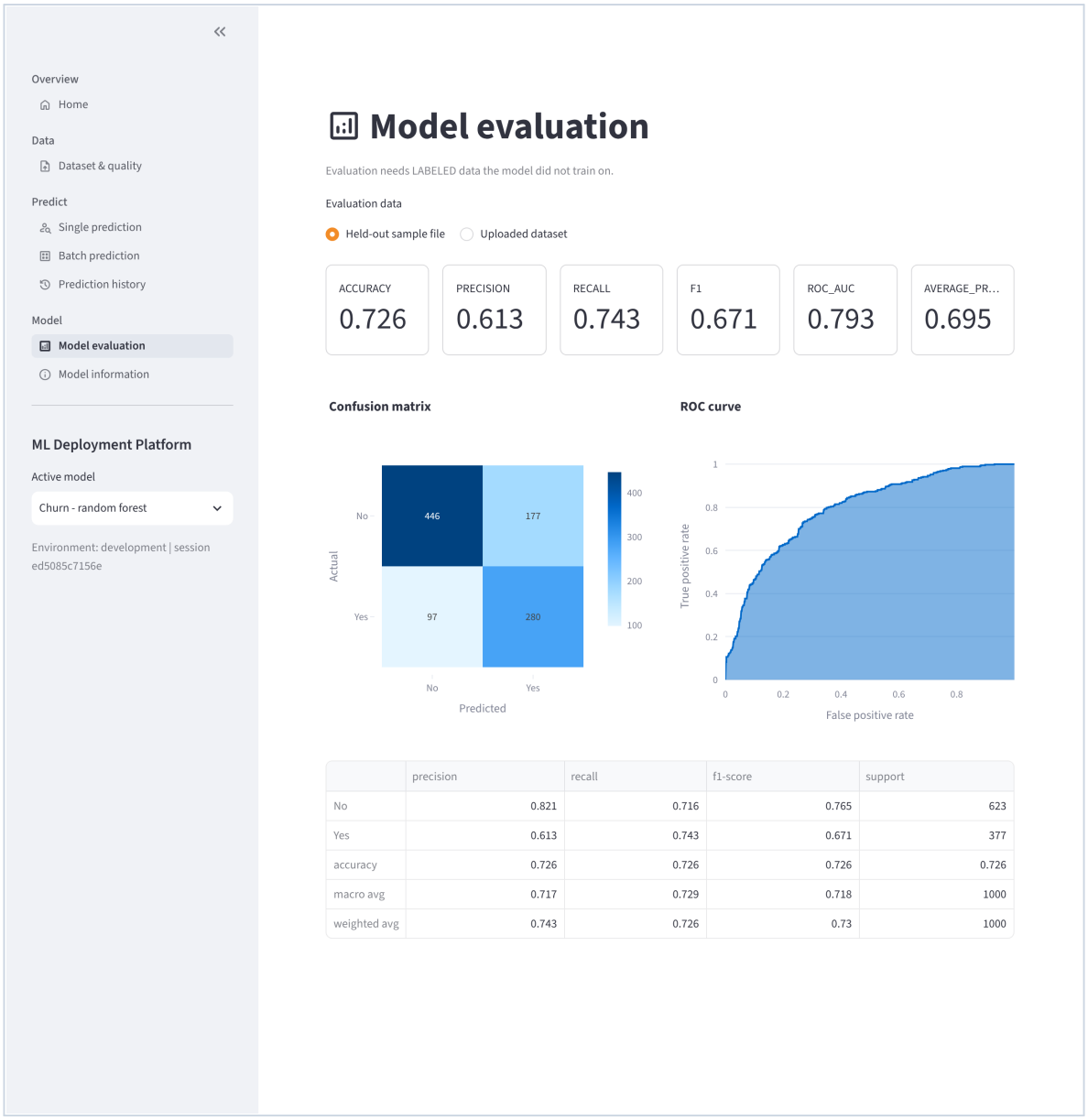


Figure 22.8: Model evaluation on held-out data

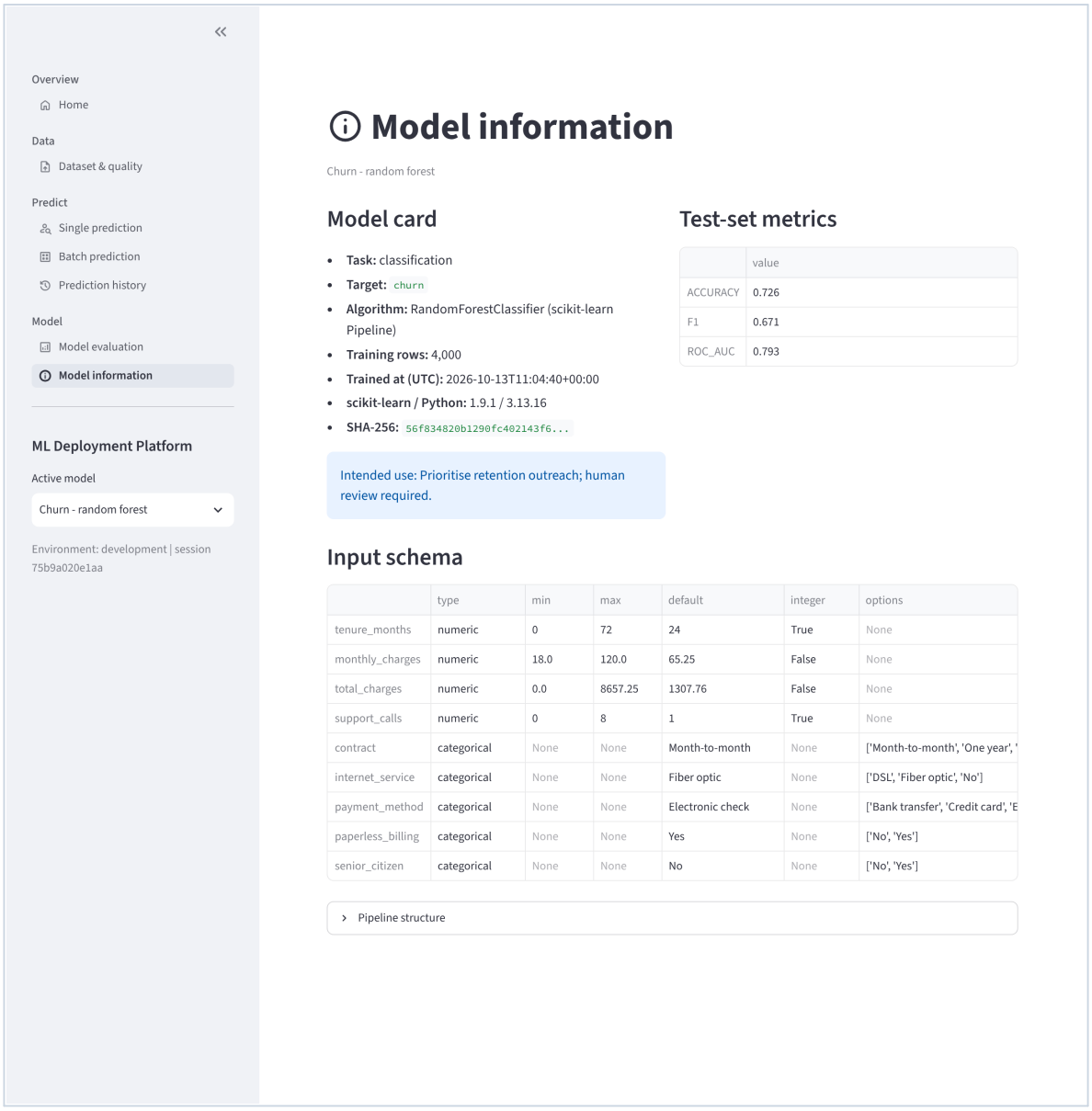


Figure 22.9: Model information: model card, metrics and input schema

22.8 Testing checklist

- [ ] `pytest -q` passes: every page renders for every registered model.
- [ ] Uploading a file with a missing column shows the column name on the quality page and blocks batch scoring.
- [ ] Non-numeric values in numeric columns are reported, not crashed on.
- [ ] Switching the active model clears stale batch results.
- [ ] History persists after restarting the server (database file / Postgres).
- [ ] Excel and PDF downloads open without warnings.
- [ ] Evaluation metrics on the held-out file match the training metadata.
- [ ] Renaming a model file (or editing its hash) shows a clear load error.
- [ ] An exception inside a page shows the friendly message and is logged.
- [ ] Layout works at phone width and in dark mode.

22.9 Deployment checklist

- [ ] Set `environment = "production"` in settings.toml (hides debug details).
- [ ] Provide `DATABASE_URL` for PostgreSQL via secrets/environment.
- [ ] Commit `models/` (small) or download artifacts at start-up with integrity checks.
- [ ] Build and run the Docker image: `docker build -t ml-platform .` and `docker run -p 8501:8501 -e DATABASE_URL=... ml-platform`.
- [ ] Or deploy on Community Cloud with main file `capstone/app.py` and secrets.
- [ ] Restrict access (`st.login()` or private app) if data is not public.
- [ ] Verify the health endpoint `/_stcore/health` and logs after deployment.

22.10 Future enhancements

- MLflow model registry and experiment tracking instead of `registry.json`.
- Background batch jobs with a queue for very large files.
- Drift monitoring page (PSI per feature) with alerts.
- Role-based access: analysts predict, admins register models.
- SHAP explanations on the single-prediction page.
- A FastAPI service sharing `services/` for system-to-system predictions.
- Internationalisation (Hindi/Marathi labels) and accessibility review.

22.11 Presenting the capstone as a portfolio project

Element	What to include
Repository	Clean structure, README with screenshots/GIF, setup in under five commands, licence
Live demo	A deployed URL (with sample data) in the README and on LinkedIn
Problem framing	Who uses it, which decision it supports, what success looks like
Engineering	Pipelines, registry, validation, tests, CI, Docker: say why each exists
Evaluation	Held-out metrics, baseline comparison, known limitations
Responsible ML	Intended use, data privacy, human oversight
Talking points	One hard bug you fixed, one design trade-off, what you would build next

TRY IT

Record a two-minute screen video: upload the sample file, show the quality report, score it, download the PDF, and open the evaluation page. Add it to your README and resume.

APPENDIX A

Glossary, Resources and Final Checklist

A.1 Glossary

Term	Meaning
AppTest	Streamlit's testing framework (`streamlit.testing.v1`) that runs an app headlessly and lets tests set widgets and inspect elements.
Artifact	A file produced by training: model, encoder, metadata, metrics.
Batch scoring	Running a model on many rows at once, usually from a file.
Cache key	The hash of a cached function's code and arguments that decides whether a stored result can be reused.
Callback	A function passed to `on_click`/`on_change` that runs before the next script run.
ColumnTransformer	scikit-learn object that applies different preprocessing to different columns.
Container (Docker)	A running instance of an image.
Container (Streamlit)	A layout block (`st.container`) that groups elements.
Data drift	A change in the distribution of input data compared with training.
Deserialisation	Turning a saved file back into a Python object; unsafe for untrusted pickles.
Fragment	A function decorated with `@st.fragment` that reruns independently.
Held-out data	Labeled data not used for training, reserved for evaluation.
Image (Docker)	A packaged, read-only filesystem with code and dependencies.
Inference	Using a trained model to make predictions.
MAE / RMSE	Mean absolute error / root mean squared error (regression).
Material icon	Icon referenced as `:material/name:` in Streamlit.
Metadata (model)	JSON describing a model: features, classes, metrics, versions, fingerprint.
OIDC	OpenID Connect, the login standard used by `st.login()`.
Pipeline	scikit-learn object chaining preprocessing and a model so they are fitted and saved together.
Prediction interval	A range expected to contain a future observation with a stated probability, produced by the model's uncertainty estimate.
PSI	Population Stability Index, a drift measure.
Registry (model)	The list of approved models and how to load them.
Rerun	One top-to-bottom execution of a Streamlit script.
ROC-AUC	Area under the ROC curve; ranking quality of a binary classifier.
Secrets	Credentials supplied via `st.secrets` or environment variables.
Session	One browser tab's connection to a Streamlit server, with its own state.
Session state	`st.session_state`: per-session key-value store.

Term	Meaning
Session affinity	Load-balancer setting that keeps a user on one server instance.
Threshold	Probability cut-off that turns a score into a class label.
WebSocket	The persistent browser-server connection Streamlit uses.
Widget	An interactive input element that returns a value.

A.2 Official documentation resources

Resource	URL
Streamlit documentation	https://docs.streamlit.io/
Streamlit API reference	https://docs.streamlit.io/develop/api-reference
Streamlit tutorials	https://docs.streamlit.io/develop/tutorials
Streamlit deployment	https://docs.streamlit.io/deploy
Streamlit release notes	https://docs.streamlit.io/develop/quick-reference/release-notes
Community Cloud status and limitations	https://docs.streamlit.io/deploy/streamlit-community-cloud/status
Community Cloud app dependencies	https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/app-dependencies
scikit-learn documentation	https://scikit-learn.org/stable/
scikit-learn model persistence	https://scikit-learn.org/stable/model_persistence.html
TensorFlow / Keras	https://www.tensorflow.org/
PyTorch documentation	https://pytorch.org/docs/stable/index.html
FastAPI	https://fastapi.tiangolo.com/
SQLAlchemy	https://docs.sqlalchemy.org/
Docker documentation	https://docs.docker.com/
Plotly Python	https://plotly.com/python/

> **NOTE**
> API details, hosting limits and version-specific behaviour change over time. This guide was verified against Streamlit 1.65.0 (October 2026); consult the release notes when upgrading.

A.3 Final deployment checklist

Model

- ☐ Trained by a reproducible script from a clean environment.
- ☐ Evaluated on held-out data; metrics and baseline recorded.
- ☐ Preprocessing inside the saved pipeline or a shared module.
- ☐ Metadata JSON: features, classes, versions, metrics, fingerprint.
- ☐ Intended use and limitations documented.

Application

- ☐ Model loaded with `@st.cache_resource` and integrity check.

- [ ] All inputs bounded and validated; training-range warnings.
- [ ] Uploads limited by type, size and rows; parsed safely.
- [ ] Clear errors; details logged, not shown to viewers.
- [ ] Long work shows spinner/progress/status.
- [ ] Results downloadable; history persisted if required.

Security

- [ ] No secrets in code, Git or images.
- [ ] No untrusted pickle/joblib loading.
- [ ] Authentication for non-public data; per-user data filtering.
- [ ] Logs contain no unnecessary personal data.
- [ ] Dependencies pinned and scanned.

Operations

- [ ] `requirements.txt` pinned; app runs from a fresh clone.
- [ ] Tests pass in CI.
- [ ] Docker image builds; health check passes.
- [ ] Secrets configured on the platform.
- [ ] Monitoring for errors, latency and drift in place.
- [ ] Rollback plan: previous image/model kept.